SWAT+ Documentation

SWAT+ Documentation

docs.swat.tamu.edu

Contents

SWAT+ Documentation **12**
About this site . . . 12

Getting Started **13**
Pages . . . 13
What it computes . . . 13
What it is used for . . . 14
How it is run . . . 14
Source and license . . . 14
Code . . . 14
Inputs . . . 14
Routing . . . 15
Management . . . 15
Aquifers . . . 15
Outputs . . . 15
Migration . . . 15
License . . . 16
How to cite . . . 16
Source . . . 16
Versioning . . . 16
Tagging policy . . . 16
Where to find what changed . . . 16
Option 1: Prebuilt executable . . . 17
Option 2: Build from source . . . 17
Supporting tools . . . 17
Verifying the install . . . 18
What it is . . . 18
Prerequisites . . . 18
Run it . . . 18
What you should see . . . 18
If it fails . . . 19
Next . . . 19
The master file: `file.cio` . . . 19
File groups . . . 20
Next . . . 22
What you need . . . 22
Step 1: Delineate the watershed (QSWAT+) . . . 22
Step 2: Create HRUs . . . 22
Step 3: Build the input files (SWAT+ Editor) . . . 23
Step 4: Run . . . 23
Step 5: Look at output . . . 23
Step 6: Calibrate . . . 23
Common first-run problems . . . 23
Overview . . . 24

Pages in this section . . . 24
How to read these pages . . . 24
Overview . . . 24
Process equations . . . 25
Switches and parameters . . . 25
Implementation . . . 25
Related pages . . . 26
Overview . . . 26
Process equations . . . 26
Switches and parameters . . . 27
Implementation . . . 27
Related pages . . . 28
Overview . . . 28
Process equations . . . 28
Switches and parameters . . . 30
Implementation . . . 30
Related pages . . . 31
Overview . . . 31
Process equations . . . 31
Switches and parameters . . . 32
Implementation . . . 33
Related pages . . . 33
Overview . . . 33
Process equations . . . 33
Switches and parameters . . . 35
Implementation . . . 35
Related pages . . . 36
Overview . . . 36
Process equations . . . 36
Land use management . . . 38
Switches and parameters . . . 38
Implementation . . . 39
Related pages . . . 39
Overview . . . 39
Process equations . . . 40
In-stream nutrient routing (QUAL2E) . . . 42
Switches and parameters . . . 42
Implementation . . . 43
Related pages . . . 43
Overview . . . 43
Process equations . . . 44
Switches and parameters . . . 45
Implementation . . . 46
Related pages . . . 46
Overview . . . 47
Process equations . . . 47
Switches and parameters . . . 48
Implementation . . . 49
Related pages . . . 49
Overview . . . 49
Process equations . . . 49
Switches and parameters . . . 51
Implementation . . . 51
Related pages . . . 52
Overview . . . 52
Process equations . . . 52

Switches and parameters . . . 54
Implementation . . . 54
Related pages . . . 55
Overview . . . 55
Process equations . . . 55
Switches and parameters . . . 56
Implementation . . . 57
Related pages . . . 58
Overview . . . 58
Process equations . . . 58
Switches and parameters . . . 59
Implementation . . . 60
Related pages . . . 61
Overview . . . 61
Process equations . . . 61
Switches and parameters . . . 62
Implementation . . . 62
Related pages . . . 63
Overview . . . 63
Process equations . . . 64
Switches and parameters . . . 66
Implementation . . . 66
Related pages . . . 66
Simulation control . . . 67
Basin parameters . . . 67
Spatial objects and connectivity . . . 67
Structural BMPs . . . 67
Climate . . . 67
Soils and land use . . . 67
Initial conditions . . . 68
Channels, aquifers, reservoirs, wetlands . . . 68
Groundwater . . . 68
Operation and management databases . . . 68
Water allocation . . . 68
Solute transport . . . 68
Pollutants . . . 68
Verification procedure . . . 69
Purpose . . . 69
Source . . . 69
Format . . . 69
Categories and fields . . . 69
Example . . . 70
Related files . . . 71
Purpose . . . 71
Source . . . 71
Format . . . 71
Example . . . 72
Related files . . . 72
Purpose . . . 72
Source . . . 72
Format . . . 72
Example . . . 74
Related files . . . 74
Purpose . . . 74
Source . . . 75
Format . . . 75

Example . . . 76
Related files . . . 76
Purpose . . . 76
Source . . . 76
Format . . . 76
Side effects after read . . . 77
Example . . . 77
Related files . . . 78
Purpose . . . 78
Source . . . 78
Format . . . 78
Example . . . 81
Related files . . . 81
Purpose . . . 81
Source . . . 81
Activation . . . 81
Format . . . 82
Example . . . 85
Calibration . . . 85
Related files . . . 85
Purpose . . . 85
Source . . . 85
Activation . . . 86
Format . . . 86
Example . . . 88
Calibration . . . 88
Related files . . . 88
Purpose . . . 88
Source . . . 89
Format . . . 89
Example . . . 90
Related files . . . 91
Purpose . . . 91
Source . . . 91
Format . . . 91
Example . . . 92
Related files . . . 93
Purpose . . . 93
Source . . . 93
hydrology.hyd . . . 93
topography.hyd . . . 95
field.fld . . . 97
shade_factor.shf . . . 99
Related files . . . 101
Purpose . . . 101
Source . . . 101
recall.rec (master database) . . . 101
salt_recall.rec (salt side database) . . . 102
cs_recall.rec (constituent side database) . . . 103
Element and region files . . . 103
Example . . . 103
Related files . . . 104
Purpose . . . 104
weather-sta.cli . . . 104
weather-wgn.cli . . . 105
pcp.cli . . . 107

tmp.cli . . . 107
slr.cli . . . 108
hmd.cli . . . 108
wnd.cli . . . 109
pet.cli . . . 109
atmodep.cli . . . 110
Atmospheric deposition . . . 111
Related files . . . 112
Purpose . . . 112
Source . . . 113
Format . . . 113
Example . . . 114
Related files . . . 114
Purpose . . . 114
Source . . . 114
Format . . . 114
Example . . . 115
Related files . . . 115
Purpose . . . 116
Source . . . 116
Format . . . 116
Example . . . 116
Related files . . . 116
Purpose . . . 116
Source . . . 117
Format . . . 117
Example . . . 119
Related files . . . 119
Purpose . . . 119
Source . . . 119
Format . . . 119
Example . . . 123
Related files . . . 123
Purpose . . . 123
Source . . . 123
Format . . . 123
Example . . . 124
Related files . . . 125
cntable.lum . . . 125
cons_practice.lum . . . 126
ovn_table.lum . . . 128
Related files . . . 130
Purpose . . . 130
Source . . . 130
channel.cha . . . 130
channel-lte.cha . . . 131
hyd-sed-lte.cha . . . 131
initial.cha . . . 133
temperature.cha . . . 133
Related files . . . 135
Purpose . . . 135
Source . . . 135
hydrology.cha . . . 135
sediment.cha . . . 137
nutrients.cha . . . 139
Related files . . . 142

Purpose . . . 142
Source . . . 142
aquifer.aqu . . . 142
initial.aqu . . . 144
Related files . . . 144
Purpose . . . 145
Source . . . 145
reservoir.res . . . 145
initial.res . . . 145
hydrology.res . . . 146
sediment.res . . . 148
nutrients.res . . . 148
weir.res . . . 149
Related files . . . 151
Purpose . . . 151
Source . . . 151
wetland.wet . . . 151
hydrology.wet . . . 152
Related files . . . 154
Purpose . . . 154
Source . . . 154
File format . . . 154
harv.ops . . . 154
graze.ops . . . 156
irr.ops . . . 158
chem_app.ops . . . 160
fire.ops . . . 162
sweep.ops . . . 164
Example . . . 166
Related files . . . 166
Purpose . . . 166
Source . . . 166
File format . . . 166
Example . . . 167
Related files . . . 168
Purpose . . . 168
Source . . . 168
Format . . . 168
Example . . . 171
Related files . . . 171
Purpose . . . 171
Source . . . 171
Format . . . 171
Example . . . 173
manure_db.frt . . . 173
manure_om.frt . . . 174
Related files . . . 175
Purpose . . . 175
Source . . . 175
Format . . . 175
Example . . . 177
Related files . . . 177
What replaces it . . . 177
Migration . . . 177
Related files . . . 178
Purpose . . . 178

Source . . . 178
Format . . . 178
Example . . . 178
Related files . . . 179
Purpose . . . 179
Source . . . 179
Format . . . 179
Example . . . 180
Related files . . . 180
management.sch . . . 180
puddle.ops . . . 182
transplant.plt . . . 184
co2.out and co2_yr.dat . . . 186
object.prt . . . 186
Purpose . . . 187
Source . . . 188
plant.ini . . . 188
soil_plant.ini . . . 192
om_water.ini . . . 194
Related files . . . 196
Purpose . . . 196
Source . . . 196
tiledrain.str . . . 197
septic.str . . . 199
septic.sep . . . 199
filterstrip.str . . . 200
grassedww.str . . . 202
bmpuser.str . . . 204
satbuffer.str . . . 206
urban.urb . . . 206
Related files . . . 208
Purpose . . . 208
Activation . . . 208
Files . . . 208
Related . . . 209
Purpose . . . 210
Activation . . . 210
Files . . . 210
Related . . . 211
Purpose . . . 211
Activation . . . 211
Files . . . 211
Related . . . 211
Purpose . . . 212
Activation . . . 212
Format conventions . . . 212
File overview . . . 212
`codes.gw`: feature flags and run controls . . . 213
`zones.gw`: aquifer and streambed zones . . . 213
`cells.gw`: cell properties and initial state . . . 214
`cellcon.gw`: cell-to-cell connections . . . 214
`outputs.gw`: output control . . . 214
`chancell.gw`: channel-to-cell links . . . 215
`gwflow.con`: channel-cell spatial connections . . . 215
`hrucell.gw`: HRU-to-cell mapping . . . 215
`wetland.gw`: wetland bed thickness . . . 215

`hru_pump.gw`: HRU pump-observation list . . . 215
Other conditional files . . . 216
`cell_sol.gw`: per-cell initial solute concentrations . . . 216
New flat optional files . . . 216
Example . . . 218
Related . . . 219
Purpose . . . 219
Activation . . . 219
Files . . . 219
Related . . . 221
Purpose . . . 221
Activation . . . 221
Files . . . 221
Related . . . 223
Purpose . . . 223
How it fits together . . . 224
Activation . . . 224
Files . . . 225
Related . . . 229
Naming convention . . . 230
Per-group pages . . . 231
Output index file . . . 231
Filename grammar . . . 231
Scopes . . . 232
Groups . . . 232
Intervals . . . 232
print.prt toggle . . . 233
Subdaily output . . . 233
Always-on outputs . . . 233
Purpose . . . 233
Source . . . 233
Format . . . 233
Aggregation . . . 235
Example . . . 235
Related files . . . 235
Purpose . . . 235
Legacy channel files . . . 235
sd_channel files . . . 237
Channel sediment budget and morphology . . . 238
Related files . . . 238
Aquifer . . . 239
Reservoir . . . 239
Crop yield . . . 240
Plant and weather . . . 241
Related files . . . 242
Purpose . . . 242
Source . . . 242
File naming . . . 242
Per-family print.prt gating . . . 242
Legacy CSU carbon outputs . . . 243
HRU files . . . 244
LSU files . . . 245
Optional supporting file . . . 246
Related pages . . . 246
Purpose . . . 246
Source . . . 247

Output files . . . 247
print.prt toggles . . . 249
Related pages . . . 249
simulation.out . . . 249
diagnostics.out . . . 250
checker.out . . . 250
files_out.out . . . 251
erosion.out . . . 251
success.fin . . . 251
erosion.txt . . . 252
Recommended checks after every run . . . 252
Related files . . . 252
test/spcheck.py . . . 252
test/swat_output_to_csv_files.py . . . 253
Related files . . . 255
Pages . . . 255
Generate the build files . . . 255
Pick a compiler . . . 255
Debug vs release . . . 256
Compile and link . . . 256
Install . . . 256
Linux note on libgfortran . . . 256
Required software . . . 257
Starting Visual Studio . . . 257
Cloning the repository . . . 257
Configuring, compiling, running . . . 258
Resources . . . 259
Cost . . . 259
Starting an instance . . . 259
Building . . . 259
Running and debugging . . . 260
Using your own dataset . . . 260
Stopping the instance . . . 261
Deleting the instance . . . 261
How it works . . . 261
Running the tests . . . 262
Adding a new scenario . . . 262
CI . . . 263
How the version is read . . . 263
Listing available tags . . . 263
Building at a specific tag . . . 263
How the version reaches the source . . . 264
Why tag-based versioning . . . 264
Interoperability and portability . . . 265
Readability . . . 265
Robustness . . . 266
Arrays . . . 266
Dynamic memory and pointers . . . 266
Looping . . . 267
Functions and procedures . . . 267
I/O . . . 267
Obsolescent features to avoid . . . 268
Source files . . . 268
References . . . 268

Code Of Conduct **270**
Where work happens . . . 270

Reporting a bug . . . 270
Submitting a change . . . 270
What reviewers look for . . . 270
CI . . . 271
Documentation changes . . . 271
Pages . . . 271
Components . . . 271
HRU water balance . . . 272
Channel sediment . . . 272
Plant growth . . . 272
Outputs . . . 273
Input files . . . 273
Related pages . . . 273
Source . . . 273
Workflow . . . 273
Object types . . . 274
Re-initialization . . . 274
Related pages . . . 274
Purpose . . . 274
Source . . . 275
cal_parms.cal . . . 275
calibration.cal . . . 275
Carbon parameters . . . 277
Related files . . . 285
Source . . . 285
Two condition syntaxes . . . 285
Supported variables . . . 285
Regionalization . . . 286
Decision-table conditional updates . . . 286
Parameter dispatch . . . 286
Related pages . . . 286
Pattern . . . 286
sensitivityAPI . . . 286
Roll your own . . . 287
Related pages . . . 287
Pages . . . 287
Prerequisites . . . 288
file.cio: the index of every other input . . . 288
simulation group . . . 288
basin group . . . 289
climate group . . . 289
connect, hru, hydrology groups . . . 289
hru_parm_db, ops, lum groups . . . 290
init, soils, decision_table groups . . . 290
carbon database . . . 290
Run the model . . . 291
Read the outputs . . . 291
What to change next . . . 292
Prerequisites . . . 292
file.cio: every group used . . . 292
time.sim and print.prt . . . 293
codes.bsn . . . 293
climate . . . 293
hru.con: a single HRU . . . 293
hru-data.hru . . . 294
landuse.lum and management.sch . . . 294

aquifer.aqu and aquifer.con . . . 294
channel-lte.cha and hyd-sed-lte.cha . . . 294
Run the model . . . 294
Read the outputs . . . 294
What to change next . . . 295
What it reads . . . 295
What it draws . . . 296
How to run it . . . 296
What you should see . . . 296
Related . . . 296
Add a new HRU . . . 296
Swap the PET method . . . 297
Switch to Green-Ampt infiltration . . . 297
Change the simulation period . . . 297
Enable the carbon model . . . 298
How it is built . . . 298
When to use FORD vs this site . . . 299
Related . . . 299
Pages . . . 299
When in doubt . . . 299
Missing input references (diagnostics.out) . . . 299
Errors written to stdout . . . 301
Failure modes that are not text errors . . . 301
Related . . . 302
What to compare . . . 302
Symptom: precip is wrong . . . 302
Symptom: surq_gen is too high or too low . . . 302
Symptom: ET is too high or too low . . . 303
Symptom: management never fires . . . 303
Symptom: percolation is zero or saturated . . . 303
Symptom: bad latq, qtile, or surq_runon . . . 303
Symptom: ru_aa.txt and basin_wb_aa.txt disagree . . . 304
Workflow . . . 304
Related . . . 304
Build with `-j` . . . 304
Choose an optimized build type . . . 304
Daily versus sub-daily routing . . . 304
The cost of enabling many outputs . . . 305
Object count . . . 305
I/O . . . 305
Profiling . . . 305
Summary . . . 305

SWAT+ Documentation

A watershed model. Plain-text inputs. Plain-text outputs. Open source.

Get started What is SWAT+

- **Getting started**

`Install SWAT+, run the reference dataset, and learn the shape of a project folder.`

- **Model theory**

`Hydrology, soil, plants, channels, aquifers, reservoirs, nutrients, carbon, pesticides, salts.`

- **Input reference**

`Every input file, with columns and units verified against the Fortran source.`

- **Output reference**

`Every output file group, with column lists and aggregation rules.`

- **Developer guide**

`Build from source, test, tag releases, contribute.`

- **Calibration**

`Soft and hard calibration workflows.`

- **Tutorials**

`Slow walks through the Ames_sub1 and Osu_1hru reference datasets.`

- **FAQ**

`Common runtime errors, water-balance issues, performance tips.`

About this site

Every input-reference and output-reference page is verified against the SWAT+ Fortran source at a specific commit, recorded in the page frontmatter. Known bugs and source/dataset discrepancies are documented in the **Important** section at the bottom of each affected page. Source: github.com/swat-model/swatplus.

Getting Started

This section gets you from a fresh machine to a running SWAT+ simulation.

If you have never run the model before, follow the pages in order:

1. Install. Get the SWAT+ executable on your machine.
2. Quick start: Ames_sub1. Run the small reference dataset shipped with the source repository.
3. Project folder anatomy. Understand what is in a SWAT+ project directory and what each group of files does.
4. Your first watershed. Build a project for your own area using QSWAT+ and run it.

If you only want to read about the model first, skip ahead to Model theory.

SWAT+ is a watershed model. It simulates water, sediment, nutrient, pesticide, and crop yield response to climate and management at scales from a single field up to a large river basin.

This section covers what the model is, how it differs from the original SWAT, how to cite it, and how releases are numbered.

Pages

- What is SWAT+
- SWAT vs SWAT+
- Citing and license
- Release notes

The Soil and Water Assessment Tool Plus (SWAT+) is a public-domain watershed model. It is jointly developed by the USDA Agricultural Research Service (USDA-ARS) and Texas A&M AgriLife Research, with contributions from Colorado State University, the USDA-ARS Hydrology and Remote Sensing Laboratory, and other institutions.

SWAT+ simulates the quantity and quality of surface water and groundwater at scales from small watersheds to large river basins. It is used to evaluate the effects of land use, land management practices, soil, climate, and climate change on water resources.

What it computes

For each hydrologic response unit (HRU), routing reach, aquifer, reservoir, and wetland in a watershed, SWAT+ computes a daily mass balance for:

- water (precipitation, evapotranspiration, surface runoff, lateral flow, percolation, return flow, channel routing)
- sediment (detachment, transport, deposition)
- nutrients (nitrogen and phosphorus cycling in soil, plant uptake, transport in surface and subsurface flow, in-stream transformations)
- carbon (soil organic carbon pools)
- pesticides (application, decay, transport)
- salts and other constituents
- plant growth (biomass, yield, water and nutrient stress)

Output is reported daily, monthly, yearly, or as an annual average.

What it is used for

Typical applications:

- assessing the effect of management practices on water yield and water quality
- evaluating soil erosion and non-point source pollution
- estimating crop yield response to climate and management
- supporting watershed planning and total maximum daily load (TMDL) studies
- climate change impact studies on water resources

How it is run

SWAT+ is a Fortran command-line executable. It reads a folder of plain-text input files that describe the watershed, climate, soil, land use, management, and simulation period. It writes output as plain-text files in the same folder.

Most users do not write the input files by hand. They build a project with one of the supporting tools:

- **QSWAT+** is a QGIS plugin that delineates the watershed from a DEM, creates HRUs, and writes the input files.
- **SWAT+ Editor** is a desktop application for inspecting and editing input parameters in an existing project.

After the tools write the project folder, the SWAT+ executable is run from that folder.

Source and license

The source code is hosted at https://github.com/swat-model/swatplus and released under LGPL-2.1.

SWAT+ is the modernised successor to SWAT (Soil and Water Assessment Tool). The science is largely the same. The code structure, input file layout, and connectivity model are different.

The SWAT (2012) column below is sourced from the legacy model repository at `swat-model/swat2012`.

Code

	SWAT (2012)	SWAT+
Language	Fortran (fixed-form)	Fortran free-form (`.f90`)
Structure	Large shared common blocks	Modular, files grouped per process (`aqu_*`, `channel_*`, `hru_*`, `gwflow_*`, …)
Source	Distributed as archives	Git on GitHub, public, LGPL-2.1
Build	Hand-written Makefiles	CMake

The CMake build for SWAT+ uses `-free` (Intel) or `-ffree-line-length-none` (GNU) and glob-compiles `src/*.f90` into a single executable. See `CMakeLists.txt` in the source repository.

Inputs

SWAT2012 used a set of input file extensions tied to subbasins (`.sub`, `.rte`, `.gw`, `.sol`, etc.) with one file per HRU and subbasin.

SWAT+ replaces this with an object-based layout. The watershed is a graph of objects (HRUs, channels, aquifers, reservoirs, point sources, recall objects, decision tables) that are wired together through connectivity files (`.con`). The connectivity slots are defined by the `input_con` type in `src/input_file_module.f90`:

```
hru.con, hru-lte.con, rout_unit.con, gwflow.con, aquifer.con,
aquifer2d.con, channel.con, reservoir.con, recall.con, exco.con,
delratio.con, outlet.con, chandeg.con
```

Inputs are grouped by purpose, not by spatial unit:

- climate: `*.cli`, `*.pcp`, `*.tmp`, `weather-sta.cli`, `weather-wgn.cli`
- land use management: `landuse.lum`, `management.sch`, `cntable.lum`, `cons_practice.lum`, `ovn_table.lum`
- soils: `soils.sol`, `nutrients.sol`
- decision tables: `lum.dtl` (rule-based management triggers)
- hydrology: `hydrology.hyd`, `topography.hyd`
- basin-level: `codes.bsn`, `parameters.bsn`, `time.sim`, `print.prt`
- master index: `file.cio`

Routing

SWAT2012 routed water from subbasin to subbasin in a fixed order based on subbasin numbering.

SWAT+ uses the `*.con` files to describe arbitrary connectivity between HRUs, routing units, channels, aquifers, reservoirs, and exit points. Routing follows the graph defined in those files. This makes it possible to route landscape units (LSUs) within an HRU to a channel via a routing unit, or to send flow to a wetland or reservoir before the channel.

Management

SWAT2012 used `mgt` files with a fixed sequence of operations by date and heat unit.

SWAT+ separates the schedule (`management.sch`) from rule-based triggers (`*.dtl`, decision tables). A decision table evaluates conditions (soil water, plant heat units, day of year, etc.) at each time step and fires an operation when the condition is met. The same schedule can be reused across HRUs with different decision tables driving when operations occur.

Aquifers

SWAT2012 had a single shallow-aquifer storage per subbasin.

SWAT+ supports 1D aquifer objects, defined in `src/aquifer_module.f90` and read by `src/aqu_read.f90`, with one aquifer per landscape unit or group. It also includes a 2D groundwater module rooted in `src/aqu2d_read.f90`, `src/aqu2d_init.f90`, and the `gwflow_*` source files, where the aquifer is discretised in plan view and exchanges water laterally with channels, the unsaturated zone, reservoirs, and other objects.

Outputs

SWAT2012 wrote a fixed set of files (`output.hru`, `output.sub`, `output.rch`, `output.std`).

SWAT+ writes one file per object type and aggregation level, controlled by `print.prt`. Examples from `refdata/Ames_sub1`:

```
basin_wb_aa.txt        basin water balance, annual average
hru_wb_aa.txt          HRU water balance, annual average
basin_cha_aa.txt       channel, annual average
basin_aqu_aa.txt       aquifer, annual average
```

The aggregation suffix is `_day`, `_mon`, `_yr`, or `_aa`. Each object row in `print.prt` toggles `daily`, `monthly`, `yearly`, and `avann` independently with `y` or `n`.

Migration

There is no automatic converter from a SWAT2012 project to a SWAT+ project. The recommended path is to rebuild the project in QSWAT+ from the same DEM, soils, land use, and climate inputs, then transfer calibrated parameter changes by hand.

License

SWAT+ is released under the GNU Lesser General Public License, version 2.1 (LGPL-2.1). The full text is in the `LICENSE` file in the source repository.

How to cite

Cite the software release using the DOI:

> Arnold, J.G., Čerkasova, N., White, M.J., Bailey, R., Thorp, K., Jeong, J., Zhang, X., Ugraskan, T., van Griensven, A., Rathjens, H., Raj, C., Cai, X., Geter, W.F., David, O., Carlson, J.R., Le, K.N. *Soil and Water Assessment Tool Plus (SWAT+)*. https://doi.org/10.5281/zenodo.14983534

The DOI `10.5281/zenodo.14983534` resolves to the current release; each tagged release also has its own DOI on Zenodo.

The machine-readable citation lives in `CITATION.cff` in the source repository. GitHub renders a "Cite this repository" button from this file.

Source

- Code: https://github.com/swat-model/swatplus
- Older versions (pre-GitHub): https://bitbucket.org/blacklandgrasslandmodels/modular_swatplus

SWAT+ releases are tagged on the GitHub releases page. Each tag corresponds to a Zenodo DOI under the concept DOI `10.5281/zenodo.14983534`.

Versioning

Releases are identified by a git tag. Tag names look like `61.0` or `61.1` for the current series, and earlier revisions in the reference data appear as `60.5.7`. The tag is read at configure time by `git describe --tags` (see `CMakeLists.txt`) and projected into the executable name and into the banner string in `main.f90`.

The executable name produced by CMake follows the pattern:

```
swatplus-<tag>-<compiler>-<arch>[-Rel|-Dbg]
```

where `<compiler>` is `gnu`, `ifo` (Intel `ifort`), or `ifx` (Intel LLVM), and `<arch>` is the host architecture. On Windows the executable has a `.exe` extension. This is defined in `CMakeLists.txt` as:

```
set(SWATPLUS_EXE "swatplus-${SWAT_VERSION}-${FFC}-${AR}${TY}")
```

A build between tags resolves to a `git describe` string of the form `<tag>-<n>-g<sha>`, for example `61.1-1-g92b4413`. See `doc/Tagging.md` in the source repository.

Tagging policy

See Tagging and releases in the developer guide for the rules a contributor follows when cutting a new tag.

Where to find what changed

For each release, the GitHub release page lists merged pull requests since the previous tag. The full commit history is available via:

```
git log <previous-tag>..<current-tag> --oneline
```

A change log curated for users (input file changes, output format changes, behaviour changes) is maintained in this section as releases are cut.

You have two options.

1. **Download a prebuilt executable** if you only want to run SWAT+.
2. **Build from source** if you want to modify the model or use a development version.

Option 1: Prebuilt executable

Prebuilt binaries for Windows, Linux, and macOS are attached to each GitHub release.

1. Go to https://github.com/swat-model/swatplus/releases.
2. Pick the latest release (or a specific tagged version).
3. Under **Assets**, download the file that matches your operating system and architecture.

Binaries are named `swatplus-<version>-<compiler>-<os>_<arch>` with an optional `-Rel` or `-Dbg` suffix and a `.exe` extension on Windows. The compiler tag is `gnu` (gfortran), `ifo` (Intel `ifort`), or `ifx` (Intel `ifx`). Examples:

- Linux x86_64, gfortran: `swatplus-3.0.4-gnu-lin_x86_64`
- Windows AMD64, gfortran: `swatplus-3.0.4-gnu-win_amd64.exe`
- macOS arm64, gfortran: `swatplus-3.0.4-gnu-mac_arm64`

Put the file somewhere on your `PATH`, or keep it next to your project folder. On Linux and macOS, make it executable:

```
chmod +x swatplus-3.0.4-gnu-lin_x86_64
```

Run it once with no arguments to confirm it starts:

```
./swatplus-3.0.4-gnu-lin_x86_64
```

It will look for input files in the current directory and exit if it does not find them. That is the expected behavior outside of a project folder.

Option 2: Build from source

You need:

- `git`
- `cmake` (3.22 or newer)
- a Fortran compiler: `gfortran`, Intel `ifort`, or Intel `ifx`
- `make` or `ninja`

Clone the source:

```
git clone https://github.com/swat-model/swatplus.git
cd swatplus
```

Configure and build:

```
cmake -B build
cmake --build build -j 8
```

The executable is written to `build/` using the `swatplus-<version>-<compiler>-<os>_<arch>` pattern (with `.exe` on Windows). To pick a specific compiler at configure time, pass `-D CMAKE_Fortran_COMPILER=gfortran` (or `ifort`, `ifx`).

To install it system-wide:

```
cmake --install build --prefix ~/bin
```

For platform-specific build instructions (Visual Studio on Windows, VS Code Codespaces, Intel compilers), see the Developer guide.

Supporting tools

To build SWAT+ inputs from a DEM, soil map, and land use map, install:

- **QSWAT+** (QGIS plugin): https://swat.tamu.edu/software/plus/. Delineates the watershed, defines HRUs, and writes the input files.
- **SWAT+ Editor** (desktop app): edits parameters in an existing project. Distributed from the same page.

The SWAT+ executable itself does not need either of these to run. They produce the input files that the executable reads.

Verifying the install

Use the reference dataset in `swatplus/refdata/Ames_sub1` to confirm the executable runs end-to-end. See Quick start: Ames_sub1.

Run a complete SWAT+ simulation in a few minutes using the small reference dataset shipped with the source repository.

What it is

`Ames_sub1` is a twelve-HRU dataset near Ames, Iowa. It simulates 1975 through 2020 on a daily time step (see `time.sim`). The dataset is in the SWAT+ source tree at `refdata/Ames_sub1`.

Prerequisites

- A SWAT+ executable. See Install.
- A copy of the `Ames_sub1` folder. If you cloned the source repository, it is at `swatplus/refdata/Ames_sub1`. Otherwise, download it from the link above.

Run it

1. Copy the executable into the `Ames_sub1` folder, or note its full path.
2. From a terminal, change into the `Ames_sub1` folder:

   ```
   cd path/to/Ames_sub1
   ```

3. Run the model:

   ```
   # Linux / macOS
   ./swatplus-3.0.4-gnu-lin_x86_64

   # Windows
   swatplus-3.0.4-gnu-win_amd64.exe
   ```

A simulation of this size takes a few seconds to a minute depending on your machine.

What you should see

While running, SWAT+ writes progress to the console and to `simulation.out`. When it finishes successfully, the last lines of `simulation.out` contain:

```
Execution successfully completed
```

The same line is written to `success.fin`.

The folder now contains a set of output files. With the default `print.prt` for this dataset, every `objects` row is set to `avann = y`, so each enabled object group writes an annual-average file. The annual-average files that appear in `refdata/Ames_sub1` after a successful run include:

```
basin_wb_aa.txt           basin water balance
basin_nb_aa.txt           basin nutrient balance
basin_ls_aa.txt           basin landscape (sediment)
basin_pw_aa.txt           basin plant and water stress
basin_aqu_aa.txt          basin aquifer
basin_cha_aa.txt          basin channel (legacy routing)
basin_sd_cha_aa.txt       basin channel (SWAT-DEG routing)
basin_res_aa.txt          basin reservoir
basin_psc_aa.txt          basin point-source / recall
basin_crop_yld_aa.txt     basin crop yields
crop_yld_aa.txt           crop yields (HRU detail)
hru_wb_aa.txt             HRU water balance
hru_nb_aa.txt             HRU nutrient balance
hru_ls_aa.txt             HRU landscape balance
hru_pw_aa.txt             HRU plant and water stress
```

Open `basin_wb_aa.txt` to see the simulated water balance. The columns are documented in Water balance outputs.

If it fails

- **`Error opening file...`** Check that you are running the executable from inside the `Ames_sub1` folder, not from its parent.
- **`Permission denied`** on Linux or macOS. Run `chmod +x` on the executable.
- The model exits before writing output. Check `diagnostics.out` and `checker.out` for the first error.

For other errors, see FAQ and troubleshooting.

Next

- Read Project folder anatomy to learn what each file in `Ames_sub1` does.
- Walk through the dataset slowly in the Ames_sub1 walkthrough.
- Build your own watershed in Your first watershed.

A SWAT+ project is a single flat folder of plain-text files. The executable reads everything in that folder and writes outputs back into the same folder.

There are no subdirectories. The grouping below is by purpose, not by location.

The master file: `file.cio`

`file.cio` is the index. It tells SWAT+ which file names to read for each category of input. Every line names a category and the file(s) to read. The literal token `null` means "no file for this slot". Full column reference: `file.cio`.

Excerpt from `Ames_sub1/file.cio`:

```
file.cio: AMES
simulation        time.sim          print.prt         null              object.cnt        null
basin             codes.bsn         parameters.bsn
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli           tmp.cli
connect           hru.con           null              null              null              null
hru               hru-data.hru      null
aquifer           nbull             null
hydrology         hydrology.hyd     topography.hyd    null
hru_parm_db       plants.plt        fertilizer.frt    tillage.til       pesticide.pes     null
ops               harv.ops          graze.ops         irr.ops           chem_app.ops      fire.ops
lum               landuse.lum       management.sch    cntable.lum       cons_practice.lum ovn_table.lum
init              plant.ini         soil_plant.ini    null              null              null
```

```
soils             soils.sol         nutrients.sol     null              null
decision_table    lum.dtl           null              null              null
regions           ls_unit.ele       null              null              null              null
```

Read this file first when you open a project. It tells you what every other file is for.

File groups

Simulation control

File	What it controls
`time.sim`	Start and end dates, time step
`print.prt`	Which output files to write, and at which aggregation (`_day`, `_mon`, `_yr`, `_aa`)
`object.cnt`	Object counts (how many HRUs, channels, aquifers, etc.)

Basin parameters

File	What it controls
`codes.bsn`	Method switches (which equation set to use for ET, runoff, routing, …)
`parameters.bsn`	Basin-wide numeric parameters

Climate

File	What it controls
`weather-sta.cli`	List of weather stations and which files belong to each
`weather-wgn.cli`	Weather generator parameters (used to fill missing data)
`pcp.cli`, `tmp.cli`, …	Indexes pointing to the actual `.pcp`, `.tmp`, etc. data files
`<name>.pcp`, `<name>.tmp`	The daily precipitation and temperature time series

Spatial objects and connectivity

File	What it controls
`hru.con`	HRU connectivity (which object each HRU drains to)
`chandeg.con`, `aquifer.con`, `reservoir.con`	Channel, aquifer, reservoir connectivity
`rout_unit.con`	Routing unit connectivity
`hru-data.hru`	Per-HRU pointers to topography, soils, land use, management

Hydrology

File	What it controls
`hydrology.hyd`	Per-HRU hydrologic parameters
`topography.hyd`	Per-HRU slope, slope length, field length

Soils

File	What it controls
`soils.sol`	Soil layer properties (bulk density, available water, conductivity, …)
`nutrients.sol`	Initial soil nutrient pools

Land use and management

File	What it controls
`landuse.lum`	List of land use scenarios and the files that define each
`management.sch`	Operation schedules (planting, harvest, fertilizer, …)
`cntable.lum`	Curve number lookup
`cons_practice.lum`	Conservation practice factor
`ovn_table.lum`	Manning's n for overland flow
`lum.dtl`	Decision tables that trigger operations based on conditions

Plant, fertiliser, tillage, pesticide databases

File	What it controls
`plants.plt`	Plant parameter database
`fertilizer.frt`	Fertiliser composition
`tillage.til`	Tillage operation effects
`pesticide.pes`	Pesticide properties

Initial conditions

File	What it controls
`plant.ini`	Starting plant communities by HRU
`soil_plant.ini`	Starting soil water, residue, plant biomass

Aquifers, reservoirs, channels

File	What it controls
`aquifer.aqu`	Aquifer parameters
`reservoir.res`, `initial.res`	Reservoir parameters and initial state
`channel-lte.cha`, `hyd-sed-lte.cha`	Legacy channel routing
`hydrology.cha`, `sediment.cha`, `nutrients.cha`	SWAT+ channel routing parameters

Outputs (written by the model)

Files ending in `_day.txt`, `_mon.txt`, `_yr.txt`, or `_aa.txt`. The prefix tells you the spatial scope (`basin_`, `region_`, `lsunit_`, `hru_`, `channel_`, `aquifer_`, `reservoir_`, …) and the underscore code tells you the variable group:

Code	Variables
`_wb`	water balance
`_nb`	nutrient balance
`_ls`	landscape (sediment)
`_pw`	plant and water stress
`_aqu`	aquifer
`_cha` / `_sd_cha`	channel
`_res`	reservoir

See Output reference for column details.

Diagnostics

File	What it tells you
`simulation.out`	Run progress and successful completion message
`diagnostics.out`	Warnings and unexpected values during the run
`checker.out`	Validation messages produced before the simulation starts
`area_calc.out`, `files_out.out`, `erosion.txt`	Setup summaries

Always check these three (`simulation.out`, `diagnostics.out`, `checker.out`) when a run does not behave as expected.

Next

The full per-file reference lives in Input reference, with one page per file and a real example pulled from `Ames_sub1`.

This walks through the workflow to build a SWAT+ project for an area of your own, run it, and look at the output. It does not click through QSWAT+ screen by screen. It covers the steps and what each one produces so you know what to expect.

What you need

Data:

- A digital elevation model (DEM) covering your area, in a projected coordinate system, with no holes.
- A soil map, either SSURGO (US), HWSD (global), or a custom raster paired with a soil property lookup table.
- A land use map, either NLCD (US), ESA WorldCover (global), or a custom raster paired with a SWAT+ land use lookup table.
- A daily climate record (precipitation and temperature at minimum, optionally solar radiation, wind, humidity) from one or more stations covering the simulation period.
- An outlet point (the gauge or stream location you want to drain to).

Software:

- QGIS with the QSWAT+ plugin. See Install.
- SWAT+ Editor.
- The SWAT+ executable.

Step 1: Delineate the watershed (QSWAT+)

In QGIS, create a new QSWAT+ project and load the DEM. QSWAT+ will:

1. Fill sinks in the DEM.
2. Compute flow direction and accumulation.
3. Build the stream network at a threshold drainage area you choose.
4. Subdivide the watershed into subbasins at stream junctions.
5. Snap your outlet point to the stream network.

Output: a polygon layer of subbasins and a polyline layer of channels (reaches).

Step 2: Create HRUs

Overlay the land use map, soil map, and a slope class layer on the subbasins. Each unique combination of (subbasin, land use, soil, slope class) is a candidate HRU. QSWAT+ writes the HRUs and the connectivity files.

You can keep all candidate HRUs (high spatial detail, slow run) or filter by area threshold (faster, less detail).

Output: HRU definitions and `*.con` connectivity files.

Step 3: Build the input files (SWAT+ Editor)

Open the project in SWAT+ Editor. The editor will:

1. Import climate station data and write the `.cli`, `.pcp`, `.tmp`, and `weather-wgn.cli` files. Stations without data get filled by the weather generator.
2. Populate soil layer parameters from the soil property table.
3. Populate plant, fertiliser, tillage, and pesticide databases.
4. Let you assign management schedules to each land use.
5. Write the final input folder.

Set the simulation period in `time.sim` (typically include a one- to three-year warm-up at the start) and the output toggles in `print.prt`.

Output: a folder of plain-text SWAT+ inputs, ready to run.

Step 4: Run

Copy the SWAT+ executable into the project folder. From a terminal:

```
cd path/to/your/project
./swatplus-<version>-gnu-lin_x86_64
```

Watch `simulation.out` for the `Execution successfully completed` message. Check `diagnostics.out` and `checker.out` for warnings.

Step 5: Look at output

Start with the basin water balance:

`basin_wb_aa.txt`

The annual-average precipitation, evapotranspiration, surface runoff, lateral flow, percolation, and water yield should be in the range you expect for your climate. If precipitation is wildly off, your climate file mapping is wrong. If ET dominates everything, check land use and plant parameters.

Then look at the channel output (`basin_sd_cha_aa.txt` or the per-channel `channel_sd_aa.txt`) for streamflow.

If you have measured streamflow at the outlet, compare to `channel_sd_day.txt` for the gauge channel.

Step 6: Calibrate

The first run is rarely a good match to observed data. Move on to Calibration to adjust parameters and improve the fit.

Common first-run problems

- **No precipitation anywhere.** The climate station coordinates are outside the watershed extent, so QSWAT+ assigned every HRU to the weather generator instead of the station. Check `weather-sta.cli`.
- **Crops never grow.** The management schedule is not assigned, or the planting date is outside the simulation period.
- **All flow is surface runoff.** Curve numbers are too high. Check `cntable.lum` and the soil hydrologic group.
- **Streamflow is zero downstream.** A channel object has no upstream connection in `chandeg.con` or `hru.con`. Inspect the connectivity.

Overview

This section describes how SWAT+ represents the physical and management processes that move water, sediment, nutrients, and other constituents through a watershed. Each page focuses on one process cluster, lists the governing relationships, the basin codes and parameters that switch behaviour, and the source modules in `swatplus/src/` that implement it.

The clusters group together because they share state, run in the same daily order, or are configured through the same input files. The water balance cluster (HRU hydrology, soils, climate, erosion) feeds the routing cluster (channels, aquifers, reservoirs and wetlands, recall objects). The biogeochemistry cluster (plant growth, nutrient cycling, carbon, pesticides, salts and constituents) sits inside the HRU. Decision tables drive management and release rules across all clusters.

Pages in this section

Cluster	Page	What it covers
Water balance	Overview and water balance	Daily HRU water balance, object flow chart, sequencing
Water balance	Climate and weather	Measured, simulated, and lapsed weather; PET methods
Water balance	Hydrology	Runoff (CN, Green-Ampt), lateral flow, percolation, channel routing
Water balance	Soil water and temperature	Soil profile water, soil temperature, frozen soil
Water balance	Erosion and sediment	MUSLE on land, channel erosion and deposition
Routing	Channels	Channel routing methods, sediment transport, in-stream water quality
Routing	Aquifers	1D recession aquifer and 2D `gwflow` cell-based groundwater
Routing	Reservoirs and wetlands	Reservoir water balance, release rules, sediment and nutrient processes; HRU wetlands
Routing	Recall and point sources	Recall time series, export coefficients, delivery ratios
Routing	Decision tables	Runtime condition evaluation and action firing
Biogeochemistry	Plant growth and land use management	Heat units, biomass, yield, scheduled management
Biogeochemistry	Nutrient cycling	Soil N and P pools, transformations, transport
Biogeochemistry	Carbon	Static, CFARM, and Century carbon models
Biogeochemistry	Pesticides	Application, partitioning, decay, transport
Biogeochemistry	Salts and constituents	Salt ion chemistry and generic constituents

How to read these pages

Each page follows the same shape. The Overview names the process and the role it plays in the simulation. Process equations give the governing relationships in plain math. Switches and parameters lists the `codes.bsn` and `parameters.bsn` entries that change the result, with defaults from the source. Implementation lists the source files in `swatplus/src/` that perform the calculation. Related pages cross-link to the input and output references.

The Fortran source at `swatplus/src/` is the source of truth. Where this section conflicts with the source, the source wins. Anything still uncertain is marked `TODO: verify`.

Overview

SWAT+ solves a daily water balance for every hydrologic response unit (HRU). The land-phase balance partitions precipitation into surface runoff, lateral flow, percolation, evapotranspiration, and change in soil water storage. Channel routing, aquifer recharge, and reservoir storage build on top of this HRU-level balance. The HRU loop in `hru_control.f90` is the entry point for the daily land-phase calculation.

Process equations

The daily water balance for an HRU is

```
P = Q_surf + Q_lat + Q_perc + ET + dS
```

Term	Meaning
`P`	Daily precipitation reaching the soil surface (after canopy interception and snow processes)
`Q_surf`	Surface runoff routed to the channel during the day
`Q_lat`	Lateral subsurface flow leaving the soil profile
`Q_perc`	Percolation below the bottom of the soil profile (recharge to the shallow aquifer)
`ET`	Actual evapotranspiration (canopy evaporation, transpiration, soil evaporation, sublimation)
`dS`	Change in soil profile water storage for the day

Subsidiary terms that the model also tracks at the HRU level and accounts for in the balance:

- Tile flow `Q_tile`, computed in the percolation pass.
- Wetland and pothole storage changes (`wet_control`, ponded HRU types).
- Snow water equivalent, accumulated and melted in `sq_snom.f90`.
- Surface runoff stored over one day through the `surlag` lag (`sq_surfst.f90`).

The HRU water balance feeds the basin-scale and channel-scale balances. Outputs at HRU, landscape unit (LSU), and basin levels are written by the `*_output.f90` modules.

Switches and parameters

Switch / parameter	File	Effect
`bsn_cc%pet`	`codes.bsn`	Selects PET method (see Climate and Weather)
`bsn_cc%gampt`	`codes.bsn`	Curve number vs Green-Ampt infiltration (see Hydrology)
`bsn_cc%rte`	`codes.bsn`	Channel routing method
`bsn_prm%surlag`	`parameters.bsn`	Surface runoff lag, default 4.0 days
`bsn_prm%ffcb`	`parameters.bsn`	Initial soil water as fraction of field capacity, default 0.0
`hyd_db%esco`, `hyd_db%epco`	`hydrology.hyd`	Soil evap and plant uptake compensation factors
`hyd_db%canmx`	`hydrology.hyd`	Maximum canopy storage (mm)
`hyd_db%lat_ttime`	`hydrology.hyd`	Lateral flow travel time

Implementation

Source modules in `swatplus/src/` that implement the land-phase water balance:

- `hru_control.f90` orchestrates the daily HRU loop: canopy interception, snow, PET, surface runoff, percolation, lateral storage, evapotranspiration, and output accumulation.
- `sq_canopyint.f90` computes canopy interception storage.
- `sq_snom.f90` accumulates and melts snowpack.
- `sq_volq.f90` dispatches to curve number (`sq_daycn.f90`) or Green-Ampt (`sq_greenampt.f90`) for surface runoff.
- `swr_percmain.f90` is the master driver for layered soil percolation and lateral flow; it calls `swr_percmicro.f90`, `swr_percmacro.f90`, and `swr_satexcess.f90`.
- `swr_substor.f90` lags lateral and tile flow on the way to the channel.

- `sq_surfst.f90` applies `surlag` to delay surface runoff over more than one day.
- `et_pot.f90` and `et_act.f90` compute potential and actual evapotranspiration.
- `stor_surfstor.f90` updates surface storage from the depression-storage routine.
- `basin_output.f90`, `hru_output.f90`, and the `*_output.f90` family write the per-object water balance to `output.std` and the individual `*_wb` files.

Related pages

- Hydrology for surface runoff, lateral flow, percolation, and channel routing.
- Climate and Weather for the precipitation and PET inputs that drive the balance.
- Soil Water and Temperature for soil profile storage.
- Aquifers for what happens to `Q_perc` after it leaves the soil profile.
- `codes.bsn` and `parameters.bsn` for switches and coefficients.
- Water balance output for the corresponding output files.

Overview

SWAT+ drives the land phase with daily values of precipitation, maximum and minimum temperature, solar radiation, relative humidity, and wind speed. Each variable can come from a gauge file, from the weather generator, or from a mix of the two on a per-station basis. Potential evapotranspiration can be computed inside the model from three methods or read from a daily file.

Process equations

Inputs from gauge files

Each weather station listed in `weather-sta.cli` names up to six measured-data files: precipitation (`*.pcp`), temperature (`*.tmp`), solar radiation (`*.slr`), humidity (`*.hmd`), wind (`*.wnd`), and a precomputed PET file (`*.pet`). The strings `null` or a missing file fall back to weather-generator output for that variable. The master index of each measured variable across all stations is held in `pcp.cli`, `tmp.cli`, `slr.cli`, `hmd.cli`, `wnd.cli`, and `pet.cli`.

Weather generator

Stochastic weather is generated from monthly statistics held in `weather-wgn.cli` (one block per WGN station). Generation routines:

- Precipitation occurrence and amount: `cli_pgen.f90`, sub-daily disaggregation in `cli_pgenhr.f90`.
- Air temperature: `cli_tgen.f90`.
- Solar radiation: `cli_slrgen.f90`.
- Relative humidity: `cli_rhgen.f90`.
- Wind speed: `cli_wndgen.f90`.
- Driver and initialisation: `cli_clgen.f90`, `cli_initwgn.f90`, `cli_weatgn.f90`.

Optional elevation lapsing of precipitation and temperature is applied in `cli_lapse.f90` when `bsn_cc%lapse = 1`, using `bsn_prm%plaps` (mm/km, default 0.0) and `bsn_prm%tlaps` (deg C/km, default 6.5).

Potential evapotranspiration

The PET method is selected by `bsn_cc%pet` in `codes.bsn`. The dispatch is in `et_pot.f90`:

`bsn_cc%pet`	Method
0	Priestley-Taylor
1	Penman-Monteith
2	Hargreaves
3	Read daily PET from file

Priestley-Taylor (`case (0)` in `et_pot.f90`) uses

```
PET = alpha * (D / (D + g)) * Rn / lambda
```

with `alpha = 1.28`, slope of saturation vapour pressure curve `D`, psychrometric constant `g`, net radiation `Rn`, and latent heat of vaporization `lambda`. Net radiation is built from incoming solar radiation, albedo (with a snow adjustment when `sno_mm > 0.5 mm`), and a net long-wave term.

Penman-Monteith (`case (1)`) uses

```
PET = (D * Rn + rho * g * VPD / rv) / (lambda * (D + g * (1 + rc / rv)))
```

with aerodynamic resistance `rv`, canopy resistance `rc`, air density `rho`, and vapour pressure deficit `VPD`. For the reference PET, `rv = 114 / (u * (170/1000)**0.2)` and `rc = 49 / (1.4 - 0.4 * CO2 / 330)`. Maximum plant ET is computed separately using canopy-specific resistance values.

Hargreaves (`case (2)`) uses

```
PET = 0.0023 * (Ra / lambda) * (Tave + 17.8) * sqrt(Tmax - Tmin)
```

with extraterrestrial radiation `Ra = solradmx * 37.59 / 30`.

The read-in option (`case (3)`) takes the daily PET value from the gauge file pointed to by the station and applies no further computation in this routine. When `bsn_cc%pet == 3`, `pet.cli` is opened in `basin_read_cc.f90`.

A final scaling `pet_day = hru(j)%hyd%pet_co * pet_day` is applied at the bottom of `et_pot.f90`. The basin-level multiplier `bsn_prm%petco_pmpt` (default 100.0, percent) is used during calibration for Penman-Monteith and Priestley-Taylor.

Actual ET (canopy evaporation, transpiration, soil evaporation) is then partitioned in `et_act.f90` against `PET`, `esco`, and `epco`.

Switches and parameters

Switch / parameter	Default	File	Effect
`bsn_cc%pet`	0 (Priestley-Taylor)	`codes.bsn`	PET method
`bsn_cc%lapse`	0 (off)	`codes.bsn`	Apply elevation lapse to precipitation and temperature
`bsn_cc%petfile`	`pet.cli`	`codes.bsn`	Name of the PET index file when `pet = 3`
`bsn_prm%plaps`	0.0 mm/km	`parameters.bsn`	Precipitation lapse rate
`bsn_prm%tlaps`	6.5 deg C/km	`parameters.bsn`	Temperature lapse rate
`bsn_prm%petco_pmpt`	100.0 %	`parameters.bsn`	PET adjustment for Penman-Monteith and Priestley-Taylor
`bsn_prm%igen`	5	`parameters.bsn`	Random seed selection for the weather generator
`hyd_db%pet_co`	1.0	`hydrology.hyd`	Per-HRU PET multiplier

Implementation

Source modules in `swatplus/src/`:

- `cli_staread.f90` reads `weather-sta.cli` (the index of weather stations and their measured-data filenames).
- `cli_wgnread.f90` reads `weather-wgn.cli` (the index of WGN station parameter sets).
- `cli_pmeas.f90`, `cli_tmeas.f90`, `cli_smeas.f90`, `cli_hmeas.f90`, `cli_wmeas.f90`, `cli_petmeas.f90` read the measured `*.pcp`, `*.tmp`, `*.slr`, `*.hmd`, `*.wnd`, and `*.pet` files for each station.
- `cli_clgen.f90`, `cli_initwgn.f90`, `cli_weatgn.f90` drive the weather generator. Variable generators: `cli_pgen.f90`, `cli_tgen.f90`, `cli_slrgen.f90`, `cli_rhgen.f90`, `cli_wndgen.f90`.

- `cli_lapse.f90` applies elevation lapsing.
- `cli_precip_control.f90` coordinates daily and sub-daily precipitation steps.
- `et_pot.f90` computes potential ET (Priestley-Taylor, Penman-Monteith, Hargreaves, read).
- `et_act.f90` computes actual ET and partitions soil evaporation across layers.
- `basin_read_cc.f90` opens the PET index file when `bsn_cc%pet == 3`.

Related pages

- `codes.bsn` for PET and lapse switches.
- `parameters.bsn` for lapse rates and PET adjustment.
- `pcp`, `tmp`, `cli` for climate input file formats.
- Overview and Water Balance for how PET feeds the daily HRU balance.

Overview

The hydrology cluster is what moves water from precipitation through HRUs and along the channel network. Each HRU produces surface runoff, lateral subsurface flow, and percolation. Surface runoff and lateral flow lag through storage terms before reaching the receiving channel. Channels route the inflow to the outlet using one of two routing schemes selected by `bsn_cc%rte`.

Process equations

Surface runoff: curve number vs Green-Ampt

The infiltration / runoff partition is set by `bsn_cc%gampt` in `codes.bsn`. The dispatch lives in `sq_volq.f90`:

```
if (bsn_cc%gampt == 0) then
  call sq_daycn
else
  call sq_greenampt
end if
```

Curve number (`bsn_cc%gampt = 0`, daily time step). The SCS curve number method computed in `sq_daycn.f90`:

```
S = 25400 / CN - 254
Ia = 0.2 * S
Q_surf = (P - Ia)^2 / (P + 0.8 * S)   when P > Ia
       = 0                            otherwise
```

`CN` is updated daily for soil moisture in `sq_dailycn.f90`. The retention `S` shape is set per HRU by `hyd_db%cn3_swf` (soil water at curve number condition III, where 0 = field capacity and 0.99 = near saturation). A separate frozen-soil adjustment uses `bsn_prm%cn_froz` (default 0.000862). For urban HRUs, an impervious-area curve number of 98 is mixed by the impervious fraction `fcimp`.

Green-Ampt (`bsn_cc%gampt = 1`, sub-daily). Used when sub-daily precipitation is available. The infiltration rate at cumulative infiltration `F` is

```
f = K * (1 + psi * dtheta / F)
```

with hydraulic conductivity `K`, wetting-front suction `psi`, and initial moisture deficit `dtheta`. The implementation iterates on cumulative infiltration each sub-daily time step in `sq_greenampt.f90`. Initial abstraction for urban HRUs is capped at `bsn_prm%urb_init_abst` (default 1.0 mm).

Surface runoff lag. Generated runoff `Q_surf` is held by `sq_surfst.f90` so that only a fraction reaches the channel on the day of the event:

```
brt = 1 - exp(-surlag / t_conc)
qday = (lagged_storage + Q_surf) * brt
```

bsn_prm%surlag (default 4.0 days) and the HRU time of concentration t_conc together set brt.

Lateral subsurface flow

Lateral flow is computed layer by layer in swr_percmicro.f90 from the gravity-drained water above field capacity. Each layer produces a lateral component latlyr and a vertical percolation component sepday. The layer values are summed into latq(j) in swr_percmain.f90:

```
latq(j) = latq(j) + latlyr
```

Lateral flow is then lagged across the basin via swr_substor.f90:

```
latq_out = lagged_storage * lat_ttime
```

where hyd_db%lat_ttime is the lateral travel-time factor read from hydrology.hyd. A linear adjustment hyd_db%latq_co (default 0.3) scales daily lateral flow.

Percolation and return flow

The master driver swr_percmain.f90 walks the soil profile from top to bottom. For each layer:

1. Add gravity-drained water from the overlying layer.
2. Compute sw_excess = st - fc (storage above field capacity).
3. Compute layer percolation sepday and lateral flow latlyr using a storage routing formula with characteristic time TT = (sat - fc) / K_sat. The implementation is in swr_percmicro.f90; macropore flow is handled in swr_percmacro.f90.
4. Subtract sepday, latlyr, and tile flow from layer storage.

Percolation from the bottom of the soil profile (sepbtm) recharges the shallow aquifer. Return flow (baseflow back to the channel) is computed in the aquifer module from the shallow aquifer storage. See Aquifers.

Tile drainage from the layer is computed with swr_drains.f90 (DRAINMOD equations, bsn_cc%tdrn = 1) or swr_origtile.f90 (drawdown-days equation, bsn_cc%tdrn = 0). Saturated profile redistribution sits in swr_satexcess.f90.

Channel routing: variable storage vs Muskingum

bsn_cc%rte selects the channel routing method:

bsn_cc%rte	Method
0	Variable storage coefficient
1	Muskingum

Both methods run in ch_rtmusk.f90 (it dispatches on bsn_cc%rte inside its sub-daily loop, despite the file name).

Variable storage. Outflow at the sub-step is

```
scoef = bsn_prm%scoef * 2 * dt / (2 * ttime + dt)
outflow = scoef * tot_storage
```

The travel time ttime is interpolated from the rating curve at the current inflow and outflow. bsn_prm%scoef (default 1.0) is the channel storage coefficient.

Muskingum. The classical three-coefficient form:

```
O2 = c1 * I2 + c2 * I1 + c3 * O1
```

with c1, c2, c3 derived from the Muskingum storage time constant Km and weighting factor X. bsn_prm%msk_co1 (default 0.75), bsn_prm%msk_co2 (default 0.25), and bsn_prm%msk_x (default 0.20) control the calibration of Km from the bankfull and low-flow reach response times.

Transmission losses are subtracted after routing using the channel bed hydraulic conductivity sd_ch%chk over the reach length and wetted perimeter. Reach evaporation uses bsn_prm%evrch (default 0.60).

Switches and parameters

Switch / parameter	Default	File	Effect
bsn_cc%gampt	0	codes.bsn	0 = curve number; 1 = Green-Ampt
bsn_cc%rte	0	codes.bsn	0 = variable storage; 1 = Muskingum
bsn_cc%tdrn	0	codes.bsn	0 = drawdown-days tile flow; 1 = DRAINMOD
bsn_cc%wtdn	0	codes.bsn	Shallow water table method
bsn_cc%crk	0	codes.bsn	1 = compute crack flow
bsn_cc%sed_det	0	codes.bsn	Peak rate method (NRCS UH vs half-hour rainfall)
bsn_prm%surlag	4.0 days	parameters.bsn	Surface runoff lag
bsn_prm%scoef	1.0	parameters.bsn	Variable-storage channel coefficient
bsn_prm%msk_co1	0.75	parameters.bsn	Muskingum bankfull storage coefficient
bsn_prm%msk_co2	0.25	parameters.bsn	Muskingum low-flow storage coefficient
bsn_prm%msk_x	0.20	parameters.bsn	Muskingum weighting factor
bsn_prm%evrch	0.60	parameters.bsn	Reach evaporation adjustment
bsn_prm%cn_froz	0.000862	parameters.bsn	Frozen soil CN adjustment
bsn_prm%urb_init_abst	1.0 mm	parameters.bsn	Urban max initial abstraction (Green-Ampt)
bsn_prm%adj_pkr	1.0	parameters.bsn	Subbasin peak rate adjustment
bsn_prm%prf	484.0	parameters.bsn	Peak rate factor for peak rate equation
bsn_prm%uhalpha	1.0	parameters.bsn	Gamma-UH alpha coefficient
hyd_db%cn3_swf	per HRU	hydrology.hyd	Soil water at CN3
hyd_db%lat_ttime	per HRU	hydrology.hyd	Lateral flow travel time
hyd_db%latq_co	0.3	hydrology.hyd	Lateral flow linear adjustment
hyd_db%perco	per HRU	hydrology.hyd	Percolation linear adjustment

Implementation

Source modules in swatplus/src/:

- sq_volq.f90 dispatches surface runoff calculation.
- sq_daycn.f90 computes daily curve-number runoff; sq_dailycn.f90 adjusts CN for soil moisture each day.
- sq_greenampt.f90 computes sub-daily Green-Ampt infiltration and runoff.
- sq_surfst.f90 applies the surface runoff lag.
- sq_canopyint.f90 removes canopy interception before infiltration.
- swr_percmain.f90 is the percolation master; swr_percmicro.f90 handles micropore flow, swr_percmacro.f90 macropore flow.
- swr_substor.f90 lags lateral and tile flow.
- swr_satexcess.f90 redistributes water above saturation.
- swr_drains.f90 and swr_origtile.f90 compute tile flow.
- ch_rtday.f90 routes daily channel flow (older path).
- ch_rtmusk.f90 runs the sub-daily Muskingum and variable-storage routing.
- sd_channel_control3.f90 is the master channel control routine.

- `cn2_init.f90` and `cn2_init_all.f90` initialise the curve number per HRU.

Related pages

- `codes.bsn` for routing and runoff switches.
- `parameters.bsn` for routing coefficients.
- `hru` for HRU pointers to `hydrology.hyd`.
- Aquifers for what happens to percolation after it leaves the soil.
- Channels for channel geometry and rating curves.
- Soil Water and Temperature for the soil profile representation.
- Water balance output.

Overview

Every HRU has a discretised soil profile of up to ten layers. Each layer holds physical properties (depth, bulk density, hydraulic conductivity, sand-silt-clay-rock fractions, organic content) and state variables (water storage, temperature, layer chemistry). The soil profile is the central reservoir for the HRU water balance: surface runoff is partitioned at the top of layer 1, percolation drains the bottom, and lateral and tile flow leave each layer between. Soil temperature is updated daily for each layer and is used to gate decomposition, nitrification, and biological processes.

Process equations

Soil layers

The layer geometry is held in `soil_module.f90` (`type soil_profile`). Each layer carries:

- `d` (mm) cumulative depth from surface
- `thick` (mm) layer thickness
- `bd` (Mg/m^3) bulk density (default 1.3, bounded to 2.0)
- `k` (mm/hr) saturated hydraulic conductivity (defaulted by hydrologic group A=50, B=20, C=5, D=2 when missing)
- `awc` (mm/mm) available water capacity, bounded to `[0.005, 0.8]`
- `clay`, `silt`, `sand`, `rock` fractions
- `wp`, `up`, `por`, `fc`, `st` water content terms (see below)

Initialisation lives in `soil_phys_init.f90`. Defaults applied there:

```
wp  = 0.4 * clay * bd / 100          ! wilting point (m^3/m^3)
up  = wp + awc                       ! upper limit (field capacity, m^3/m^3)
por = 1 - bd / 2.65                  ! porosity from bulk density
fc  = thick * (up - wp)              ! field capacity water in mm
st  = fc * ffc                       ! initial layer water as fraction of FC
```

The basin-wide initial soil water fraction `bsn_prm%ffcb` (default 0.0) seeds layer storage at the start of the run.

Available water capacity

The available water capacity of a layer is the water held between field capacity and wilting point:

`AWC_layer (mm) = thick * (up - wp) = thick * awc`

The profile sum is `soil(j)%s%sumfc`. This drives plant water uptake (compared against `epco`) and soil evaporation (compared against `esco`).

Daily soil water balance

For each layer, the daily update is

```
st(t+1) = st(t) + inflow - percolation - lateral - tile - soil_evap - transpiration
```

Inflow to layer 1 is infiltration `P - Q_surf`. Inflow to deeper layers is percolation from the layer above. Gravity drainage is allowed when `st > fc`. The implementation runs in `swr_percmain.f90` and `swr_percmicro.f90` (see Hydrology).

Soil temperature

Soil temperature at the centre of each layer is computed daily in `stmp_solt.f90` from the soil surface temperature, an annual mean air temperature, and a depth-dependent damping function.

Maximum damping depth (mm), from bulk density:

```
f  = avbd / (avbd + 686 * exp(-5.63 * avbd))
dp = 1000 + 2500 * f
```

Soil water scaling factor:

```
ww = 0.356 - 0.144 * avbd
wc = sw / (ww * d_bottom)
```

Daily damping depth:

```
b  = ln(500 / dp)
dd = dp * exp(b * ((1 - wc) / (1 + wc))^2)
```

Bare soil surface temperature:

```
st0   = (solrad * (1 - albedo) - 14) / 20
tbare = tave + 0.5 * (tmax - tmin) * st0
```

Cover lagging factor (residue, biomass, snow):

```
cover = above_ground_mass + residue_mass
bcv   = cover / (cover + exp(7.563 - 1.297e-4 * cover))
```

If `sno_mm > 0`, `bcv` is taken as the maximum of the residue and snow lagging factors. Surface temperature with cover:

```
tcov           = bcv * tmp_layer2 + (1 - bcv) * tbare
soil%tmp_srf   = 0.5 * (tbare + tcov)
```

Layer temperature with the lag coefficient `tlag = 0.8`:

```
zd  = depth_center / dd
df  = zd / (zd + exp(-0.8669 - 2.0775 * zd))
tmp = tlag * tmp_prev + (1 - tlag) * (df * (tmp_an - tmp_srf) + tmp_srf)
```

where `tmp_an` is the long-term annual mean air temperature for the WGN station. Layer temperature feeds frozen-soil flags, decomposition rates, and septic system temperature corrections.

Switches and parameters

Switch / parameter	Default	File	Effect
`bsn_prm%ffcb`	0.0	`parameters.bsn`	Initial soil water as fraction of field capacity
`bsn_prm%cn_froz`	0.000862	`parameters.bsn`	Frozen soil CN adjustment for infiltration
`bsn_cc%wtdn`	0	`codes.bsn`	Shallow water table depth algorithm
`hyd_db%esco`	per HRU	`hydrology.hyd`	Soil evaporation compensation factor (0-1)

Switch / parameter	Default	File	Effect
hyd_db%epco	per HRU	hydrology.hyd	Plant water uptake compensation factor (0-1)
hyd_db%canmx	per HRU	hydrology.hyd	Maximum canopy storage
Layer fields bd, awc, k, clay, silt, sand, rock, cbn	per layer	sol	Soil layer physical properties

Implementation

Source modules in swatplus/src/:

- soil_module.f90 defines the soil_profile derived type and per-layer physical and chemical fields.
- soil_phys_init.f90 initialises layer water-retention parameters (wp, up, por, fc, st) and applies the hydrologic-group default conductivities.
- solt_db_read.f90 reads the soil temperature lookup data.
- swr_percmain.f90 updates layer water storage through the daily percolation pass.
- swr_satexcess.f90 redistributes water above saturation.
- stmp_solt.f90 computes daily soil layer temperatures.
- basin_sw_init.f90 initialises soil water using ffcb at simulation start.
- et_act.f90 removes soil evaporation from layers using esco.
- albedo.f90 updates surface albedo, which feeds the bare soil temperature.

Related pages

- sol for the soil layer input file.
- Hydrology for percolation and lateral flow that change layer storage.
- Plant growth and land use management for transpiration, which removes layer water.
- Nutrient cycling for layer chemistry and temperature dependence of mineralisation.
- Carbon for the Century-style organic matter pools driven by soil temperature.

Overview

Soil loss at the HRU is computed with the Modified Universal Soil Loss Equation (MUSLE). The HRU sediment yield is then routed through the channel network, where each reach can erode its bed and banks or deposit suspended sediment. The peak runoff rate that drives MUSLE comes from one of two methods set by bsn_cc%sed_det. Channel sediment processes use the rating curve and reach geometry from the SWAT+ degradation channel.

Process equations

MUSLE for HRU sediment yield

The MUSLE form used in ero_ysed.f90:

```
sed = 10 * (Q_surf * q_peak * A_ha) ^ 0.56 * cklsp
```

Symbol	Meaning
sed	Daily sediment yield (t)
Q_surf	Daily surface runoff volume (mm)
q_peak	Peak runoff rate (m^3/s)
A_ha	HRU area (ha)
cklsp	Combined cover (C), management (P), topographic (LS), and soil erodibility (K) factor

cklsp is built per HRU as cklsp = usle_cfac * usle_mult, where usle_mult = K * LS * P * 11.8 is set up at HRU initialisation and usle_cfac is the dynamic cover factor from ero_cfactor.f90.

The sediment yield is reduced under snow cover using:

sed = sed / exp(3 * sno_mm / 25.4)

and is set to zero when the snowpack exceeds 100 mm.

For reference, USLE itself is computed alongside for output as usle = 1.292 * usle_ei * cklsp / 11.8, where usle_ei is the rainfall erosivity from ero_eiusle.f90.

Peak runoff rate

The peak flow q_peak is selected by bsn_cc%sed_det in ero_pkq.f90:

- bsn_cc%sed_det = 0 (default): NRCS dimensionless unit hydrograph with peak rate factor bsn_prm%prf (default 484.0).
- bsn_cc%sed_det = 1: half-hour rainfall intensity method using precip_half_hr from the climate or WGN data.

NRCS form (default):

```
q_peak (cfs) = prf / 6578.6 * A_ha * qday / tconc
q_peak (cms) = q_peak (cfs) / 35.3
```

Half-hour form:

```
altc      = 1 - exp(2 * tconc * ln(1 - alpha_half))
q_peak    = altc * qday / tconc * A_km^2 / 3.6   (m^3/s)
```

with alpha_half = precip_half_hr as a fraction of daily rainfall.

bsn_prm%adj_pkr (default 1.0) is the subbasin peak rate adjustment factor.

Sediment in lateral and tile flow

swr_latsed.f90 adds a sediment load to the HRU yield from lateral and tile flow:

sed_lat = lat_sed * (Q_lat + Q_tile) * A_ha / 100000

with hyd_db%lat_sed (g/L) read from hydrology.hyd. Particle size split (sand, silt, clay, small aggregate, large aggregate) follows the soil texture detachment fractions.

Overland sediment routing

ero_ovrsed.f90 partitions HRU sediment yield by particle size class and applies an enrichment ratio. When bsn_cc%gampt > 0 (sub-daily simulations), the routine also distributes sediment in time using the sub-daily runoff pattern.

Channel sediment transport

Channel sediment processes run in sd_channel_sediment3.f90 (the active routine for SWAT+ degradation channels). Two erosion sources and one deposition sink are computed each day.

Critical velocity for bank erosion is built from a clay cohesion term and a vegetation cover term:

```
cohesion  = -87.1 + 42.82 * ch_clay - 0.261 * ch_clay^2 + 0.029 * ch_clay^3
veg       = exp(-5 * chd) * 4500 * cov
bd_fac    = max(0.001, 0.03924 * ch_bd * 1000 - 1000)
cohes_fac = 0.021 * cohesion + veg
v_cr      = vcr_coef * log10(2200 * chd) * sqrt(0.0004 * (bd_fac + cohes_fac))
```

with ch_clay (channel clay percent), ch_bd (channel bed bulk density), chd (channel depth), and cov (channel cover). The mean reach velocity adjusted for meander curvature is

```
rad_curv = (12 * chw) * sinu^1.5 / (13 * (sinu - 1)^0.5)
v_bend   = v * (1 / rad_curv + 1)
v_rch    = 0.33 * v_bend + 0.66 * v
```

Bank erosion is computed when `v_rch > v_cr`:

```
ebank_m = 1 / (1 / v_vc + 1 / m_exhaust)
ebank_t = 1000 * ebank_m * chd * arc_len * ch_bd
```

with `arc_len = 0.25 * chl` and `m_exhaust = 0.0002 * chw`.

Bed erosion is allowed when channel slope exceeds an equilibrium slope. Critical shear comes from the median bed particle `d50`. Acting shear is

```
shear_btm = 9800 * dep * chs   (Pa)
```

with `dep` the flow depth and `chs` the channel slope.

Channel deposition is taken as a bedload fraction of bank erosion:

```
ch_dep_sed = wash_bed_fr * bank_ero_sed
```

Suspended sediment in the routed hydrograph carries associated organic N and P using channel-specific concentrations (`n_conc`, `p_conc`) and enrichment factors (`n_dep_enr`, `p_dep_enr`).

Note: the legacy `bsn_prm%spcon` and `bsn_prm%spexp` parameters are listed in `basin_module.f90` as "not used". The active sediment routing in `sd_channel_sediment3.f90` is based on the velocity and shear formulation above, not on a sediment transport capacity power law.

Switches and parameters

Switch / parameter	Default	File	Effect
`bsn_cc%sed_det`	0	`codes.bsn`	0 = NRCS UH peak rate, 1 = half-hour intensity
`bsn_cc%gampt`	0	`codes.bsn`	Triggers sub-daily sediment distribution when > 0
`bsn_prm%prf`	484.0	`parameters.bsn`	Peak rate factor for the NRCS method
`bsn_prm%adj_pkr`	1.0	`parameters.bsn`	Subbasin peak rate adjustment
`bsn_prm%eros_spl`	0.0	`parameters.bsn`	Splash erosion coefficient (0.9-3.1)
`bsn_prm%rill_mult`	0.0	`parameters.bsn`	Rill erosion coefficient
`bsn_prm%eros_expo`	0.0	`parameters.bsn`	Exponent for overland flow erosion
`bsn_prm%c_factor`	0.0	`parameters.bsn`	Scaling for overland C-factor
`bsn_prm%ch_d50`	0.0	`parameters.bsn`	Median particle diameter of main channel (mm)
`hyd_db%lat_sed`	per HRU	`hydrology.hyd`	Sediment concentration in lateral flow (g/L)
`hyd_db%erorgn`, `hyd_db%erorgp`	per HRU	`hydrology.hyd`	Organic N and P enrichment ratios
`sd_ch%ch_clay`, `ch_bd`, `cov`, `d50`, `vcr_coef`, `wash_bed_fr`	per channel	`sd-channel`	Channel sediment properties

Implementation

Source modules in `swatplus/src/`:

- `ero_ysed.f90` implements MUSLE for HRU daily sediment yield.
- `ero_pkq.f90` computes the peak runoff rate used by MUSLE.
- `ero_cfactor.f90` updates the dynamic cover and management factor.
- `ero_eiusle.f90` computes rainfall erosivity for the reference USLE.
- `ero_ovrsed.f90` distributes sediment in time and by particle size class for sub-daily simulations.
- `swr_latsed.f90` adds sediment from lateral and tile flow to the HRU yield.
- `sd_channel_sediment3.f90` is the active channel sediment routine: bank erosion, bed erosion, deposition, and associated nutrients.
- `sd_channel_control3.f90` is the channel master that calls the sediment routine.
- `nut_psed.f90` handles sediment-bound phosphorus on land.

Related pages

- Hydrology for surface runoff and peak rate.
- Channels for reach geometry, rating curve, and routing.
- `sd-channel` for channel sediment properties.
- `parameters.bsn` for basin-level erosion coefficients.
- Channel sediment output for the corresponding output.

Overview

SWAT+ grows one or more plants in a community on each HRU. Daily potential biomass is a function of intercepted photosynthetically active radiation. That potential is reduced by water, temperature, nitrogen, phosphorus, and aeration stresses. Phenology is tracked by accumulated plant heat units (PHU). The plants on an HRU are scheduled by `management.sch`, and land use plus structural BMPs are assigned through `landuse.lum`. Decision tables (`lum.dtl`) drive conditional operations such as auto-irrigation, auto-fertilisation, and land use change.

Process equations

Heat unit accumulation

Each plant accumulates a fraction of total heat units to maturity per day:

```
delg   = max(0, (tave - t_base) / phumat)
phuacc = phuacc + delg
```

`t_base` is the plant's minimum growth temperature (`plants.plt`), `phumat` is the heat units to maturity. When `phuacc` reaches 1.0 the plant is mature. `phuacc_p` is the perennial counterpart. Source: `pl_nut_demand.f90`.

Biomass accumulation

Daily potential biomass is

```
bioday = beadj * par
```

where `par` is intercepted PAR (MJ/m^2) and `beadj` is radiation use efficiency adjusted for CO2 and vapour pressure deficit. The CO2 adjustment uses the two-point curve described by `bio_e`, `bioehi`, `co2hi` in `plants.plt`. When vapour pressure deficit exceeds 1.0 kPa, RUE declines at the plant-specific rate `wavp`, with a floor of `0.27 * bio_e`. Realised biomass is

```
bm_grow = bioday * reg
reg     = min(strsw, strst, strsn, strsp, strsa[, strss])
```

`reg` is the most limiting stress factor. Plant carbon mass is fixed at `0.42 * biomass`. Source: `pl_biomass_gro.f90`.

Leaf area index

LAI follows a two-point optimal curve defined by `(frgrw1, laimx1)` and `(frgrw2, laimx2)` from `plants.plt`. The model computes a fraction `f` of maximum LAI from `phuacc`:

```
f = phuacc / (phuacc + exp(leaf1 - leaf2 * phuacc))
```

leaf1 and leaf2 are shape parameters fitted from the two points. Before phuacc reaches dlai (fraction of season at which leaves decline), LAI grows toward laimax:

```
deltaLAI = (f - laimxfr) * laimax * (1 - exp(5*(LAI - laimax))) * sqrt(strsw)
```

After phuacc > dlai, LAI declines following dlai_rate. Canopy height is chtmx * sqrt(f). For perennials, LAI is scaled by tree age through laixco_tree and mat_yrs. Source: pl_leaf_gro.f90, pl_leaf_senes.f90.

Water stress and uptake

Potential transpiration epmax(ipl) is distributed through the rooting depth using an exponential extraction function with parameters uptake%water_dis and uptake%water_norm. Water uptake from each layer is

```
sum = epmax * (1 - exp(-water_dis * gx / root_dep)) / water_norm
wuse = sum - sump * (1 - epco)
```

where gx is depth to the bottom of the current layer. Layers below the root zone do not contribute. The compensation factor epco (0-1) controls how much a deeper layer can compensate when a shallower layer is dry. Water stress is

```
strsw = sum_wuse / epmax
```

Aeration stress activates when soil water exceeds field capacity:

```
satco = (sw - sumfc) / (sumul - sumfc)
strsa = 1 - scparm / (scparm + exp(2.9014 - 0.03867*scparm))
```

where scparm = 100 * (satco - aeration) / (1.0001 - aeration). Source: pl_waterup.f90.

Temperature stress

```
rto   = (tave - t_base) / (t_opt - t_base)
strst = sin(pi/2 * rto)
```

strst = 0 if tave <= t_base or if tmin <= annual mean - 15. Source: pl_tstr.f90.

Nitrogen and phosphorus stress and uptake

Plant N and P demands follow optimal concentration curves with pltnfr1, pltnfr2, pltnfr3 and pltpfr1, pltpfr2, pltpfr3 (see plants.plt). N and P are taken from soil layers with a depth-distribution function controlled by n_updis and p_updis in parameters.bsn. If supply is below demand, strsn and strsp reduce growth. Legumes fix N when soil supply is short:

```
fxr   = min(1, fxw, fxn) * fxg
fixn  = nfix_co * fxr * deficit + (1 - nfix_co) * deficit
fixn  = min(nfixmx, fixn)
```

nfix_co is the legume coefficient (0.5 for legumes, 0 for non-legumes). nfixmx (default 20 kg N/ha/day, parameters.bsn) caps daily fixation. Source: pl_nup.f90, pl_pup.f90, pl_nfix.f90, pl_nupd.f90, pl_pupd.f90.

Harvest

Yield is harveff * seed_mass. The harvest index is bounded below by wsyf and accumulates from zero up to hvsti as phuacc increases past 0.5. Harvest types in harv.ops are grain, biomass, residue, tree, tuber, peanuts, stripper, picker. Pesticide is removed in yield in proportion to pl_yield%m / pl_mass(j)%tot(ipl)%m. Source: mgt_harvgrain.f90, mgt_harvbiomass.f90, mgt_harvresidue.f90, mgt_harvtuber.f90, mgt_killop.f90.

Land use management

`landuse.lum`

Each HRU points to a row in `landuse.lum`. That row carries the plant community name, curve number group, USLE conservation practice, tillage practice, and pointers to the management schedule, tile drainage code, septic code, filter strip, grass waterway, and BMP user practice. `hru_lum_init.f90` copies these pointers onto each HRU at the start of simulation. Source: `hru_lum_init.f90`, `landuse_data_module.f90`.

`management.sch`

For each schedule, operations are listed in calendar or PHU-trigger order. Operation codes used by `mgt_sched.f90` include:

Code	Operation
`plnt`	Plant a single species (or community)
`harv`	Harvest (grain / biomass / residue / tree / tuber etc.)
`kill`	Kill the plant, transfer all biomass to residue
`hvkl`	Harvest and kill in the same operation
`till`	Tillage, mixes residue and pools using `till.til`
`irrm`	Irrigation, applies depth from `irr.ops`
`fert`	Mineral or organic fertiliser (`fertilizer.frt`)
`manu`	Manure application
`pest`	Pesticide application
`graz`	Grazing
`cnup`	Curve number update
`burn`	Prescribed burn
`swep`	Street sweeping
`dwm`	Drainage water management depth
`weir`	Set or adjust weir height
`mons`	Begin or end monsoon trigger window

PHU-triggered operations fire when `phuacc` of the plant in column `op2` crosses the value in column `phu`. Date-triggered operations fire on the given month and day. Source: `mgt_sched.f90`, `mgt_operations_module.f90`.

Decision tables (`lum.dtl`)

Each row in `landuse.lum` may also reference one or more decision tables in `lum.dtl`. Decision tables are conditional rule sets that fire actions when their conditions are met. Each table has `conds` conditions, `alts` alternative outcomes, and `acts` actions. On each day in `hru_control.f90` the model loops over the HRU's `num_autos` auto operations and calls `conditions` then `actions` for each. Typical actions include auto-irrigation, auto-fertilisation, planting based on PHU triggers, and land use change. The decision table reader is `dtbl_lum_read.f90`; the run-time logic is in `conditions.f90` and `actions.f90`.

See the Decision tables theory page for the rule grammar.

Switches and parameters

File	Field	Effect
`codes.bsn`	`nostress`	0 = all stresses applied, 1 = no stress, 2 = no N/P stress
`codes.bsn`	`cswat`	Selects soil carbon model. The dynamic CENTURY/SWAT-C model (=2) changes residue and tillage routines
`parameters.bsn`	`co2`	Ambient CO2 concentration (ppm), default 400
`parameters.bsn`	`nfixmx`	Maximum daily N fixation (kg/ha), default 20
`parameters.bsn`	`n_updis`, `p_updis`	Depth-distribution shape for plant N/P uptake

File	Field	Effect
parameters.bsn	evlai	LAI above which only transpiration occurs, default 3.0
plants.plt	bio_e, bioehi, co2hi, wavp	RUE and its CO2 / VPD response
plants.plt	blai, frgrw1, laimx1, frgrw2, laimx2, dlai, dlai_rate	LAI development curve
plants.plt	t_base, t_opt	Temperature response
plants.plt	hvsti, wsyf	Harvest index and water-stress floor
plants.plt	chtmx, rdmx	Maximum canopy height and root depth
plants.plt	aeration	Aeration stress threshold (fraction above FC)
plants.plt	pltnfr1..3, pltpfr1..3	N and P optimal concentration curves
plants.plt	nfix_co	Legume coefficient
plants.plt	mat_yrs, laixco_tree, bmx_peren, leaf_tov_min, leaf_tov_max	Perennial / tree parameters

Implementation

Plant growth: pl_grow.f90, pl_biomass_gro.f90, pl_leaf_gro.f90, pl_leaf_senes.f90, pl_root_gro.f90, pl_seed_gro.f90, pl_partition.f90, pl_community.f90, pl_waterup.f90, pl_tstr.f90, pl_nut_demand.f90, pl_nup.f90, pl_pup.f90, pl_nupd.f90, pl_pupd.f90, pl_nfix.f90, pl_dormant.f90, pl_mortality.f90, pl_rootfr.f90.

Plant data and initialisation: plant_data_module.f90, plant_module.f90, plant_parm_read.f90, plant_init.f90, plant_all_init.f90, plant_transplant_read.f90.

Management operations: mgt_sched.f90, mgt_operatn.f90, mgt_plantop.f90, mgt_harvgrain.f90, mgt_harvbiomass.f90, mgt_harvresidue.f90, mgt_harvtuber.f90, mgt_killop.f90, mgt_biomix.f90, mgt_newtillmix_cswat0.f90, mgt_newtillmix_cswat1.f90, mgt_newtillmix_wet.f90, mgt_tillfactor.f90, mgt_transplant.f90, mgt_operations_module.f90, mgt_read_*.f90.

Plant operation utilities: pl_fert.f90, pl_fert_wet.f90, pl_manure.f90, pl_graze.f90, pl_burnop.f90.

Land use: hru_lum_init.f90, hru_lum_init_all.f90, landuse_data_module.f90, lum_read*.f90.

Decision tables: dtbl_lum_read.f90, conditions.f90, actions.f90, conditional_module.f90.

Related pages

- landuse.lum
- plants.plt
- fertilizer.frt
- management.sch
- lum.dtl
- codes.bsn
- parameters.bsn
- Decision tables
- Nutrient cycling
- Carbon

Overview

Nitrogen and phosphorus in SWAT+ move among organic, mineral, and plant pools in each soil layer and are transported to channels by surface runoff, lateral flow, tile drainage, and percolation. Once in a channel they are transformed by the QUAL2E in-stream water quality routines. The legacy soil P model (one stable pool) and the Vadas and White (2010) model (multiple pools) are selectable through bsn_cc%sol_p_model.

Process equations

Soil pools

Each soil layer carries the following N pools:

- nitrate `mn(k)%no3`
- ammonium `mn(k)%nh4`
- active organic humus N `hact(k)%n`
- stable organic humus N `hsta(k)%n`
- fresh organic residue N (per plant) `pl(ipl)%rsd(k)%n`

P pools per layer:

- labile mineral `mp(k)%lab`
- active mineral `mp(k)%act`
- stable mineral `mp(k)%sta`
- humus organic P `hsta(k)%p`
- fresh residue P `pl(ipl)%rsd(k)%p`

Combined temperature and moisture factor

Most rates are scaled by a combined factor `csf = sqrt(cdg * sut)` where

```
sut = max(0.05, 0.1 + 0.9 * sqrt(st/fc))
cdg = max(0.1, 0.9 * tmp / (tmp + exp(9.93 - 0.312*tmp)) + 0.1)
```

`st` is soil water, `fc` field capacity, `tmp` layer temperature. Mineralisation only occurs when `tmp > 0`. Source: `nut_nminrl.f90`.

Humus mineralisation

A fixed fraction `nactfr = 0.02` of organic N is held in the active pool. Flow between the active and stable pools maintains that fraction:

```
rwn = 1e-5 * (hact*n * (1/nactfr - 1) - hsta*n)
```

Active humus N mineralises to nitrate:

```
hmn = cmn * csf * hact%n
```

with `cmn` from `parameters.bsn` (default 0.003). Mineralised P from humus:

```
hmp = 1.4 * hmn * hsta%p / (hsta%n + hact%n)
```

Source: `nut_nminrl.f90`.

Residue decomposition

For each plant residue pool in each layer:

```
cnr  = rsd%c / rsd%n              # C:N ratio (capped at 500)
cnrf = exp(-0.693 * (cnr - 25)/25)
cpr  = rsd%c / rsd%p              # C:P ratio (capped at 5000)
cprf = exp(-0.693 * (cpr - 200)/200)
ca   = min(cnrf, cprf, 1)
decr = max(decr_min, min(1, rsdco_pl * ca * csf))
decomp = decr * rsd
```

80% of decomposed residue N and P goes to mineral pools (NO3 and labile P), 20% to humus. `rsdco_pl` is per plant (`plants.plt`); `decr_min` is in `parameters.bsn`. Source: `nut_nminrl.f90`, `cbn_rsd_decomp.f90`.

Nitrification and ammonia volatilisation

Computed in `nut_nitvol.f90`. Temperature factor `tf = 0.41 * (tmp - 5) / 10` activates when `tf >= 0.001`. The water factor `swf` ramps from 0 at wilting point to 1 at field capacity + 25%. The depth factor `dpf = 1 - dmid/(dmid + exp(4.706 - 0.0305*dmid))` increases nitrification with depth and decreases volatilisation. Combined coefficients:

```
akn = tf * swf                 # nitrification
akv = tf * dpf * cecf          # volatilisation (cecf = 0.15)
rnv = nh4 * (1 - exp(-akn - akv))
rnit = rnv * (1 - exp(-akn)) / [(1 - exp(-akn)) + (1 - exp(-akv))]
rvol = rnv - rnit
```

NO3 increases by `rnit`, NH4 decreases by `rnit + rvol`.

Denitrification

In `nut_nminrl.f90`:

```
if (sut >= sdnco):
    wdn = no3 * (1 - exp(-cdn * cdg * cbn/100))
```

where `cbn` is layer organic carbon (%), `cdn` is the exponential rate coefficient and `sdnco` is the saturation threshold, both in `parameters.bsn`. Defaults: `cdn = 1.40`, `sdnco = 1.30`.

Nitrate leaching and surface transport

In `nut_nlch.f90`, the layer mixes nitrate with mobile water:

```
vv = perc + flat + (surfq if layer 1) + (qtile if tile layer)
co = no3 * (1 - exp(-vv / (porosity_term))) / vv
```

Surface runoff carries `nperco * co * surfq`, lateral flow carries `co * flat`, tile flow uses `nperco_lchtile * co`, and the remainder percolates to the next layer. `nperco` and `nperco_lchtile` come from `parameters.bsn` (defaults 0.10 and 0.50).

Organic N and P transport with sediment

`nut_orgn.f90` and `nut_orgnc.f90` compute sediment-bound organic N. Concentration in the surface 10 mm is multiplied by `sedyld` and an enrichment ratio. The default enrichment ratio uses the CREAMS form

```
enratio = 0.78 * (0.1 * sedyld / (area * surfq)) ^ -0.2468
```

clipped to 3.0 (`pest_enrsb.f90`). Sediment-attached labile P comes from `nut_psed.f90`.

Soluble P loss and leaching

`nut_solp.f90` partitions surface labile P between water and soil via `phoskd` (`parameters.bsn`, default 175):

```
surqsolp = lab * surfq / (bd * d * phoskd + 1)
```

Leaching uses a similar partition with `pperco` (default 10.0):

```
vap  = -prk / (0.01 * st + 0.1 * pperco * bd)
plch = 0.001 * lab * (1 - exp(vap))
```

Mineral P transformations

`nut_pminrl.f90` (legacy model, `sol_p_model = 0`) flows P among labile, active, and stable pools to maintain the ratio `psp / (1 - psp)` between labile and active. Slow active-to-stable flow uses `bk = 0.01`. `nut_pminrl2.f90` is the Vadas and White (2010) reformulation, selected by `sol_p_model = 1`.

Plant N fixation

See plant growth. Fixation only kicks in when soil supply is below demand and the plant has `nfix_co > 0`. Capped daily by `nfixmx` (`parameters.bsn`, default 20 kg N/ha).

Atmospheric N deposition

Added daily through `nut_nrain.f90` using ammonium and nitrate concentrations in precipitation.

Plant uptake distribution

Uptake from a layer follows an exponential function of depth controlled by `n_updis` and `p_updis` in `parameters.bsn` (defaults 20). The plant fixes any unmet demand if it is a legume.

In-stream nutrient routing (QUAL2E)

When `bsn_cc%qual2e = 0`, channels use the full QUAL2E water quality formulation in `ch_watqual4.f90`. Simulated state variables are:

- Algae as chlorophyll-a
- Organic N, ammonia, nitrite, nitrate
- Organic P, dissolved P
- CBOD and dissolved oxygen

First-order rates and Arrhenius temperature corrections (theta factors) follow QUAL2E:

```
rate(T) = rate(20) * theta^(T - 20)
```

Default theta values applied in `ch_watqual4.f90`:

Process	Symbol	Theta
Algal growth, respiration	thgra, thrho	1.047
Settling	thrs1-thrs5	1.024 to 1.074
Biological constants	thbc1-thbc4	1.047 to 1.083
Reaeration / decay	thrk1-thrk4	1.024 to 1.060

Key channel nutrient parameters (`channel-nut.cha`):

- `rs1..rs5` settling rates (organic N, organic P, NH4, alg)
- `bc1..bc4` biological rate constants (NH4 to NO2, NO2 to NO3, organic to ammonia hydrolysis, organic P to dissolved P)
- `rk1..rk4` reaeration / BOD decay / SOD
- `ai0, ai1, ai2` stoichiometric ratios for algae
- `lambda0, lambda1, lambda2` light extinction
- `mumax, rhoq` algal growth and respiration

When `bsn_cc%qual2e = 1`, the channel uses simplified transformations.

Switches and parameters

File	Field	Default	Effect
`codes.bsn`	`sol_p_model`	0	0 = original P model, 1 = Vadas and White (2010)
`codes.bsn`	`qual2e`	0	0 = full QUAL2E, 1 = simplified transformations
`parameters.bsn`	`cmn`	0.003	Active organic N mineralisation rate
`parameters.bsn`	`cdn`	1.40	Denitrification exponential rate

File	Field	Default	Effect
parameters.bsn	sdnco	1.30	Saturation threshold for denitrification (frac of FC)
parameters.bsn	nperco	0.10	Nitrate percolation/runoff partition
parameters.bsn	nperco_lchtile	0.50	Nitrate coefficient for tile and bottom-layer leaching
parameters.bsn	phoskd	175	P soil partitioning coefficient
parameters.bsn	pperco	10.0	P percolation coefficient
parameters.bsn	psp	0.40	P availability index
parameters.bsn	rsdco	0.05	Residue decomposition coefficient
parameters.bsn	decr_min	0.01	Minimum daily residue decay
parameters.bsn	n_updis, p_updis	20	Plant uptake depth distribution
parameters.bsn	nfixmx	20	Maximum daily N fixation (kg/ha)

Implementation

Soil N: nut_nminrl.f90, nut_nitvol.f90, nut_nlch.f90, nut_orgn.f90, nut_orgnc.f90, nut_orgnc2.f90, nut_nrain.f90, nut_denit.f90, nut_np_flow.f90.

Soil P: nut_pminrl.f90, nut_pminrl2.f90, nut_solp.f90, nut_psed.f90.

Plant uptake and fixation: pl_nup.f90, pl_pup.f90, pl_nupd.f90, pl_pupd.f90, pl_nfix.f90, pl_nut_demand.f90.

Soil pool transfer: solt_db_read.f90, cbn_rsd_decomp.f90, cbn_rsd_transfer.f90, cbn_surfrsd_decomp.f90.

In-stream: ch_watqual4.f90, ch_rtday.f90, ch_read_nut.f90, channel_module.f90.

Enrichment: pest_enrsb.f90.

Related pages

- parameters.bsn
- codes.bsn
- fertilizer.frt
- Carbon
- Plant growth
- Channels

Overview

SWAT+ exposes two soil carbon paths today. The choice is set by the integer bsn_cc%cswat in codes.bsn:

cswat	Model	Pools
0	Static (legacy)	Single residue + humus; C is not tracked as a state
1	C-FARM	Reserved for the C-FARM model; not implemented. Treated as off (no carbon model, no carbon.bsn required).
2	Active CENTURY/SWAT-C module	Three litter pools and three soil C pools (microbial, slow, passive) following Parton et al. (1993, 1994); driven by cbn_zhang2.f90

The carbon model controls how plant residue, manure, tillage, and grazing inputs are routed into the soil and the subsequent CO2, N and P fluxes.

Carbon activation value changed to 2

The dynamic carbon model now activates on `cswat = 2`, matching legacy SWAT numbering (1 = C-FARM, 2 = CENTURY). It was previously selected by `cswat = 1`. `carbon_bsn_read.f90` returns early for any value other than 2, so `cswat = 1` is now inert: no carbon model and no `carbon.bsn` required. This is an input-compatibility break: existing `codes.bsn` files (including refdata and the SWAT+ Editor default) left at `carbon = 1` will silently run with the carbon model off and must be changed to `carbon = 2`.

Process equations

Static (cswat = 0)

Mineralisation and immobilisation are computed in `nut_nminrl.f90`. There is no explicit soil C state. Plant residue carbon is implicit through the `0.42 * biomass` conversion. Residue decomposes at rate

```
decr = max(decr_min, min(1, rsdco_pl * ca * csf))
```

with 80% of mineralised N going to NO3 and 20% to active humus N. See nutrient cycling.

Active carbon module (cswat = 2)

Selected when `bsn_cc%cswat = 2`. The active path runs the full Century pool transformation in `cbn_zhang2.f90`, fed by residue inputs, microbial decomposition, and tillage-driven mixing. Basin-wide and per-layer coefficients come from `carbon.bsn` and `carbon_lyr.bsn`; both files are **required** when `cswat = 2` and are read by `carbon_bsn_read.f90`. The tillage method is set by `bsn_cc%idc_till`:

idc_till	Method
1	DSSAT-style
2	EPIC-style
3	Kemanian (default)
4	DNDC-style

In `hru_control.f90`, when `cswat = 2`:

```
call cbn_surfrsd_decomp
call cbn_rsd_transfer
call cbn_zhang2
```

Surface residue decomposes in `cbn_surfrsd_decomp.f90` using the combined temperature-moisture factor `csf` from the N cycle. Below-ground residue is partitioned to soil pools through `cbn_rsd_transfer.f90`. The `cbn_zhang2.f90` routine then runs the Century-style multi-pool turnover that C-FARM and Century share.

The active path also adds the plant biomass increment as carbon (`hpc_d(j)%npp_c`) and accumulates respiratory CO2.

Tillage residue mixing uses `mgt_newtillmix_cswat1.f90` (vs `mgt_newtillmix_cswat0.f90` for the static model). Tillage activity decays over `till_eff_days` days (a column of `carbon.bsn`), controlled per layer by `soil(j)%ly(ly)%tillagef_tillmix`.

Century pool structure (active path)

The Century scheme implemented in `cbn_zhang2.f90`. Pools:

- Litter: structural (`ls`), metabolic (`lm`), with the structural pool further split into lignin (`lsl`) and non-lignin
- Soil organic carbon: microbial biomass (`bm`), slow humus (`hs`), passive humus (`hp`)

For each pool, potential transformation under optimal conditions uses a first-order rate (the `*_para` rates in `carbon_module.f90`). Daily realised transformation is

```
rate(layer) = rate_optimal * cs
cs = sqrt(cdg * sut) * xbm * ox     (cs capped at 10)
```

`cdg` and `sut` are the temperature and water factors from the N cycle. `ox` is the oxygen factor

```
ox = 1 - OX_aa * depth / (depth + exp(OX_aa - OX_bb * depth))
```

with `OX_aa = 10.0`, `OX_bb = 0.035`. `xbm` is the texture and structure factor: 1 in the surface litter and `1 - 0.75 * (silt + clay)` elsewhere.

Default Parton optimal rates (per day):

Pool	Surface	Subsurface
Microbial biomass (BMR)	0.0164	0.02
Slow humus (HSR)	-	0.000548
Passive humus (HPR)	-	0.000012
Metabolic litter (LMR)	0.0405	0.0507
Structural litter (LSR)	0.0107	0.0132

CO2 allocations on transformation:

From	To CO2 (surface)	To CO2 (subsurface)
Biomass	0.60	0.85 - 0.68 * (silt+clay)
Metabolic litter	0.60	0.55
Non-lignin structural	0.60	0.55
Lignin structural	0.30	0.30
Slow humus	0.55	0.55
Passive humus	0.55	0.55

Biomass-to-passive humus allocation in subsurface layers: `0.003 + 0.032 * clay`. Slow-to-passive allocation: `max(0.001, PRMT_45 - 0.00009 * clay)` with `PRMT_45 = 0.003`.

Microbial-biomass leaching uses a water-balance form

```
abl = 1 - exp(-vflow / (0.01 * st + KOC * bd))
```

with `KOC = 4000` (organic carbon partition coefficient, `part_DOC_para`).

Lignin shape factor for structural litter: `xlslf = exp(-3 * lslf)` where `lslf` is lignin fraction of structural litter.

C:N ratios are kept within Parton bounds for each pool and the transformation can release or consume mineral N depending on the deficit (`cpn1..cpn5` and `sum1..sum5`).

The Parton defaults are overridden by the per-layer rates and CO2 fractions supplied in `carbon_lyr.bsn`; the residue and basin-wide tunables come from `carbon.bsn`. The legacy `carb_coefs.cbn` and `basins_carbon.tes` files (and their readers `carbon_coef_read.f90` and `carbon_read.f90`) have been removed.

DOC and DIC

The Century implementation accumulates dissolved organic carbon (DOC) and dissolved inorganic carbon (DIC) percolation. Defaults `peroc_DIC_para = 0.95`, `peroc_DOC_para = 0.70`, `hlife_doc_para = 50` days for DOC half-life in groundwater. Source: `carbon_module.f90`, `cbn_zhang2.f90`.

Switches and parameters

File	Field	Default	Effect
codes.bsn	cswat	0	0 = static, 1 = C-FARM (reserved, inert), 2 = active CENTURY/SWAT-C module
codes.bsn	idc_till	3	Tillage method when cswat = 2
carbon.bsn	28 basin-wide scalars	see file	Required when cswat = 2; covers initial pool fractions, water coupling, manure partition, biomixing and tillage curves, temperature thresholds, residue C:N / C:P controls, and the slow-humus init method
carbon_lyr.bsn	per-layer rate constants and CO2 fractions	see file	Required when cswat = 2; one row per layer group
carbon_layers.prt	cb_n_layers	max HRU layers	Optional; fixes the number of layers in per-layer carbon output. If absent, defaults to the largest soil layer count across all HRUs

Notable Century coefficients (from carbon_module.f90):

Name	Default	Meaning
CFB_para	0.42	Carbon fraction of residue
er_POC_para	1.5	POC enrichment ratio
peroc_DOC_para	0.70	DOC percolation coefficient
peroc_DIC_para	0.95	DIC percolation coefficient
part_DOC_para	4000	DOC partition coefficient
hlife_doc_para	50 days	DOC half-life in groundwater
ALSLCO2_para	0.30	Lignin-to-CO2 allocation
APCO2_para	0.55	Passive humus to CO2
ASCO2_para	0.55	Slow humus to CO2
BMR_para_sur / BMR_para_sub	0.0164 / 0.02	Microbial biomass turnover (d^-1)
HPR_para	0.000012	Passive humus turnover (d^-1)
HSR_para	0.000548	Slow humus turnover (d^-1)
PRMT_45_para	0.003	Slow-to-passive allocation coefficient
PRMT_51_para	1.0	Microbial activity coefficient for top layer
OX_aa_para	10.0	Oxygen factor coefficient
OX_bb_para	0.035	Oxygen factor coefficient

Implementation

Core: cbn_zhang2.f90 (Century pool transformations), cbn_surfrsd_decomp.f90 (surface residue), cbn_rsd_decomp.f90 (sub-surface residue, used in legacy paths), cbn_rsd_transfer.f90 (residue routing into Century pools).

Inputs: carbon_module.f90 (data types and module-scope tunables), carbon_bsn_read.f90 (reads both carbon.bsn and carbon_lyr.bsn), carbon_layers_read.f90 (optional carbon_layers.prt).

Tillage: mgt_newtillmix_cswat0.f90, mgt_newtillmix_cswat1.f90, mgt_newtillmix_wet.f90, mgt_tillfactor.f90, mgt_biomix.f90.

Driver in hru_control.f90 switches by bsn_cc%cswat. Outputs are written by output_landscape_init.f90, hru_carbon_output.f90, lsu_carbon_output.f90, and soil_nutcarb_write.f90. Output gating is per-family through the hru_cb_* and lsu_cb_* rows of print.prt; the old cbn_diagnostics flag has been retired.

Related pages

- codes.bsn. The cswat switch.
- carbon.bsn. Basin-wide carbon scalars.
- carbon_lyr.bsn. Per-layer carbon coefficients.

- Carbon outputs. HRU and LSU file families.
- Parameter change files. 40 wired carbon calibration parameters.
- Nutrient cycling.
- Plant growth.

Overview

SWAT+ tracks a configurable set of pesticides as constituents. Each pesticide is described by sorption, solubility, half-lives, and (optionally) one or more daughter products. Applications are scheduled by the management table and split between the canopy and the top two soil layers. Once in the system the pesticide decays exponentially on foliage and in the soil, washes off the canopy in rain, leaches with percolation, partitions to runoff water and sediment, and may be taken up by plants. Daughter products are accumulated as the parent decays.

Process equations

Application

Per application, ground cover is

```
gc = (1.99532 - erfc(1.333 * lai_sum - 2)) / 2.1
```

clamped to [0, 1]. Of the applied mass `pest_kg`:

- `gc * pest_kg` goes to plant foliage, partitioned among plants in proportion to their LAI
- `(1 - gc) * surf_frac * pest_kg` goes to soil layer 1
- `(1 - gc) * (1 - surf_frac) * pest_kg` goes to soil layer 2

`surf_frac` is the surface application fraction read from `chem_app.ops`. Source: `pest_apply.f90`.

Decay

First-order decay in soil and on foliage:

```
decay_s = exp(-ln(2) / soil_hlife)
decay_f = exp(-ln(2) / foliar_hlife)
pest(t+1) = pest(t) * decay
```

The decay coefficients `pestcp(k)%decay_s` and `pestcp(k)%decay_f` are pre-computed once from `soil_hlife` and `foliar_hlife` in `pesticide.pst`. Daughter pesticides accumulate decayed mass scaled by the molecular weight ratio:

```
metab_decay = pst_decay * soil_fr * (mol_wt_daughter / mol_wt_parent)
```

Source: `pest_decay.f90`.

Foliar washoff

Each rain day, a fraction `washoff` of the pesticide on each plant's foliage is washed to soil layer 1:

```
pest_soil = washoff * pl_on
```

`washoff` is per pesticide in `pesticide.pst`. Source: `pest_washp.f90`.

Soil partitioning

In each layer, the pesticide is split between adsorbed and dissolved phases using a `Kd` partition computed from soil organic carbon:

```
kd   = koc * cbn / 100
zdb1 = ul + kd * bd * thick
```

koc is the organic-carbon-normalised partition coefficient. zdb1 is the effective partitioning depth in mm. The dissolved concentration available for transport is

```
co = pest * (1 - exp(-vf / zdb1)) / vf
co = min(solub / 100, co)
```

where vf is total flow through the layer (prk + flat, plus surfq in layer 1 and qtile in the tile layer). Source: pest_lch.f90.

Surface runoff, lateral flow, tile flow, percolation

Surface runoff carries

```
csurf = percop * co
surq  = csurf * surfq
```

Lateral and tile flow use co directly (layer 1 lateral uses csurf). Percolation moves co * prk to the next layer or out of the profile at the bottom layer. percop is the basin percolation coefficient (parameters.bsn, default 0.5). Source: pest_lch.f90.

Sediment-attached transport

For the top layer, sediment-attached pesticide is

```
conc = 100 * kd * pest / (zdb1 + 1e-10)
sed_pest = 0.001 * sedyld * conc * er / area_ha
```

The enrichment ratio er is either the channel-specific erorgn or the CREAMS-derived value from pest_enrsb.f90. Source: pest_pesty.f90.

Plant uptake

Daily uptake from each layer is proportional to water uptake by the plant:

```
pest_up = pl_uptake * uptake(ly) / st(ly) * pest(ly)
```

pl_uptake is the per-pesticide fraction in pesticide.pst. Pesticide on the plant is removed in harvest in proportion to the harvested biomass fraction. Source: pest_pl_up.f90, mgt_harvgrain.f90.

In-channel and in-aquifer

Channel pesticide reactions use aquatic and benthic half-lives, settling and resuspension velocities, burial, volatilisation, and benthic decay. Source: ch_rtpest.f90, ch_pesticide_module.f90, aqu_pesticide_module.f90, pest_cha_res_read.f90.

Switches and parameters

File	Field	Default	Effect
parameters.bsn	percop	0.5	Pesticide percolation coefficient (0-1). 0 = no pesticide in surface runoff
pesticide.pst	koc	per pest	Organic-carbon-normalised partition (mL/g)
pesticide.pst	washoff	per pest	Fraction washed from foliage per rain day
pesticide.pst	foliar_hlife	per pest	Foliar half-life (days)
pesticide.pst	soil_hlife	per pest	Soil half-life (days)
pesticide.pst	aq_hlife, ben_hlife	per pest	Aquatic and benthic half-lives (days)
pesticide.pst	solub	per pest	Water solubility (mg/L)
pesticide.pst	pl_uptake	per pest	Fraction taken up by plant

File	Field	Default	Effect
pesticide.pst	aq_volat, aq_resus, aq_settle, ben_bury	per pest	Channel kinetics
chem_app.ops	surf_frac	per op	Surface application fraction (rest to layer 2)
pest_hru_aqu.ini	initial mass	per HRU	Initial pesticide mass per HRU and layer

Implementation

Land phase: pest_apply.f90, pest_decay.f90, pest_washp.f90, pest_lch.f90, pest_pesty.f90, pest_pl_up.f90, pest_enrsb.f90, pest_soil_tot.f90.

Inputs: pest_parm_read.f90, pest_hru_aqu_read.f90, pest_metabolite_read.f90, pesticide_data_module.f90.

Constituent driver: constituent_mass_module.f90, cs_db%num_pests controls activation.

HRU output: hru_pesticide_output.f90.

Channel and aquifer: ch_rtpest.f90, ch_pesticide_module.f90, aqu_pesticide_module.f90, aqu_pest_output_init.f90, aqu_pesticide_output.f90, pest_cha_res_read.f90, cha_pesticide_output.f90.

The driver in hru_control.f90 calls (in order) pest_apply (via management), then daily: pest_washp, pest_decay, pest_lch, pest_pesty, pest_pl_up.

Related pages

- parameters.bsn
- management.sch
- Plant growth
- Channels
- Erosion and sediment

Overview

SWAT+ tracks two optional families of conservative-but-reactive constituents:

- Salt ions (rtb salt): SO4, Ca, Mg, Na, K, Cl, CO3, HCO3 (eight ions)
- Trace constituents (rtb cs): SeO4, SeO3, boron

Both families share the same flow paths as nitrate: surface runoff, lateral flow, tile drainage, percolation, plant uptake, rainfall and dry deposition, irrigation, and fertiliser/amendment inputs. Salt ions are subject to equilibrium chemistry (precipitation-dissolution of CaCO3, MgCO3, CaSO4, MgSO4, NaCl and cation exchange between Na, Ca, Mg, K). Selenium and boron use simple linear sorption and a first-order reaction term. Salinity stress can reduce crop water uptake.

These modules are active only when their respective databases are present, controlled by cs_db%num_salts and cs_db%num_cs (set during cs_read_* setup).

Process equations

Soil-water transport

For both salts and trace constituents the layer-level transport routine mirrors nut_nlch.f90. For each salt ion (or cs species) m:

```
vv = perc + flat + (surfq if layer 1) + (qtile if tile layer)
co = mass * (1 - exp(-vv / ul)) / vv
surq_mass  = co * surfq             # layer 1 only
```

```
latq_mass  = co * flat
tile_mass  = co * qtile            # at tile layer
perc_mass  = co * prk              # next layer or out of profile
```

Lateral, tile, and percolation are first-order in mobile water with porosity `ul`. No partition coefficient is applied to salt ions; sorbed selenium and boron use `cs_sorb_hru.f90`. Source: `salt_lch.f90`, `cs_lch.f90`.

Salt chemistry in soil

`salt_chem_hru.f90` and `salt_chem_soil_single.f90` solve equilibrium speciation per layer using:

- Activity coefficients via ionic strength (Davies / extended Debye-Huckel form)
- Precipitation-dissolution of five minerals: CaCO3 (Ksp = 3.07e-9), MgCO3 (4.79e-6), CaSO4 (7.89e-5), MgSO4 (7.24e-3), NaCl (37.3)
- Complexation reactions (CaCO3, MgCO3, CaSO4, MgSO4, NaCl)
- Cation exchange between Na, Ca, Mg, K through `cationexchange`

These run iteratively until ion concentrations converge. The same scheme is applied to the aquifer in `salt_chem_aqu.f90` using a separate set of Ksp values (currently the same defaults).

Salinity stress on plants

When `salt_tol_sim = 1` and at least one salt ion is simulated, `pl_waterup.f90` computes a salinity reduction factor per layer using the FAO yield-vs-ECe relationship:

```
soil_TDS_sat = sum(soil_salt_conc) at saturation
soil_ECe = soil_TDS_sat / salt_tds_ec
if (soil_ECe > a):
    reduc_salt = (100 - b * (soil_ECe - a)) / 100
```

`a` is the salt tolerance threshold (dS/m), `b` is the slope (yield loss per dS/m above the threshold). Both are per-plant parameters read from `salt_plants.slt`. If `salt_soil_type = 1` (CaSO4 / gypsiferous), the threshold is increased by 2 dS/m. The minimum `reduc_salt` across rooted layers becomes the plant salt stress factor `strss`.

`salt_effect = 1` applies salinity stress after the other stresses are minimised; `salt_effect = 2` (default) includes it in the joint minimum.

Salt uptake by plants

When `salt_uptake_on = 1`, daily salt removal by plants is read from a user file (`salt_uptake.slt`) and applied through `salt_uptake.f90`. Otherwise no plant uptake of salt occurs.

Trace constituent reactions and sorption

`cs_rctn_hru.f90` applies first-order reactions to SeO4, SeO3, and boron. Coefficients come from `cs_reactions.cs`. `cs_sorb_hru.f90` runs an equilibrium linear sorption between dissolved and solid-phase pools per layer, parameterised by partition coefficients in `cs_data_module.f90`. Constituent uptake follows the nitrate-style uptake routine `cs_uptake.f90`.

Salt and constituent inputs

- Atmospheric: wet deposition (`salt_rain.f90`, `cs_rain.f90`) and dry deposition (read from `atmodep_salt`, `atmodep_cs`), monthly or annual
- Fertiliser: salt-bearing fertilisers from `salt_fert.f90`, `cs_fert.f90` (read into `fert_salt`, `fert_cs`)
- Road salt: `salt_roadsalt.f90`, applied seasonally per HRU
- Irrigation: salt and cs concentrations in irrigation water (`salt_irrig.f90`, `cs_irrig.f90`)
- Initial soil conditions: `salt_hru_init.f90`, `cs_hru_init.f90`

In-channel and aquifer

Channels route salt ions through `ch_salt_module.f90` and `ch_salt_output.f90`; constituents through `ch_cs_module.f90` and `ch_cs_output.f90`. Aquifer fluxes are handled in `salt_aquifer.f90`, `salt_chem_aqu.f90`, `cs_aquifer.f90`, `cs_rctn_aqu.f90`, `cs_sorb_aqu.f90`. Reservoirs and wetlands have parallel implementations (`res_salt.f90`, `wet_salt.f90`).

Switches and parameters

Salts

File	Field	Default	Effect
`salt_module.f90`	`salt_uptake_on`	0	1 = enable user-defined daily salt uptake
`salt_data_module.f90`	`salt_tol_sim`	0	1 = salinity stress on plant water uptake
`salt_data_module.f90`	`salt_soil_type`	0	1 = CaSO4 (gypsiferous), 2 = NaCl
`salt_data_module.f90`	`salt_effect`	0	1 = applied after other stresses, 2 = joint minimum
`salt_data_module.f90`	`salt_tds_ec`	input	TDS-to-EC conversion factor
`salt_data_module.f90`	`Ksp11..Ksp51`	see source	Soil mineral solubility products
`salt_data_module.f90`	`Ksp12..Ksp52`	same defaults	Aquifer mineral solubility products
`salt_plants.slt`	`a`, `b` per plant	input	Salt tolerance threshold and slope
`salt_recall.slt` and other recall files	input	Constituent loads on point sources	

Constituents (cs)

File	Field	Effect
`cs_data_module.f90`	partition coefficients	Linear sorption per constituent
`cs_reactions.cs`	first-order rate coefficients	Constituent reaction rates
`cs_uptake.cs`	uptake fractions per plant	Plant uptake of cs
`cs_fert.cs`	constituent loads in fertiliser	Inputs from fertiliser
`cs_irrig.cs`	constituent concentration in irrigation	Inputs from irrigation

Implementation

Salts

Soil and HRU: `salt_balance.f90`, `salt_lch.f90`, `salt_chem_hru.f90`, `salt_chem_soil_single.f90`, `salt_uptake.f90`, `salt_rain.f90`, `salt_fert.f90`, `salt_fert_wet.f90`, `salt_irrig.f90`, `salt_roadsalt.f90`, `salt_hru_init.f90`.

Inputs: `salt_aqu_read.f90`, `salt_cha_read.f90`, `salt_fert_read.f90`, `salt_hru_read.f90`, `salt_irr_read.f90`, `salt_plant_read.f90`, `salt_roadsalt_read.f90`, `salt_uptake_read.f90`, `salt_urban_read.f90`.

Modules: `salt_module.f90`, `salt_data_module.f90`.

Channel and water bodies: `ch_salt_module.f90`, `ch_salt_output.f90`, `res_salt.f90`, `res_salt_module.f90`, `res_salt_output.f90`, `res_read_salt_cs.f90`, `res_read_saltdb.f90`, `wet_salt.f90`, `wet_salt_output.f90`, `wet_read_salt_cs.f90`.

Aquifer: `salt_aquifer.f90`, `salt_chem_aqu.f90`, `aqu_salt_output.f90`.

Climate, recall, export coefficients, delivery ratios: `cli_read_atmodep_salt.f90`, `recall_read_salt.f90`, `recall_salt.f90`, `dr_read_salt.f90`, `exco_read_salt.f90`, `header_salt.f90`.

Output: `hru_salt_output.f90`, `ru_salt_output.f90`, `output_ls_salt_module.f90`.

Trace constituents (cs)

Soil and HRU: `cs_balance.f90`, `cs_lch.f90`, `cs_rctn_hru.f90`, `cs_sorb_hru.f90`, `cs_uptake.f90`, `cs_rain.f90`, `cs_fert.f90`, `cs_fert_wet.f90`, `cs_irrig.f90`, `cs_hru_init.f90`.

Inputs: `cs_aqu_read.f90`, `cs_cha_read.f90`, `cs_fert_read.f90`, `cs_hru_read.f90`, `cs_irr_read.f90`, `cs_plant_read.f90`, `cs_reactions_read.f90`, `cs_uptake_read.f90`, `cs_urban_read.f90`.

Modules: `cs_module.f90`, `cs_data_module.f90`.

Channel: `ch_cs_module.f90`, `ch_cs_output.f90`, `cs_str_output.f90`.

Aquifer: `cs_aquifer.f90`, `cs_rctn_aqu.f90`, `cs_sorb_aqu.f90`, `aqu_cs_output.f90`.

Reservoirs and wetlands: `res_cs_module.f90`, `res_cs_output.f90`, `wet_cs_output.f90`.

HRU and RU output: `hru_cs_output.f90`, `ru_cs_output.f90`.

Related pages

- Nutrient cycling
- Plant growth
- Aquifers
- Channels
- Recall and point sources

Overview

Channels move water, sediment, and constituents along the drainage network from upstream HRUs and aquifers to the outlet. Each channel object has a geometry (width, depth, slope, length, side slope, Manning's n) read from `hydrology-cha.cha` and `sd_channel.cha`, and a rating curve built from that geometry. Daily inflow is the sum of all upstream object outflows. Daily outflow is computed by one of two routing methods selected by `bsn_cc%rte`. After routing, the channel runs sediment transport, in-stream water quality, and optional pesticide, salt, and constituent routines.

Process equations

Channel routing: variable storage vs Muskingum

`bsn_cc%rte` in `codes.bsn` selects the method:

`bsn_cc%rte`	Method
0	Variable storage coefficient
1	Muskingum

Both run inside `ch_rtmusk.f90` (the file name reflects history; the routine dispatches on `bsn_cc%rte` inside its sub-step loop). The older daily-only path in `ch_rtday.f90` uses the variable storage coefficient and is retained for the legacy channel object.

Variable storage. At each sub-step the outflow fraction is

```
scoef = bsn_prm%scoef * 2 * dt / (2 * ttime + dt)
outflow = scoef * total_storage
```

where `ttime` is the travel time interpolated from the channel rating curve at the current flow rate. `bsn_prm%scoef` (default 1.0) calibrates the storage coefficient.

Muskingum. The three-coefficient form:

```
O2 = c1 * I2 + c2 * I1 + c3 * O1
```

with c1, c2, c3 derived from the storage time constant Km and the weighting factor X. Km is computed from the bankfull and low-flow reach response times scaled by bsn_prm%msk_co1 (default 0.75) and bsn_prm%msk_co2 (default 0.25). X = bsn_prm%msk_x (default 0.20). The number of sub-daily steps and stability sub-steps is sd_ch(jrch)%msk%nsteps and sd_ch(jrch)%msk%substeps.

After routing, transmission losses are subtracted using the channel bed conductivity sd_ch%chk (mm/h) applied over the wetted perimeter and reach length. Reach evaporation uses bsn_prm%evrch (default 0.60) applied to PET over the channel surface area.

Sediment transport: legacy vs sd_channel

Two sediment routines exist.

Legacy stream power transport (channel_module.f90). Older channels carry the maximum sediment concentration as a power function of the peak flow velocity:

```
conc_max = spcon * v_peak ^ spexp
```

spcon and spexp live in bsn_prm as legacy parameters; in current basin_module.f90 they are flagged as not used by the modern routine. TODO: verify whether the legacy channel path still references these.

Velocity and shear (sd_channel_sediment3.f90). The current sd_channel path computes:

1. Peak channel flow rate from the daily flow and a peak ratio sd_ch%pk_rto adjusted for drainage area.
2. Channel velocity from Manning's equation at the peak rate, using the rating curve to get cross-section area and hydraulic radius.
3. Critical bank velocity vel_cr from the bank cohesion and bulk density factors, combined with the side-slope geometry.
4. Bank erosion as a function of the ratio vel / vel_cr raised to sd_chd1%bed_exp (default 1.5), distributed along the meander arc length scaled by sd_chd1%arc_len_fr (default 1.2).
5. Bed erosion from a critical shear stress comparison using the median bed particle diameter sd_chd%d50 and the bulk density sd_chd%ch_bd.
6. Deposition computed in the flood plain when the flow exceeds bankfull, with a trap efficiency trap_eff set by the flood plain Manning's n sd_chd%fpn and slope sd_chd%fps.

Eroded sediment is split between channel and bank; a fraction sd_chd1%wash_bed_fr (default 0.1) of bank erosion is washload that leaves the reach. Particle-class fractions (silt, clay, sand, large aggregates, gravel) follow the bed composition.

In-stream water quality (QUAL2E)

Nutrient transformations are computed in ch_watqual4.f90. The implementation is the QUAL2E equation set: algal growth limited by light, nutrient availability, and temperature; oxygen balance between algal photosynthesis, respiration, BOD decay, nitrification, and reaeration; nitrogen cycling from organic N to ammonium to nitrite to nitrate; phosphorus cycling from organic P to mineral P with settling and benthic sources.

Temperature corrections use the standard theta factors hard-coded in the routine (thgra = 1.047, thrho = 1.047, thbc1 = 1.083, etc.). Reaeration uses bc1, bc2, bc3, bc4, rs1 through rs5, and rk1 through rk4 read from nutrients-cha.nut.

When bsn_cc%qual2e = 1, a simplified soluble-to-particulate transformation is applied after ch_watqual4 runs, overriding the QUAL2E organic and mineral exchange:

```
delta_orgn = (1 - exp(-n_sol_part * ttime)) * orgn_in
delta_orgp = (1 - exp(-p_sol_part * ttime)) * sedp_in
```

n_sol_part and p_sol_part come from sd_channel.cha (defaults 0.01 each). ttime is the channel travel time from the rating curve.

Pesticides, salts, constituents

After water quality, the channel runs ch_rtpest.f90 for pesticides and the salt and constituent routines if those modules are active. See Pesticides and Salts and constituents.

Switches and parameters

Switch / parameter	Default	File	Effect
bsn_cc%rte	0	codes.bsn	0 = variable storage; 1 = Muskingum
bsn_cc%qual2e	0	codes.bsn	1 = simplified N and P sol-to-part overrides QUAL2E
bsn_prm%scoef	1.0	parameters.bsn	Variable-storage channel coefficient
bsn_prm%msk_co1	0.75	parameters.bsn	Muskingum bankfull storage coefficient
bsn_prm%msk_co2	0.25	parameters.bsn	Muskingum low-flow storage coefficient
bsn_prm%msk_x	0.20	parameters.bsn	Muskingum weighting factor
bsn_prm%evrch	0.60	parameters.bsn	Reach evaporation factor
bsn_prm%spcon	0.0	parameters.bsn	Legacy stream power coefficient (not used by sd_channel)
bsn_prm%spexp	0.0	parameters.bsn	Legacy stream power exponent (not used by sd_channel)
bsn_prm%ch_d50	0.0 mm	parameters.bsn	Basin-wide channel median particle size
sd_chd%chw	per channel	sd-channel	Channel width (m)
sd_chd%chd	per channel	sd-channel	Channel depth (m)
sd_chd%chs	per channel	sd-channel	Channel slope (m/m)
sd_chd%chl	per channel	sd-channel	Channel length (km)
sd_chd%chn	per channel	sd-channel	Channel Manning's n
sd_chd%chk	per channel	sd-channel	Channel bottom conductivity (mm/h)
sd_chd%cov	per channel	sd-channel	Channel cover factor (0-1)
sd_chd%d50	per channel	sd-channel	Channel median sediment size (mm)
sd_chd%ch_bd	per channel	sd-channel	Bed and bank bulk density (t/m^3)
sd_chd%ch_clay	per channel	sd-channel	Bed and bank clay percent
sd_chd1%pk_rto	1.0	sd-channel	Ratio of peak to mean daily flow
sd_chd1%n_setl	0.5	sd-channel	N settling ratio to sediment settling
sd_chd1%p_setl	0.5	sd-channel	P settling ratio to sediment settling
sd_chd1%n_sol_part	0.01	sd-channel	Simplified soluble to particulate N rate
sd_chd1%p_sol_part	0.01	sd-channel	Simplified soluble to particulate P rate
sd_chd1%bed_exp	1.5	sd-channel	Bed erosion exponential coefficient
sd_chd1%arc_len_fr	1.2	sd-channel	Fraction of arc length where bank erodes
sd_chd1%wash_bed_fr	0.1	sd-channel	Washload fraction of bank erosion

Implementation

Source modules in `swatplus/src/`:

- `sd_channel_control3.f90` is the master control routine for an `sd_channel` object. It receives the daily

inflow, calls routing, sediment, water quality, pesticide, salt, and constituent routines, and updates output structures.
- `ch_rtmusk.f90` runs the sub-daily routing loop. It dispatches on `bsn_cc%rte` and computes outflow, transmission loss, and reach evaporation.
- `ch_rtday.f90` is the daily variable-storage routing used by the legacy `channel` object.
- `sd_channel_sediment3.f90` computes channel velocity, critical velocity, bank and bed erosion, and flood plain deposition for the `sd_channel` path.
- `ch_watqual4.f90` performs the QUAL2E in-stream nutrient and oxygen calculations.
- `ch_rtpest.f90` routes pesticides through the channel.
- `ch_temp.f90` computes channel water temperature.
- `channel_module.f90` defines the legacy `channel` derived types and storage arrays.
- `sd_channel_module.f90` defines the `sd_channel` derived types: hydsed data (`sd_chd`), sediment-nutrient data (`sd_chd1`), sediment budget output, and morphology output.
- `channel_data_module.f90` holds channel parameter arrays read from input.
- `channel_velocity_module.f90` stores the rating curve and velocity bookkeeping.
- `ch_rchinit.f90` initialises the channel state at the start of simulation.
- `ch_ttcoef.f90` builds the Muskingum coefficients from the channel geometry.

Related pages

- codes.bsn for `rte` and `qual2e`.
- parameters.bsn for Muskingum and storage coefficients.
- sd-channel for channel geometry and sediment parameters.
- cha for the legacy channel object.
- Channel sediment and nutrient output.
- Hydrology for the upstream HRU inflow.
- Aquifers for return flow to channels.

Overview

The aquifer cluster receives percolation from HRU soil profiles and returns it to channels as baseflow, sends a fraction to deep seepage, and supports root uptake (revap) when the water table is shallow. Two implementations are available. The default is a 1D lumped recession aquifer with one storage per aquifer object. The alternative is the `gwflow` module, a grid-cell groundwater model with explicit head and lateral exchange. `bsn_cc%gwflow` in `codes.bsn` switches between them.

Process equations

1D recession aquifer (`bsn_cc%gwflow = 0`)

The default aquifer object holds a single storage. Each day the recharge from the soil profile is added to storage, then the water table depth, return flow, deep seepage, and revap are computed in `aqu_1d_control.f90`.

Water table depth. Storage converts to a water table depth using specific yield:

```
dep_wt = dep_bot - stor / (1000 * spyld)
```

`dep_bot` is the aquifer bottom depth (m), `spyld` is specific yield ($m^{3/m}3$), and `stor` is storage in mm.

Return flow. When the water table is at or above the threshold `flo_min`, return flow follows a recession with lag factor `alpha`:

```
flo_new = flo_prev * exp(-alpha) + rchrg * (1 - exp(-alpha))
```

`alpha_e = exp(-alpha)` is precomputed once. Return flow is capped at the available storage. Below the threshold (`dep_wt > flo_min`) return flow is zero.

Deep seepage. A fixed fraction of the daily recharge seeps to the deep aquifer:

```
seep = rchrg * seep_frac
```

where `seep_frac` is `aqu_dat%seep` from `aquifer.aqu`.

Revap. When the water table is shallower than `revap_min`, plant uptake and evaporation from the aquifer is

```
revap = pet * revap_co
```

capped at storage. Above the threshold revap is zero. `revap_co` and `revap_min` are read from `aquifer.aqu`.

Nitrate. Nitrate in recharge water mixes with the aquifer pool. Return flow nitrate is the storage-weighted concentration. A first-order decay uses the half-life `hlife_n` (default 30 days) read from `aquifer.aqu`. Nitrate seepage uses the same concentration applied to the seepage volume.

2D gwflow (`bsn_cc%gwflow = 1`)

When `bsn_cc%gwflow = 1` the lumped 1D aquifer is replaced by a cell-based groundwater model. The watershed is discretised into a grid (structured or unstructured) where each active cell carries a head, hydraulic conductivity, specific yield, and a list of connections to neighbour cells. The model couples to channels, reservoirs, wetlands, tile drains, pumping wells, and HRU recharge.

Per-cell state (in `gwflow_module.f90`):

- `head` current simulated head (m)
- `hydc` aquifer hydraulic conductivity (m/day)
- `spyd` specific yield ($m^{3/m}3$)
- `exdp` groundwater ET extinction depth (m)
- `thck` aquifer thickness, `botm` bedrock elevation, `elev` surface elevation
- `stor` available groundwater storage (m^3)

Each day the model:

1. Computes recharge from HRU or LSU connections (`gwflow_rech.f90`).
2. Computes groundwater ET and phreatophyte transpiration (`gwflow_gwet.f90`, `gwflow_phreatophyte.f90`).
3. Exchanges flow with channels using the Darcian conductance derived from the channel bed (`gwflow_channel_exch.f90`).
4. Handles flow to and from reservoirs and ponds (`gwflow_reservoir.f90`, `gwflow_pond.f90`).
5. Handles wetland exchange (`gwflow_wetland.f90`).
6. Handles tile drain capture (`gwflow_tile.f90`).
7. Handles floodplain exchange when channels overflow (`gwflow_floodplain.f90`).
8. Handles canal diversions and external inflows (`gwflow_canal.f90`, `gwflow_canal_div.f90`, `gwflow_canal_ext.f90`).
9. Handles pumping (`gwflow_pump_allo.f90`, `gwflow_pump_ext.f90`).
10. Solves the head at each cell from neighbour conductances and storage change (`gwflow_simulate.f90`).
11. Redistributes saturation-excess water if a cell head exceeds the surface (`gwflow_satexcess.f90`).
12. Routes solutes and heat through cells (`gwflow_solute.f90`, `gwflow_chem.f90`, `gwflow_heat.f90`).

Aquifer-to-channel mapping for the geomorphic baseflow option uses `aqu2d_init.f90` to sort channels by drainage area, allocate flow to channels by length and area, and `aqu2d_read.f90` to read the `aquifer_cha.con` link file.

Boundary conditions are set per cell (`gw_state(i)%stat`): 0 inactive, 1 active, 2 boundary. Constant-head and no-flow types are selected by `bc_type` in the gwflow inputs.

Switches and parameters

Switch / parameter	Default	File	Effect
`bsn_cc%gwflow`	0	`codes.bsn`	0 = 1D recession aquifer; 1 = `gwflow` grid model
`aqu_dat%flo`	0.05 mm	`aqu`	Initial flow from aquifer
`aqu_dat%dep_bot`	per aquifer	`aqu`	Depth to aquifer bottom (m)
`aqu_dat%dep_wt`	per aquifer	`aqu`	Initial water table depth (m)

Switch / parameter	Default	File	Effect
aqu_dat%bf_max	0.0 mm	aqu	Maximum daily baseflow
aqu_dat%alpha	per aquifer	aqu	Recession lag factor (1/days)
aqu_dat%revap_co	per aquifer	aqu	Revap coefficient
aqu_dat%revap_min	per aquifer	aqu	Water table depth threshold for revap (m)
aqu_dat%flo_min	per aquifer	aqu	Water table depth threshold for return flow (m)
aqu_dat%seep	per aquifer	aqu	Fraction of recharge to deep seepage
aqu_dat%spyld	per aquifer	aqu	Specific yield ($m^{3/m}3$)
aqu_dat%hlife_n	30 days	aqu	Half-life of nitrate in groundwater
aqu_dat%no3	per aquifer	aqu	Initial NO3-N concentration (ppm)
aqu_dat%minp	per aquifer	aqu	Initial mineral P concentration (ppm)
gw_state%hydc	per cell	gwflow inputs	Hydraulic conductivity (m/day)
gw_state%spyd	per cell	gwflow inputs	Specific yield ($m^{3/m}3$)
gw_state%exdp	per cell	gwflow inputs	ET extinction depth (m)
conn_type	0	gwflow inputs	1 = HRU recharge connections; 2 = LSU
bc_type	0	gwflow inputs	1 = constant head; 2 = no-flow

TODO: verify exact column order and defaults in `aquifer.aqu` and the gwflow input files against the corresponding `*_read.f90` readers.

Implementation

Source modules in `swatplus/src/`:

- `aqu_1d_control.f90` is the master daily driver for the 1D aquifer object. Adds recharge, computes water table, return flow, deep seepage, revap, and nitrate transport.
- `aqu_read.f90` reads `aquifer.aqu` into the `aqudb` and `aqu_dat` arrays.
- `aqu_read_init.f90`, `aqu_read_init_cs.f90` read initial conditions.
- `aqu_initial.f90` and `aqu_initial` set up storage and pre-compute `alpha_e`.
- `aquifer_module.f90` defines `aquifer_database`, `aquifer_data_parameters`, and `aquifer_dynamic` derived types.
- `aquifer_output.f90`, `aqu_pesticide_output.f90`, `aqu_salt_output.f90`, `aqu_cs_output.f90` print aquifer state and constituent outputs.
- `aqu2d_init.f90` orders channels by drainage area and sets up the aquifer-to-channel mapping for the geomorphic baseflow option.
- `aqu2d_read.f90` reads the `aquifer_cha.con` link file.
- `gwflow_module.f90` defines all gwflow state types and arrays.
- `gwflow_read.f90` reads the gwflow grid and parameters.
- `gwflow_simulate.f90` is the daily driver: assembles fluxes and solves for new heads.
- `gwflow_rech.f90` distributes HRU or LSU recharge to cells.
- `gwflow_channel_exch.f90`, `gwflow_chan_read.f90` handle cell-channel exchange.
- `gwflow_floodplain.f90` handles overland exchange with the channel flood plain.
- `gwflow_reservoir.f90`, `gwflow_wetland.f90`, `gwflow_pond.f90` handle waterbody exchange.
- `gwflow_tile.f90` handles tile drainage capture.
- `gwflow_canal.f90`, `gwflow_canal_div.f90`, `gwflow_canal_ext.f90` handle canal networks.
- `gwflow_pump_allo.f90`, `gwflow_pump_ext.f90` handle pumping.

- `gwflow_lateral.f90` handles lateral inflow from non-grid sources.
- `gwflow_soil.f90` couples grid cells back into HRU soil profiles.
- `gwflow_satexcess.f90` handles saturation-excess water when head reaches the surface.
- `gwflow_gwet.f90`, `gwflow_phreatophyte.f90` handle groundwater ET.
- `gwflow_solute.f90`, `gwflow_chem.f90` handle nitrate, salt, and constituent transport.
- `gwflow_heat.f90` handles heat transport.
- `gwflow_output.f90` writes daily, monthly, yearly, and average-annual gwflow output.

Related pages

- `codes.bsn` for the `gwflow` switch.
- `aqu` for the 1D aquifer input file.
- Hydrology for percolation that becomes aquifer recharge.
- Channels for baseflow return to streams.
- Aquifer, reservoir, and plant output.

Overview

Reservoirs are standalone water body objects in the routing network. They receive inflow from upstream channels, HRUs, or recall objects, store it, and release it according to a decision table in `res_rel.dtl`. Each reservoir runs sediment settling, in-pond nutrient processes, and optional pesticide, salt, and constituent processes.

Wetlands are a separate object type linked to individual HRUs. They sit on the HRU surface, intercept surface runoff before it reaches the channel, and run a similar water balance and sediment-nutrient set as reservoirs. Wetland storage is configured per HRU in `wetland.wet`.

Process equations

Reservoir water balance

The daily water balance for a reservoir in `res_control.f90` is

```
stor_new = stor_prev + inflow + precip - outflow - evap - seep - irrig
```

where:

- `inflow` is the upstream hydrograph (`ht1`).
- `precip` is precipitation onto the water surface from the assigned weather station, using the current surface area.
- `outflow` is the release computed by `res_hydro.f90`.
- `evap` is potential evaporation times the water surface area, optionally adjusted by the reservoir evaporation coefficient.
- `seep` is seepage through the bed, computed from the bed hydraulic conductivity over the surface area.
- `irrig` is water withdrawn for irrigation through the water allocation module.

Surface area as a function of storage is interpolated from the principal volume `pvol_m3` and emergency volume `evol_m3` defined for the reservoir.

Release rules via `res_rel.dtl`

Releases are not hard-coded. Each reservoir points to a decision table in `res_rel.dtl`. The table lists conditions (volume above principal, day of year, downstream demand, …) and corresponding actions (weir release, target release, percent of volume, irrigation supply, …). At each daily step `res_hydro.f90`:

1. Walks the action list (`d_tbl%acts`).
2. For each action, evaluates whether all condition alternatives are satisfied (`d_tbl%act_hit`).
3. Fires the corresponding release equation.

Release types include:

- `weir_release` for a weir of height `wbody_prm%weir_hgt` and width `wbody_prm%weir_wid`. The discharge over time follows the weir equation with the daily volume above the weir crest as the head source.
- `pvol_release` and `evol_release` for releases triggered when storage exceeds the principal or emergency volume.
- Doell-style storage-based release with coefficient `kr` and exponent `alpha` (Jose T 2025 extension in `res_hydro.f90`).
- Hanazaki monthly-target release using monthly inflow and demand.
- HYPE seasonal sine and linear storage modulation for hydropower-style operations.

The weir branch calls `res_weir_release.f90` to integrate over sub-daily steps.

Reservoir sediment

`res_sediment.f90` computes settling using a first-order rate. If the in-reservoir sediment concentration (ppm) exceeds the equilibrium concentration `wbody_prm%sed%nsed`, the excess is reduced by the settling coefficient `wbody_prm%sed_stlr_co`:

```
sed_ppm_new = (sed_ppm - nsed) * sed_stlr_co + nsed
```

After settling, sand, large aggregates, silt, clay, and gravel are recomputed; all of the sand-sized fraction is assumed to deposit. Outflow sediment leaves at the post-settling concentration.

Reservoir nutrients

`res_nutrient.f90` runs an in-pond nutrient set: settling of organic N and organic P, mineralisation, nitrification, and algal growth limited by light and nutrients. Soluble nutrients track storage-weighted concentration and leave at the outflow concentration. Settling rates are read from `nutrients-res.nut`.

Reservoir pesticides, salts, constituents

`res_pest.f90`, `res_salt.f90`, and `res_cs.f90` run when those constituent modules are active. See Pesticides and Salts and constituents.

Wetlands on HRUs

Wetlands are configured per HRU via `wet_initial.f90` and `wetland.wet`. Each wetland has:

- Principal volume `wet_ob%pvol` (m^3) computed from `wet_hyd%pdep` (principal depth, mm) times the HRU area.
- Emergency volume `wet_ob%evol` (m^3) from `wet_hyd%edep`.
- Principal and emergency surface areas `psa` and `esa`.
- Optional weir geometry `weir_hgt` and `weir_wid` read from `weir.res`.

The wetland state `wet(iihru)` is the same `water_body` type used for reservoirs. In `hru_control.f90` the HRU surface runoff is routed into the wetland (`wet(j) = wet(j) + ht1`), the wetland runs its water balance and release (using the same `res_hydro` machinery on a wetland), and the outflow is added to the HRU outgoing hydrograph. Storage to depth uses a quadratic relation from `wet_hyd%bcoef` and `wet_hyd%ccoef`:

```
depth^2 * ccoef + depth * bcoef = (1 - flo / pvol)
```

with the positive root taken.

Wetlands run their own sediment, nutrient, pesticide, salt, and constituent routines using the same module code paths as reservoirs.

Switches and parameters

Switch / parameter	Default	File	Effect
res_hyd%iyres	per reservoir	hydrology-res.res	Year reservoir comes online
res_hyd%mores	per reservoir	hydrology-res.res	Month reservoir comes online
pvol_m3	per reservoir	hydrology-res.res	Principal volume (m^3)
evol_m3	per reservoir	hydrology-res.res	Emergency volume (m^3)
wbody_prm%sed%nsed	per reservoir	sediment-res.sed	Equilibrium sediment concentration (ppm)
wbody_prm%sed_stlr_co	per reservoir	sediment-res.sed	Sediment settling coefficient
wbody_prm%weir_hgt	per reservoir	weir.res	Weir height (m)
wbody_prm%weir_wid	per reservoir	weir.res	Weir width (m)
res_ob%release_dtl	per reservoir	dtl (res_rel.dtl)	Decision table name for release rules
wet_hyd%pdep	per HRU wetland	wetland.wet	Principal depth (mm)
wet_hyd%edep	per HRU wetland	wetland.wet	Emergency depth (mm)
wet_hyd%psa	per HRU wetland	wetland.wet	Principal surface area fraction of HRU
wet_hyd%esa	per HRU wetland	wetland.wet	Emergency surface area fraction of HRU
wet_hyd%bcoef	per HRU wetland	wetland.wet	Storage-depth quadratic coefficient
wet_hyd%ccoef	per HRU wetland	wetland.wet	Storage-depth quadratic coefficient

TODO: verify default values in hydrology-res.res, sediment-res.sed, nutrients-res.nut, weir.res, and wetland.wet against their respective _read.f90 readers.

Implementation

Source modules in swatplus/src/:

- res_control.f90 is the master daily driver for a reservoir object. Adds inflow, computes the water balance, calls release, sediment, nutrient, pesticide, salt, and constituent routines.
- res_hydro.f90 evaluates the release decision table and computes outflow. Houses weir, principal-volume, emergency-volume, Doell, Hanazaki, and HYPE release branches.
- res_weir_release.f90 integrates weir discharge over sub-daily steps.
- res_rel_conds.f90 evaluates release conditions specific to reservoirs.
- res_sediment.f90 computes in-reservoir sediment settling.
- res_nutrient.f90 runs in-reservoir nutrient transformations.
- res_pest.f90 handles reservoir pesticides.
- res_salt.f90, res_cs.f90 handle salts and constituents.
- res_initial.f90, res_allo.f90, res_objects.f90 set up reservoir state at simulation start.
- res_read.f90, res_read_hyd.f90, res_read_sed.f90, res_read_nut.f90, res_read_weir.f90, res_read_init.f90, res_read_conds.f90, res_read_elements.f90, res_read_csdb.f90, res_read_saltdb.f90, res_read_salt_cs.f90 read the reservoir input files.
- reservoir_module.f90 defines reservoir types including wet_ob for wetlands.
- reservoir_data_module.f90 and reservoir_conditions_module.f90 hold parameter and conditions arrays.
- reservoir_output.f90, res_pesticide_output.f90, res_salt_output.f90, res_cs_output.f90 write reservoir outputs.
- wet_initial.f90 initialises a wetland on each flagged HRU.
- wet_all_initial.f90 initialises all wetlands at simulation start.
- wet_fp_init.f90 sets up the wetland-floodplain link to channels.
- wet_read.f90, wet_read_hyd.f90, wet_read_salt_cs.f90 read wetland input files.
- wet_irrp.f90 handles irrigation withdrawal from a wetland.

- `wet_cs.f90`, `wet_cs_output.f90`, `wet_salt.f90`, `wet_salt_output.f90` handle wetland constituents and salts.

Related pages

- Decision tables for how release rules are evaluated.
- `dtl` for the decision table input file format.
- `res` for the reservoir input files.
- `wet` for the wetland input files.
- Channels for downstream routing.
- Aquifer, reservoir, and plant output.

Overview

Three related object types let SWAT+ inject water and constituent loads into the routing network from outside the simulated land surface: recall objects, export coefficient objects, and delivery ratio objects.

- **Recall** objects deliver a time series of flow and mass (NO3, NH3, organic N, mineral P, organic P, sediment, pesticides, salts, constituents) read from `recall.rec` and supporting per-object data files. They are the standard point source representation, including municipal and industrial discharges, atmospheric deposition points, or measured imports.
- **Export coefficient** objects deliver an average annual flow and mass per unit area, computed once and applied each day. They are useful where only annual loading rates are known.
- **Delivery ratio** objects scale incoming loads on a routing unit basis (delivery from HRUs to channels), either calculated from time of concentration or set to a fixed fraction.

All three share a common reader infrastructure under `recall_module.f90` because internally they are all stored as recall databases.

Process equations

Recall objects

The recall reader (`recall_read.f90`) opens the file pointed to in `recall.rec` for each recall object and selects an allocation pattern based on the time step token:

`tstep`	Storage shape	Meaning
`sub`	`hyd_flo(step*366, nbyr)` plus `hd(366, nbyr)`	Sub-daily values per simulation year
`day`	`hd(366, nbyr)`	One record per day
`mo`	`hd(12, nbyr)`	One record per month
`yr`	`hd(1, nbyr)`	One record per year

At runtime the recall driver looks up the current day, month, or year and copies the stored hydrograph (`recall(irec)%hd(...)`) into the object's outgoing hydrograph. For sub-daily, flow is stored as `m^3/s` and converted to volume per step (`flo * 86400 / time%step`).

Recall objects can be linked to channel objects so the volume is added to the channel inflow, or to other receiving objects through the connectivity file `recall.con`.

Negative-flow recall objects represent diversions: `recall_nut.f90` and `recall_salt.f90` remove constituent mass from the source channel in proportion to the diverted volume and the source-water concentration.

Export coefficient objects

Export coefficient objects (`exco_module.f90`) are stored in the same recall database type with one record per object that holds average annual flow and constituent loads. The reader chain (`exco_db_read.f90`, `exco_read_om.f90`,

exco_read_pest.f90, exco_read_path.f90, exco_read_hmet.f90, exco_read_salt.f90) populates separate organic-mineral, pesticide, pathogen, heavy metal, and salt files keyed by exco_*_name. Each day the export coefficient flow is divided by 365.25 (TODO: verify exact daily fraction) and added to the receiving object.

Delivery ratio objects

Delivery ratios scale incoming loads from a routing-unit element by a fraction between 0 and 1. The element database dr_db in dr_module.f90 lists the organic-mineral, pesticide, pathogen, heavy metal, and salt delivery files for each delivery ratio object.

In dr_ru.f90 each element of a routing unit gets a dr value built from the dr_name keyword:

```
if (ru_elem(ielem)%dr_name == "calc" .or. "0") then
   ! compute from time of concentration
   rto = tconc(ihru) / ru_tc(iru)
   rto = min(1.0, sqrt(rto))
   ru_elem(ielem)%dr = rto + hz
   ru_elem(ielem)%dr%flo = 1.
else if (ru_elem(ielem)%dr_name == "full" .or. "1") then
   ru_elem(ielem)%dr = 1. + hz
end if
```

Three cases:

- "calc" or "0": ratio is the square root of the HRU time of concentration over the routing unit time of concentration, capped at 1. Flow itself is delivered in full (dr%flo = 1).
- "full" or "1": full delivery, ratio is 1 for everything.
- A named dr_db entry: ratios are read from the corresponding _om, _pest, _path, _hmet, and _salt files.

Delivery ratios apply only to the mass components (sediment, N, P, pesticide, salt), not to flow.

Switches and parameters

Parameter	File	Effect
recall_db%org_min%name	recall.rec	Organic and mineral data file for this recall object
recall_db%org_min%units	recall.rec	mass or conc
recall_db%org_min%tstep	recall.rec	sub, day, mo, or yr
recall_db%pest%name	recall.rec	Pesticide data file
recall_db%path%name	recall.rec	Pathogen data file
recall_db%hmet%name	recall.rec	Heavy metal data file
recall_db%salt%name	recall.rec	Salt data file
recall_db%constit%name	recall.rec	Generic constituent data file
exco_db%om_file	exco.exc	Organic-mineral export coefficient file
exco_db%pest_file	exco.exc	Pesticide export coefficient file
exco_db%salts_file	exco.exc	Salt export coefficient file
dr_db%om_file	delratio.del	Organic-mineral delivery ratio file
dr_db%pest_file	delratio.del	Pesticide delivery ratio file
dr_db%salts_file	delratio.del	Salt delivery ratio file
ru_elem%dr_name	rout_unit.def	Delivery ratio choice: calc, full, or a named dr_db entry

TODO: verify exact file names and column layouts against the source readers and current refdata.

Implementation

Source modules in swatplus/src/:

- `recall_module.f90` defines `constituent_file_data` and `recall_databases`. The same `recall_db` type is used for recall, exco, and dr objects.
- `recall_read.f90` reads `recall_db.rec` (the database index) and dispatches per-object reads. For each recall it opens the organic-mineral file, allocates the storage for the selected time step, and stores hydrographs by day, month, or year.
- `recall_read_salt.f90` reads the salt data file.
- `recall_read_cs.f90` reads the generic constituent data file.
- `recall_nut.f90` removes nutrient mass from a source channel for negative-flow recall objects (diversions).
- `recall_salt.f90` does the same for salts.
- `recall_cs.f90` does the same for constituents.
- `recall_output.f90` writes recall object output.
- `exco_module.f90` defines `export_coefficient_datafiles` and the per-constituent name and number arrays.
- `exco_db_read.f90` reads `exco.exc`.
- `exco_read_om.f90`, `exco_read_pest.f90`, `exco_read_path.f90`, `exco_read_hmet.f90`, `exco_read_salt.f90` read the per-constituent export coefficient files.
- `dr_module.f90` defines `delivery_ratio_datafiles` and the per-constituent name and number arrays.
- `dr_db_read.f90` reads `delratio.del`.
- `dr_read_om.f90`, `dr_read_pest.f90`, `dr_path_read.f90`, `dr_read_hmet.f90`, `dr_read_salt.f90` read the per-constituent delivery ratio files.
- `dr_ru.f90` computes the per-element delivery ratio for `calc` and `full` cases and assigns named delivery ratios at runtime.

Related pages

- Channels for the receiving object.
- Decision tables for management triggers that may use recall objects.
- Nutrient cycling for the channel and HRU receiving the recalled mass.
- TODO: link to input pages for `recall.rec`, `exco.exc`, and `delratio.del` once they exist.

Overview

Decision tables drive most of the data-driven behaviour in SWAT+: scheduled and conditional management on land use objects, reservoir release rules, scenario land use change, and flow control. A decision table lists conditions, alternative rule sets across those conditions, and the actions to fire when an alternative is matched. The runtime walks every condition, narrows the active alternatives, and fires each action whose action-outcome entry under any surviving alternative is `y`.

There are four decision table classes, each with its own file and its own array of in-memory tables:

Class	Input file	Array	Use
Land use management	`lum.dtl`	`dtbl_lum`	Plant, fert, till, harvest, kill operations on HRUs
Reservoir release	`res_rel.dtl`	`dtbl_res`	Daily reservoir release rules
Scenario land use	`scen_lu.dtl`	`dtbl_scen`	Land use change scenarios
Flow control	`flo_con.dtl`	`dtbl_flo`	Channel and routing flow rules

All four share the same derived type `decision_table` defined in `conditional_module.f90`.

Process equations

Data model

A decision table holds:

- `conds`, `alts`, `acts`: number of conditions, alternatives, and actions.
- `cond(conds)`: each condition has a variable name (`var`), an object reference (`ob`, `ob_num`), a comparison limit (`lim_const`, `lim_var`, `lim_op`).
- `alt(conds, alts)`: a string per condition per alternative giving the comparison operator (`<`, `>`, `<=`, `>=`, `=`, `/=`) or `-` for "any".
- `act(acts)`: each action has a type name (`typ`), an object reference, a name, an option, a constant `const`, an application fraction `fp`, and a file pointer.
- `act_outcomes(acts, alts)`: a `y` or `n` per action per alternative giving whether to fire the action if that alternative survives.
- `act_typ(acts)`, `act_app(acts)`: integer pointers resolved at read time (for instance, fertiliser type from the fert database, harvest operation type).
- `act_hit(alts)`: scratch array set during evaluation. Starts as `y` for every alternative; conditions flip it to `n` when not satisfied.

Condition evaluation

The driver `conditions.f90` walks `d_tbl%cond(1:conds)` and uses a `select case` on the condition variable name. Supported variables include:

```
w_stress n_stress p_stress phu_plant phu_base0 precip_cur precip_next
plant_gro plant_name_gro days_plant days_harv days_irr days_act
day_start irr_year slope soil_water tile_flo aqu_dep biom_lai
month jday year_seq year_rot year_cal year_dur prob prob_unif
prob_norm land_use weed_pres ...
```

(The full set is the case list in `conditions.f90`.)

For each variable the routine fetches the current value (from plant, soil, climate, time, or scenario state) and calls one of the comparison helpers:

- `cond_real.f90` for real-valued comparisons.
- `cond_real_c.f90` for real-valued comparisons against a constant in a paired condition.
- `cond_integer.f90`, `cond_integer_c.f90` for integer-valued comparisons.

`cond_real.f90` implements the alternative narrowing:

```
do ialt = 1, d_tbl%alts
  if (d_tbl%alt(ic,ialt) /= "-" .and. d_tbl%act_hit(ialt) == "y") then
    if (d_tbl%alt(ic,ialt) == "<"  .and. var_cur >= var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == ">"  .and. var_cur <= var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == "<=" .and. var_cur >  var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == ">=" .and. var_cur <  var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == "="  .and. var_cur /= var_tbl) d_tbl%act_hit(ialt) = "n"
    if (d_tbl%alt(ic,ialt) == "/=" .and. var_cur == var_tbl) d_tbl%act_hit(ialt) = "n"
  end if
end do
```

After every condition has run, an alternative still marked `y` is a matched alternative. Any number of alternatives can match.

Action firing

The driver `actions.f90` walks `d_tbl%act(1:acts)`. For each action it scans alternatives:

```
do iac = 1, d_tbl%acts
  action = "n"
```

```
  do ial = 1, d_tbl%alts
    if (d_tbl%act_hit(ial) == "y" .and. d_tbl%act_outcomes(iac,ial) == "y") then
      action = "y"
      exit
    end if
  end do
  if (action == "y") then
    select case (d_tbl%act(iac)%typ)
      case ("manure_demand") ...
      case ("irr_demand")    ...
      case ("irrigate")      ...
      case ("fertilize")     ...
      case ("manure")        ...
      case ("till")          ...
      case ("plant")         ...
      case ("harvest")       ...
      case ("harvest_kill")  ...
      case ("kill")          ...
      case ("burn")          ...
      case ("graze")         ...
      case ("pest_apply")    ...
      case ("lu_change")     ...
      case ("weir_release")  ...
      case ("res_release")   ...
      ! many more
    end select
  end do
end do
```

The fire decision is "any matched alternative wants this action": as soon as one surviving alternative has `y` in the action-outcome cell, the action fires. The action-outcome matrix lets you write rules where some alternatives share actions and others differ.

For management actions, the action implementation calls the relevant operation routine. Examples:

- `case ("plant")` resolves the plant from the community database, sets the start of the heat-unit accumulation, and calls the planting initialiser.
- `case ("fertilize")` looks up `d_tbl%act_typ(iac)` in `fertdb`, applies the fertiliser, and updates pools through `pl_fert`.
- `case ("till")` looks up the tillage implement, calls `mgt_newtillmix*` to redistribute residue and nutrients.
- `case ("harvest")` looks up the harvest operation type and dispatches to `mgt_harvbiomass`, `mgt_harvgrain`, `mgt_harvresidue`, `mgt_harvtuber`, or related routines.
- `case ("lu_change")` calls `hru_lum_init` and `chg_par` to switch the land use class on the HRU and re-initialise dependent state.

For reservoir release actions, the action implementation lives in `res_hydro.f90` rather than `actions.f90`. See Reservoirs and wetlands.

Probability and land use change

Two condition variables modify the alternative narrowing with random draws:

- `prob` and `prob_unif` compare a uniform random number to a constant probability. The probability passes through `d_tbl%frac_app`.
- `land_use` paired with a probabilistic action distributes applications across HRUs of a land-use class. The reader counts HRUs in the matching class (`hru_lu`, `ha_lu`) and the runtime tracks how many have been applied (`hru_lu_cur`, `hru_act_left`).

This is how stochastic management (apply fertiliser to 30 percent of corn HRUs in May, for instance) is implemented.

Switches and parameters

Decision tables are configured through the `*.dtl` input files, not through `codes.bsn` or `parameters.bsn` switches. The behaviour of each table is controlled by:

Parameter	File	Effect
`dtbl%name`	`*.dtl`	Table name referenced by HRUs, reservoirs, or scenarios
`dtbl%conds`	`*.dtl`	Number of conditions
`dtbl%alts`	`*.dtl`	Number of alternatives (rule columns)
`dtbl%acts`	`*.dtl`	Number of actions
`dtbl%cond%var`	`*.dtl`	Condition variable name; see case list in `conditions.f90`
`dtbl%cond%ob`	`*.dtl`	Object the condition is evaluated against
`dtbl%cond%lim_const`	`*.dtl`	Constant value the condition variable is compared to
`dtbl%cond%lim_var`	`*.dtl`	Limit-variable name (e.g. `pvol`, `evol`, `fc`, `ul`)
`dtbl%cond%lim_op`	`*.dtl`	Limit operator (`*`, `+`, `-`)
`dtbl%alt(ic, ialt)`	`*.dtl`	Comparison operator for each condition under each alternative
`dtbl%act%typ`	`*.dtl`	Action type; see case list in `actions.f90`
`dtbl%act%ob`	`*.dtl`	Object the action is applied to
`dtbl%act%name`	`*.dtl`	Action name (e.g. operation name in `harv.ops`, `fert.frt`, …)
`dtbl%act_outcomes(iac, ialt)`	`*.dtl`	`y` or `n` whether action `iac` fires under alternative `ialt`

Implementation

Source modules in `swatplus/src/`:

- `conditional_module.f90` defines `conditions_var`, `actions_var`, and `decision_table` derived types, plus the four runtime arrays `dtbl_lum`, `dtbl_res`, `dtbl_scen`, `dtbl_flo` and the working pointer `d_tbl`.
- `dtbl_lum_read.f90` reads `lum.dtl` into `dtbl_lum`. Allocates conditions, alternatives, actions, and outcome matrices, resolves named operation pointers (`act_typ`) to database indices.
- `dtbl_res_read.f90` reads `res_rel.dtl` into `dtbl_res`.
- `dtbl_scen_read.f90` reads `scen_lu.dtl` into `dtbl_scen`.
- `dtbl_flocon_read.f90` reads `flo_con.dtl` into `dtbl_flo`.
- `conditions.f90` evaluates conditions for the current `d_tbl`. Dispatches on `d_tbl%cond(ic)%var` and calls `cond_real` or `cond_integer` for narrowing.
- `cond_real.f90`, `cond_real_c.f90`, `cond_integer.f90`, `cond_integer_c.f90` implement the comparison operators and update `d_tbl%act_hit`.
- `actions.f90` fires actions whose `act_outcomes` entry is `y` for any surviving alternative. Houses the management action select-case (plant, fert, till, harvest, kill, burn, graze, lu_change, …).
- `hru_dtbl_actions_init.f90` initialises decision table linkages on HRUs at simulation start.
- `res_rel_conds.f90` is the reservoir-specific conditions evaluator used inside `res_hydro.f90`.

The call site sets `d_tbl => dtbl_lum(idtl)` (or `dtbl_res`, `dtbl_scen`, `dtbl_flo`) so that `conditions.f90` and `actions.f90` operate on the right table.

Related pages

- `*.dtl` input file for the file format used by all four classes.
- Reservoirs and wetlands for release rules driven by `res_rel.dtl`.
- Plant growth and land use management for the management actions fired by `lum.dtl`.

This section documents every SWAT+ input file. Each page lists:

- the file's purpose
- the Fortran reader and the derived type it populates
- the column layout, units, defaults
- a real example from the reference datasets
- known bugs or quirks in an `Important` section at the bottom

Every page is verified against the SWAT+ source at a specific commit, recorded in the page frontmatter as `verified_at_commit`. The source code is the only authoritative reference; example datasets are downstream and can drift.

Simulation control

- `file.cio`. Master index. Names every other input file.
- `time.sim`. Simulation period and time step.
- `print.prt`. Output selection and aggregation.
- `object.cnt`. Basin name, areas, spatial object counts.

Basin parameters

- `codes.bsn`. Method switches.
- `parameters.bsn`. Numeric coefficients.

Spatial objects and connectivity

- `*.con`. Connectivity files (hru.con, chandeg.con, aquifer.con, reservoir.con, …).
- `hru-data.hru`. Per-HRU pointer table.
- `*.hyd, field.fld, shade_factor.shf`. Per-HRU hydrology, topography, field geometry, and shade factor. Referenced by `hru-data.hru`.
- `recall`. Point sources and inlets. Master `recall.rec` plus salt and constituent side databases and the region/element files.

Structural BMPs

- `*.str, septic.sep, urban.urb`. Tile drains, septic systems and effluent, filter strips, grassed waterways, user-defined BMPs, saturated buffers, urban land cover.

Climate

- `*.cli`. Index files: weather-sta.cli, weather-wgn.cli, pcp.cli, tmp.cli, slr.cli, hmd.cli, wnd.cli, pet.cli, atmodep.cli.
- `*.pcp`. Precipitation data files.
- `*.tmp`. Temperature data files.
- `*.tem`. Legacy temperature file extension.
- `snow.sno`. Snow parameter database.

Soils and land use

- `soils.sol`. Soil profile and layer properties.
- `landuse.lum`. Land-use management.
- `*.lum` lookup tables. Curve number, conservation practice, and overland-flow Manning's n tables referenced by `landuse.lum`.

Initial conditions

- *.ini. General initial conditions: `plant.ini` (plant communities), `soil_plant.ini` (soil water, residue, plant biomass), `om_water.ini` (organic matter in water bodies). Solute-side init files are covered in their module pages under Solute transport and Pollutants.

Channels, aquifers, reservoirs, wetlands

- *.cha. Legacy channel files.
- sd_channel. SWAT+ routing channel inputs (`hydrology.cha`, `sediment.cha`, `nutrients.cha`).
- *.aqu. Aquifer files (`aquifer.aqu`, `initial.aqu`).
- *.res. Reservoir files.
- *.wet. Wetland files.

Groundwater

- gwflow.*. 2D cell-based groundwater module inputs (mesh, channel coupling, tiles, ponds, canals, solutes, heat).

Operation and management databases

- management.sch and minor inputs. Fixed-schedule management, puddle.ops, transplant.plt, co2.out, object.prt.
- *.ops. harv, graze, irr, chem_app, fire, sweep.
- *.dtl. Decision tables (lum.dtl, res_rel.dtl, scen_lu.dtl, flo_con.dtl).
- plants.plt. Plant parameter database.
- fertilizer.frt. Fertilizer composition and the manure type and organic-matter databases (`manure_db.frt`, `manure_om.frt`).
- manure_allo.mnu. Manure allocation. Sources, demands, and the rule that distributes manure between them.
- tillage.til. Tillage operations.
- carb_coefs.cbn. Century carbon coefficients.
- treatment.trt. Legacy treatment placeholder.

Water allocation

- water_allocation.wro and the *.wal transport files. Water-rights and inter-object water transfer module: master allocation file plus canal, pipe, tower, treatment, use, out-of-basin source and receiver inputs.

Solute transport

- salt module. rtb-salt inputs: per-object initial conditions and applied loads for the eight salt ions.
- cs module. rtb-cs inputs: master `constituents.cs` plus per-object initial conditions and applied loads for selenium, boron, and other constituents.

Pollutants

- pesticide module. `pesticide.pes` parameter database, per-HRU and per-channel initial conditions, and the parent-daughter metabolite chain.
- pathogen module. `pathogens.pth` parameter database and per-HRU and per-channel initial conditions.
- metals module. Per-HRU initial heavy metal concentrations.

Verification procedure

The procedure for verifying a new or updated page is in SOURCE-OF-TRUTH.md in the docs working folder.

Purpose

file.cio is the master index. SWAT+ opens this file first. Each line tells the model which file to read for one input category. Every other input file in the project is reached through file.cio.

Source

- Reader: src/readcio_read.f90
- File-name fields and defaults: src/input_file_module.f90

Format

- Line 1: title line. Skipped by the reader.
- Lines 2 to 31: one line per input category. Each line starts with a category label (a free-form token that the reader reads but discards), followed by a fixed number of file-name fields whose order matches the corresponding in_* type in input_file_module.f90.
- Line 32: a single line for the optional output path.
- A field set to the literal token null disables the corresponding file.

The category labels written by tools like QSWAT+ and SWAT+ Editor are conventional. The reader does not validate them; only field order and count matter.

Categories and fields

The table lists every category in the order the reader expects them, the number of file-name fields, and the default file names defined in input_file_module.f90.

#	Category label	Fields (defaults from input_file_module.f90)
1	simulation	time.sim, print.prt, object.prt, object.cnt, constituents.cs
2	basin	codes.bsn, parameters.bsn
3	climate	weather-sta.cli, weather-wgn.cli, pet.cli, pcp.cli, tmp.cli, slr.cli, hmd.cli, wnd.cli, atmodep.cli
4	connect	hru.con, hru-lte.con, rout_unit.con, gwflow.con, aquifer.con, aquifer2d.con, channel.con, reservoir.con, recall.con, exco.con, delratio.con, outlet.con, chandeg.con
5	channel	initial.cha, channel.cha, hydrology.cha, sediment.cha, nutrients.cha, channel-lte.cha, hyd-sed-lte.cha, temperature.cha
6	reservoir	initial.res, reservoir.res, hydrology.res, sediment.res, nutrients.res, weir.res, wetland.wet, hydrology.wet
7	routing_unit	rout_unit.def, rout_unit.ele, rout_unit.rtu, rout_unit.dr
8	hru	hru-data.hru, hru-lte.hru
9	exco	exco.exc, exco_om.exc, exco_pest.exc, exco_path.exc, exco_hmet.exc, exco_salt.exc
10	recall	recall.rec

#	Category label	Fields (defaults from input_file_module.f90)
11	dr	delratio.del, dr_om.del, dr_pest.del, dr_path.del, dr_hmet.del, dr_salt.del
12	aquifer	initial.aqu, aquifer.aqu
13	herd	animal.hrd, herd.hrd, ranch.hrd
14	water_rights	water_allocation.wro, element.wro, water_rights.wro
15	link	chan-surf.lin, aqu_cha.lin
16	hydrology	hydrology.hyd, topography.hyd, field.fld
17	structural	tiledrain.str, septic.str, filterstrip.str, grassedww.str, bmpuser.str
18	hru_parm_db	plants.plt, fertilizer.frt, tillage.til, pesticide.pes, pathogens.pth, metals.mtl, salt.slt, urban.urb, septic.sep, snow.sno
19	ops	harv.ops, graze.ops, irr.ops, chem_app.ops, fire.ops, sweep.ops
20	lum	landuse.lum, management.sch, cntable.lum, cons_practice.lum, ovn_table.lum
21	chg	cal_parms.cal, calibration.cal, codes.sft, wb_parms.sft, water_balance.sft, ch_sed_budget.sft, ch_sed_parms.sft, plant_parms.sft, plant_gro.sft
22	init	plant.ini, soil_plant.ini, om_water.ini, pest_hru.ini, pest_water.ini, path_hru.ini, path_water.ini, hmet_hru.ini, hmet_water.ini, salt_hru.ini, salt_water.ini
23	soils	soils.sol, nutrients.sol, soils_lte.sol
24	decision_table	lum.dtl, res_rel.dtl, scen_lu.dtl, flo_con.dtl
25	regions	ls_unit.ele, ls_unit.def, ls_reg.ele, ls_reg.def, ls_cal.reg, ch_catunit.ele, ch_catunit.def, ch_reg.def, aqu_catunit.ele, aqu_catunit.def, aqu_reg.def, res_catunit.ele, res_catunit.def, res_reg.def, rec_catunit.ele, rec_catunit.def, rec_reg.def
26	pcp_path	one path string
27	tmp_path	one path string
28	slr_path	one path string
29	hmd_path	one path string
30	wnd_path	one path string
31	output_path	one path string (read as the whole line after the label)

Categories 1 to 30 are read with list-directed I/O. Category 31 (output_path) is read with format (A) and the value is taken from the trailing portion of the line after the first space.

Example

Top of refdata/Ames_sub1/file.cio:

```
file.cio: AMES
simulation        time.sim          print.prt         null              object.cnt        null
basin             codes.bsn         parameters.bsn
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli           tmp.cli
```

```
connect           hru.con           null              null              null              null
channel           null       null       null       null       null       null       null       null
reservoir         null       null       null       null       null       null       null       null
routing_unit      null       null       null       null
hru               hru-data.hru      null
exco              null              null              null              null              null
recall            null
dr                null              null              null              null              null
aquifer           nbull             null
```

Notes on the example:

- Line 2 (`simulation`): only `time.sim`, `print.prt`, and `object.cnt` are in use. Slots 3 and 5 (`object.prt`, `constituents.cs`) are set to `null`.
- Line 4 (`climate`): only stations, weather generator, precipitation, and temperature are active. PET, solar, humidity, wind, and atmospheric deposition are disabled.
- Line 14 (`aquifer`): the dataset writes `nbull` for the initial-aquifer file. This is the file name in this project, not a default and not a typo of `null`. The model opens whatever file name appears here.

Related files

Every input file referenced in `file.cio`. Start from the Input reference index.

Purpose

`time.sim` sets the simulation start date, end date, and the sub-daily time step. All time-driven processes (climate reads, routing, output aggregation) use these values.

Source

- Reader: `src/time_read.f90`
- Type definition: `src/time_module.f90`, `type time_current`

Format

- Line 1: title line. Skipped.
- Line 2: header line. Skipped.
- Line 3: five integer values, list-directed.

#	Name	Type	Description
1	day_start	int	Julian day of year on which the simulation starts (1 to 366)
2	yrc_start	int	Calendar year on which the simulation starts
3	day_end	int	Julian day of year on which the simulation ends
4	yrc_end	int	Calendar year on which the simulation ends
5	step	int	Sub-daily time step code (see below)

Sub-daily time step values

The type definition in `time_module.f90` documents these step codes:

Value	Meaning
0	daily
1	12-hour increment
24	hourly

Value	Meaning
96	15 minutes
1440	per minute

Defaults and derived values

The reader applies these adjustments after parsing:

- `step <= 0` is forced to `1`.
- `day_start <= 0` is forced to `1`.
- `time%nbyr` is set to `yrc_end - yrc_start + 1`.
- `time%mo`, `time%day_mo`, and `time%mo_start` are computed from `day_start` via the `xmon` helper.
- `time%yrc` is initialised to `yrc_start`.

Example

`refdata/Ames_sub1/time.sim`:

```
time.sim:
day_start  yrc_start   day_end   yrc_end      step
       0       1975         0      2020         0
```

This runs from 1 Jan 1975 through 31 Dec 2020 (the zero values for `day_start` and `day_end` are interpreted as the first and last day of the year). The `step` value of `0` is forced to `1` by the reader.

Related files

- `print.prt`. Controls which outputs are written within this simulation window.

Purpose

`print.prt` controls every output the model writes. It sets the print window in calendar time, the daily print interval, the average-annual print intervals, three database-output toggles, four "other output" toggles, and one row per output object with four print-frequency flags (daily, monthly, yearly, average annual).

If `print.prt` is missing or set to `null` in `file.cio`, the model still runs but writes no time-stepped output.

Source

- Reader: `src/basin_print_codes_read.f90`
- Type definitions: `src/basin_module.f90`. See `type basin_print_codes` (`pco`) and `type print_interval`.

Format

The file has fixed top-of-file blocks followed by one row per output object. There are two object-row modes, selected by the `use_obj_lbls` flag in the third block. The reader skips a header line before each block.

Block 1: window

- Line 1: title (skipped).
- Line 2: header (skipped).
- Line 3: six integers.

#	Name	Description	Default if unset
1	nyskip	number of warm-up years to skip in average-annual summaries	0
2	day_start	julian day to start daily printing	forced to 1 if 0
3	yrc_start	calendar year to start daily printing	`time%yrc` from `time.sim`
4	day_end	julian day to end daily printing	forced to 366 if 0
5	yrc_end	calendar year to end daily printing	`time%yrc + time%nbyr`
6	interval	days between daily prints	forced to 1 if `<= 0`

Block 2: average-annual print intervals

- Line 4: header (skipped).
- Line 5: `aa_numint` followed by `aa_numint` year values. If `aa_numint = 0`, only the value is read.

`aa_numint` is the number of end-years to use for average-annual outputs. When it is greater than zero, average-annual statistics are produced once for each interval defined by the listed end years.

Block 3: database output toggles

- Line 6: header (skipped).
- Line 7: three single-character flags.

Name	Values	Meaning
csvout	`n`, `y`	also write `.csv` versions of outputs
use_obj_lbls	`n`, `y`	object-row mode (see Object rows)
cdfout	`n`, `y`	also write netCDF (`.nc`) versions of outputs

Block 4: other output toggles

- Line 8: header (skipped).
- Line 9: four single-character flags.

Name	Values	Meaning
crop_yld	`a`, `y`, `b`, `n`	crop yields: `a` average annual, `y` yearly, `b` both, `n` no print
mgtout	`n`, `y`	write `mgt_out.txt` (management operations log)
hydcon	`n`, `y`	write `hydcon.out` (hydrograph connections)
fdcout	`n`, `y`	flow duration curve. Read but the type comment notes "NOT ACTIVE"

Object rows

- Line 10: header (`objects daily monthly yearly avann`), skipped.
- Lines 11 and following: one row per output object. Each row has five fields: a name and four flags (`d`, `m`, `y`, `a`) with values `n` or `y`.

The reader interprets the rows in one of two modes:

- `use_obj_lbls = n` (positional): the reader expects rows in a fixed order. The name column is read but discarded. Missing or extra rows cause silent skips of later objects when the loop hits end-of-file.
- `use_obj_lbls = y` (labelled): rows can be in any order. The reader matches the first column against a list of recognised names. Repeated rows for the same object are flagged via `print_prt_error`. Unknown names cause the model to print an error and `error stop`.

Recognised object names (labelled mode)

Group	Names
Basin	basin_wb, basin_nb, basin_ls, basin_pw, basin_aqu, basin_res, basin_cha, basin_sd_cha, basin_psc
Region	region_wb, region_nb, region_ls, region_pw, region_aqu, region_res, region_sd_cha, region_psc, water_allo
LSU	lsunit_wb, lsunit_nb, lsunit_ls, lsunit_pw
HRU	hru_wb, hru_nb, hru_ls, hru_pw, hru_cb, hru_cb_vars
HRU-lte	hru-lte_wb, hru-lte_nb, hru-lte_ls, hru-lte_pw
Other	channel, channel_sd, aquifer, reservoir, recall, hyd, ru, pest
Salt	basin_salt, hru_salt, ru_salt, aqu_salt, channel_salt, res_salt, wetland_salt
Constituents	basin_cs, hru_cs, ru_cs, aqu_cs, channel_cs, res_cs, wetland_cs
gwflow	gwflow_wb, gwflow_flux, gwflow_heat, gwflow_solute, gwflow_obs, gwflow_pump

Each row's four flags map to the `print_interval` fields `d`, `m`, `y`, `a` (daily, monthly, yearly, average annual). Value `y` enables that aggregation. Value `n` disables it.

The HRU carbon rows `hru_cb` and `hru_cb_vars` drive the legacy CSU carbon output files (alongside the standardised per-family carbon outputs). SWAT+ Editor does not write these rows; the CSU team adds them by hand. The `hru_cb` flag letter selects detail (`l` = full per-layer set with diagnostics, `y` = light profile set). See the carbon outputs reference.

Example

`refdata/Ames_sub1/print.prt`:

```
print.prt:
nyskip      day_start  yrc_start  day_end   yrc_end   interval
0           0               1975         0      2020          1
aa_int_cnt
0
csvout        use_obj_lbls  cdfout
n             n             n
crop_yld      mgtout        hydcon        fdcout
b             y             n             n
objects                  daily       monthly        yearly         avann
basin_wb                     n             n             n             y
basin_nb                     n             n             n             y
basin_ls                     n             n             n             y
basin_pw                     n             n             n             y
basin_aqu                    n             n             n             y
...
```

Window: print over 1975 to 2020, daily interval 1, no warm-up. `aa_int_cnt = 0` means a single average-annual interval over the whole period. CSV and netCDF off. Crop yields in both average-annual and yearly form. Management log on. `use_obj_lbls = n` so the rows are in fixed order.

Related files

- `time.sim`. Defines the simulation period that bounds these print windows.
- Output reference. Documents each output file produced when the corresponding row is enabled.

Purpose

`object.cnt` declares the basin name, the landscape and total areas, and how many of each spatial object type (HRU, channel, aquifer, reservoir, and so on) the project contains. The model uses these counts to allocate the `ob`,

obcs, obcs_alloc, and obom arrays before any object data is read.

If object.cnt is missing or set to null in file.cio, the model prints Cannot find object.cnt input file and stops.

Source

- Reader: src/basin_read_objs.f90
- Type definitions:
  - src/basin_module.f90, type basin_inputs (bsn)
  - src/hydrograph_module.f90, type spatial_objects (sp_ob)

Format

- Line 1: title line. Skipped.
- Line 2: header line. Skipped.
- Line 3: 21 values read into bsn and sp_ob, list-directed.

#	Field	Maps to	Type	Description
1	name	bsn%name	char(25)	basin name (free-form)
2	ls_area	bsn%area_ls_ha	real	landscape area, hectares
3	tot_area	bsn%area_tot_ha	real	total area, hectares
4	objs	sp_ob%objs	int	total object count (sum of all object types)
5	hru	sp_ob%hru	int	number of HRUs
6	hru_lte	sp_ob%hru_lte	int	number of HRU-lte objects
7	ru	sp_ob%ru	int	number of routing units
8	gwflow	sp_ob%gwflow	int	number of gwflow cells
9	aqu	sp_ob%aqu	int	number of aquifer objects
10	chan	sp_ob%chan	int	number of legacy channels
11	res	sp_ob%res	int	number of reservoirs
12	recall	sp_ob%recall	int	number of recall objects
13	exco	sp_ob%exco	int	number of export-coefficient objects
14	dr	sp_ob%dr	int	number of delivery-ratio objects
15	canal	sp_ob%canal	int	number of canals
16	pump	sp_ob%pump	int	number of pumps
17	outlet	sp_ob%outlet	int	number of outlets
18	chandeg	sp_ob%chandeg	int	number of SWAT-deg channels (SWAT+ routing)
19	aqu2d	sp_ob%aqu2d	int	2D aquifer count (currently not used per the type comment)
20	herd	sp_ob%herd	int	herd count (currently not used per the type comment)
21	wro	sp_ob%wro	int	water-rights object count (currently not used per the type comment)

gwflow side effect

After reading, if bsn_cc%gwflow == 1 (set via codes.bsn), the reader looks for gwflow.chancells. If that file exists and contains river cells, the reader:

- sets sp_ob%gwflow to the number of river cells,
- recomputes sp_ob%objs to sp_ob%objs + nriv - sp_ob%aqu,
- sets in_con%gwflow_con to gwflow.con,
- sets sp_ob%aqu to 0 and in_con%aqu_con to null.

If bsn_cc%gwflow == 1 but gwflow.chancells is missing or empty, gwflow is silently turned off.

Example

`refdata/Ames_sub1/object.cnt`:

```
object.cnt:
name                    ls_area      tot_area       obj       hru      lhru       rtu       mfl       aqu
demo                       1.           1            12         12         0         0         0         0
```

Notes on the example:

- `name = demo`, `ls_area = 1` ha, `tot_area = 1` ha. The fields are floats; the trailing `.` makes that explicit in this dataset.
- `objs = 12` and `hru = 12`. Every spatial object in this project is an HRU.
- The header uses shortened labels (`obj`, `lhru`, `rtu`, `mfl`, `dlr`, `can`, `pmp`, `out`, `lcha`, `hrd`). These are tool conventions; the reader does not parse the header.

Related files

- `codes.bsn`. Sets `bsn_cc%gwflow`, which triggers the gwflow side effect above.
- `file.cio`. The `simulation` category points to `object.cnt`.
- Object-specific connectivity files (`hru.con`, `chandeg.con`, `aquifer.con`, `reservoir.con`, …). These must agree with the counts in `object.cnt`.

Purpose

`codes.bsn` holds the method-selection switches that apply to the whole basin: the PET method, the routing method, the carbon model, the soil P model, and so on. It is read into `bsn_cc` (`type basin_control_codes`) early in the run and then consulted by every process module.

Source

- Reader: `src/basin_read_cc.f90`
- Type definition: `src/basin_module.f90`, `type basin_control_codes`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Line 3: 26 fields, list-directed, read into `bsn_cc` in declaration order.

#	Field	Type	Default	Description
1	petfile	char(16)	`pet.cli`	Potential ET filename. Used when `pet = 3` (read PET from file).
2	wwqfile	char(16)	`""`	Watershed stream water-quality filename.
3	pet	int	0	PET method. `0` Priestley-Taylor, `1` Penman-Monteith, `2` Hargreaves. (Method `3` triggers reading `pet.cli`.)
4	nam1	int	0	Not used.
5	crk	int	0	Crack flow. `1` to compute flow in cracks.
6	swift_out	int	0	SWIFT output. `0` do not write, `1` write `swift_hru.inp`.
7	sed_det	int	0	Peak rate method. `0` NRCS dimensionless hydrograph with PRF, `1` half-hour rainfall intensity method.
8	rte	int	0	Water routing method. `0` variable storage, `1` Muskingum.
9	deg	int	0	Not used.
10	wq	int	0	Not used.

#	Field	Type	Default	Description
11	nostress	int	0	Plant stress. `0` all stresses applied, `1` turn off all plant stress, `2` turn off nutrient stress only.
12	cn	int	0	Not used.
13	cfac	int	0	Not used.
14	cswat	int	0	Carbon model. `0` static (legacy mineralization), `1` C-FARM (reserved, not implemented), `2` dynamic CENTURY/SWAT-C model.
15	lapse	int	0	Lapse-rate adjustment for precip and temperature. `0` no, `1` yes.
16	uhyd	int	1	Unit hydrograph. `0` triangular, `1` gamma function.
17	sed_ch	int	0	Not used.
18	tdrn	int	0	Tile drainage equation. `0` drawdown-days, `1` DRAINMOD.
19	wtdn	int	0	Shallow water-table depth. `0` original routine (fill to upper limit), `1` DRAINMOD water-table depth.
20	sol_p_model	int	0	Soil P model. `0` original SWAT P model, `1` Vadas and White (2010).
21	gampt	int	0	Infiltration method. `0` curve number, `1` Green-Ampt.
22	atmo	char(1)	`a`	Not used.
23	smax	int	0	Not used.
24	qual2e	int	0	Instream nutrient routing. `0` QUAL2E, `1` QUAL2E with simplified transformations.
25	gwflow	int	0	gwflow module. `0` inactive, `1` active. Triggers extra logic in `object.cnt`.
26	idc_till	int	3	Tillage method when `cswat = 2`. `1` DSSAT, `2` EPIC, `3` Kemanian, `4` DNDC.

The "Not used" fields are read from the file but the model does not consult them. Tools commonly write `0`.

Side effects after read

If `pet = 3`, the reader opens `pet.cli` and reads a title and header line to position the file pointer. The actual PET values are read later by the climate routines.

Example

`refdata/Ames_sub1/codes.bsn`:

`codes.bsn`:

pet_file	wq_file	pet	event	crack	swift_out	sed_det	rte_cha	deg_cha
null	null	2	0	0	0	0	0	0

The header column names written by SWAT+ Editor (`event`, `rte_cha`, `i_fpwet`, …) differ from the field names in the source (`crk`, `rte`, `qual2e`, …). Order is what the reader cares about. Mapping the example to the table:

- `pet = 2`: Hargreaves.
- `cswat = 2`: dynamic CENTURY/SWAT-C carbon model. (Header label `carbon`.) The example file above still carries `carbon = 1`, which is now the reserved C-FARM slot and runs no carbon model; datasets that want the dynamic model must set `carbon = 2`.
- `uhyd = 1`: gamma-function unit hydrograph.
- `idc_till = 3`: Kemanian tillage method.

The `null` strings in columns 1 and 2 disable the PET and water-quality file inputs.

Related files

- parameters.bsn. Numeric basin parameters consumed by the methods selected here.
- object.cnt. Reads gwflow state when `gwflow = 1`.
- pet.cli. Used when `pet = 3`.

Purpose

`parameters.bsn` holds the basin-wide numeric coefficients (initial soil water, surface runoff lag, nutrient cycling rates, channel routing factors, lapse rates, CO2, and so on). Values are read into `bsn_prm` (`type basin_parms`) and consulted by hydrology, nutrient, erosion, and routing routines.

Source

- Reader: src/basin_read_prm.f90
- Type definition: src/basin_module.f90, `type basin_parms`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Line 3: 44 values, list-directed, read into `bsn_prm` in declaration order. The last two values are integers; the rest are reals.

#	Field	Type	Default	Units	Description
1	evlai	real	3.0	none	leaf area index above which no soil evaporation occurs
2	ffcb	real	0.0	fraction	initial soil water content as a fraction of field capacity
3	surlag	real	4.0	days	surface runoff lag time
4	adj_pkr	real	1.0	none	peak rate adjustment factor in the subbasin
5	prf	real	484.0	none	peak rate factor for the peak rate equation
6	spcon	real	0.0	none	not used
7	spexp	real	0.0	none	not used
8	cmn	real	0.003	1/day	rate factor for mineralization of active organic N
9	n_updis	real	20.0	none	nitrogen uptake distribution parameter
10	p_updis	real	20.0	none	phosphorus uptake distribution parameter
11	nperco	real	0.10	0..1	nitrate percolation coefficient
12	pperco	real	10.0	0..1	phosphorus percolation coefficient
13	phoskd	real	175.0	none	phosphorus soil partitioning coefficient
14	psp	real	0.40	none	phosphorus availability index
15	rsdco	real	0.05	none	residue decomposition coefficient
16	percop	real	0.5	0..1	pesticide percolation coefficient
17	msk_co1	real	0.75	none	Muskingum storage time constant for bankfull depth
18	msk_co2	real	0.25	none	Muskingum storage time constant for low flow
19	msk_x	real	0.20	none	Muskingum weighting factor for inflow vs outflow
20	nperco_lchtile	real	0.5	none	N concentration coefficient for tile flow and bottom-layer leaching
21	evrch	real	0.60	none	reach evaporation adjustment factor
22	scoef	real	1.0	0..1	channel storage coefficient
23	cdn	real	1.40	none	denitrification exponential rate coefficient
24	sdnco	real	1.30	none	denitrification threshold (fraction of field capacity)
25	bact_swf	real	0.15	fraction	fraction of manure containing active colony-forming units
26	tb_adj	real	0.0	none	adjustment factor for sub-daily unit hydrograph base time
27	cn_froz	real	0.000862	none	parameter for frozen-soil adjustment on infiltration/runoff
28	dorm_hr	real	-1.0	hours	dormancy time threshold
29	plaps	real	0.0	mm/km	precipitation lapse rate per km of elevation difference
30	tlaps	real	6.5	deg C/km	temperature lapse rate per km of elevation difference
31	nfixmx	real	20.0	kg/ha/day	maximum daily nitrogen fixation
32	decr_min	real	0.01	1/day	minimum daily residue decay
33	rsd_covco	real	0.75	none	residue cover factor for computing cover fraction
34	urb_init_abst	real	1.0	none	maximum initial abstraction for urban areas (Green-Ampt)

#	Field	Type	Default	Units	Description
35	petco_pmpt	real	100.0	percent	PET adjustment for Penman-Monteith and Priestley-Taylor
36	uhalpha	real	1.0	none	alpha coefficient for the gamma-function unit hydrograph
37	eros_spl	real	0.0	none	coefficient of splash erosion (typical range 0.9 to 3.1)
38	rill_mult	real	0.0	none	rill erosion coefficient
39	eros_expo	real	0.0	none	exponential coefficient for overland flow
40	c_factor	real	0.0	none	cover and management factor scaling for overland-flow erosion
41	ch_d50	real	0.0	mm	median particle diameter of the main channel
42	co2	real	400.0	ppm	CO2 concentration at start of simulation
43	day_lag_mx	int	0	days	maximum days to lag hydrographs for HRU, RU, and channels on non-draining soils
44	igen	int	5	none	random generator code. `0` use defaults, `1` generate new numbers every run

Example

refdata/Ames_sub1/parameters.bsn:

```
parameters.bsn: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
  lai_noevap       sw_init      surq_lag      adj_pkrt  adj_pkrt_sed       lin_sed       exp_sed      orgn
     3.00000       0.50000       4.00000       1.00000       1.00000       0.00000       1.00000       0.0
```

Notes:

- Tool header names (`lai_noevap`, `sw_init`, `surq_lag`, `adj_pkrt`, `orgn_min`, `n_uptake`, `evap_adj`, `denit_exp`, `denit_frac`, `man_bact`, `adj_uhyd`, `n_fix_max`, `rsd_decay`, `rsd_cover`, `uhyd_alpha`, `splash`, `rill`, `surq_exp`, `cov_mgt`, `cha_d50`, `day_lag_max`) do not match the field names in the source (`evlai`, `ffcb`, `surlag`, `adj_pkr`, `cmn`, `n_updis`, `evrch`, `cdn`, `sdnco`, `bact_swf`, `tb_adj`, `nfixmx`, `decr_min`, `rsd_covco`, `uhalpha`, `eros_spl`, `rill_mult`, `eros_expo`, `c_factor`, `ch_d50`, `day_lag_mx`). The reader uses position only.
- Position 8 in this dataset is `orgn_min = 0.0`. In the source this is `cmn`, with default `0.003`. The example overrides the default to zero, which disables active organic-N mineralization.
- Position 7 is `exp_sed = 1.0`. This is `spexp` in the source, marked "not used".
- Positions 28 to 30 (`dorm_hr`, `plaps`, `tlaps`) are all zero in this dataset. The default for `tlaps` is `6.5` deg C/km; this dataset disables temperature lapse.

Related files

- codes.bsn. Method switches that select which of these parameters are active.

Purpose

`carbon.bsn` holds the basin-wide scalar coefficients for the SWAT+ carbon module. The reader fills `org_frac`, `cb_wtr_coef`, `man_coef`, and `org_con` (all defined in carbon_module.f90), the basin-scope tillage variables in tillage_data_module.f90 (`bio_consf`, `till_consf`, `bmix_a/b/c`, `tillmix_a/b/c`, `till_eff_days`), the `photo_degrade_factor` in plant_data_module.f90, and the five residue-decomposition tunables (`n_act_frac`, `cnr_cap`, `cnr_ref`, `cpr_cap`, `cpr_ref`) promoted from hard-coded constants into module-scope reals in `carbon_module.f90`.

`carbon.bsn` replaces the legacy `carb_coefs.cbn` (keyword-style) and works alongside the per-layer file carbon_lyr.bsn, which holds the rate constants and CO2 fractions that differ between the surface and subsurface layer groups.

Source

- Reader: src/carbon_bsn_read.f90
- Filename slot: src/input_file_module.f90, `type input_basin`, field `carbon_bsn = "carbon.bsn"`.
- Target types: `organic_fractions`, `carbon_water_coef`, `manure_coef`, `organic_controls` in src/carbon_module.f90; module-scope reals `n_act_frac`, `cnr_cap`, `cnr_ref`, `cpr_cap`, `cpr_ref` in the same module; `bio_consf`, `till_consf`, `till_eff_days`, `bmix_a/b/c`, `tillmix_a/b/c` in src/tillage_data_module.f90; `photo_degrade_factor` in src/plant_data_module.f90.

Activation

The reader runs only when `bsn_cc%cswat == 2` (set in codes.bsn). When `cswat == 2` the file is **required**: if `carbon.bsn` (or its companion `carbon_lyr.bsn`) is missing or its data row cannot be parsed, the reader writes an error message to stdout and unit 9001 and calls `error stop`. There is no silent fallback to compiled defaults. `carbon_bsn_read.f90` returns early for any other `cswat` value, so `cswat = 1` (the reserved C-FARM slot) does not read this file.

The dynamic carbon model activates on `cswat = 2`, matching legacy SWAT numbering (1 = C-FARM, 2 = CENTURY). It was previously `cswat = 1`; existing `codes.bsn` files left at `carbon = 1` now run with the carbon model off and must be changed to `carbon = 2`.

`carbon.bsn` is listed in the BASIN row of `file.cio` alongside `codes.bsn` and `parameters.bsn` (three columns). `carbon_lyr.bsn` rides at a fixed name beside it and does not appear in `file.cio`.

> **Note**
>
> The legacy `cbn_diag` first column is retired. Output gating now lives entirely in `print.prt` through the `hru_cb_*` and `lsu_cb_*` rows (see the carbon outputs reference).

Format

- Line 1: title (skipped).
- Line 2: column header (skipped).
- Line 3: 28 values, list-directed, read into the target fields in the order below. (The 28th value, `mathers_method`, was added when carbonDev merged to main; 27-value files from before the merge fail to parse and must add the trailing column.)

Values are space-delimited. Position is significant; the column header is decorative.

#	Field	Type	Target	Units	Description
1	init_seq	real	org_frac%frac_seq	fraction	initial fraction of total soil carbon held in the sequestered pool
2	init_microb	real	org_frac%frac_hum_microb	fraction	initial microbial-humus fraction of non-sequestered carbon
3	init_slow	real	org_frac%frac_hum_slow	fraction	initial slow-humus fraction of non-sequestered carbon
4	init_passive	real	org_frac%frac_hum_passive	fraction	initial passive-humus fraction of non-sequestered carbon
5	koc_c	real	cb_wtr_coef%prmt_21	none	partition coefficient for carbon loss in water and sediment (formerly prmt_21)
6	solc_ratio	real	cb_wtr_coef%prmt_44	0..1	ratio of soluble C in surface runoff to soluble C in percolate (formerly prmt_44)
7	till_eff_days	real	till_eff_days	days	number of days a tillage event remains effective
8	manure_c_frac	real	man_coef%rtof	none	manure organic N and P partition between fresh and stable pools (formerly rtof)
9	bio_consol	real	bio_consf	none	biological consolidation factor for soil moisture mixing
10	till_consol	real	till_consf	none	tillage consolidation factor for soil moisture mixing
11	tmpf_eqn	int	org_con%tmpf	none	temperature factor selector (1, 2, or 3) used in cbn_zhang2.f90
12	watf_eqn	int	org_con%watf	none	water factor selector (1 or 2) used in cbn_zhang2.f90
13	t_cbn_min	real	org_con%tn	deg C	minimum temperature bound for tmpf2 (lower limit of biological activity)
14	t_cbn_opt	real	org_con%top	deg C	optimum temperature for tmpf2
15	t_cbn_max	real	org_con%tx	deg C	maximum temperature bound for tmpf2 (upper limit of biological activity)
16	bmix_a	real	bmix_a	none	biological mixing curve coefficient A (mgt_tillfactor.f90 tillf option 4)
17	bmix_b	real	bmix_b	none	biological mixing curve coefficient B
18	bmix_c	real	bmix_c	none	biological mixing curve coefficient C
19	tillmix_a	real	tillmix_a	none	tillage mixing curve coefficient A
20	tillmix_b	real	tillmix_b	none	tillage mixing curve coefficient B
21	tillmix_c	real	tillmix_c	none	tillage mixing curve coefficient C
22	sfc_rsd_photodeg	real	photo_degrade_factor	1/day	surface residue photodegradation rate factor
23	n_act_frac	real	n_act_frac	fraction	fraction of organic N held in the active humus pool (drives active-to-stable flow in nut_nminrl.f90)
24	cnr_cap	real	cnr_cap	none	upper cap on residue C:N ratio before computing the decomposition factor
25	cnr_ref	real	cnr_ref	none	reference C:N ratio where the C:N decomposition factor equals 1
26	cpr_cap	real	cpr_cap	none	upper cap on residue C:P ratio before computing the decomposition factor
27	cpr_ref	real	cpr_ref	none	reference C:P ratio where the C:P decomposition factor equals 1

#	Field	Type	Target	Units	Description
28	`mathers_method`	int	`org_frac%mathers_method`	flag	slow-humus pool initialization: `0` original humus-slow init (default), `1` Mathers method (clay+silt based, from “Updating carbon pool initialization with DSSAT-CENTURY”). Used in `soil_nutcarb_init`.

The reader does not interpret column 1-4 as a probability simplex; the four `init_*` values are used as supplied. The four target fields hold the fractional partitioning of soil carbon into sequestered, microbial-humus, slow-humus, and passive-humus pools at simulation start.

The `tmpf_eqn` (col 11) and `watf_eqn` (col 12) integers select between the temperature and water response equations encoded in `cbn_zhang2.f90`. They are integers in the file but the reader does not enforce that; the consuming code treats them as `int(real)`.

Example

`refdata/Lafayette/carbon.bsn`:

```
carbon.bsn: basin-wide scalar carbon parameters (one record)
   init_seq   init_microb   init_slow   init_passive   koc_c   solc_ratio   till_eff_days   manure_c_frac
       0.95         0.02         0.44           0.54    1000          0.5             100            0.50          0.1
```

Calibration

Eighteen of the 28 fields are wired in `cal_parm_select.f90` and may be touched through `calibration.cal`: `init_seq`, `init_microb`, `init_slow`, `init_passive`, `koc_c`, `solc_ratio`, `manure_c_frac`, `bio_consol`, `till_consol`, `sfc_rsd_photodeg`, `n_act_frac`, `cnr_cap`, `cnr_ref`, `cpr_cap`, `cpr_ref`, `t_cbn_min`, `t_cbn_opt`, `t_cbn_max`. The six bmix/tillmix curve coefficients are also wired. The selector integers (`tmpf_eqn`, `watf_eqn`) and the integer-valued `till_eff_days` are not calibrated.

> **Note**
>
> `proc_cal` must run **before** `soil_nutcarb_init` for the `init_*` calibration values to take effect. The current `main.f90` already orders these correctly; do not reorder.

Related files

- `carbon_lyr.bsn`. Per-layer rate constants and CO2 allocation fractions.
- `codes.bsn`. The `cswat` switch that gates this file.
- `print.prt`. Output gating for the carbon families.
- Soil carbon (model theory). Equations and pool structure.
- Parameter change files. Calibration entries for the wired carbon parameters.

Purpose

`carbon_lyr.bsn` holds the carbon coefficients that differ between soil layer groups. It populates `carbdb(layer_id)` (rate constants for the litter, microbial, slow, and passive pools) and `org_allo(layer_id)` (the CO2 fractions produced by each pool transformation). The two layer groups currently in use are:

- `layer 1` (surface litter and topsoil)
- `layer 2` (all subsurface layers)

The reader indexes directly into the `carbdb(:)` and `org_allo(:)` arrays. Both are sized `dimension(2)` in `carbon_module.f90` today; a future expansion to additional layer groups requires only a larger array allocation, not a reader change.

Source

- Reader: same routine as `carbon.bsn`, `src/carbon_bsn_read.f90`. The per-layer block is parsed in the second half of the subroutine.
- Target types: `carbon_inputs` (`carbdb`) and `organic_allocations` (`org_allo`) in `src/carbon_module.f90`.
- Filename: hardcoded in the reader as `carbon_lyr.bsn`. **Not listed in `file.cio`** — it rides at a fixed name beside `carbon.bsn`.

Activation

Read only when `bsn_cc%cswat == 2` (the dynamic CENTURY/SWAT-C model; the activation value moved from 1 to 2). Like `carbon.bsn`, the file is **required**: an absent file, an unparseable row, or fewer data rows than the size of `carbdb` (currently 2) triggers `error stop` with a clear message.

Format

- Line 1: title (skipped).
- Line 2: column header (skipped).
- Lines 3..: one row per layer group. Each row begins with an integer `layer` id (1 = top, 2 = subsurface) followed by 11 real coefficients. The reader loops until end-of-file. Rows with an out-of-range `layer` id (less than 1 or greater than `size(carbdb)`) are skipped with a warning to unit 9001 and do not count toward the required row count.

#	Field	Type	Target	Units	Description
1	`layer`	int	(array index)	none	layer group id; 1 = top (surface litter + topsoil), 2 = all subsurface layers
2	`hp_rate`	real	`carbdb(layer)%hp_rate`	1/day	passive humus transformation rate under optimal conditions
3	`hs_rate`	real	`carbdb(layer)%hs_rate`	1/day	slow humus transformation rate under optimal conditions
4	`microb_rate`	real	`carbdb(layer)%microb_rate`	1/day	microbial biomass transformation rate under optimal conditions
5	`meta_rate`	real	`carbdb(layer)%meta_rate`	1/day	metabolic litter transformation rate under optimal conditions
6	`str_rate`	real	`carbdb(layer)%str_rate`	1/day	structural litter transformation rate under optimal conditions
7	`microb_top_rate`	real	`carbdb(layer)%microb_top_rate`	1/day	top-layer microbial activity adjustment
8	`hs_hp`	real	`carbdb(layer)%hs_hp`	none	slow-to-passive humus allocation coefficient
9	`a1co2`	real	`org_allo(layer)%a1co2`	fraction	decomposed metabolic litter routed to CO2
10	`asco2`	real	`org_allo(layer)%asco2`	fraction	decomposed slow humus routed to CO2
11	`apco2`	real	`org_allo(layer)%apco2`	fraction	decomposed passive humus routed to CO2
12	`abco2`	real	`org_allo(layer)%abco2`	fraction	decomposed microbial biomass routed to CO2

Example

refdata/Lafayette/carbon_lyr.bsn:

```
carbon_lyr.bsn: layer-keyed carbon coefficients (one row per layer group; 1 = top, 2 = subsurface; extend
   layer       hp_rate       hs_rate   microb_rate   meta_rate   str_rate   microb_top_rate   hs_hp   a1co
       1      1.2e-05      2.92e-04        0.0164      0.0405     0.0107            0.0164    0.05   0.60
       2      1.2e-05      1.81e-04          0.02      0.0507     0.0134              0.02    0.05   0.55
```

Calibration

All 11 per-layer fields are wired in `cal_parm_select.f90`. The calibration target is the `carbdb(ly)` or `org_allo(ly)` element selected by the `ly` index in the calibration record (see Parameter change files):

- `hp_rate`, `hs_rate`, `microb_rate`, `meta_rate`, `str_rate`, `microb_top_rate`, `hs_hp` target `carbdb(ly)%*`.
- `a1co2`, `asco2`, `apco2`, `abco2` target `org_allo(ly)%*`.

Each `case` branch in `cal_parm_select.f90` guards `ly` to the range `[1, size(carbdb)]` (or `[1, size(org_allo)]`), so out-of-range layer indices in a calibration row are silently no-op rather than crashing.

Related files

- `carbon.bsn`. Basin-wide scalar carbon parameters.
- `codes.bsn`. The `cswat` switch.
- Soil carbon (model theory). Pool definitions and Century-style transformation equations.

Purpose

The `.con` files describe the spatial-object topology of the project. Every spatial object (HRU, channel, aquifer, reservoir, routing unit, recall, export coefficient, delivery ratio, outlet, gwflow cell) gets one row in its corresponding `.con` file. The row gives the object its sequential number, name, GIS id, drainage area, location, elevation, parameter pointers, weather station, optional constituent and overbank pointers, and the list of downstream objects that receive its outflow. The model uses these rows to build the routing graph in `hyd_connect`.

Each object type has its own `.con` file but every `.con` file is read by the same routine, `hyd_read_connect`, and is stored in the same array `ob`. The names of the files are listed in `type input_con` in `input_file_module.f90`:

File	Object type passed to `hyd_read_connect`	Default name
`hru.con`	`hru`	`in_con%hru_con`
`hru-lte.con`	`hru_lte`	`in_con%hruez_con`
`rout_unit.con`	`ru`	`in_con%ru_con`
`gwflow.con`	`gwflow`	`in_con%gwflow_con`
`aquifer.con`	`aqu`	`in_con%aqu_con`
`channel.con`	`chan`	`in_con%chan_con`
`chandeg.con`	`chandeg`	`in_con%chandeg_con`
`reservoir.con`	`res`	`in_con%res_con`
`recall.con`	`recall`	`in_con%rec_con`
`exco.con`	`exco`	`in_con%exco_con`
`delratio.con`	`dr`	`in_con%delr_con`
`outlet.con`	`outlet`	`in_con%out_con`

A 13th name, `aquifer2d.con` (`in_con%aqu2d_con`), is declared in `type input_con` but is not currently passed to `hyd_read_connect` from `hyd_connect.f90`.

Source

- Reader: src/hyd_read_connect.f90
- Dispatcher: src/hyd_connect.f90 (calls `hyd_read_connect` once per object type)
- Type definition: src/hydrograph_module.f90 (`type object_connectivity`, array `ob`)
- File name pointers: src/input_file_module.f90 (`type input_con`)

Format

- Line 1: title. Skipped.
- Line 2: header. Skipped.
- Line 3 and beyond: one record per object. The loop runs from `nspu1` to `nspu1 + nspu - 1`, where `nspu` is the count for this object type taken from `sp_ob` (set in `object.cnt`). The number of rows in the file must equal `nspu`.

Each record has a fixed leading block of 13 fields followed by `out_tot` connection tuples. The reader makes two list-directed reads of the same line: the first pass reads up to `src_tot`, then if `src_tot > 0` the reader backspaces and re-reads with the connection tuples appended.

Fixed leading block

#	Column (typical header)	Maps to	Type	Description
1	id / NUMB	`ob(i)%num`	int	Object number within this object type. Sequential. Becomes the spatial-object pointer used everywhere else (`hru` column of `hru-data.hru`, channel id in `chandeg.con`, etc).
2	name / NAME	`ob(i)%name`	char(16)	Object name. Free-form.
3	gis_id / GIS_ID	`ob(i)%gis_id`	int*8	GIS identifier from the source watershed delineation.
4	area / AREA_HA	`ob(i)%area_ha`	real (ha)	Drainage area. For HRU, routing unit, and recall objects this value is also copied into `ob(i)%area_ha_calc`. For other objects `area_ha_calc` is initialized to 0 and is filled in by `hyd_connect` from the routing topology.
5	lat / LAT	`ob(i)%lat`	real (deg)	Latitude.
6	lon / LONG	`ob(i)%long`	real (deg)	Longitude.
7	elev / ELEV	`ob(i)%elev`	real (m)	Elevation.
8	hru / aqu / lcha / rtu / RECALL	`ob(i)%props`	int	Pointer to the parameter record for this object. For `hru.con` it points into `hru-data.hru`; for `chandeg.con` into the SWAT-deg channel database; for `aquifer.con` into `aquifer.aqu`; and so on.
9	wst / WST	`ob(i)%wst_c`	char(50)	Weather station name. Resolved by `search(wst_n, db_mx%wst, wst_c, wst)` to the integer `ob(i)%wst` after the record is read.
10	cst / CONSTIT	`ob(i)%constit`	int	Pointer to constituent data (pesticides, pathogens, metals, salts). `0` means no constituent data.

#	Column (typical header)	Maps to	Type	Description
11	ovfl / OVERFLOW	`ob(i)%props2`	int	Overbank-connectivity pointer to a landscape unit. `0` means no overbank connection.
12	rule / RULESET	`ob(i)%ruleset`	char(16)	Name of a decision table in `flo_con.dtl` that controls the outflow fraction. Empty string means no rule.
13	out_tot / OUT_TOT	`ob(i)%src_tot`	int	Number of downstream (source) connections that follow on this line. May be 0.

Per-connection tuple

If `out_tot > 0`, the reader expects `out_tot` groups of four values after column 13, all on the same line:

Offset	Column (typical header)	Maps to	Type	Description
+1	obj_typ / OBTYP_OUT	`ob(i)%obtyp_out(k)`	char(3)	Type of the downstream object. Values used in `hyd_connect`: `hru`, `ru`, `aqu`, `cha`, `sdc` (SWAT-deg channel), `res`, `rec`, `exc`, `dr`, `out`.
+2	obj_id / HTYPNO_OUT	`ob(i)%obtypno_out(k)`	int	Sequential number of the downstream object within its type.
+3	hyd_typ / HTYP_OUT	`ob(i)%htyp_out(k)`	char(3)	Which outflow hydrograph from this object flows to the target. Values include `tot` (total), `rhg` (recharge), `sur` (surface), `lat` (lateral), `til` (tile). The set depends on the source object type; see `type hd_tot` in `hydrograph_module.f90`.
+4	frac / FRAC_OUT	`ob(i)%frac_out(k)`	real	Fraction of that hydrograph routed to the target. The fractions for one source object should sum to 1.0. The reader does not check this.

If `out_tot == 0`, the reader allocates `ob(i)%obtypno_out(1)` and sets it to 0. No connections are recorded.

gwflow side effect

After the connection block is read, if `bsn_cc%gwflow == 1` and any of the connections target an aquifer (`obtyp_out(k) == "aqu"`), the reader decrements `ob(i)%src_tot` by 1. The aquifer connection is left in the arrays but `src_tot` no longer counts it. This is part of the gwflow override that replaces standard aquifer routing with grid-cell routing (see `object.cnt`, gwflow side effect).

Example

`refdata/0su_1hru/aquifer.con`:

```
aquifer.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       aqu
       1  aqu001                   1      10.00000      35.64206     127.38513     322.26395         1
```

Notes:

- One aquifer object, named `aqu001`, with `area_ha = 10.0` and `out_tot = 1`.
- The single connection sends `tot` (total) outflow from this aquifer to SWAT-deg channel number 1 (`sdc 1 tot 1.00000`), fraction 1.0.

`refdata/Osu_1hru/rout_unit.con` shows a row with `out_tot = 2`:

```
rout_unit.con: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                gis_id          area           lat           lon          elev       rtu
       1  rtu001                   1       10.0000      35.52014     127.32787     113.51276         1
```

The routing unit `rtu001` sends total flow to SWAT-deg channel 1 (fraction 1.0) and recharge flow (`rhg`) to aquifer 1 (fraction 1.0). The two fractions need not sum to 1.0 because they apply to different hydrograph types (`tot` and `rhg`).

`refdata/Ames_sub1/hru.con` is the trivial case with `out_tot = 0` on every row:

```
hru.con: AMES
      id  name                gis_id          area           lat           lon          elev       hru
       1  hru0001                  1         1.005         41.20        -96.63         347.5         1
       2  hru0002                  1         1.005         41.20        -96.63         347.5         2
```

When `out_tot = 0` the row stops at column 13 and no connection tuples follow. Routing for these HRUs is then expected to be supplied through a separate routing unit definition.

Related files

- `object.cnt`. Sets `sp_ob%hru`, `sp_ob%chan`, etc, which determine how many rows `hyd_read_connect` expects in each `.con` file.
- `hru-data.hru`. The `hru` column in `hru.con` points into the rows of this file.
- `hru-lte.hru`, `rout_unit.def`, `aquifer.aqu`, `channel.cha`, `chandeg.cha`, `reservoir.res`, `recall.rec`, `exco.exc`, `delratio.del`. Each `.con` file points into one of these property files via the `props` column.
- `flo_con.dtl`. Decision tables named in the `ruleset` column.
- `weather-sta.cli`. Source of the names in the `wst` column.
- `gwflow.chancells`. When `bsn_cc%gwflow == 1`, the aquifer connection is silently dropped from `src_tot` after reading.

Purpose

`hru-data.hru` is the pointer table for every HRU in the project. Each record names one HRU and lists the parameter sets it draws from: topography, hydrology, soils, land use management, soil and plant initialization, surface storage, snow, and field. The names in this file are looked up in the corresponding parameter files (`topography.hyd`, `hydrology.hyd`, `soils.sol`, `landuse.lum`, `plant.ini`, `snow.sno`, `field.fld`). The integer indices resolved by those lookups are stored in `hru_db(i)%dbs` and used throughout the simulation to fetch HRU properties.

If the file is missing or set to `null` in `file.cio`, the reader allocates `hru_db(0:0)` and no HRU data is loaded.

Source

- Reader: `src/hru_read.f90`
- Type definition: `src/hru_module.f90` (`type hru_databases_char`, `type hydrologic_response_unit_db`)
- File name pointer: `src/input_file_module.f90` (`in_hru%hru_data`, default `"hru-data.hru"`)

Format

- Line 1: title. Skipped.
- Line 2: header. Skipped.

- Line 3 and beyond: one record per HRU. The reader makes two passes. The first pass scans the file to find the largest `id` and allocates `hru_db(0:imax)`. The second pass reads each record list-directed into `k, hru_db(i)%dbsc`, where `k` is a throwaway read of the id and `i` is the id parsed from the same line.

Record layout: 10 list-directed values per HRU.

#	Column	Maps to	Type	Description
1	id	record index `i`	int	HRU number. Used as the array index into `hru_db`. Does not have to be sequential. The largest `id` in the file sets the array size.
2	name	`hru_db(i)%dbsc%name`	char(40)	HRU name. Free-form identifier.
3	topo	`hru_db(i)%dbsc%topo`	char(40)	Topography record name. Looked up in `topography.hyd`. Missing names are logged to `diagnostics.out` as "not found (topography.hyd)".
4	hydro	`hru_db(i)%dbsc%hyd`	char(40)	Hydrology record name. Looked up in `hydrology.hyd`. Missing names are logged as "not found (hydrograph.hyd)".
5	soil	`hru_db(i)%dbsc%soil`	char(40)	Soil record name. Looked up in `soils.sol`. Missing names are logged as "not found (soils.sol)".
6	lu_mgt	`hru_db(i)%dbsc%land_use_mgt`	char(40)	Land use and management record name. Looked up in `landuse.lum`. Missing names are logged as "not found (landuse.lum)".
7	soil_plant_init	`hru_db(i)%dbsc%soil_plant_init`	char(40)	Soil and plant initialization record name. Looked up in `plant.ini`. Missing names are logged as "not found (plant.ini)". A match also cross-walks the initial soil-test, pesticide, pathogen, heavy-metal, salt, and constituent records named inside the `plant.ini` entry.
8	surf_stor	`hru_db(i)%dbsc%surf_stor`	char(40)	Surface storage (reservoir or wetland) record name. The reader stores the name but does not resolve a pointer for it in `hru_read`. Use `null` if the HRU has no surface storage.
9	snow	`hru_db(i)%dbsc%snow`	char(40)	Snow record name. Looked up in `snow.sno`. The literal `null` is allowed and suppresses the not-found warning.
10	field	`hru_db(i)%dbsc%field`	char(40)	Field record name. Looked up in `field.fld`. The literal `null` is allowed and suppresses the not-found warning.

The reader does not enforce a count line. The number of HRUs comes from the maximum `id` seen during the scan pass, so a gap in the numbering allocates unused slots in `hru_db`.

Example

`refdata/Ames_sub1/hru-data.hru`:

```
hru-data.hru:
      id  name                      topo            hydro            soil           lu_mgt   soil_
       1  hru0001              topohru0001          hyd0001        soil_01-h1       cosy_lum        
       2  hru0002              topohru0002          hyd0002        soil_02          mntill_corn_lum 
       3  hru0003              topohru0003          hyd0003        soil_03-h3       cosy_lum        
       4  hru0004              topohru0004          hyd0004        soil_04          cosy_lum        
```

Notes on the example:

- HRU 2 uses `mntill_corn_lum` while the rest use `cosy_lum`. The land use record drives tillage, plant cover, and management operations.

- `surf_stor` and `field` are `null` for every HRU. None of the HRUs has a pond, wetland, or tile-drained field assigned.
- `snow001` is the same record for every HRU. The snow record is a per-elevation-band parameter set.

Related files

- `object.cnt`. The `hru` count must be at least the largest `id` in this file.
- `hru.con`. One row per HRU, in the same id order.
- `topography.hyd`, `hydrology.hyd`, `soils.sol`, `landuse.lum`, `plant.ini`, `snow.sno`, `field.fld`. Targets of the name pointers above.
- `file.cio`. The `hru` category line points to this file.

Purpose

This page documents the per-HRU hydrology and topography parameter files: `hydrology.hyd`, `topography.hyd`, and `field.fld`, plus the closely related shade-factor file `shade_factor.shf`. File names for the first three are set in `type input_hydrology` in `src/input_file_module.f90`; the shade file is set in `type shade_factor` in the same module.

Records in `hydrology.hyd`, `topography.hyd`, and `field.fld` are referenced by name from `hru-data.hru` (columns `hydro`, `topo`, and the field-name slot). Every HRU points at one record from each. Records in `shade_factor.shf` are used by the channel temperature routine in `src/ch_temp.f90`.

Each file is optional. If the slot in `file.cio` is `null` or the file is missing, the reader allocates a zero-length array and skips on. All four readers skip two header lines, then read one record per parameter set.

Source

- File names: `src/input_file_module.f90`, `type input_hydrology`, `type shade_factor`
- Readers:
  - `src/hydrol_read.f90` (`hydrology.hyd`)
  - `src/topo_read.f90` (`topography.hyd`)
  - `src/field_read.f90` (`field.fld`)
  - `src/shade_factor_read.f90` (`shade_factor.shf`)
- Type definitions: `src/hydrology_data_module.f90`, `src/topography_data_module.f90`, `src/hydrograph_module.f90`

hydrology.hyd

Per-HRU hydrology parameter sets (lateral flow, canopy, evaporation compensation, percolation, enrichment ratios). Read into `hyd_db` (`type hydrology_db` in `src/hydrology_data_module.f90`).

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“”	parameter set name
2	lat_ttime	real	days	0.0	exponential of the lateral flow travel time (0..120)
3	lat_sed	real	g/L	0.0	sediment concentration in lateral flow
4	canmx	real	mm H2O	0.0	maximum canopy storage
5	esco	real	none	0.0	soil evaporation compensation factor (0..1)
6	epco	real	none	0.0	plant water uptake compensation factor (0..1)
7	erorgn	real	none	0.0	organic N enrichment ratio (0 = model calculates per event)
8	erorgp	real	none	0.0	organic P enrichment ratio (0 = model calculates per event)
9	cn3_swf	real	none	0.0	soil water at CN3 (0 = field capacity, 0.99 = near saturation)
10	biomix	real	none	0.0	biological mixing efficiency, applied at end of each calendar year
11	perco	real	none	0.0	percolation coefficient, linear adjustment to daily perc (0..1)
12	lat_orgn	real	ppm	0.0	organic N concentration in lateral flow
13	lat_orgp	real	ppm	0.0	organic P concentration in lateral flow
14	pet_co	real	none	1.0	radiation coefficient used in Hargreaves equation
15	latq_co	real	none	0.3	lateral soil flow coefficient, linear adjustment to daily lateral flow (0..1)

Example from `refdata/Osu_1hru/hydrology.hyd`:

```
hydrology.hyd: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                 lat_ttime       lat_sed       can_max          esco          epco   orgn_enrich   org
hyd0001                0.00000       0.00000       1.00000       0.95000       0.50000       0.00000
```

Editor column names (`can_max`, `harg_pet`, `bio_mix`) differ from source field names. The reader uses position only.

topography.hyd

Per-HRU topography parameter sets. Read into `topo_db` (`type topography_db` in `src/topography_data_module.f90`).

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“default”	parameter set name
2	slope	real	m/m	0.02	average slope steepness in HRU
3	slope_len	real	m	50.0	average slope length for erosion
4	lat_len	real	m	50.0	slope length for lateral subsurface flow
5	dis_stream	real	m	100.0	average distance to stream
6	dep_co	real	none	1.0	deposition coefficient

Example from `refdata/Osu_1hru/topography.hyd`:

```
topography.hyd: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        slp       slp_len       lat_len      dist_cha         depos
topohru0001             0.01078      10.00000      10.00000     121.00000       0.00000
```

The same type and array are also used for landscape units; `hru-data.hru` and the subbasin objects both reference entries in `topo_db` by name.

field.fld

Per-field geometry used by the wind-erosion routine. Read into `field_db` (`type fields_db` in `src/topography_data_module.f9`

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“default”	parameter set name
2	length	real	m	500.0	field length for wind erosion
3	wid	real	m	100.0	field width for wind erosion
4	ang	real	deg	30.0	field angle for wind erosion

Example from `refdata/Osu_1hru/field.fld` (first four records shown):

```
field.fld: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        len            wd           ang
fld011                500.00000     100.00000      30.00000
fld012                500.00000     100.00000      30.00000
fld031                500.00000     100.00000      30.00000
fld032                500.00000     100.00000      30.00000
```

The reader uses a `read .. backspace .. read` pattern, but the file format is one record per line, same as the other files on this page.

shade_factor.shf

Time- and landscape-unit-resolved shade factor used by the channel water-temperature routine. Read into `shf_db` (`type shade_factor_data` in `src/hydrograph_module.f90`). Activated per channel by `sf_on = 1` in `temperature.cha`; when `sf_on = 0` the channel uses the calibration-file value instead.

#	Column	Type	Units	Default	Description
1	jday	int	day of year	0	day of year the record applies to
2	lsu	int	none	0	landscape unit number
3	value	real	none	0.0	shade factor value

In `src/ch_temp.f90` the array is scanned for the matching `(jday, lsu)` pair and the `value` is assigned to `ssff`. There is no name column.

No example dataset in `refdata/` ships a `shade_factor.shf` file.

Related files

- `file.cio`. The `hru` line selects `hydrology.hyd`, `topography.hyd`, and `field.fld`; the shade file is listed separately.
- `hru-data.hru`. Every HRU references one entry from each of `hydrology.hyd`, `topography.hyd`, and `field.fld`.
- `temperature.cha`. Controls whether `shade_factor.shf` is consulted (`sf_on` flag).

Purpose

The recall module reads externally provided time series and exposes them as spatial objects in the SWAT+ object graph. A recall object is a point source, an outlet, or a basin inlet whose flow and constituent loads are read from disk instead of simulated. Each recall has its own time step (subdaily, daily, monthly, or annual). The hydrograph delivered by a recall feeds the receiving object through `recall.con`, exactly the same way that an HRU or a channel feeds its downstream link.

Inputs come in two layers:

- A master database file (`recall.rec`) lists every recall object and points to the per-object side files for organic and mineral constituents and, optionally, for pesticides, pathogens, heavy metals, salts, and other constituents.
- Optional parallel database files (`salt_recall.rec`, `cs_recall.rec`) carry the time series of salt-ion and constituent loads when those modules are active.
- Optional spatial-element files (`rec_catunit.def`, `rec_reg.def`, `rec_catunit.ele`) group recall objects into regions used for output aggregation and soft calibration.

Source

- Master reader: `src/recall_read.f90` (`recalldb_read` and `recall_read`)
- Salt reader: `src/recall_read_salt.f90`
- Constituent reader: `src/recall_read_cs.f90`
- Element reader: `src/rec_read_elements.f90`
- Type definitions: `src/recall_module.f90` (`type recall_databases`, `type constituent_file_data`), `src/hydrograph_module.f90` (`type recall_hydrograph_inputs`), `src/constituent_mass_module.f90` (`type recall_salt_inputs`, `type recall_cs_inputs`, `type recall_pesticide_inputs`).

recall.rec (master database)

Lists every recall object and names the side files that hold its time-series data.

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per recall object. Two passes: the first finds the maximum object number, the second reads the records.

Each record is read into `recall_db(i)` (`type recall_databases`):

#	Field	Type	Description
1	id	int	Sequential id. Used as the array index.
2	name	char(13)	Recall object name. Referenced by `recall.con` and by the `obj_typ = rec` slots in other connectivity files.

#	Field	Type	Description
3	org_min	record	Organic and mineral side file. Three subfields read together: `name` (filename), `units` (`mass` or `conc`), `tstep` (`sub`, `day`, `mo`, `yr`).
4	pest	record	Pesticide side file (same three subfields).
5	path	record	Pathogen side file.
6	hmet	record	Heavy-metal side file.
7	salt	record	Salt side file.
8	constit	record	Constituent (other) side file.

`type constituent_file_data` has three character fields read in order: `name(25)`, `units(13)`, `tstep(13)`. List-directed reads consume three tokens per record slot. After reading each row, the master reader calls `recall_read(i)` which opens the file in `recall_db(i)%org_min%name` and reads the time series.

Per-object time series

`recall_read` opens the organic-mineral side file and dispatches on `tstep`:

`tstep`	Storage	Records per year
`sub`	`recall(irec)%hyd_flo(step*366, nbyr)` plus `recall(irec)%hd(366, nbyr)`	Subdaily. Flow is converted from m^3/s to m^3 per subdaily step.
`day`	`recall(irec)%hd(366, nbyr)`	One row per Julian day.
`mo`	`recall(irec)%hd(12, nbyr)`	One row per month.
`yr`	`recall(irec)%hd(1, nbyr)`	One row per year.

Side-file format (`day`, `mo`, `yr`):

- Line 1: title.
- Line 2: `nbyr` (number of years).
- Line 3: header (skipped).
- Lines 4+: per time step, `jday`, `mo`, `day_mo`, `iyr`, `ob_typ`, `ob_name`, then a 22-field `hyd_output` record (flow plus organic, mineral, and physical constituents).

Each `hyd_output` row supplies, in order: `flo`, `sed`, `orgn`, `sedp`, `no3`, `solp`, `chla`, `nh3`, `no2`, `cbod`, `dox`, `san`, `sil`, `cla`, `sag`, `lag`, `grv`, `temp`. Flow is in m^3/s. Sediment is in metric tons. Nutrients and other constituents are in kg. See `type hyd_output` for the full column list.

The reader fast-forwards the side file to the row whose `iyr` matches `time%yrc` (simulation start year). Rows before the simulation start are skipped; rows after `time%yrc_end` end the read. Each row sets `recall(irec)%start_yr` to the first matching year and `recall(irec)%end_yr` to the last row read.

pest.com (recall pesticide side database)

When `pest.com` exists in the working directory, `recall_read` reads it as a parallel pesticide database. It is structured like `recall.rec`: a master file naming per-pesticide files. Each per-pesticide file has its own `nbyr` and time step (1=day, 2=mo, 3=yr) and fills `rec_pest(i)%hd_pest(istep, nbyr)`. The exact column count of the per-pesticide payload depends on `db_mx%pestcom` (the number of pesticides in the constituent database).

salt_recall.rec (salt side database)

Mirror of `recall.rec` for salt loads, read when the salt module is active.

- Line 1: title.
- Line 2: header.
- Lines 3+: one record per recall object. Fields: `id`, `name`, `typ` (1=day, 2=mo, 3=yr, 4=exco crosswalk), `filename`.

For `typ` in `1..3`, the side file is opened with:

- Line 1: title. If the literal token is `Incoming` the source is flagged as originating from outside the watershed (`pts_type = 2`); otherwise `pts_type = 1` (within the watershed).
- Line 2: `nbyr`.
- Line 3: header.
- Lines 4+: `jday`, `mo`, `day_mo`, `iyr`, `ob_typ`, `ob_name`, then `cs_db%num_salts` real values (one per salt ion). The order of ions follows `constituents.cs`.

Each side file fills `rec_salt(i)%hd_salt(istep, nbyr)%salt(1..num_salts)`. For `typ = 4`, the salt record is crosswalked to an `exco` entry; no side file is opened.

cs_recall.rec (constituent side database)

Same structure as `salt_recall.rec`, but for the other-constituent module. Fields per record: `id`, `name`, `typ`, `filename`. Side files have the same five-line header pattern; payload rows carry `cs_db%num_cs` real values that fill `rec_cs(i)%hd_cs(istep, nbyr)%cs(1..num_cs)`. The `Incoming` flag in the title is honoured identically.

Element and region files

Three files in `file.cio`'s `regions` slot group recall objects for output aggregation and soft calibration. Names default to `rec_catunit.def`, `rec_reg.def`, and `rec_catunit.ele` (set by `in_regs%def_psc`, `in_regs%def_psc_reg`, and `in_regs%ele_psc`). All three are optional.

rec_catunit.def (output regions)

Defines the regions used to print recall output. Format:

- Line 1: title.
- Line 2: number of regions, `mreg`.
- Line 3: header.
- Lines 4+: per region: `id`, `name`, `area_ha`, `nspu`, then `nspu` element ids.

When `nspu > 0`, the listed element ids are passed to `define_unit_elements` and stored in `pcu_out(i)%num`. When `nspu = 0`, the region defaults to every recall in the basin (`sp_ob%recall` elements).

rec_reg.def (calibration regions)

Same format as `rec_catunit.def`. Populates `pcu_reg(i)` instead of `pcu_out(i)`. Used for soft-calibration grouping and per-type output.

rec_catunit.ele (element table)

Lists every element that can appear in `rec_catunit.def` or `rec_reg.def`. Format:

- Line 1: title.
- Line 2: header.
- Lines 3+: per element: `id`, `name`, `obtyp`, `obtypno`, `bsn_frac`, `ru_frac`, `reg_frac`.

`obtyp` is the object-type tag (typically `rec`) and `obtypno` is the recall object number. The three fractions are the share of the element in the basin, in its routing unit, and in its region.

Example

`refdata/Osu_1hru/recall.rec`:

```
recall.rec
NUM     NAME    TYPE     FNAME    OBJ_TYP     OBJ_NAME       flo     sed    orgn    sedp     no3    solp
   1    PSTYR    3      RECYR.        hru         hru1   12120.5       0       0       0       0       0
```

```
   1       1          1     2011         hru          hru2     9000.7        0        0        0        0        0
```

This dataset stores one recall object (PSTYR) whose annual hydrograph lives in a side file. The trailing 22-column row is the actual hyd_output payload that the side file would contain. In a clean layout, the side file (RECYR. or similar) is its own file: a title line, a year count, a header, then one or more annual rows.

Related files

- recall.con. Spatial-connectivity entries for each recall object. Links the recall to its receiving spatial object.
- file.cio. Names the master recall.rec (in_rec%recall_rec) and the element files (in_regs%def_psc, in_regs%def_psc_reg, in_regs%ele_psc).
- salt module, cs module. Document the salt and other-constituent modules whose recall side databases parallel recall.rec.

Purpose

The .cli files are index files. Each one names a set of climate inputs that SWAT+ should load. They sit between file.cio, which names the index files themselves, and the actual time series files (*.pcp, *.tmp, *.slr, *.hmd, *.wnd, *.pet, weather-generator stations, atmospheric deposition).

The full set of index files is defined by type input_cli in src/input_file_module.f90:

Slot	Default name	Reader	Purpose
weat_sta	weather-sta.cli	cli_staread.f90	Weather stations: one row per station, columns are pointers to a wgn station and to per-variable gages.
weat_wgn	weather-wgn.cli	cli_wgnread.f90	Weather generator parameters: location plus 12 monthly statistics.
pet_cli	pet.cli	cli_petmeas.f90	List of measured PET station files.
pcp_cli	pcp.cli	cli_pmeas.f90	List of measured precipitation station files.
tmp_cli	tmp.cli	cli_tmeas.f90	List of measured temperature station files.
slr_cli	slr.cli	cli_smeas.f90	List of measured solar radiation station files.
hmd_cli	hmd.cli	cli_hmeas.f90	List of measured relative humidity station files.
wnd_cli	wnd.cli	cli_wmeas.f90	List of measured wind speed station files.
atmo_cli	atmodep.cli	cli_read_atmodep.f90	Atmospheric N deposition: control record plus one or more stations.

In file.cio the slot value is either a filename or the literal token null. Every reader skips its work when the slot is null or when the file does not exist. The per-variable readers (pcp.cli, tmp.cli, slr.cli, hmd.cli, wnd.cli, pet.cli) all have the same shape: title, header, then one filename per line. The actual time-series files are documented separately.

All readers skip a title line and a header line at the top of the file. Records are list-directed (free format), so columns are separated by whitespace.

weather-sta.cli

One record per weather station. Each row binds a station name to a weather generator and to gage names for each climate variable.

Source

- Reader: src/cli_staread.f90
- Type: src/climate_module.f90, `type weather_station` (column 1) and `type weather_codes_station_char` (columns 2 to 9)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: 9 fields per record, list-directed.

The reader call is `read (107,*,iostat=eof) wst(i)%name, wst(i)%wco_c`. The fields of `wco_c` are filled in declaration order.

#	Column	Type	Description
1	name	char(50)	Weather station name. Referenced by hru.con and other spatial-object files.
2	wgn	char(50)	Name of the weather generator station in `weather-wgn.cli`.
3	pgage	char(50)	Name of the precipitation gage in `pcp.cli`, or `sim` to generate.
4	tgage	char(50)	Name of the temperature gage in `tmp.cli`, or `sim` to generate.
5	sgage	char(50)	Name of the solar radiation gage in `slr.cli`, or `sim` to generate.
6	hgage	char(50)	Name of the relative humidity gage in `hmd.cli`, or `sim` to generate.
7	wgage	char(50)	Name of the wind speed gage in `wnd.cli`, or `sim` to generate.
8	petgage	char(50)	Name of the measured PET station in `pet.cli`, or `null`.
9	atmodep	char(50)	Name of the atmospheric deposition station in `atmodep.cli`, or `null`.

Each gage name is matched against the station list of the corresponding `.cli` file. A missing match (and a value other than `sim` for the generated variables, or `null` for pet/atmodep) is written to the error log at unit 9001.

Example

`refdata/Ames_sub1/weather-sta.cli`:

```
weather-sta.cli:
name                          wgn              pcp              tmp              slr              hm
wgn_sta1                  wgn_001        ames.pcp         ames.tem              sim              si
```

The header column at position 8 is labelled `wnd_dir` by SWAT+ Editor. The reader stores that value in `petgage` (the PET gage slot).

weather-wgn.cli

One record per weather generator station. Each record is a station header line plus a 12-row monthly statistics block.

Source

- Reader: src/cli_wgnread.f90
- Type: src/climate_module.f90, `type weather_generator_db`

Format

- Line 1: title (skipped).
- Per station, 14 lines:
  - Line A: station name, latitude, longitude, elevation, rain_yrs.
  - Line B: header for the monthly block (skipped).
  - Lines C1..C12: one line per month, 14 values each.

Number of stations is determined by reading to end of file.

Station header line:

#	Field	Type	Units	Description
1	name	char	-	Generator station name. Matched by `wgn` in `weather-sta.cli`.
2	lat	real	degrees	Latitude of the generator station.
3	long	real	degrees	Longitude of the generator station.
4	elev	real	m	Elevation of the generator station.
5	rain_yrs	real	years	Number of years of record used to compute `rainhmx`.

Monthly block, 12 rows (Jan to Dec), 14 columns:

#	Field	Type	Units	Description
1	tmpmx	real	deg C	Average daily maximum air temperature.
2	tmpmn	real	deg C	Average daily minimum air temperature.
3	tmpstdmx	real	deg C	Standard deviation of daily maximum air temperature.
4	tmpstdmn	real	deg C	Standard deviation of daily minimum air temperature.
5	pcpmm	real	mm	Total precipitation in the month.
6	pcpstd	real	mm/day	Standard deviation of daily precipitation.
7	pcpskw	real	none	Skew coefficient of daily precipitation.
8	pr_wd	real	none	Probability of a wet day after a dry day.
9	pr_ww	real	none	Probability of a wet day after a wet day.
10	pcpd	real	days	Average number of days of precipitation in the month.
11	rainhmx	real	mm	Maximum 0.5-hour rainfall in the month.
12	solarav	real	MJ/m^2/day	Average daily solar radiation.
13	dewpt	real	deg C	Average daily dew point temperature.
14	windav	real	m/s	Average daily wind speed.

Example

`refdata/Ames_sub1/weather-wgn.cli`:

```
weather-wgn.cli: Ames
wgn001      37.48     -100.83   887.00    10
   TMPMX   TMPMN  TMPSTDMX TMPTDMN  PCPMM  PCPSTD  PCPSKW   PR_WD   PR_WW   PCPD  RAINHMX SOLARAV  DEWPT
    7.61   -8.13    8.06    5.99     7.4     3.3    0.19    0.07    0.25    2.65     5.1   10.39   -8.14
   11.07   -5.53    8.06    5.34    10.2     5.1    2.67    0.08    0.27    2.86     1.8   13.41   -5.58
   15.22   -1.87    7.99    5.22    28.1     8.9    1.64    0.11    0.34    4.43     7.6   17.85   -4.71
   21.42    4.12    6.42    4.82    32.0     7.9    1.65    0.14    0.34    5.25    24.6    21.7    0.93
```

```
 25.97    9.93    5.68    4.29    83.7    12.4    1.71    0.21    0.42    8.24    26.9   23.09    7.81
 31.78   15.51    4.96    3.62    71.6    13.0     0.9    0.17    0.41    6.71    40.4   26.15   12.49
 34.28   18.34    3.68    2.51    65.0    12.4    1.25    0.18    0.33    6.56    37.3   25.85   14.72
 33.17   17.22    3.89    2.59    65.0    13.0    1.26    0.19    0.27     6.4    32.0   22.54   14.29
 29.04   12.36    5.12    4.25    43.5    13.5    3.71    0.13    0.31    4.76    18.0   20.32    9.38
 23.43    5.54    6.17    4.63    29.8    11.9    1.31    0.08    0.32    3.26    10.4   15.04    3.48
 14.34   -1.71    6.76    4.75    20.1     6.9    0.58    0.09    0.34     3.6    10.4   15.46   -2.96
  9.17   -6.26    7.09    5.01     8.0     3.6    1.24    0.07    0.24    2.61    16.3    9.76   -6.38
```

This file holds one generator station (`wgn001`).

pcp.cli

List of measured precipitation station files. Each row points to one `*.pcp` file in the project directory (or under the path set by `in_path_pcp%pcp`).

Source

- Reader: src/cli_pmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`pcp` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.pcp` time-series file. The same string is used as the station key (`pcp_n(i)`) that `weather-sta.cli` matches against.

The reader makes two passes over the file. The first pass reads the field into `pcp_n(i)` (the lookup name). The second pass reads the same field into `pcp(i)%filename` and opens that file.

Example

`refdata/Ames_sub1/pcp.cli`:

```
pcp.cli
FILENAME
ames.pcp
```

One station, file `ames.pcp` in the project directory.

tmp.cli

List of measured temperature station files.

Source

- Reader: src/cli_tmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`tmp` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.tmp` time-series file. Used as the station key (`tmp_n(i)`).

Optional weather path prefix from `in_path_tmp%tmp` (see `file.cio`).

Example

`refdata/Ames_sub1/tmp.cli`:

```
tmp.cli
FILENAME
ames.tmp
```

slr.cli

List of measured solar radiation station files.

Source

- Reader: src/cli_smeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`slr` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.slr` time-series file. Used as the station key (`slr_n(i)`).

Example

```
slr.cli
FILENAME
ames.slr
```

The reference dataset `refdata/Ames_sub1` does not ship a `slr.cli` because the weather-station rows use `sim` for solar radiation. The example shape above is what the reader expects when the file is present.

hmd.cli

List of measured relative humidity station files.

Source

- Reader: src/cli_hmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`hmd` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.hmd` time-series file. Used as the station key (`hmd_n(i)`).

Example

```
hmd.cli
FILENAME
ames.hmd
```

wnd.cli

List of measured wind speed station files.

Source

- Reader: src/cli_wmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`wnd` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.wnd` time-series file. Used as the station key (`wnd_n(i)`).

Example

```
wnd.cli
FILENAME
ames.wnd
```

pet.cli

List of measured potential ET station files. Used when codes.bsn sets `pet = 3`.

Source

- Reader: src/cli_petmeas.f90
- Type: src/climate_module.f90, `type climate_measured_data` (`petm` array)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one field per record.

#	Column	Type	Description
1	filename	char(50)	Name of a `*.pet` time-series file. Used as the station key (`petm_n(i)`).

Example

```
pet.cli
FILENAME
ames.pet
```

atmodep.cli

Atmospheric N deposition for one or more stations. Unlike the per-variable index files, this file contains the deposition values themselves, not pointers to data files.

Source

- Reader: src/cli_read_atmodep.f90
- Types: src/climate_module.f90, `type atmospheric_deposition_control` (line 3) and `type atmospheric_deposition` (per-station block)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Line 3: control record, 5 fields.
- Then, per station, a block whose layout depends on `timestep`.

Control record (line 3):

#	Field	Type	Description
1	num_sta	int	Number of deposition stations to read.
2	timestep	char(2)	`aa` (average annual, scalar values), `mo` (monthly time series), or `yr` (annual time series).
3	mo_init	int	First month of the time series (1 to 12). Used when `timestep = mo`.
4	yr_init	int	First year of the time series. Used for `mo` and `yr`.
5	num	int	Number of records per station (length of the time series). Set to `1` for `aa`.

For `timestep = aa`, each station occupies 5 lines:

Line	Type	Description
1	char(50)	Station name (used as `atmo_n(i)`).
2	real	`nh4_rf` - average annual ammonia concentration in rainfall (mg/L).
3	real	`no3_rf` - average annual nitrate concentration in rainfall (mg/L).
4	real	`nh4_dry` - average annual ammonia dry deposition (kg/ha/yr).
5	real	`no3_dry` - average annual nitrate dry deposition (kg/ha/yr).

For `timestep = mo`, each station occupies 5 lines (name plus four monthly series of length `num`):

Line	Description
1	Station name.

Line	Description
2	`nh4_rfmo(1..num)` - ammonia in rainfall by month (mg/L).
3	`no3_rfmo(1..num)` - nitrate in rainfall by month (mg/L).
4	`nh4_drymo(1..num)` - ammonia dry deposition by month (kg/ha/yr).
5	`no3_drymo(1..num)` - nitrate dry deposition by month (kg/ha/yr).

For `timestep = yr`, each station occupies 5 lines (name plus four annual series of length `num`):

Line	Description
1	Station name.
2	`nh4_rfyr(1..num)` - ammonia in rainfall by year (mg/L).
3	`no3_rfyr(1..num)` - nitrate in rainfall by year (mg/L).
4	`nh4_dryyr(1..num)` - ammonia dry deposition by year (kg/ha/yr).
5	`no3_dryyr(1..num)` - nitrate dry deposition by year (kg/ha/yr).

When `num_sta = 0` (no stations) or the file is named `null` in `file.cio` or absent, the reader allocates an empty array and skips deposition.

Example

Minimal `aa` form, one station:

```
atmodep.cli
num_sta  timestep  mo_init  yr_init  num
      1        aa        1     2000    1
station_001
1.0
0.2
0.0
0.0
```

Atmospheric deposition

Three readers populate the atmospheric deposition state. They share the per-station list set up by `atmodep.cli` and the control record `atmodep_cont` (`type atmospheric_deposition_control`). The three files cover N, salts, and other constituents.

atmodep.cli (nitrogen)

Documented above. Read by src/cli_read_atmodep.f90. Fills `atmodep(0:num_sta)` (`type atmospheric_deposition`) with NH4 and NO3 wet (rainfall) concentrations and dry deposition rates. The reader also computes `atmodep_cont%ts` (the index into the time series corresponding to the first simulation step) for `mo` and `yr` time steps, and sets `atmodep_cont%first = 0` once aligned.

salt_atmo.cli (salt ions)

Read by src/cli_read_atmodep_salt.f90 when `cs_db%num_salts > 0`. Per-station wet and dry deposition for each salt ion in `constituents.cs`. Filled into `atmodep_salt(iadep)%salt(isalt)` (`type atmospheric_deposition_cs` within `type object_deposition_cs`).

Layout:

- Lines 1 to 6: skipped. The reader consumes six lines as commentary before processing stations.
- Per station, block layout depends on `atmodep_cont%timestep` (set by `atmodep.cli`):

For `timestep = aa`:

Line	Description
1	Station name (character, ignored for cross-matching; positional).
2..1+num_salts	`salt_ion_label`, then one real: `rf` (rainfall concentration, mg/L) for each ion.
…next num_salts	`salt_ion_label`, then one real: `dry` (dry deposition, kg/ha/yr) for each ion.

For `timestep = mo`:

Line	Description
1	Station name.
2..1+num_salts	`salt_ion_label`, then `num` reals: monthly wet concentrations (mg/L).
…next num_salts	`salt_ion_label`, then `num` reals: monthly dry deposition (kg/ha/yr).

For `timestep = yr`: same shape as monthly with annual series of length `num`.

When the file exists, the reader sets the module-level flag `salt_atmo = "y"`. When `cs_db%num_salts = 0` or the file is missing, salt deposition is skipped.

cs_atmo.cli (other constituents)

Read by src/cli_read_atmodep_cs.f90 when `cs_db%num_cs > 0`. Same shape as `salt_atmo.cli` but without per-ion labels.

Layout:

- Lines 1 to 3: skipped.
- Per station:

For `timestep = aa`:

Line	Description
1	Station name (real-typed scratch read; positional).
2..1+num_cs	One real per line: wet concentration (mg/L) for each constituent.
…next num_cs	One real per line: dry deposition (kg/ha) for each constituent.

For `timestep = mo` and `timestep = yr`, each constituent gets one line of `num` reals for wet, and one line of `num` reals for dry, in the same order as `constituents.cs`.

When the file exists, the reader sets `cs_atmo = "y"`.

Related files

- file.cio. Names every `.cli` file via `type input_cli`.
- codes.bsn. `pet = 3` triggers reading `pet.cli`.
- `*.pcp`, `*.tmp`, `*.slr`, `*.hmd`, `*.wnd`, `*.pet`. The actual time-series files listed in the per-variable `.cli` files.
- salt module, cs module. Define the salt and other-constituent lists that drive `salt_atmo.cli` and `cs_atmo.cli`.

Purpose

A `*.pcp` file holds the measured precipitation time series for a single gauge. One file per station. The filename is listed in pcp.cli and the same filename string is matched as the gauge key referenced from `pgage` in weather-sta.cli. The reader supports either daily values or subdaily values at a fixed time step.

Source

- Reader: src/cli_pmeas.f90
- Type definition: src/climate_module.f90, `type climate_measured_data` (the `pcp` array)

Format

The reader uses list-directed reads (free format, whitespace-separated). The first two lines are skipped. The third line is the station header. From line four to end of file are the time-step records.

Line 1: title

Single string, skipped.

Line 2: column header

Skipped. SWAT+ Editor writes `NBYR  TSTEP  LAT  LONG  ELEV`.

Line 3: station header

Read by `read (108,*,iostat=eof) pcp(i)%nbyr, pcp(i)%tstep, pcp(i)%lat, pcp(i)%long, pcp(i)%elev`.

#	Field	Type	Units	Description
1	nbyr	int	years	Number of years of record in the file. Used to size the in-memory time-series array.
2	tstep	int	minutes	Time step. `0` means daily. A positive value is the subdaily interval in minutes and must match `time%step` from time.sim.
3	lat	real	degrees	Latitude of the gauge.
4	long	real	degrees	Longitude of the gauge.
5	elev	real	m	Elevation of the gauge.

Records (line 4 onward)

The record layout depends on `tstep`.

For daily records (`tstep = 0`):

#	Field	Type	Units	Description
1	year	int	calendar year	Calendar year of the record.
2	jday	int	day-of-year	Julian day (1 to 365 or 366).
3	pcp	real	mm	Daily precipitation depth. Use a value at or below `-97` to flag missing data; the reader replaces it with a generated value.

For subdaily records (`tstep > 0`):

#	Field	Type	Units	Description
1	year	int	calendar year	Calendar year.
2	jday	int	day-of-year	Julian day.
3	mo	int	month	Month number.
4	day_mo	int	day	Day of the month.
5	ihr	int	step index	Sub-day step index (1 to `1440 / tstep`).
6	pcp	real	mm	Precipitation depth for the time step.

Records are read in order. The reader advances year-by-year using the change in `jday` (it expects the year to change after `jday = 365` or `366`).

Missing data

The daily reader treats any value at or below `-97` as missing. Downstream in `cli_precip_control.f90`, when `precip_next <= -97.` the value is replaced by the weather generator. The classic flag written by SWAT+ Editor and the legacy tools is `-99`.

Example

First lines of `refdata/Ames_sub1/ames.pcp`:

```
AME.pcp
 NBYR    TSTEP       LAT      LONG      ELEV
   121        0     42.04    -93.89    316
1900 1 -99
1900 2 -99
1900 3 -99
...
1983 184 0
1983 185 15.5
1983 186 0
```

Station header: 121 years, daily time step (`0`), latitude 42.04, longitude -93.89, elevation 316 m. Early years are missing (`-99`) and will be filled by the weather generator.

Related files

- `pcp.cli`. Lists each `*.pcp` filename.
- `weather-sta.cli`. Binds a precipitation gauge to a weather station via `pgage`.
- `file.cio`. Names the `pcp.cli` index and the optional path prefix `in_path_pcp%pcp` applied to every `*.pcp` filename.
- `time.sim`. For subdaily files, `time%step` must equal the file's `tstep`.

Purpose

A `*.tmp` file holds the measured daily temperature time series for a single gauge. One file per station. The filename is listed in `tmp.cli` and the same filename string is matched as the gauge key referenced from `tgage` in `weather-sta.cli`. The reader records daily maximum and minimum air temperature.

Source

- Reader: `src/cli_tmeas.f90`
- Type definition: `src/climate_module.f90`, `type climate_measured_data` (the `tmp` array; `ts` holds daily max, `ts2` holds daily min)

Format

List-directed reads (free format, whitespace-separated). The first two lines are skipped. The third line is the station header. From line four to end of file are the daily records.

Line 1: title

Single string, skipped.

Line 2: column header

Skipped. SWAT+ Editor writes `NBYR  TSTEP  LAT  LONG  ELEV`.

Line 3: station header

Read by `read (108,*,iostat=eof) tmp(i)%nbyr, tmp(i)%tstep, tmp(i)%lat, tmp(i)%long, tmp(i)%elev`.

#	Field	Type	Units	Description
1	nbyr	int	years	Number of years of record in the file.
2	tstep	int	minutes	Time step indicator. Stored but the reader expects daily values regardless.
3	lat	real	degrees	Latitude of the gauge.
4	long	real	degrees	Longitude of the gauge.
5	elev	real	m	Elevation of the gauge.

Records (line 4 onward)

Daily values only:

#	Field	Type	Units	Description
1	year	int	calendar year	Calendar year of the record.
2	jday	int	day-of-year	Julian day (1 to 365 or 366).
3	tmax	real	deg C	Daily maximum air temperature. Stored in `tmp(i)%ts(jday, year_index)`.
4	tmin	real	deg C	Daily minimum air temperature. Stored in `tmp(i)%ts2(jday, year_index)`.

Records are read sequentially. The reader detects the year change after `jday = 365` or `366` and increments its internal year index.

While reading, the routine accumulates monthly sums into `tmp(i)%max_mon` and `tmp(i)%min_mon` (per the `xmon` call) and at end of file divides by the total day count to get the average monthly maximum and minimum. A `-99` flag is treated like any other value at this stage, so missing-data flags carry into the monthly statistics unless the file is clean over the simulation window.

Example

First lines of `refdata/Ames_sub1/ames.tem` (the temperature time series for the Ames reference dataset; the same record layout applies to a `*.tmp` file):

```
AME.Tmp
 NBYR    TSTEP      LAT     LONG     ELEV
   121        0    42.04   -93.89   316
1900    1   -99 -99
1900    2   -99 -99
...
1983    185 32.2    10.6
1983    186 26.7    11.1
```

Station header: 121 years, time step `0`, latitude 42.04, longitude -93.89, elevation 316 m. Early years carry the `-99` missing-data flag for both max and min.

Related files

- `tmp.cli`. Lists each temperature gauge filename.
- `weather-sta.cli`. Binds a temperature gauge to a weather station via `tgage`.
- `file.cio`. Names the `tmp.cli` index and the optional path prefix `in_path_tmp%tmp` applied to every temperature filename.
- `*.tem`. Same record layout, alternate filename extension used by some toolchains.

Purpose

A `*.tem` file is a temperature time-series file with the same record layout as a *.tmp. The `.tem` extension is used by the reference datasets shipped with SWAT+ (for example `refdata/Ames_sub1/ames.tem`) and by some external preprocessors. There is no separate reader for `.tem`. The temperature reader at `src/cli_tmeas.f90` opens whatever filename is supplied; the extension is not inspected.

Source

- Reader: src/cli_tmeas.f90
- Type definition: src/climate_module.f90, `type climate_measured_data`

The reader is reached only when the filename appears in tmp.cli. The `tgage` slot in weather-sta.cli is matched against the filename listed in `tmp.cli` via the `tmp_n` lookup array, so both files must agree on the string (including the `.tem` extension if used).

Format

Identical to a daily `*.tmp` file. List-directed reads, whitespace-separated.

- Line 1: title (skipped).
- Line 2: column header (skipped). Conventionally `NBYR  TSTEP  LAT  LONG  ELEV`.
- Line 3: station header (`nbyr`, `tstep`, `lat`, `long`, `elev`).
- Line 4 onward: one record per day with year, Julian day, daily maximum temperature, daily minimum temperature.

See *.tmp for the field tables.

Example

`refdata/Ames_sub1/ames.tem` (excerpt):

```
AME.Tmp
 NBYR    TSTEP       LAT      LONG      ELEV
   121        0     42.04    -93.89    316
1900    1   -99 -99
1900    2   -99 -99
...
1983    185 32.2    10.6
1983    186 26.7    11.1
```

Related files

- *.tmp. Same content, alternate extension. The reader is shared.
- tmp.cli. Lists each temperature filename (with whatever extension).
- weather-sta.cli. The `tgage` column must match a filename entry in `tmp.cli`.

Purpose

`snow.sno` is the snow parameter database. Each row defines a named set of snow accumulation and melt parameters. HRUs reference a row by name through the `snow` column of hru-data.hru, which the reader resolves to an index into the `snodb` array. If `hru-data.hru` carries `null` for an HRU's snow entry, that HRU uses the default `snow_parameters` values.

Source

- Reader: src/snowdb_read.f90
- Type definition: src/hru_module.f90, `type snow_parameters` (the `snodb` array)
- Filename slot: `in_parmdb%snow` in src/input_file_module.f90 (default `snow.sno`).

Format

List-directed reads (free format, whitespace-separated). The first two lines are skipped. From line three to end of file each line is one snow parameter row. The reader counts records on a first pass, allocates `snodb(0:imax)`, then reads each row into `snodb(isno)`.

- Line 1: title (skipped). Editor convention: `snow.sno: written by SWAT+ editor ...`.
- Line 2: column header (skipped). Editor convention: `name fall_tmp melt_tmp melt_max melt_min tmp_lag snow_h2o cov50 snow_init`.
- Line 3+: one row per parameter set, 9 fields each.

The fields appear in the order declared in `type snow_parameters`:

#	Field	Type	Units	Default	Description
1	name	char(40)	-	(none)	Parameter set name. Referenced from the `snow` column of `hru-data.hru`.
2	falltmp	real	deg C	0.0	Snowfall temperature. Precipitation falls as snow when the mean air temperature is at or below this threshold.
3	melttmp	real	deg C	0.5	Snowmelt base temperature. Snowmelt only occurs when the snowpack temperature exceeds this value.
4	meltmx	real	mm/deg C/day	4.5	Maximum melt rate factor for snow during the year (occurs near 21 June).
5	meltmn	real	mm/deg C/day	0.5	Minimum melt rate factor for snow during the year (occurs near 21 December).
6	timp	real	none	0.8	Snowpack temperature lag factor (0 to 1). Higher values give more weight to the current day's air temperature.
7	covmx	real	mm H2O	25.0	Snow water content above which the ground is fully covered by snow.
8	cov50	real	none	0.5	Fraction of `covmx` at which the ground is 50% snow-covered. Shape factor for the snowpack areal depletion curve.
9	init_mm	real	mm H2O	0.0	Initial snow water content at the start of the simulation.

The defaults above are the values assigned in `type snow_parameters` at declaration. They are not values the reader supplies, so every row of `snow.sno` must list all nine fields.

Example

`refdata/Ames_sub1/snow.sno`:

```
snow.sno: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                  fall_tmp      melt_tmp      melt_max      melt_min       tmp_lag      snow_h2o
snow001                1.00000       0.50000       4.50000       4.50000       1.00000       1.00000
```

One parameter set named `snow001`. Note that the example value for `meltmn` (column 5) is `4.5`, identical to `meltmx`. The defaults shipped in source set `meltmn = 0.5`.

Related files

- hru-data.hru. References a `snow.sno` row by name in its `snow` column. `null` selects the type defaults.
- file.cio. Names the `snow.sno` file in the `init` group via `in_parmdb%snow`. Set to `null` to skip reading (no snow database is loaded; HRUs that point at a snow name will fall back as described above).

Purpose

`soils.sol` defines every soil class used in the watershed. Each class has a profile header (name, number of layers, hydrologic group, rooting depth, anion exclusion fraction, crack volume, texture string) and one row per soil layer describing depth, bulk density, available water capacity, saturated hydraulic conductivity, organic carbon, particle-size fractions, rock content, albedo, USLE K, electrical conductivity, calcium carbonate, and pH. HRUs reference these classes through `hru-data.hru`. Values feed `soil_phys_init`, `soil_nutcarb_init`, runoff, percolation, erosion, and plant uptake routines.

Source

- Reader: src/soil_db_read.f90
- Type definition: src/soil_data_module.f90, `type soil_database` (which holds `type soil_profile_db` plus an allocatable array of `type soilayer_db`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Each soil is a multi-record block:
  - One profile record with 7 fields.
  - `nly` layer records that follow, each with 14 fields. `nly` is read from the second field of the profile record and used to allocate the layer array.
- Records are list-directed (free format). Column positions in the reference example are cosmetic indentation only; the reader does not depend on them.

Profile record (7 fields)

Read into `soildb(i)%s` (`type soil_profile_db`).

#	Field	Type	Units	Default	Description
1	snam	char(20)	none	blank	soil class name. Referenced from `hru-data.hru`
2	nly	int	none	1	number of soil layers that follow this record
3	hydgrp	char(16)	none	“A”	hydrologic soil group (A, B, C, or D)
4	zmx	real	mm	1500.0	maximum rooting depth
5	anion_excl	real	fraction	0.5	fraction of porosity from which anions are excluded
6	crk	real	fraction	0.01	crack volume potential of the soil
7	texture	char(16)	none	blank	texture descriptor string (free-form label)

Layer record (14 fields, repeated `nly` times)

Read into `soildb(i)%ly(j)` (`type soilayer_db`) for `j = 1..nly`.

#	Field	Type	Units	Default	Description
1	z	real	mm	1500.0	depth from soil surface to the bottom of the layer
2	bd	real	Mg/m^3	1.3	moist bulk density
3	awc	real	mm H2O / mm soil	0.2	available water capacity
4	k	real	mm/hr	10.0	saturated hydraulic conductivity
5	cbn	real	%	2.0	organic carbon content of the layer
6	clay	real	%	10.0	clay fraction of mineral material
7	silt	real	%	60.0	silt fraction of mineral material
8	sand	real	%	30.0	sand fraction of mineral material
9	rock	real	%	0.0	rock fragment content of the layer
10	alb	real	none	0.1	moist soil albedo
11	usle_k	real	none	0.2	USLE soil erodibility (K) factor
12	ec	real	dS/m	0.0	electrical conductivity
13	cal	real	%	0.0	CaCO3 content
14	ph	real	none	0.0	soil pH

The reader applies one fix-up after reading: if the first layer is thinner than 20 mm and either the profile has only one layer or the second layer is deeper than 20 mm, the depth of layer 1 is forced to 20 mm.

Example

Excerpt from `refdata/Ames_sub1/soils.sol`. The first soil class has four layers:

```
soils.sol Ames
   name                NLY  HYD_GRP        ZMX     ANION_EXCL      CRK       TEXTURE        DEPTH         BD
soil_01                  4        B   2000.000          0.500    0.500       L- L- L- L
                                                                                             50.00      1.260
                                                                                            150.00      1.420
                                                                                            840.00      1.500
                                                                                           2000.00      1.600
```

Notes:

- The profile record `soil_01 4 B 2000.000 0.500 0.500 "L- L- L- L"` declares 4 layers. The next four physical lines are the layer records.
- Each layer record has 14 numeric values. The `TEXTURE` token on the profile line (`L- L- L- L` here) is one space-separated token only because the reader reads `texture` as a single character field; the writer pads it with hyphen-separated horizon codes.
- Header tokens in the file (`DEPTH`, `BD`, `AWC`, `K`, `CBN`, `CLAY`, `SILT`, `SAND`, `ROCK`, `ALB`, `USLE_K`, `EC`, `CAL`, `PH`) are read past as a single skipped line. The column order is fixed by the source.

Related files

- `hru-data.hru`. Each HRU points to a soil class name in `soils.sol`.
- `nutrients.sol`. Soil chemistry initialization paired with a soil class.
- `plants.plt`. Rooting depth in `soils.sol` (`zmx`) caps plant root depth.

Purpose

`landuse.lum` is the master table of land use and management classes. Each row defines one class by name and lists string pointers to records in other files: the plant community (`plants.ini`), management schedule (`management.sch`), curve number entry (`cntable.lum`), conservation practice (`cons_practice.lum`), urban land use (`urban.urb`), overland-flow Manning's n class (`ovn_table.lum`), tile drainage (`tiledrain.str`), septic (`septic.str`), filter strip (`filterstrip.str`), grassed waterway (`grassedww.str`), and user BMP (`bmpuser.str`). HRUs reference a row in this table through `hru-data.hru`. After reading, the reader resolves every non-`null` string pointer to an integer index stored in `lum_str`; unresolved names are logged to `simulation.out` (unit 9001).

Source

- Reader: `src/landuse_read.f90`
- Type definition: `src/landuse_data_module.f90`, `type land_use_management` (instances stored in `lum`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- One record per land use class. Records are list-directed (free format). All 14 fields are read on every row.
- Any pointer field that should be empty must be the literal string `null`. The reader treats `null` as "no link" and does not try to resolve it.

#	Field	Type	Default	Description
1	name	char(40)	blank	name of the land use and management class. Referenced from `hru-data.hru`
2	cal_group	char(40)	blank	calibration group label. Not currently used by the solver
3	plant_cov	char(40)	blank	plant community pointer into `plants.ini`. Resolved to `lum_str%plant_cov`
4	mgt_ops	char(40)	blank	management schedule pointer into `management.sch`. Resolved to `lum_str%mgt_ops`
5	cn_lu	char(40)	blank	curve number entry pointer into `cntable.lum`. Resolved to `lum_str%cn_lu`
6	cons_prac	char(40)	blank	conservation practice pointer into `cons_practice.lum`. Resolved to `lum_str%cons_prac`
7	urb_lu	char(40)	blank	urban land use pointer into `urban.urb` (e.g. `URLD`, `URHD`). Resolved during urban init
8	urb_ro	char(40)	blank	urban runoff model. `usgs_reg` for USGS regression equations, `buildup_washoff` for build-up/wash-off
9	ovn	char(40)	blank	overland-flow Manning's n class pointer into `ovn_table.lum`
10	tiledrain	char(40)	blank	tile drainage pointer into `tiledrain.str`. Resolved to `lum_str%tiledrain`
11	septic	char(40)	blank	septic pointer into `septic.str`. Resolved to `lum_str%septic`
12	fstrip	char(40)	blank	filter strip pointer into `filterstrip.str`. Resolved to `lum_str%fstrip`
13	grassww	char(40)	blank	grassed waterway pointer into `grassedww.str`. Resolved to `lum_str%grassww`
14	bmpuser	char(40)	blank	user-defined BMP pointer into `bmpuser.str`. Resolved to `lum_str%bmpuser`

The reader logs a line to `simulation.out` for every non-`null` pointer that fails to match a name in the target file. The simulation continues with the link unresolved (index zero), which means the corresponding behaviour is skipped.

Example

Excerpt from `refdata/Ames_sub1/landuse.lum`:

```
landuse.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                          cal_group          plnt_com                                  mgt
cosy_lum                           null         cosy_comm                              mgt_01
agrr_lum                           null         agrr_comm                           AGRR_rot
frst_lum                           null         frst_comm                              null
urld_lum                           null              null                              null
```

Notes:

- The header tokens written by SWAT+ Editor (`plnt_com`, `cn2`, `urban`, `ov_mann`, `tile`, `sep`, `vfs`, `grww`, `bmp`) do not match the field names in the source (`plant_cov`, `cn_lu`, `urb_lu`, `ovn`, `tiledrain`, `septic`, `fstrip`, `grassww`, `bmpuser`). The reader skips the header and uses position only.
- `urld_lum` is an urban class. Its `plant_cov` and `mgt_ops` are both `null`; the row links instead to `URLD` in `urban.urb` and selects `buildup_washoff` as the urban runoff model.
- `frst_lum` has no management schedule (`mgt_ops = null`). Forest classes typically rely on the plant community initialization alone.

Related files

- hru-data.hru. Each HRU points to one `landuse.lum` row by name.
- cntable.lum. Curve number table referenced by the `cn_lu` field. Each row holds a name and four curve numbers (hydrologic conditions A through D).
- cons_practice.lum. Conservation practice table referenced by `cons_prac`. Holds USLE P factor and maximum slope length per practice.
- ovn_table.lum. Overland-flow Manning's n table referenced by `ovn`. Holds mean, minimum, and maximum n per class.
- plants.ini. Target of `plant_cov`.
- management.sch. Target of `mgt_ops`.
- `urban.urb`. Target of `urb_lu`.
- `tiledrain.str`, `septic.str`, `filterstrip.str`, `grassedww.str`, `bmpuser.str`. Structural BMP files targeted by the last five pointer columns.

Three lookup files supply numeric parameters that landuse.lum references by name: `cntable.lum` (SCS curve numbers), `cons_practice.lum` (USLE P factor and maximum slope length), and `ovn_table.lum` (overland-flow Manning's n). Each file has the same shape: a title line, a header line, then one record per row. Rows are read with list-directed I/O, so any column after the last typed field is ignored by the solver.

cntable.lum

Curve number table. Each entry holds four SCS-CN values for hydrologic soil groups A, B, C, and D.

Source

- Reader: src/cntbl_read.f90
- Type: src/landuse_data_module.f90, `type curvenumber_table` (instances stored in `cn`)
- Sized into `db_mx%cn_lu` after the file is read.

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- One record per curve number entry. Fields are list-directed.

#	Field	Type	Default	Description
1	name	char(40)	blank	curve number entry name. Referenced from `landuse.lum` field `cn_lu`
2	cn(1)	real	30.0	curve number for hydrologic soil group A
3	cn(2)	real	55.0	curve number for hydrologic soil group B
4	cn(3)	real	70.0	curve number for hydrologic soil group C
5	cn(4)	real	77.0	curve number for hydrologic soil group D

Curve numbers are dimensionless and bounded by 0 and 100. SWAT+ Editor writes additional `description`, `treat`, and `cond_cov` columns after `cn(4)`. The reader does not consume them.

Example

Excerpt from `refdata/Ames_sub1/cntable.lum`:

```
cntable.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                      cn_a          cn_b          cn_c          cn_d  description
fal_bare               77.00000      86.00000      91.00000      94.00000  Fallow
rc_strow_g             67.00000      78.00000      85.00000      89.00000  Row_crops
wood_f                 36.00000      60.00000      73.00000      79.00000  Woods
urban                  98.00000      98.00000      98.00000      98.00000  Paved_parking_lots_roofs_drivewa
```

cons_practice.lum

Conservation practice table. Supplies the USLE support practice factor (P) and the maximum slope length used for length-slope factor calculations.

Source

- Reader: src/cons_prac_read.f90
- Type: src/landuse_data_module.f90, `type conservation_practice_table` (instances stored in `cons_prac`)
- Sized into `db_mx%cons_prac` after the file is read.

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- One record per practice. Fields are list-directed.

#	Field	Type	Default	Units	Description
1	name	char(40)	blank	-	practice name. Referenced from `landuse.lum` field `cons_prac`
2	pfac	real	1.0	-	USLE support practice factor (P). Dimensionless, 0 to 1
3	sl_len_mx	real	1.0	m	maximum slope length for the practice

SWAT+ Editor writes a `description` column after `sl_len_mx`. The reader does not consume it.

Example

Excerpt from `refdata/Ames_sub1/cons_practice.lum`:

```
cons_practice.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                    usle_p   slp_len_max  description
up_down_slope          1.00000     121.00000  Up_and_down_slope
cross_slope            0.75000     121.00000  Cross_slope_tillage
contour_farming        0.50000     121.00000  Contour_tillage
strip_contour          0.25000     121.00000  Strip_cropping_contour
ter_3-8_sodout         0.50000      76.00000  terraces_3-8%_slopes_sod-outlet
```

ovn_table.lum

Overland-flow Manning's n table. Supplies a mean, minimum, and maximum n per land use class for sheet-flow routing across the HRU.

Source

- Reader: src/overland_n_read.f90
- Type: src/landuse_data_module.f90, `type overlandflow_n_table` (instances stored in `overland_n`)
- Sized into `db_mx%ovn` after the file is read.

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- One record per class. Fields are list-directed.

#	Field	Type	Default	Units	Description
1	name	char(40)	blank	-	class name. Referenced from `landuse.lum` field `ovn`
2	ovn	real	0.5	s m^(-1/3)	mean Manning's n for overland flow
3	ovn_min	real	0.5	s m^(-1/3)	minimum Manning's n for overland flow
4	ovn_max	real	0.5	s m^(-1/3)	maximum Manning's n for overland flow

SWAT+ Editor writes a `description` column after `ovn_max`. The reader does not consume it.

Example

Excerpt from `refdata/Ames_sub1/ovn_table.lum`:

```
ovn_table.lum: written by SWAT+ editor v2.3.3 on 2023-10-25 08:58 for SWAT+ rev.60.5.7
name                  ovn_mean       ovn_min       ovn_max  description
convtill_nores         0.09000       0.06000       0.12000  Conventional_tillage_no_residue
convtill_res           0.19000       0.16000       0.22000  Conventional_tillage_residue
notill_2-9res          0.30000       0.17000       0.47000  No_till_2-9_t/ha_residue
forest_med             0.60000       0.50000       0.70000  Forest_medimum_good
urban_asphalt          0.01100       0.01100       0.01100  Urban_asphalt
```

Related files

- landuse.lum. Master land-use management table. Its `cn_lu`, `cons_prac`, and `ovn` columns name rows in these three files.
- hru-data.hru. Selects a `landuse.lum` row per HRU, which in turn selects rows in these three tables.

Purpose

This page documents the legacy channel input files: `channel.cha` (original reach pointer), `channel-lte.cha` (SWAT+ deg routing pointer), `hyd-sed-lte.cha` (combined hydrology and sediment table used by the deg routine), `initial.cha` (initial-state pointer), and `temperature.cha` (water temperature parameters). All file names are set in `type input_cha` in `src/input_file_module.f90`. Each file is independently optional: if the slot in `file.cio` is `null` or the file is missing, the reader allocates an empty array and skips on.

The three companion tables that these pointer files reference (`hydrology.cha`, `sediment.cha`, `nutrients.cha`) are documented on the sd_channel page.

Source

- File names: src/input_file_module.f90, `type input_cha`
- Readers:
  - src/ch_read.f90 (`channel.cha`)
  - src/sd_channel_read.f90 (`channel-lte.cha`)
  - src/sd_hydsed_read.f90 (`hyd-sed-lte.cha`)
  - src/ch_read_init.f90 (`initial.cha`)
  - src/ch_read_temp.f90 (`temperature.cha`)
- Type definitions: src/channel_data_module.f90, src/sd_channel_module.f90

Every reader skips two header lines, then reads one record per channel definition.

channel.cha

Pointer file for the original SWAT-style reach routing. Each record names a channel and points to entries in `initial.cha`, `hydrology.cha`, `sediment.cha`, and `nutrients.cha`. Read into `ch_dat_c` (`type channel_data_char_input`).

#	Column	Type	Description
1	id	int	record number (echoed back)
2	name	char(16)	channel name
3	init	char(16)	name of an `initial.cha` entry, or `null`
4	hyd	char(16)	name of a `hydrology.cha` entry, or `null`
5	sed	char(16)	name of a `sediment.cha` entry, or `null`

#	Column	Type	Description
6	nut	char(16)	name of a `nutrients.cha` entry, or `null`

After reading, the reader resolves each character pointer to an integer index by scanning `ch_init`, `ch_hyd`, `ch_sed`, and `ch_nut`. Missing names are written to the diagnostics file (unit 9001) with messages like `"<name> not found (hydrology.cha)"`. The reader does not abort; the index is left at 0 and downstream code uses the default record.

No example dataset shipping with SWAT+ uses `channel.cha`. The two reference datasets at `refdata/Osu_1hru` and (if present) `refdata/Ames_sub1` configure this slot as `null` in `file.cio` and use `channel-lte.cha` instead.

channel-lte.cha

Pointer file for SWAT+ deg routing. Listed under `chandeg.con`. Each record points to entries in `initial.cha`, `hyd-sed-lte.cha`, an optional sediment table, and `nutrients.cha`. Read into `sd_dat` (`type swatdeg_datafiles`).

#	Column	Type	Description
1	id	int	record number used to position the array element
2	name	char(16)	channel name
3	cha_ini	char(16)	name of an `initial.cha` entry, or `null`
4	cha_hyd	char(16)	name of a `hyd-sed-lte.cha` entry, or `null`
5	cha_sed	char(16)	name of a sediment-nutrient entry (resolved against `sed_nut.cha` when present)
6	cha_nut	char(16)	name of a `nutrients.cha` entry, or `null`

The `cha_hyd` column is matched against names in both `sd_chd` (`hyd-sed-lte.cha`) and `sd_chd1` (`sed_nut.cha`). The `cha_sed` column is read but not resolved to an index by `sd_channel_read`; sediment behaviour follows the `hyd-sed-lte.cha` parameters.

Example from `refdata/Osu_1hru/channel-lte.cha`:

```
channel-lte.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                        cha_ini           cha_hyd           cha_sed           cha_nut
       1  cha01                      initcha1          hydcha01              null          nutcha1
       2  cha03                      initcha1          hydcha03              null          nutcha1
       3  cha04                      initcha1          hydcha04              null          nutcha1
```

hyd-sed-lte.cha

Combined hydrology and bank-erosion parameters for the SWAT+ deg routing routine. One record per channel hydrology profile. Read into `sd_chd` (`type swatdeg_hydsed_data`).

#	Column	Type	Units	Default	Description
1	name	char(25)	none	“”	parameter set name (referenced by `channel-lte.cha`)
2	order	int	none	0	stream order
3	chw	real	m	0.0	channel width
4	chd	real	m	0.0	channel depth
5	chs	real	m/m	0.0	channel slope
6	chl	real	km	0.0	channel length
7	chn	real	none	0.0	channel Manning’s n
8	chk	real	mm/hr	0.0	channel bottom hydraulic conductivity
9	bank_exp	real	none	0.0	bank erosion exponent (column `erod_fact` in editor output)
10	cov	real	0..1	0.0	channel cover factor
11	sinu	real	none	0.0	sinuosity (channel length / straight-line length)
12	vcr_coef	real	none	0.0	critical velocity coefficient (column `eq_slp` in editor output)
13	d50	real	mm	0.0	median sediment diameter
14	ch_clay	real	percent	0.0	clay percent of bank and bed
15	carbon	real	percent	0.0	carbon percent of bank and bed
16	ch_bd	real	t/m^3	0.0	dry bulk density
17	chss	real	none	0.0	channel side slope
18	bankfull_flo	real	m^3/s	0.0	bank-full flow rate (column `bed_load` in editor output)
19	fps	real	m/m	0.000001	flood plain slope
20	fpn	real	none	0.1	flood plain Manning’s n
21	n_conc	real	mg/kg	0.0	nitrogen concentration in channel bank
22	p_conc	real	mg/kg	0.0	phosphorus concentration in channel bank
23	p_bio	real	fraction	0.0	fraction of bank P that is bioavailable

Example from `refdata/Osu_1hru/hyd-sed-lte.cha`:

```
hyd-sed-lte.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                          order            wd            dp           slp           len          mann
hydcha01                          4      27.17726       0.99164       0.00146       3.42760       0.05000
```

Editor column names differ from source field names. The reader uses position only.

initial.cha

Initial state pointer file. Each record names a set of initial-condition files for organic-mineral mass, pesticides, pathogens, heavy metals, and salts. Read into `ch_init` (`type channel_init_datafiles`).

#	Column	Type	Description
1	name	char(16)	initial-state set name
2	org_min	char(16)	initial organic-mineral input file name, or `null`
3	pest	char(16)	initial pesticide input file name, or `null`
4	path	char(16)	initial pathogen input file name, or `null`
5	hmet	char(16)	initial heavy metals input file name, or `null`
6	salt	char(16)	initial salt input file name, or `null`

The reader uses unformatted list-directed reads with no leading integer id column. The record is read directly into the derived type, so the first token on each line is the `name`.

Example from `refdata/Osu_1hru/initial.cha`:

```
initial.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        org_min             pest             path             hmet             salt
initcha1                    no_init             null             null             null             null
```

A separate file `initial.cha_cs` is read by `ch_read_init_cs` for salt and conservative-species initial conditions. The file name is hard-coded in the reader (not in `type input_cha`).

temperature.cha

Water temperature parameters used by the channel temperature routine. Read into `w_temp` (`type water_temperature_data`). The `name` field is `character(len=13)` in the source; longer names will be truncated.

#	Column	Type	Units	Default	Description
1	name	char(13)	none	“”	parameter set name
2	sno_mlt	real	none	1.0	coefficient on snowmelt temperature contribution
3	gw	real	none	1.0	coefficient on groundwater temperature contribution
4	sur_lat	real	none	1.0	coefficient on surface and lateral flow temperature contribution
5	sno_lag	real	days	2.0	air temperature lag to snowmelt (1..3)
6	gw_lag	real	days	250.0	air temperature lag to groundwater flow (200..365)
7	surf_lag	real	days	5.0	air temperature lag to surface runoff (2..5)
8	lat_lag	real	days	10.0	air temperature lag to lateral flow (5..10)
9	lat_lag_coef	real	none	0.75	lateral air-lag coefficient
10	surf_lag_coef	real	none	0.75	surface air-lag coefficient (also used for snow)
11	gw_lag_coef	real	none	0.75	groundwater air-lag coefficient
12	hex_coef1	real	none	0.75	dew point calibration
13	hex_coef2	real	none	1.5	channel geometry calibration
14	sf_on	int	none	0	shade factor file activation. `1` = read from file, `0` = use value from calibration file
15	ssff	real	0..1	0.5	shade factor value (used when `sf_on = 0`)

No example dataset in `refdata/` contains `temperature.cha`. The slot is `null` in `file.cio` for the included reference datasets.

Related files

- `file.cio`. The `channel` line selects which of these files are read.
- `chandeg.con`. Connectivity file that references `channel-lte.cha` entries by name.
- `channel.con`. Connectivity file that references `channel.cha` entries by name.
- sd_channel companion tables (`hydrology.cha`, `sediment.cha`, `nutrients.cha`).

Purpose

This page documents the three per-channel parameter tables that hold hydraulic, sediment, and nutrient coefficients used inside the channel routing modules: `hydrology.cha`, `sediment.cha`, and `nutrients.cha`. Each is a flat database. Records are referenced by name from the pointer files `channel.cha` (legacy) and `channel-lte.cha` (SWAT+ deg). When the SWAT+ deg routine routes water and constituents through reaches listed under `chandeg.con`, the parameters in these tables drive the in-channel processes.

File names are declared in `type input_cha` in `src/input_file_module.f90` (`hyd`, `sed`, `nut`). Each file is optional. If the slot in `file.cio` is `null` or the file is missing, the reader allocates an empty array and the matching pointer is left at index 0.

Source

- File names: `src/input_file_module.f90`, `type input_cha`
- Readers:
  - `src/ch_read_hyd.f90`
  - `src/ch_read_sed.f90`
  - `src/ch_read_nut.f90`
- Type definitions: `src/channel_data_module.f90`, types `channel_hyd_data`, `channel_sed_data`, `channel_nut_data`

Every reader skips two header lines, then reads one record per parameter set.

hydrology.cha

Hydraulic and bank parameters for the original SWAT-style reach routine. Read into `ch_hyd` (`type channel_hyd_data`). The reader rewrites a few fields after read: `alpha_bnk` is replaced with `exp(-alpha_bnk)`, `s` is floored at 0.0001, `n` is clamped to `[0.01, 0.70]`, `l` is floored at 0.0010, `wdr` defaults to 3.5 if non-positive, and `side` defaults to 2.0 if non-positive.

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“default”	parameter set name
2	w	real	m	2.0	average width of main channel
3	d	real	m	0.5	average depth of main channel
4	s	real	m/m	0.01	average slope of main channel
5	l	real	km	0.1	main channel length in subbasin
6	n	real	none	0.05	Manning’s n
7	k	real	mm/hr	0.01	effective hydraulic conductivity of channel alluvium
8	wdr	real	m/m	6.0	channel width-to-depth ratio
9	alpha_bnk	real	days	0.03	alpha factor for bank storage recession (read as days, stored as `exp(-alpha_bnk)`)
10	side	real	none	0.0	channel side slope (horizontal per unit vertical)

No example dataset in `refdata/` ships with `hydrology.cha`. The included reference datasets configure this slot as `null` in `file.cio` and use `hyd-sed-lte.cha` instead.

sediment.cha

Channel sediment routing parameters. Read into `ch_sed` (`type channel_sed_data`). The 12 monthly `erod` values follow the scalar fields. The reader clamps several values: cover and erodibility factors are bounded by the routing equation choice, bank and bed `d50` default to 50 and 500 micrometers if non-positive, bank and bed bulk density default to 1.40 and 1.50 g/cc if non-positive, and bank and bed jet-test erodibility default to 0.2 (or `0.2 / sqrt(tc_*)` if a critical shear stress is given). If all 12 monthly `erod` values are effectively zero, the reader fills the monthly array with `cov1`.

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“”	parameter set name
2	eqn	int	none	0	sediment routing method. `0` original SWAT, `1` Bagnold, `2` Kodatie, `3` Molinas-Wu, `4` Yang
3	cov1	real	none	0.1	channel erodibility factor (0..1 when `eqn = 0`, 0..25 otherwise)
4	cov2	real	none	0.1	channel cover factor (same bounds as `cov1`)
5	bnk_bd	real	g/cc	0.0	bulk density of channel bank sediment (1.1..1.9)
6	bed_bd	real	g/cc	0.0	bulk density of channel bed sediment (1.1..1.9)
7	bnk_kd	real	cm^3/N/s	0.0	bank erodibility from jet test
8	bed_kd	real	cm^3/N/s	0.0	bed erodibility from jet test
9	bnk_d50	real	micrometers	0.0	bank median particle diameter (clamped to 10000 max)
10	bed_d50	real	micrometers	0.0	bed median particle diameter (clamped to 10000 max)
11	tc_bnk	real	N/m^2	0.0	critical shear stress of channel bank
12	tc_bed	real	N/m^2	0.0	critical shear stress of channel bed
13..24	erod(1..12)	real	none	0.0	monthly channel erodibility factor. `0.0` = non-erosive, `1.0` = no resistance to erosion

No example dataset in `refdata/` ships with `sediment.cha`.

nutrients.cha

In-stream nutrient cycling parameters. Read into `ch_nut` (`type channel_nut_data`). After read, several rate constants are scaled by the routing time step (`time%step`) so that values defined per day are converted to per-substep. The light half-saturation `k_l` is converted from kJ/(m^2 min) to MJ/(m^2 hr) by multiplying by 0.06. Defaults are applied (in place of zero) for every coefficient.

#	Column	Type	Units	Default	Description
1	name	char(16)	none	“”	parameter set name
2	onco	real	ppm	0.0	channel organic N concentration
3	opco	real	ppm	0.0	channel organic P concentration
4	rs1	real	m/day	1.0	local algal settling rate at 20 deg C
5	rs2	real	(mg disP-P)/(m^2 day)	0.05	benthos source rate for dissolved P at 20 deg C
6	rs3	real	(mg NH4-N)/(m^2 day)	0.5	benthos source rate for ammonia at 20 deg C
7	rs4	real	1/day	0.05	organic N settling rate at 20 deg C
8	rs5	real	1/day	0.05	organic P settling rate at 20 deg C
9	rs6	real	1/day	2.5	settling rate for arbitrary non-conservative constituent
10	rs7	real	(mg ANC)/(m^2 day)	2.5	benthal source rate for arbitrary non-conservative constituent
11	rk1	real	1/day	1.71	CBOD deoxygenation rate at 20 deg C
12	rk2	real	1/day	1.0	reaeration rate (Fickian) at 20 deg C
13	rk3	real	1/day	2.0	CBOD loss rate due to settling at 20 deg C
14	rk4	real	(mg O2)/(m^2 day)	0.0	sediment oxygen demand at 20 deg C
15	rk5	real	1/day	1.71	coliform die-off rate
16	rk6	real	1/day	1.71	decay rate for arbitrary non-conservative constituent
17	bc1	real	1/hr	0.55	NH3 to NO2 biological oxidation rate at 20 deg C
18	bc2	real	1/hr	1.1	NO2 to NO3 biological oxidation rate at 20 deg C
19	bc3	real	1/hr	0.21	organic N to ammonia hydrolysis rate at 20 deg C
20	bc4	real	1/hr	0.35	organic P to dissolved P decay rate at 20 deg C
21	lao	real	none	2.0	QUAL2E light averaging option. Only option 2 is implemented in SWAT
22	igropt	int	none	2	algal growth limiting equation. 1 multiplicative, 2 limiting nutrient, 3 harmonic mean
23	ai0	real	ug chla / mg alg	50.0	ratio of chlorophyll-a to algal biomass
24	ai1	real	mg N / mg alg	0.08	nitrogen fraction of algal biomass
25	ai2	real	mg P / mg alg	0.015	phosphorus fraction of algal biomass
26	ai3	real	mg O2 / mg alg	1.60	rate of O2 production per unit of algal photosynthesis
27	ai4	real	mg O2 / mg alg	2.0	rate of O2 uptake per unit of algal respiration
28	ai5	real	mg O2 / mg N	3.5	rate of O2 uptake per unit of NH3 oxidation
29	ai6	real	mg O2 / mg N	1.07	rate of O2 uptake per unit of NO2 oxidation
30	mumax	real	1/hr	2.0	maximum specific algal growth rate at 20 deg C
31	rhoq	real	1/day	2.5	algal respiration rate
32	tfact	real	none	0.3	fraction of solar radiation that is photosynthetically active
33	k_l	real	kJ/(m^2 min) read, stored as MJ/(m^2 hr)	0.75	light half-saturation coefficient
34	k_n	real	mg N / L	0.02	Michaelis-Menten N half-saturation constant

#	Column	Type	Units	Default	Description
35	k_p	real	mg P / L	0.025	Michaelis-Menten P half-saturation constant
36	lambda0	real	1/m	1.0	non-algal portion of the light extinction coefficient
37	lambda1	real	1/(m * ug chla/L)	0.03	linear algal self-shading coefficient
38	lambda2	real	(1/m)(ug chla/L)^(-2/3)	0.054	nonlinear algal self-shading coefficient
39	p_n	real	none	0.5	algal preference factor for ammonia

Example from `refdata/Osu_1hru/nutrients.cha`:

```
nutrients.cha: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                      plt_n         ptl_p       alg_stl      ben_disp      ben_nh3n      ptln_stl      
nutcha1                 0.00000       0.00000       1.00000       0.05000       0.50000       0.05000
```

Editor column names (`plt_n`, `ptl_p`, `alg_stl`, …) do not match the source field names (`onco`, `opco`, `rs1`, …). The reader uses position only.

Notes on the Osu_1hru example:

- Position 11 in this dataset (`cbn_bod_co = 1.71`) is `rk1` in the source.
- Position 13 (`cbn_bod_stl = 0.36`) is `rk3`. The source default is 2.0; this dataset overrides it.
- Position 14 (`ben_bod = 2.0`) is `rk4`. The source default is 0.0; this dataset overrides it.
- Positions 21 and 22 (`q2e_lt = 2`, `q2e_alg = 2`) are `lao` (read as real) and `igropt` (read as int).

Related files

- `*.cha` legacy channel files (`channel.cha`, `channel-lte.cha`, `hyd-sed-lte.cha`, `initial.cha`, `temperature.cha`). The pointer files that select records from `hydrology.cha`, `sediment.cha`, and `nutrients.cha` by name.
- `chandeg.con`. Connectivity file for SWAT+ deg routing.
- `channel.con`. Connectivity file for the original SWAT-style routing.
- `file.cio`. The `channel` line lists the file names that occupy each slot in `type input_cha`.

Purpose

The aquifer group has two input files declared in `type input_aqu` (`src/input_file_module.f90`):

- `aquifer.aqu` defines one record per aquifer object. Each record sets baseflow recession parameters, initial water table depth, initial chemistry, and the recharge and revap controls used by the lumped (non-gwflow) aquifer module. Records are read into `aqudb` (an array of `type aquifer_database`).
- `initial.aqu` is a small table of named initial-condition pointers. Each entry points to organic-mineral, pesticide, pathogen, heavy-metal, and salt initialization files. The `init` column of each row in `aquifer.aqu` selects one of these names.

Source

- Readers: `src/aqu_read.f90`, `src/aqu_read_init.f90`
- Type definitions: `src/aquifer_module.f90`, `type aquifer_database` and `type aquifer_init_data_char`
- Filename declarations: `src/input_file_module.f90`, `type input_aqu`

aquifer.aqu

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per aquifer with list-directed I/O. The first integer on each record is the aquifer id; the reader uses it as the index into `aqudb`. The remaining columns are read into the fields of `type aquifer_database` in declaration order.

#	Field	Type	Units	Default	Description
1	id	int	none	-	aquifer id. Used as the array index into `aqudb`
2	aqunm	char(16)	none	“”	aquifer name
3	aqu_ini	char(16)	none	“”	initial-condition pointer. Cross-walks with `name` in `initial.aqu`
4	flo	real	mm	0.05	flow from aquifer in the current time step
5	dep_bot	real	m	0.0	depth from mid-slope surface to bottom of aquifer
6	dep_wt	real	m	0.0	initial depth from mid-slope surface to water table
7	no3	real	ppm N	0.0	initial nitrate-N concentration in aquifer
8	minp	real	ppm P	0.0	initial mineral phosphorus concentration in aquifer
9	cbn	real	percent	0.5	initial organic carbon in aquifer
10	flo_dist	real	m	50.0	average flow distance to stream or downstream object
11	bf_max	real	mm	0.0	maximum daily baseflow when all channels are contributing
12	alpha	real	1/day	0.0	lag factor for the groundwater recession curve
13	revap_co	real	none	0.0	revap coefficient. `evap = pet * revap_co`
14	seep	real	fraction	0.0	fraction of recharge that seeps from the aquifer to the deep store
15	spyld	real	$m^{3/m}3$	0.0	specific yield of the aquifer
16	hlife_n	real	days	30.0	half-life of nitrogen in groundwater
17	flo_min	real	m	0.0	water table depth above which return flow occurs
18	revap_min	real	m	0.0	water table depth above which revap occurs

Example

`refdata/Osu_1hru/aquifer.aqu`:

```
aquifer.aqu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
      id  name                          init        gw_flo       dep_bot        dep_wt         no3_n
       1  aqu011                     initaqu1       0.05000      10.00000       6.00000       0.00000
       2  aqu012                     initaqu1       0.05000      10.00000       6.00000       0.00000
      21  aqu_deep010                initaqu1       0.00000      10.00000      20.00000       0.00000
```

Notes:

- The tool header names (`gw_flo`, `no3_n`, `sol_p`, `carbon`, `alpha_bf`, `revap`, `rchg_dp`, `spec_yld`, `hl_no3n`) differ from the source field names (`flo`, `no3`, `minp`, `cbn`, `alpha`, `revap_co`, `seep`, `spyld`, `hlife_n`). Column order is what matters.
- The `init` column points to `initaqu1`, which must exist as a row in `initial.aqu`.
- In this dataset all shallow aquifers share the same property values. The last record (`aqu_deep010`) represents the deep aquifer with no baseflow (`flo = 0`, `flo_min = 0`, `revap_min = 0`).

initial.aqu

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per initial-condition set with list-directed I/O. Records are read into `aqu_init_dat_c` (an array of `type aquifer_init_data_char`).

#	Field	Type	Description
1	name	char(16)	initial set name. Cross-walks with `aqu_ini` in `aquifer.aqu`
2	org_min	char(16)	name of organic-mineral initial-state record
3	pest	char(16)	name of pesticide initial-state record
4	path	char(16)	name of pathogen initial-state record
5	hmet	char(16)	name of heavy-metal initial-state record
6	salt	char(16)	name of salt initial-state record

The literal `null` is accepted in slots that have no initial-state file.

Example

`refdata/Osu_1hru/initial.aqu`:

```
initial.aqu: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                       org_min             pest             path             hmet             salt
initaqu1                   no_init             null             null             null             null
```

Notes:

- The tool writes a trailing `description` column. The source type has no `description` field. List-directed reads consume tokens left to right, so the extra trailing token is harmless. Do not rely on the description being read.
- `no_init` is itself a name that must resolve in the organic-mineral initial file. `null` is treated as "no file".

Related files

- aquifer.con. Connectivity entries that map each aquifer id to its spatial location and downstream object.
- `aqu_catunit.ele` and `aqu_catunit.def`. Catchment-unit elements that group aquifers.
- Organic-mineral, pesticide, pathogen, heavy-metal, and salt initial-state files referenced by `initial.aqu`.

Purpose

The reservoir group has six input files declared in `type input_res` (`src/input_file_module.f90`):

- `reservoir.res` is the object table. One record per reservoir, naming the hydrology, sediment, nutrient, release, and initial-condition records that apply.
- `initial.res` is a table of named initial-condition pointers (organic-mineral, pesticide, pathogen, heavy-metal, salt) shared with the wetland module.
- `hydrology.res` is the hydrology database. One record per named hydrology profile.
- `sediment.res` is the sediment-quality database.
- `nutrients.res` is the nutrient settling and loss-rate database.
- `weir.res` is the weir outflow database for reservoirs that release through a weir.

All five database files are referenced by name from `reservoir.res`. Names are not positional. The reader builds an index after all files are loaded.

Source

- Readers:
  - src/res_read.f90
  - src/res_read_init.f90
  - src/res_read_hyd.f90
  - src/res_read_sed.f90
  - src/res_read_nut.f90
  - src/res_read_weir.f90
- Type definitions: src/reservoir_data_module.f90
- Filename declarations: src/input_file_module.f90, `type input_res`

reservoir.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per reservoir with list-directed I/O. The first integer on each record is the reservoir id; the reader uses it as the index into `res_dat_c`. The remaining columns are read into `type reservoir_data_char_input` in declaration order.

#	Field	Type	Description
1	id	int	reservoir id. Used as the array index into `res_dat_c`
2	name	char(25)	reservoir name
3	init	char(25)	initial-state pointer. Cross-walks with `init` in `initial.res`
4	hyd	char(25)	hydrology pointer. Cross-walks with `name` in `hydrology.res`
5	release	char(25)	release rule pointer. Either a decision-table name in `res_rel.dtl`, or a name prefixed with `ctbl_` for a conditional table
6	sed	char(25)	sediment pointer. Cross-walks with `name` in `sediment.res`
7	nut	char(25)	nutrient pointer. Cross-walks with `name` in `nutrients.res`

The `release` lookup branches on the literal prefix `ctbl_`. If present, the name is matched against `ctbl(:)%name` and the release mode is set to `c`. Otherwise it is matched against `dtbl_res(:)%name` and the release mode is set to `d`.

initial.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per initial-condition set with list-directed I/O. Records are read into `res_init_dat_c` (an array of `type reservoir_init_data_char`).

#	Field	Type	Description
1	init	char(25)	initial set name. Cross-walks with `init` in `reservoir.res` (and `wetland.wet`)
2	org_min	char(25)	name of organic-mineral initial-state record
3	pest	char(25)	name of pesticide initial-state record
4	path	char(25)	name of pathogen initial-state record
5	hmet	char(25)	name of heavy-metal initial-state record
6	salt	char(25)	name of salt initial-state record

The literal `null` is accepted in slots that have no initial-state file.

Example

`refdata/Osu_1hru/initial.res`:

```
initial.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                      org_min             pest             path             hmet             salt
initwet1                  no_init             null             null             null             null
```

The first column header in the example is `name` but the source field is `init` (the value it must match in `reservoir.res`). The source uses position. The editor also writes a trailing `description` column that is not read.

hydrology.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per hydrology profile with list-directed I/O. Records are read into `res_hyddb` (an array of `type reservoir_hyd_data`) in declaration order.

#	Field	Type	Units	Default	Description
1	name	char(25)	none	“default”	hydrology profile name
2	iyres	int	none	0	simulation year the reservoir becomes operational
3	mores	int	none	0	month the reservoir becomes operational
4	psa	real	ha	0.0	surface area when filled to principal spillway
5	pvol	real	ha-m	0.0	volume needed to fill to principal spillway. Converted to m^3 by `res_initial`
6	esa	real	ha	0.0	surface area when filled to emergency spillway
7	evol	real	ha-m	0.0	volume needed to fill to emergency spillway. Converted to m^3 by `res_initial`
8	k	real	mm/hr	0.01	hydraulic conductivity of the reservoir bottom
9	evrsv	real	none	0.7	lake evaporation coefficient
10	br1	real	none	0.0	volume-surface area coefficient (estimated by the model if zero)
11	br2	real	none	0.0	volume-surface area coefficient (estimated by the model if zero)

The reader fills in defaults when fields are zero or non-positive:

- If `pvol + evol > 0` and `pvol <= 0`, sets `pvol = 0.9 * evol`.
- If both are zero or negative, sets `pvol = 60000.0`.
- If `evol <= 0`, sets `evol = 1.11 * pvol`.
- If `psa <= 0`, sets `psa = 0.08 * pvol`.
- If `esa <= 0`, sets `esa = 1.5 * psa`.
- If `evrsv <= 0`, sets `evrsv = 0.6`.

Note the default for `evrsv` in the type definition is `0.7`, but the runtime default applied when the field is zero or negative is `0.6`.

sediment.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per sediment profile with list-directed I/O. Records are read into `res_sed` (an array of `type reservoir_sed_data`).

#	Field	Type	Units	Description
1	name	char(25)	none	sediment profile name
2	nsed	real	mg/L	normal amount of sediment in the reservoir
3	d50	real	um	median particle size of suspended and benthic sediment
4	carbon	real	percent	organic carbon in suspended and benthic sediment
5	bd	real	t/m^3	bulk density of benthic sediment
6	sed_stlr	real	none	sediment settling rate
7	velsetlr	real	m/day	sediment settling velocity

Example

`refdata/Osu_1hru/sediment.res`:

```
sediment.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    sed_amt           d50        carbon            bd       sed_stl       stl_vel
sedwet1                   10.00           5.00       0.04            0.8          0.15       0.0002
```

The editor writes `sed_amt`, `sed_stl`, `stl_vel`; the source fields are `nsed`, `sed_stlr`, `velsetlr`. Column order is what matters.

nutrients.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per nutrient profile with list-directed I/O. Records are read into `res_nut` (an array of `type reservoir_nut_data`). After reading, the reader divides the four `setlr` rates and the two `solr` rates by 365 (m/yr to m/day).

#	Field	Type	Units (file)	Description
1	name	char(25)	none	nutrient profile name
2	ires1	int	day of year	start of mid-year settling season
3	ires2	int	day of year	end of mid-year settling season
4	nsetlr1	real	m/year	nitrogen mass loss rate, mid-year period
5	nsetlr2	real	m/year	nitrogen mass loss rate, remainder of year
6	psetlr1	real	m/year	phosphorus mass loss rate, mid-year period
7	psetlr2	real	m/year	phosphorus mass loss rate, remainder of year
8	nsolr	real	m/year	loss rate for soluble nitrogen (NO3, NH3, NO2)
9	psolr	real	m/year	loss rate for soluble phosphorus
10	theta_n	real	none	temperature adjustment for nitrogen settling

#	Field	Type	Units (file)	Description
11	theta_p	real	none	temperature adjustment for phosphorus settling
12	conc_nmin	real	ppm	minimum nitrogen concentration for settling
13	conc_pmin	real	ppm	minimum phosphorus concentration for settling

Example

`refdata/Osu_1hru/nutrients.res`:

```
nutrients.res: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name               mid_start   mid_end     mid_n_stl         n_stl     mid_p_stl         p_stl       chla_c
nutwet1                    5        10       5.50000       5.50000      10.00000      10.00000       1.00000
```

Notes:

- Editor headers `mid_n_stl`, `n_stl`, `mid_p_stl`, `p_stl` correspond to source fields `nsetlr1`, `nsetlr2`, `psetlr1`, `psetlr2`.
- Editor headers `chla_co` and `secchi_co` correspond to source fields `nsolr` and `psolr` by position. These are the soluble nitrogen and phosphorus loss rates, not chlorophyll or Secchi-depth coefficients.

weir.res

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per weir profile with list-directed I/O. Records are read into `res_weir` (an array of `type reservoir_weir_outflow`).

#	Field	Type	Units	Default	Description
1	name	char(25)	none	“”	weir profile name
2	c	real	none	1.84	weir discharge linear coefficient
3	k	real	none	2.6	weir discharge exponential coefficient
4	w	real	m	2.5	weir width
5	h	real	m	0.0	height of the weir above the bottom of the impoundment

Example

`refdata/Osu_1hru/weir.res`:

```
weir.res: Reservoir weir inputs - asdf;lj
NAME   Linear_C    Exp_K     Width(m)   Depth(m)
weir1   1.83       1.50      5.00       0.0
```

Related files

- reservoir.con. Connectivity entries that place each reservoir in the routing topology.
- `res_rel.dtl`. Decision tables referenced from the `release` column of `reservoir.res`.
- wetland.wet. Wetlands share the `initial.res` table and the `hydrology.wet`, `sediment.res`, `nutrients.res`, and `weir.res` databases.

Purpose

The wetland group has two input files declared in `type input_res` (`src/input_file_module.f90`):

- `wetland.wet` is the wetland object table. One record per wetland, naming the initial-condition set, hydrology profile, release rule, sediment profile, nutrient profile, and weir profile that apply.
- `hydrology.wet` is the wetland hydrology database. One record per named hydrology profile. The geometry is HRU-relative (fraction of HRU area, depth in mm) rather than absolute area and volume as for reservoirs.

Wetlands reuse the reservoir sediment, nutrient, weir, and initial-condition databases. The wetland records in `wetland.wet` point at the same `sediment.res`, `nutrients.res`, `weir.res`, and `initial.res` tables described in reservoir.res.

Source

- Readers: src/wet_read.f90, src/wet_read_hyd.f90
- Type definitions: src/reservoir_data_module.f90, `type reservoir_data_char_input` (shared with reservoirs) and `type wetland_hyd_data`
- Filename declarations: src/input_file_module.f90, `type input_res`

wetland.wet

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per wetland with list-directed I/O. The first integer on each record is the wetland id (used as the index into `wet_dat_c`). The remaining columns are read into `type reservoir_data_char_input` in declaration order (the same type used by reservoirs).

#	Field	Type	Description
1	id	int	wetland id. Used as the array index into `wet_dat_c`
2	name	char(25)	wetland name
3	init	char(25)	initial-state pointer. Cross-walks with `init` in `initial.res`
4	hyd	char(25)	hydrology pointer. Cross-walks with `name` in `hydrology.wet`
5	release	char(25)	release rule pointer (decision-table name)
6	sed	char(25)	sediment pointer. Cross-walks with `name` in `sediment.res`
7	nut	char(25)	nutrient pointer. Cross-walks with `name` in `nutrients.res`

The example dataset writes a trailing `WEIR` column. The base type has six character fields (name, init, hyd, release, sed, nut). A weir pointer is held in the extension type `reservoir_data` (`character (len=25) :: weir`). The list-directed reader for `wetland.wet` does not consume a separate weir token into the base record; the source links wetland weirs through dedicated reservoir-conditions input. See the Important note below.

Example

`refdata/Osu_1hru/wetland.wet`:

```
wetland.       wet: written by SWAT+ edi#############for SWAT+ rev.60.5.4
     id                name         init          hyd          rel          sed          nut         WEIR
       1           paddy0001     initwet2        paddy         weir      sedwet1      nutwet1        weir1
```

Notes:

- The title line in this dataset is partially corrupted ("wetland. wet:"). The reader skips it as a single text line, so the content does not matter as long as it stays on one line.
- The `rel` column value `weir` is matched against decision-table names. If no match exists in `res_rel.dtl` (or in the `ctbl_` conditional tables), the wetland will be flagged as missing a release rule.

hydrology.wet

Header lines: 2. The reader skips a title line and a column-header line, then reads one record per hydrology profile with list-directed I/O. Records are read into `wet_hyddb` (an array of `type wetland_hyd_data`) in declaration order.

#	Field	Type	Units	Default	Description
1	name	char(25)	none	“default”	hydrology profile name
2	psa	real	fraction	0.0	fraction of HRU area at the principal spillway. Surface-inlet riser flow starts at this level
3	pdep	real	mm	0.0	average water depth at the principal spillway
4	esa	real	fraction	0.0	fraction of HRU area at the emergency spillway. Overflow into the ditch starts at this level
5	edep	real	mm	0.0	average water depth at the emergency spillway
6	k	real	mm/hr	0.01	hydraulic conductivity of the wetland bottom
7	evrsv	real	none	0.7	wetland evaporation coefficient
8	acoef	real	none	1.0	volume-surface area coefficient for HRU impoundment
9	bcoef	real	none	1.0	volume-depth coefficient for HRU impoundment
10	ccoef	real	none	1.0	volume-depth coefficient for HRU impoundment
11	frac	real	none	0.5	fraction of HRU that drains into the impoundment

The reader fills defaults when fields are zero or non-positive:

- If `psa <= 0`, sets `psa = 0.08 * wet_hyd(ires)%pdep` (uses the previously initialized `wet_hyd`, not the just-read `wet_hyddb`).
- If `esa <= 0`, sets `esa = 1.5 * wet_hyd(ires)%psa`.
- If `evrsv <= 0`, sets `evrsv = 0.6`.

Example

`refdata/Osu_1hru/hydrology.wet`:

```
hydrology.wet: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                     hru_ps          dp_ps         hru_es          dp_es              k           evap    vol
paddy                      1.00         150.00          1.00         150.00           0.5            0.75
```

Editor headers `hru_ps`, `dp_ps`, `hru_es`, `dp_es`, `evap`, `vol_area_co`, `vol_dp_a`, `vol_dp_b`, `hru_frac` correspond to source fields `psa`, `pdep`, `esa`, `edep`, `evrsv`, `acoef`, `bcoef`, `ccoef`, `frac`. Column order is what matters.

Related files

- reservoir.res and initial.res. Shared tables. `wetland.wet` records point at `initial.res` names, and the sediment, nutrient, and weir lookups use the same databases as reservoirs.
- HRU connectivity. Each wetland is attached to an HRU; the geometry in `hydrology.wet` is expressed as fractions of HRU area.

Purpose

The six `.ops` files declared in `type input_ops` are parameter databases for the management operations that `management.sch` and the `lum.dtl` decision tables can invoke. Each file is a named list of parameter sets that other files reference by `name`.

All six readers follow the same structure: skip two header lines, count records, allocate, rewind, skip two, read one record per row into the corresponding `*_db` array.

Source

- Readers: `src/mgt_read_harvops.f90`, `src/mgt_read_grazeops.f90`, `src/mgt_read_irrops.f90`, `src/mgt_read_chemapp.f90`, `src/mgt_read_fireops.f90`, `src/mgt_read_sweepops.f90`.
- Type definitions: `src/mgt_operations_module.f90`.

File format

For every file:

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3 and following: one record per operation. List-directed read. The reader counts records on a first pass, allocates the database array, rewinds, and reads the values.

The "Common" first column is the operation name; other files reference it.

harv.ops

Reader: `mgt_read_harvops.f90` → `harvop_db` (`type harvest_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	typ	char(40)	“”	none	`grain`, `biomass`, `residue`, `tree`, or `tuber`
3	hi_ovr	real	0	none	harvest index override at harvest
4	eff	real	0	fraction	harvest efficiency. Fraction of harvested yield removed; the remainder is left as residue
5	bm_min	real	0	kg/ha	minimum biomass required to allow harvest

graze.ops

Reader: `mgt_read_grazeops.f90` → `grazeop_db` (`type grazing_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	fertnm	char(40)	“”	none	manure name pointing into `fertilizer.frt`
3	manure_id	int	0	none	resolved id of `fertnm` in `fertilizer.frt`
4	eat	real	0	(kg/ha)/day	dry biomass removed by grazing per day
5	tramp	real	0	(kg/ha)/day	dry biomass removed by trampling per day
6	manure	real	0	(kg/ha)/day	dry manure deposited per day
7	biomin	real	0	kg/ha	minimum plant biomass to allow grazing

irr.ops

Reader: `mgt_read_irrops.f90` → `irrop_db` (`type irrigation_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	amt_mm	real	25.4	mm	application amount
3	eff	real	0	none	in-field application efficiency
4	surq	real	0	fraction	surface runoff ratio
5	dep_mm	real	0	mm	depth of application for subsurface irrigation
6	salt	real	0	mg/kg	total salt concentration in irrigation water
7	no3	real	0	mg/kg	nitrate concentration
8	po4	real	0	mg/kg	phosphate concentration

chem_app.ops

Reader: `mgt_read_chemapp.f90` → `chemapp_db` (`type chemical_application_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	form	char(40)	“”	none	`solid` or `liquid`
3	op_typ	char(40)	“”	none	`spread`, `spray`, `inject`, or `direct`
4	app_eff	real	0	none	application efficiency
5	foliar_eff	real	0	none	foliar efficiency
6	inject_dep	real	0	mm	injection depth
7	surf_frac	real	0	fraction	fraction in upper 10 mm of soil
8	drift_pot	real	0	none	drift potential
9	aerial_unif	real	0	none	aerial uniformity

fire.ops

Reader: `mgt_read_fireops.f90` → `fire_db` (`type fire_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	cn2_upd	real	0	none	change applied to SCS curve number II value
3	fr_burn	real	0	fraction	fraction of cover burned

sweep.ops

Reader: `mgt_read_sweepops.f90` → `sweepop_db` (`type streetsweep_operation`).

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	eff	real	0	none	removal efficiency of the sweeping operation
3	fr_curb	real	0	none	availability factor: fraction of curb length that is sweepable

Example

`refdata/Ames_sub1/harv.ops`:

```
harv.ops written by SWAT+ editor v2.3.3 on 2023-10-25
name                  typ                hi_ovr        eff        bm_min
harv_grain            grain              0.0           1.0        0.0
harv_residue          residue            0.0           1.0        0.0
```

The other five files follow the same shape with their own column sets.

Related files

- `landuse.lum` and `management.sch` reference operations by `name`.
- `*.dtl` decision tables can fire operations by `name`.
- `fertilizer.frt` provides the `manure_id` resolution target for `graze.ops`.

Purpose

A decision-table file holds one or more tables. Each table evaluates a set of conditions every time step and fires the matching action when an alternative is satisfied. SWAT+ uses decision tables to drive land-use management (planting, harvest, fertiliser, irrigation), reservoir releases, scenario land-use changes, and flow control.

The four decision-table files declared in `type input_condition` are:

- `lum.dtl`: land-use management actions.
- `res_rel.dtl`: reservoir release rules.
- `scen_lu.dtl`: scenario-driven land-use changes.
- `flo_con.dtl`: flow control rules.

All four use the same file format. They differ only in which decision-table array they fill (`dtbl_lum`, `dtbl_res`, `dtbl_scen`, `dtbl_flo`).

Source

- Readers: `src/dtbl_lum_read.f90`, `src/dtbl_flocon_read.f90` (and matching readers for reservoir-release and scenario tables).
- Type definitions: `src/conditional_module.f90`. Each table is a `decision_table` instance; its `cond`, `alt`, and `act` arrays hold conditions, alternatives, and actions.

File format

A single file holds `mdtbl` tables.

- Line 1: title (skipped).
- Line 2: `mdtbl`, the number of decision tables in the file.
- Line 3: blank line (skipped).

For each of the `mdtbl` tables:

- One header line (skipped).
- One name line: `name`, `conds`, `alts`, `acts` (table name, number of conditions, number of alternatives, number of actions).
- One header line for the conditions block (skipped).
- `conds` lines, one per condition. Each line names a conditional variable plus `alts` alternative thresholds. See Condition row below.
- One header line for the actions block (skipped).
- `acts` lines, one per action. See Action row below.

Condition row

Each condition row is read into a `condition_var` record with the following fields, then `alts` alternative thresholds are read into the `alt(ic, 1..alts)` array.

Fields on a condition row:

#	Field	Type	Description
1	var	char	conditional variable name (for example `land_use`, `plant_gro`, `phu_plant`, `soil_water`, `prob_unif`)
2	obj	char	object the variable applies to
3	obj_num	int	object number
4	lim_var	char	limit-variable name
5	lim_op	char	comparison operator
6	lim_const	real	comparison constant
7+	alt(1..alts)	char	one alternative threshold per alternative column

When `var == "prob_unif"`, the reader rereads the row and treats the second token as the probability fraction `frac_app` rather than the standard condition layout.

Action row

#	Field	Type	Description
1	act_typ	char	action type (for example `plant`, `harvest`, `fert`, `irrigate`, `lu_change`)
2	obj	char	target object
3	obj_num	int	target object number
4	name	char	action name pointing into the matching ops or input database
5	option	char	option tag
6	const	real	numeric constant for the action
7	const2	real	second numeric constant
8	fp	char	flag or pointer
9+	act_hit(1..alts)	char	one outcome flag per alternative column

Notes

- The reader allocates `cond(conds)`, `alt(conds, alts)`, `act(acts)`, and `act_hit(alts)` based on the header values. A mismatch between declared counts and actual rows truncates or overruns these arrays.
- After all tables are read, `lum.dtl` does a second pass to count HRUs whose `land_use_mgt_c` matches any `land_use` condition. The counts feed probabilistic operations.

Example

Top of `refdata/Ames_sub1/lum.dtl`:

```
lum.dtl: written by SWAT+ editor for SWAT+ rev.60.5.7
8

corn_grain
name                 conds   alts   acts
pl_hv_corn               2      2      2
var                   obj   obj_num   lim_var   lim_op   lim_const   alt1   alt2
plant_gro           plant         1   tree      >        0           y      n
phu_plant           plant         1                               >  1.0    n      y
act_typ             obj   obj_num   name                 option   const   const2   fp   outcome1   outcom
plant               hru   0         corn                 below    0       0        y    y          n
harvest_kill        hru   0         harv_grain_corn      now      0       0        y    n          y
```

Reading this:

- File holds 8 tables. The first is named `pl_hv_corn` with 2 conditions, 2 alternatives, 2 actions.
- Condition 1 checks `plant_gro` on plant object 1 against `tree > 0`. Alt 1 fires when `y`, alt 2 fires when `n`.
- Condition 2 checks `phu_plant > 1.0`. Alt 1 fires when `n`, alt 2 fires when `y`.
- Action 1: `plant` HRU operation named `corn` with option `below`. Fires under alt 1, not alt 2.
- Action 2: `harvest_kill` HRU operation named `harv_grain_corn` with option `now`. Fires under alt 2, not alt 1.

The exact column layout varies a little between datasets because the reader is fully list-directed.

Related files

- `landuse.lum`. Plant and management names referenced by actions.
- `*.ops`. Operation parameter databases that the actions resolve `name` against.
- `management.sch`. Fixed schedules used alongside decision tables.

Purpose

`plants.plt` holds one parameter set per plant species the model can grow. Each record fills a `plant_db` entry in the `pldb` array. Plant growth, phenology, biomass-to-energy conversion, root and canopy geometry, nutrient uptake, and residue partition fractions all come from this file. HRU plant communities (`plant.ini`) and management operations (`plant.ops`, `landuse.lum`) reference these records by name.

Source

- Reader: `src/plant_parm_read.f90`
- Type definition: `src/plant_data_module.f90`, `type plant_db`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per plant. List-directed read into `pldb(ic)` (`type plant_db`). The last sub-object is `res_part_fracs` (`type residue_partition_fracs`), which contributes three trailing fields.

The reader has a conditional branch:

```
if (bsn_cc%nam1 == 0) then
  read (104,*,iostat=eof) pldb(ic)
else
  read (104,*,iostat=eof) pldb(ic), pl_class(ic)
end if
```

`bsn_cc%nam1` is a switch in `codes.bsn`. With the default `nam1 = 0`, the standard column list below applies. With `nam1 /= 0`, one extra `pl_class` token (a 25-character name like `row_crop`, `tree`, `grass`) is read at the end of each record.

#	Field	Type	Units	Default	Description
1	plantnm	char(40)	none	“”	plant name (used as the key from other files)
2	typ	char(18)	none	“”	plant category: `warm_annual`, `cold_annual`, `warm_annual_tuber`, `cold_annual_tuber`, `perennial`
3	trig	char(18)	none	“”	phenology trigger: `moisture_gro` or `temp_gro`
4	nfix_co	real	none	0.0	nitrogen fixation coefficient (0.5 for legumes, 0 for non-legumes)
5	days_mat	int	days	110	days to maturity. If 0, heat units run the entire growing season
6	bio_e	real	(kg/ha)/(MJ/m^2)	15.0	biomass-energy ratio
7	hvsti	real	(kg/ha)/(kg/ha)	0.76	harvest index: crop yield over above-ground biomass
8	blai	real	none	5.0	maximum potential leaf area index
9	frgrw1	real	fraction	0.05	fraction of growing season at the 1st point on the optimal LAI curve
10	laimx1	real	fraction	0.05	fraction of maximum LAI at the 1st point on the optimal LAI curve
11	frgrw2	real	fraction	0.4	fraction of growing season at the 2nd point on the optimal LAI curve
12	laimx2	real	fraction	0.95	fraction of maximum LAI at the 2nd point on the optimal LAI curve
13	dlai	real	fraction	0.99	fraction of growing season when LAI starts to decline
14	dlai_rate	real	none	1.0	exponent governing LAI decline rate
15	chtmx	real	m	6.0	maximum canopy height
16	rdmx	real	m	3.5	maximum root depth
17	t_opt	real	deg C	30.0	optimal temperature for plant growth
18	t_base	real	deg C	10.0	minimum (base) temperature for plant growth
19	cnyld	real	kg N / kg yield	0.0015	fraction of nitrogen in yield
20	cpyld	real	kg P / kg yield	0.0003	fraction of phosphorus in yield
21	pltnfr1	real	kg N / kg biomass	0.006	nitrogen uptake parameter 1 (emergence)
22	pltnfr2	real	kg N / kg biomass	0.002	nitrogen uptake parameter 2 (mid-season)
23	pltnfr3	real	kg N / kg biomass	0.0015	nitrogen uptake parameter 3 (maturity)
24	pltpfr1	real	kg P / kg biomass	0.0007	phosphorus uptake parameter 1 (emergence)
25	pltpfr2	real	kg P / kg biomass	0.0004	phosphorus uptake parameter 2 (mid-season)
26	pltpfr3	real	kg P / kg biomass	0.0003	phosphorus uptake parameter 3 (maturity)
27	wsyf	real	(kg/ha)/(kg/ha)	0.01	lower bound of harvest index under water stress
28	usle_c	real	none	0.001	minimum value of the USLE C factor for water erosion
29	gsi	real	m/s	0.002	maximum stomatal conductance
30	vpdfr	real	kPa	4.0	vapor pressure deficit at which `gmaxfr` is valid
31	gmaxfr	real	none	0.75	fraction of maximum stomatal conductance achieved at `vpdfr`
32	wavp	real	none	8.0	rate of decline in radiation use efficiency per unit VPD

#	Field	Type	Units	Default	Description
33	co2hi	real	uL CO2 / L air	660.0	elevated CO2 concentration at the 2nd point on the RUE-CO2 curve
34	bioehi	real	(kg/ha)/(MJ/m^2)	16.0	biomass-energy ratio at the elevated CO2 level (`co2hi`)
35	rsdco_pl	real	none	0.05	plant residue decomposition coefficient
36	alai_min	real	$m^{2/m}2$	0.75	minimum LAI during winter dormant period
37	laixco_tree	real	none	0.3	coefficient used to estimate maximum LAI during tree growth
38	mat_yrs	int	years	10	years to maturity (forced to at least 1 by the reader)
39	bmx_peren	real	metric tons/ha	1000.0	maximum biomass for forest
40	ext_coef	real	none	0.65	light extinction coefficient
41	leaf_tov_min	real	months	12.0	perennial leaf turnover rate with minimum stress
42	leaf_tov_max	real	months	3.0	perennial leaf turnover rate with maximum stress
43	bm_dieoff	real	fraction	0.0	above-ground biomass that dies off at dormancy
44	rsr1	real	fraction	0.0	initial root-to-shoot ratio at the beginning of the growing season
45	rsr2	real	fraction	0.0	root-to-shoot ratio at the end of the growing season
46	pop1	real	plants/m^2	0.0	plant population at the 1st point on the population-LAI curve
47	frlai1	real	fraction	0.0	fraction of maximum LAI at the 1st point on the population-LAI curve
48	pop2	real	plants/m^2	0.0	plant population at the 2nd point on the population-LAI curve
49	frlai2	real	fraction	0.0	fraction of maximum LAI at the 2nd point on the population-LAI curve
50	frsw_gro	real	fraction	0.5	30-day sum of P-PET that triggers monsoon-season growth in tropical plants
51	aeration	real	none	0.2	aeration stress factor
52	rsd_pctcov	real	none	0.0	residue factor for the percent cover equation
53	rsd_covfac	real	none	0.0	residue factor for the surface cover (C factor) equation
54	res_part_fracs%meta_frac	real	fraction	0.85	metabolic fraction of residue
55	res_part_fracs%str_frac	real	fraction	0.15	structural fraction of residue
56	res_part_fracs%lig_frac	real	fraction	0.12	lignin fraction of residue

When `cswat = 2` in `codes.bsn`, the three residue partition fractions are reinterpreted into above- and below-ground metabolic, structural, and lignin pools (`cswat_1_part_fracs`) in `plant_parm_read`. When `cswat /= 2`, the reader overwrites the read values with hard-coded defaults of 0.85, 0.15, and 0.12.

Example

First two records of `refdata/Ames_sub1/plants.plt`:

```
plants.plt
name  plnt_typ  gro_trig  nfix_co  days_mat  bm_e  harv_idx  lai_pot  frac_hu1  lai_max1  frac_hu2  lai_ma
alfa  perennial  temp_gro  0.5  0  20  0.9  4  0.15  0.01  0.5  0.95  0.9  0.5  0.9  3  20  4  0.02  0.003
cana  warm_annual  temp_gro  0  100  34  0.3  4.5  0.15  0.02  0.45  0.95  0.5  0.3  1.3  1.4  21  5  0.03
```

Notes on the example:

- Header column names written by the editor (`plnt_typ`, `gro_trig`, `bm_e`, `harv_idx`, `lai_pot`, `frac_hu1`, `frac_n_yld`, `frac_n_em`, `stcon_max`, `frac_stcon`, `ru_vpd`, `co2_hi`, `bm_e_hi`, `plnt_decomp`, `lai_min`, `bm_tree_acc`, `yrs_mat`, `bm_tree_max`, `ext_co`, `leaf_tov_mn`, `leaf_tov_mx`, `rt_st_beg`, `rt_st_end`, `plnt_pop1`, `frac_lai1`, `frac_sw_gro`, `avg_lig_frac`, `ab_lig_frac`, `bg_lig_frac`) do not match the field names in the source. The reader uses position only.
- The trailing `description` column is not part of `plant_db`. List-directed reads stop after the type is filled, so the description is silently discarded.
- For `alfa`, `days_mat = 0` is the explicit "use heat units for the full season" signal.

Related files

- `codes.bsn`. `bsn_cc%nam1` selects whether `pl_class` is read after each record. `bsn_cc%cswat` selects how residue partition fractions are interpreted.
- `plant.ini`. Lists which plants are present in each HRU at the start of simulation, keyed by `plantnm`.
- `landuse.lum` and `plant.ops`. Reference plant names from this file in management schedules.

Purpose

`fertilizer.frt` lists every fertilizer product the simulation can apply, with its mass fractions of mineral nitrogen, mineral phosphorus, organic nitrogen, organic phosphorus, and the share of mineral N that is in the NH3 form. Each record fills one `fertilizer_db` entry in the `fertdb` array. Fertilizer operations (in `chem_app.ops` and the management decision tables) reference entries here by name.

Source

- Reader: `src/fert_parm_read.f90`
- Type definition: `src/fertilizer_data_module.f90`, `type fertilizer_db`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per fertilizer product. List-directed read into `fertdb(it)` (`type fertilizer_db`).

#	Field	Type	Units	Default	Description
1	fertnm	char(16)	none	“”	fertilizer name (key referenced by `chem_app.ops` and management decision tables)
2	fminn	real	kg min-N / kg fertilizer	0.0	fraction of fertilizer that is mineral N (NO3 + NH3)
3	fminp	real	kg min-P / kg fertilizer	0.0	fraction of fertilizer that is mineral P
4	forgn	real	kg org-N / kg fertilizer	0.0	fraction of fertilizer that is organic N
5	forgp	real	kg org-P / kg fertilizer	0.0	fraction of fertilizer that is organic P
6	fnh3n	real	kg NH3-N / kg min-N	0.0	fraction of mineral N content that is NH3

The five real fractions must sum to less than or equal to 1 for a single product. `fminn + fminp + forgn + forgp` is the total nutrient mass per unit fertilizer; the remainder is carrier mass.

Example

First lines of `refdata/Ames_sub1/fertilizer.frt`:

```
fertilizer.frt: written by SWAT+ editor v2.2.2 on 2023-10-17 14:43 for SWAT+ rev.60.5.4
name                      min_n         min_p         org_n         org_p         nh3_n         pathogens  
elem_n                  1.00000       0.00000       0.00000       0.00000       0.00000              null  
elem_p                  0.00000       1.00000       0.00000       0.00000       0.00000              null  
anh_nh3                 0.82000       0.00000       0.00000       0.00000       1.00000              null  
urea                    0.46000       0.00000       0.00000       0.00000       1.00000              null  
46_00_00                0.46000       0.00000       0.00000       0.00000       0.00000              null  
18_46_00                0.18000       0.20200       0.00000       0.00000       0.00000              null  
dairy_fr                0.00700       0.00500       0.03100       0.00300       0.99000              null  
```

Notes on the example:

- The editor writes two trailing columns (`pathogens` and `description`) that are not in `fertilizer_db`. List-directed reads stop once the type is filled, so both trailing tokens are discarded by the reader. The literal `null` in the `pathogens` column has no effect.
- For `urea`, `fminn = 0.46` and `fnh3n = 1.00`. That is, 46% of the product is mineral N, and 100% of that mineral N is in the NH3 form.
- For `18_46_00`, `fminn = 0.18` and `fminp = 0.202`. Note that the third value in the "46" position is the P (P2O5 converted to P), not P2O5 directly.

manure_db.frt

Named manure types. Each entry points to one organic-matter record in `manure_om.frt` and (optionally) to records for pesticides, pathogens, heavy metals, salts, and other constituents that may be carried with the manure. This is the lookup used by manure-application operations and by manure-allocation.md.

Source

- Reader: src/manure_db_read.f90
- Type: src/fertilizer_data_module.f90, `type manure_database` (variable `manure_db`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per manure type. Two passes: count rows, then read.

#	Field	Type	Description
1	name	char(25)	Manure-type name. Referenced by manure-application operations and by `manure_allo.mnu`.
2	org_min	char(25)	Crosswalk into `manure_om.frt`. Resolved into `iorg_min`.
3	pests	char(25)	Pesticide profile name. Pointer field `ipests` is reserved (the crosswalk loop is commented out in the reader). Use `null` when not applicable.
4	paths	char(25)	Pathogen profile name. Pointer `ipaths`.
5	hmets	char(25)	Heavy-metal profile name. Pointer `imets`.
6	salts	char(25)	Salt-ion profile name. Pointer `isalts`.
7	constit	char(25)	Other-constituent profile name. Pointer `iconstit`.
8	descrip	char(80)	Free-text description.

After reading each row, the reader crosswalks `org_min` against `manure_om(:)%name` and stores the matching index in `manure_db(it)%iorg_min`. The other crosswalks are present in the type but not wired in this reader.

Example

`refdata/Osu_1hru/manure_db.frt`:

```
manure_db.frt: manure database
name          org_min            pests  paths  hmets  salts  constit  description
beef_salt     mw_beef_liq        null   null   null   null   null     midwest_beef_liquid_salt
```

One manure type, pointing at `mw_beef_liq` in `manure_om.frt`. No pesticide, pathogen, heavy-metal, salt, or constituent profile.

manure_om.frt

Organic-matter attributes for each manure type. Holds the water fraction and the C, mineral N, mineral P, organic N, organic P, and NH3 fractions used when a manure application is partitioned into the soil pools.

Source

- Reader: src/manure_orgmin_read.f90
- Type: src/fertilizer_data_module.f90, `type manure_attributes` (variable `manure_om`)

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per manure organic-matter profile.

#	Field	Type	Units	Description
1	name	char(64)	none	Profile name. Used as the crosswalk target from `manure_db.frt`.
2	frac_water	real	kg water / (kg manure + kg water)	Water fraction of the as-applied manure.
3	fcbn	real	kg C / kg manure	Carbon fraction.
4	fminn	real	kg min-N / kg manure	Mineral N fraction (NO3 + NH3).
5	fminp	real	kg min-P / kg manure	Mineral P fraction.
6	forgn	real	kg org-N / kg manure	Organic N fraction.
7	forgp	real	kg org-P / kg manure	Organic P fraction.
8	fnh3n	real	kg NH3-N / kg min-N	Share of mineral N that is NH3.
9	description	char(64)	none	Free-text description.

Example

`refdata/Osu_1hru/manure_om.frt`:

```
manure_om.frt: manure organic matter database
name           frac_water   fcbn          fminn         fminp         forgn         forgp         fnh3n  c
mw_beef_liq    0.988        0.000780576   0.000311751   0.00081295    0.002110312   0.000365707   0.99   r
mw_beef_slur   0.926        0.016786571   0.002697842   0.00081295    0.002110312   0.000365707   0.99   r
```

Two profiles. The liquid form is nearly all water (98.8%); the slurry form is 92.6% water with much higher solids and carbon.

Related files

- `chem_app.ops`. Fertilizer applications in management schedules reference `fertnm` from `fertilizer.frt` or a manure name from `manure_db.frt`.
- `manure-allocation.md`. Allocates manure from named source objects to HRUs and other receivers.
- `codes.bsn`. The active carbon model (`cswat`) controls how N from fertilizer and manure enters the soil pools.

Purpose

`tillage.til` defines each named tillage operation. Each record fills one `tillage_db` entry in the `tilldb` array. Tillage operations referenced in `chem_app.ops` and management decision tables look up the implement by name to get the mixing efficiency, mixing depth, random roughness, and ridge geometry that drive soil-mixing and surface-roughness updates in `mgt_newtillmix_*` and `mgt_tillfactor.f90`.

The reader also scans for the entry named `biomix`. If present, its `effmix` and `deptil` are copied into the module-level `bmix_eff` and `bmix_depth`, which control the daily biological mixing computed in `mgt_biomix.f90`. If no `biomix` entry is present, `bmix_eff` defaults to 0.2 and `bmix_depth` defaults to 50 mm.

Source

- Reader: `src/till_parm_read.f90`
- Type definition: `src/tillage_data_module.f90`, `type tillage_db`

Format

- Line 1: title (skipped).
- Line 2: header (skipped).
- Lines 3+: one record per tillage operation. List-directed read into `tilldb(itl)` (`type tillage_db`).

#	Field	Type	Units	Default	Description
1	tillnm	char(16)	none	“”	tillage operation name (key from management schedules). The literal name `biomix` sets the biological mixing parameters at the module level
2	effmix	real	none	0.0	mixing efficiency of the tillage operation (fraction of soil mass redistributed within the till depth)
3	deptil	real	mm	0.0	depth of mixing caused by the tillage operation
4	ranrns	real	mm	0.0	random roughness imparted to the soil surface
5	ridge_ht	real	mm	0.0	ridge height
6	ridge_sp	real	mm	0.0	ridge interval (or row spacing)

Example

First lines of `refdata/Osu_1hru/tillage.til` (which has the title and header lines the reader expects):

```
tillage.til: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    mix_eff        mix_dp         rough      ridge_ht      ridge_sp  description
puddle                  0.980       150.000        00.000         0.000         0.000    rice_paddy_puddlin
```

The trailing `description` column is not in `tillage_db`. List-directed reads stop after the six fields are filled, so the description is silently discarded.

The Ames database has many more operations (the first records, after a correct two-line header, are):

```
chiselplow              0.65          225           20              0             0     chiselplow
cltiweed                0.3           100           15              0             0     cultiweeder
moldboardplow           0.95          250           30              0             0     mldboard
biomix                  0.01           50            0              0             0     biological_mixing
```

The `biomix` row above sets `bmix_eff = 0.01` and `bmix_depth = 50` at the module level after the file is read.

Related files

- `chem_app.ops` and `lum.dtl` (and similar decision tables). Tillage operations reference `tillnm` from this file.
- codes.bsn. The active carbon model (`cswat`) selects which `mgt_newtillmix_*` routine performs the soil pool mixing.
- `hru-data.hru` and `landuse.lum`. Identify the HRUs that receive the tillage operations defined here.

Removed

`carb_coefs.cbn` was removed. The keyword-style reader (`carbon_coef_read.f90`) and the dead companion file `basins_carbon.tes` have also been deleted. The 22 daily hard-coded coefficient overrides that previously lived in `cbn_zhang2.f90` are gone; user-supplied values now flow end-to-end.

What replaces it

Carbon inputs have been split into two flat-table files that follow the same layout convention as the rest of the SWAT+ inputs (title line, header line, data rows):

- carbon.bsn: 28 basin-wide scalar parameters in a single data row. Initial pool fractions, water coupling, manure partitioning, temperature and water factor selectors, the `t_cbn_*` triple, biomixing and tillage curves, photodegradation, residue C:N / C:P controls, and the slow-humus initialization method (`mathers_method`).
- carbon_lyr.bsn — per-layer rate constants and CO2 allocation fractions. One row per layer group (1 = top, 2 = subsurface).

Both files are required when `bsn_cc%cswat == 2` (the dynamic carbon model; the activation value moved from 1 to 2). Either missing or malformed file triggers `error stop`; there is no silent fallback to compiled defaults.

Migration

Old keyword in `carb_coefs.cbn`	New location
`cbn_diagnostics`	retired; output gating now lives in print.prt via the `hru_cb_*` and `lsu_cb_*` rows
`hp_rate`, `hs_rate`, `microb_rate`, `meta_rate`, `str_rate`, `microb_top_rate`, `hs_hp`	per-layer rows of carbon_lyr.bsn
`a1co2`, `asco2`, `apco2`, `abco2`	per-layer rows of carbon_lyr.bsn
`org_frac` (`frac_seq`, `frac_hum_microb`, `frac_hum_slow`, `frac_hum_passive`)	`init_seq`, `init_microb`, `init_slow`, `init_passive` columns of carbon.bsn
`prmt_21`	`koc_c` column of carbon.bsn
`prmt_44`	`solc_ratio` column of carbon.bsn
`till_eff_days`	`till_eff_days` column of carbon.bsn

Old keyword in carb_coefs.cbn	New location
rtof	manure_c_frac column of carbon.bsn
cbn_consolidation_factors (2 values)	bio_consol, till_consol columns of carbon.bsn
cbn_factor_approaches (2 ints)	tmpf_eqn, watf_eqn columns of carbon.bsn
tn, top, tx	t_cbn_min, t_cbn_opt, t_cbn_max columns of carbon.bsn
zz_bmix_coefs (3 values)	bmix_a, bmix_b, bmix_c columns of carbon.bsn. zz_ prefix dropped to match the column name.
zz_emix_coefs (3 values)	tillmix_a, tillmix_b, tillmix_c columns of carbon.bsn
photo_degrade_factor	sfc_rsd_photodeg column of carbon.bsn
nmbr_soil_test_layers, soil_test	retired; no replacement (sub-table was unused downstream)

Five new tunables were promoted from hard-coded constants and added to carbon.bsn:

- n_act_frac (formerly the nactfr local in nut_nminrl.f90)
- cnr_cap, cnr_ref (formerly the 500 and 25 hard-coded values in cbn_surfrsd_decomp.f90)
- cpr_cap, cpr_ref (formerly the 5000 and 200 hard-coded values in cbn_surfrsd_decomp.f90)

Related files

- carbon.bsn. Replacement for the basin-wide scalars.
- carbon_lyr.bsn. Replacement for the per-layer table.
- codes.bsn. The cswat switch. cswat == 2 and cswat == 3 are no longer accepted; set 0 (no carbon) or 1 (active carbon).
- Carbon outputs. Replacement for the cbn_diagnostics flag.

Purpose

treatment.trt is a legacy file name carried over from the SWAT2012 treatment operation table. It is written by external setup tools (QSWAT+, SWAT+ Editor) into project directories, but no SWAT+ Fortran reader consumes it. The file plays no part in a SWAT+ run.

The current treatment functionality in SWAT+ lives in the water allocation module and uses water_treat.wal, not treatment.trt. See src/water_treatment_read.f90.

Source

A search of the SWAT+ Fortran tree at commit 8a1edd4a23915a2cac1cd90c788f0d521404700e finds no open, inquire, or filename string referencing treatment.trt or any .trt extension. file.cio does not list it.

```
grep -rn "treatment.trt\|\\.trt" swatplus/src/
# no matches
```

Format

There is no defined format. The file is not parsed.

Example

Both reference datasets ship an empty treatment.trt:

```
$ wc -l refdata/Ames_sub1/treatment.trt refdata/Osu_1hru/treatment.trt
0 refdata/Ames_sub1/treatment.trt
0 refdata/Osu_1hru/treatment.trt
```

The file exists only as a placeholder.

Related files

- water_treat.wal. The active treatment input, read by `water_treatment_read`. Defines water treatment objects with storage, lag, loss fraction, and constituent links used by water allocation.

Purpose

`manure_allo.mnu` defines manure-allocation objects. Each object names one or more source locations (manure storage on a farm, a feedlot, a lagoon) and one or more demand locations (HRUs, municipal treatment, inter-basin diversion). At runtime, manure produced at the sources is distributed to the demand objects under a named rule. Each allocation object holds its own source list, demand list, monthly production, storage, and per-source withdrawal history.

The allocation file ties three databases together:

- fertilizer.frt. Each source's `manure_typ` is a key into `fertdb` (cross-walked at read time into `src%fertdb`).
- lum.dtl. For HRU demands, the rule name is a key into `dtbl_lum` (cross-walked into `trn%dtbl_num`).
- `chem_app.ops`. The first action of the matched lum decision table names a chemical-application profile in `chemapp_db`, cross-walked into `manure_amt%app_method`.

Source

- Reader: src/manure_allocation_read.f90
- Types: src/manure_allocation_module.f90
  - `type manure_allocation` (variable `mallo`)
  - `type manure_source_objects` (`mallo(i)%src(j)`)
  - `type manure_demand_objects` (`mallo(i)%trn(j)`)

Format

- Line 1: title (skipped).
- Line 2: number of allocation objects, `imax`. Stored in `db_mx%mallo_db`.
- Per allocation object, in order:
  1. Header line (skipped).
  2. Object header record: `name`, `rule_typ`, `src_obs`, `trn_obs`.
  3. Header line (skipped).
  4. `src_obs` source records.
  5. Header line (skipped).
  6. `trn_obs` demand records.

Object header

#	Field	Type	Description
1	name	char(25)	Allocation-object name.
2	rule_typ	char(25)	Allocation rule. Selects how unmet demand is split across sources.
3	src_obs	int	Number of source objects in this allocation.
4	trn_obs	int	Number of demand objects in this allocation.

Source record (one per source)

#	Field	Type	Units	Description
1	num	int	none	Sequential source id. Used as the array index.
2	mois_typ	char(3)	none	Moisture class: `wet` or `dry`.

#	Field	Type	Units	Description
3	manure_typ	char(25)	none	Manure-product name. Cross-walked against `fertdb(:)%fertnm` and stored in `fertdb`.
4	lat	real	degrees	Latitude of the source.
5	long	real	degrees	Longitude of the source.
6	stor_init	real	t	Initial manure in storage.
7	stor_max	real	t	Storage capacity.
8..19	prod_mon(1..12)	real	t/month	Average monthly manure production. Twelve values, January through December.

Demand record (one per demand)

#	Field	Type	Description
1	num	int	Sequential demand id. Used as the array index.
2	ob_typ	char(10)	Receiver type: `hru` (apply to an HRU), `muni` (municipal treatment), or `divert` (inter-basin diversion).
3	ob_num	int	Number of the receiving object of `ob_typ`.
4	dtbl	char(25)	Decision-table name. For `hru`, matched against `dtbl_lum`.
5	right	char(2)	Allocation right. `sr` for senior, `jr` for junior.

For an HRU demand, the reader sets `hru(ihru)%man_trn_dtbl` to the resolved decision-table index, and looks up the first action's `option` against `chemapp_db(:)%name` to set the application method on `mallo(imro)%trn(itrn)%manure_amt%app_method`.

Example

A two-line skeleton with one source and one HRU demand:

```
manure_allo.mnu: example
1
name            rule_typ    src_obs  trn_obs
farm_a          fixed             1        1
num  mois_typ  manure_typ  lat       long      stor_init  stor_max   prod_mon(1..12)
  1  wet       beef_salt   41.99   -93.62          0.0       500.0    5.0  5.0  6.0  6.0  7.0  7.0  7.0  7
num  ob_typ    ob_num  dtbl                right
  1  hru             1  apply_manure_dtbl      sr
```

The decision-table named `apply_manure_dtbl` must exist in `lum.dtl` and its first action must name an entry in `chem_app.ops` for the cross-walks to resolve.

Related files

- `fertilizer.frt`. Defines the manure-product names that source records reference, and the manure organic-matter database (`manure_db.frt`, `manure_om.frt`).
- `lum.dtl`. Decision tables that fire manure applications at each demand HRU.
- `chem_app.ops`. Chemical-application profiles (rate, method) selected by the decision-table action.

Five small inputs that the larger pages reference but do not document. Each is a single reader populating a single derived type.

management.sch

Fixed-schedule management. Each schedule lists scheduled operations on calendar dates or heat-unit fractions, plus a set of named auto-operation tables (decision tables in `lum.dtl`). `landuse.lum` selects a schedule by name through `mgt`.

- Reader: `src/mgt_read_mgtops.f90` (header) and `src/read_mgtops.f90` (operation rows).
- Default file name: `management.sch` (from `in_lum%management_sch` in `input_file_module.f90`).
- Target type: `management_schedule` in `src/mgt_operations_module.f90`, array `sched(:)`.

File format

- Line 1: title (skipped).
- Line 2: header (skipped).
- For each schedule:
  - One header row: `name`, `num_ops`, `num_autos`.
  - `num_autos` rows, each naming an auto-operation. If the name is `pl_hv_summer1` or `pl_hv_winter1` the row also lists one crop name; if it is `pl_hv_summer2` it lists two crop names.
  - `num_ops` operation rows, list-directed.

Schedule header

#	Field	Type	Description
1	name	char(40)	schedule name referenced by `landuse.lum`
2	num_ops	int	number of scheduled operation rows
3	num_autos	int	number of auto-operation tables

Operation row

Read into `management_ops` (see `mgt_operations_module.f90`).

#	Field	Type	Description
1	op	char(40)	operation code (see below)
2	mon	int	calendar month
3	day	int	calendar day of month
4	husc	real	heat-unit fraction trigger (0 if calendar-driven)
5	op_char	char(40)	name into the operation database keyed by `op`
6	op_plant	char(40)	name into a secondary database (plant, harvest type, chemical application)
7	op3	real	application amount or override value

Recognised `op` codes and the database that `op_char` (and where noted `op_plant`) resolves against:

Code	Action	op_char database	op_plant database
`pcom`	plant community init	`plant.ini` (`pcomdb`)	none
`plnt`	plant	`plants.plt` (`pldb`)	`transplant.plt` (`transpl`)
`harv`	harvest	(none)	`harv.ops` (`harvop_db`)
`hvkl`	harvest and kill	(none)	`harv.ops`
`kill`	kill	none	none
`till`	tillage	`tillage.til` (`tilldb`)	none
`irrm`, `irrp`	irrigation	`irr.ops` (`irrop_db`)	none
`fert`	fertilizer	`fertilizer.frt` (`fertdb`)	`chem_app.ops` (`chemapp_db`)
`manu`	manure	`manure_db.frt` (`manure_db`)	`chem_app.ops`
`pest`	pesticide	`pesticide.pes` (`pestdb`)	`chem_app.ops`
`graz`	grazing	`graze.ops` (`grazeop_db`)	none
`burn`	burn	`fire.ops` (`fire_db`)	none
`swep`	street sweep	`sweep.ops` (`sweepop_db`)	none
`skip`	end of rotation year	none	none
`prtp`	print plant variables	none	none

A `skip` row advances the internal rotation year counter; subsequent rows belong to the next year of the rotation.

Example

`refdata/Ames_sub1/management.sch`:

```
management.sch file AMES
                          NAME NUMB_OPS NUMB_AUTO OP_TYP   MON   DAY HU_SCH   OP_DATA1
 ↪  OP_DATA2  OP_DATA3
mgt_01                               305         1
                                                 irr_year_irr
                                                     till    4     1    0.0   chisplow
 ↪  null     0.000
                                                     plnt    4    28    0.0       corn
 ↪  null     0
                                                     irrm    5    15    0.0  sprinkler_ilm
 ↪  null     0.000
                                                     fert    6     1    0.0       null
 ↪  elem_n   181.400
                                                     hvkl   10     7    0.0       corn
 ↪  grain     0.000
                                                     skip    0     0    0         null
 ↪  null     0
```

Related files

- `landuse.lum` selects a schedule by name.
- `*.ops` hold the operation parameters that `op_char` and `op_plant` resolve against.
- `*.dtl` decision tables provide the `auto_name` entries.

puddle.ops

Rice paddy puddling parameters. A puddling event resets the upper soil-layer hydraulic conductivity and the in-paddy water chemistry to fixed values.

- Reader: `src/mgt_read_puddle.f90`.
- File name: hard-coded `puddle.ops`.
- Target type: `puddle_operation` in `src/mgt_operations_module.f90`, array `pudl_db(:)`.

File format

Line 1 title and line 2 header are skipped. Each remaining line is one record, list-directed.

#	Field	Type	Default	Units	Description
1	name	char(40)	“”	none	operation name
2	wet_hc	real	0	mm/h	upper-layer hydraulic conductivity after puddling
3	sed	real	0	ppm	sediment concentration after puddling
4	orgn	real	0	ppm	organic N concentration after puddling
5	sedp	real	0	ppm	organic P concentration after puddling
6	no3	real	0	ppm	NO3-N concentration after puddling
7	solp	real	0	ppm	mineral (soluble) P concentration after puddling
8	nh3	real	0	ppm	NH3 concentration after puddling
9	no2	real	0	ppm	NO2 concentration after puddling

Related files

- `*.ops` for the other operation databases that follow the same skip-two-and-read pattern.

transplant.plt

Plant transplant database. Initial state for a plant that is established by transplant rather than from seed. Operation rows with `op = plnt` look up `op_plant` against this database.

- Reader: `src/plant_transplant_read.f90`.
- File name: hard-coded `transplant.plt`.
- Target type: `plant_transplant_db` in `src/plant_data_module.f90`, array `transpl(:)`.

File format

Line 1 title and line 2 header are skipped. Each remaining line is one record.

#	Field	Type	Default	Units	Description
1	name	char(40)	`frsd`	none	transplant entry name
2	lai	real	0	$m^{2/m}2$	initial leaf area index
3	bioms	real	0	kg/ha	initial biomass
4	phuacc	real	0	fraction	initial fraction of accumulated heat units
5	fr_yrmat	real	0.05	fraction	fraction of current year of growth to years to maturity
6	pop	real	0	plants/m^2	initial plant population

Related files

- plants.plt for the underlying plant parameter database.
- management.sch resolves the op_plant field against transplant.plt for plnt operations.

co2.out and co2_yr.dat

CO2 driver. Despite the .out extension, co2_read is the routine that establishes the per-year atmospheric CO2 series used by the simulation. The reader reads co2_yr.dat if present, fills the per-year array co2y(:), and writes the resolved series to co2.out as a record of what was used.

- Reader: src/co2_read.f90.
- Input file (optional): co2_yr.dat.
- Output file: co2.out (unit 2222).
- Fallback: when co2_yr.dat is missing, every simulation year is filled with bsn_prm%co2 from parameters.bsn.

co2_yr.dat format

Line	Content
1	title (skipped)
2	integer yrs, the number of year records that follow
3	header (skipped)
4 to 3+yrs	one record per year: iyr (int, calendar year) and co2 (real, ppm)

How the series is built

For a simulation that runs time%nbyr years starting in time%yrc_start:

- If co2_yr.dat is absent or empty, every simulation year is set to bsn_prm%co2.
- If the latest year in co2_yr.dat is at or before the simulation start, every simulation year is set to that final value.
- Otherwise the file is aligned to the simulation start. Simulation years that fall before the first record use the first record's value; years that fall after the last record carry the last value forward.

co2.out format

Header YR CO2(ppm), then one row per simulation year with the calendar year and the resolved ppm value.

Related files

- codes.bsn and parameters.bsn for the co2 fallback in bsn_prm.

object.prt

Per-object output selector. Each row asks the model to write one specific hydrograph or state variable for one spatial object to a named file. Reuses the old saveconc mechanism.

- Reader: src/object_read_output.f90.
- Default file name: object.prt (from in_sim%object_prt in input_file_module.f90).
- Target type: object_output in src/hydrograph_module.f90, array ob_out(:).

File format

Line 1 title and line 2 header are skipped. Each remaining row is one selection.

#	Field	Type	Description
1	k	int	row id (the reader also uses it to size the array)
2	obtyp	char(10)	object type (see below)
3	obtypno	int	object number within `obtyp` (1-based)
4	hydtyp	char(20)	hydrograph or state code (see below)
5	filename	char(26)	output file name to open and write to

Object types recognised by the reader:

Code	Object
`hru`	HRU
`hlt`	HRU LTE
`ru`	routing unit
`res`	reservoir
`cha`	channel
`exc`	export coefficient object
`dr`	delivery ratio object
`out`	outlet
`sdc`	SWAT-Deg channel

Hydrograph or state codes:

Code	Content	Header written
`tot`	total flow	`hyd_hdr_time hyd_hdr`
`rhg`	recharge	`hyd_hdr_time hyd_hdr`
`sur`	surface	`hyd_hdr_time hyd_hdr`
`lat`	lateral	`hyd_hdr_time hyd_hdr`
`til`	tile	`hyd_hdr_time hyd_hdr`
`sol_water`	soil moisture by layer	`hyd_hdr_time sol_hdr`
`solnut_ly`	soil N and P by layer	(no header)
`solnut_pr`	soil N and P for profile	(no header)
`plant`	plant status	`hyd_hdr_time plt_hdr plt_hdr`
`cha_fp`	channel and flood-plain water balance	`hyd_hdr_time fp_hdr`

Each row opens its own file at unit `6100 + i` and writes the matching header. The simulation appends to that file as the run progresses.

Related files

- `print.prt` controls the main output selection. `object.prt` is the side channel for per-object hydrograph files.

Purpose

The initial-conditions files set the starting state of land and water objects before the first simulation day. The general files are:

- `plant.ini`. Named plant communities. One community is a list of plants with initial LAI, biomass, heat-unit accumulation, plant population, fraction of years to maturity, and residue cover. Land-use entries in `landuse.lum` point at a community name through the `plant_cov` field.
- `soil_plant.ini`. Named soil and plant initial-state records combining a starting soil-water fraction with names of nutrient, pesticide, pathogen, salt, heavy-metal, and constituent initial-condition records. `hru-data.hru` references one row of this file per HRU through `soil_plant_init`.
- `om_water.ini`. Initial organic matter and water properties for water bodies (aquifers, channels, ponds, reservoirs, wetlands). One row per named initialisation record.

The solute-specific initial-condition files (pest_hru.ini, pest_water.ini, path_hru.ini, path_water.ini, hmet_hru.ini, hmet_water.ini, salt_hru.ini, salt_water.ini, cs_hru.ini, cs_aqu.ini, cs_channel.ini) are declared in the same type input_init block but documented on the module pages linked at the bottom.

Source

Filename strings: src/input_file_module.f90, type input_init.

File	Reader	Type populated
plant.ini	src/readpcom.f90	type plant_community_db (with type plant_init_db array) in src/plant_data_module.f90
soil_plant.ini	src/soil_plant_init.f90	type soil_plant_initialize in src/hru_module.f90
om_water.ini	src/om_water_init.f90	type hyd_output in src/hydrograph_module.f90

All three readers skip line 1 (title) and line 2 (header), then read records list-directed. Column positions in the reference examples are cosmetic indentation. If the file is missing or the filename in file.cio is null, the reader allocates an empty array and continues.

plant.ini

Each plant community is a header record followed by plt_cnt plant records.

Community record (3 fields)

Read into pcomdb(icom).

#	Field	Type	Units	Default	Description
1	name	char(40)	none	“frsd_frsd”	community name. Referenced from `landuse.lum` (`plant_cov`)
2	plants_com	int	none	1	number of plant records that follow
3	rot_yr_ini	int	none	1	starting year in the rotation

Plant record (8 fields, repeated `plants_com` times)

Read into `pcomdb(icom)%pl(iplt)` (`type plant_init_db`).

#	Field	Type	Units	Default	Description
1	cpnm	char(40)	none	“frsd”	plant name. Must match a row in `plants.plt`. The reader looks the name up in `pldb` and stores the index in `db_num`. If the name is not found a message is written to `9001` (`diagnostics.out`)
2	igro	char(1)	none	“y”	land cover status. `y` = growing, `n` = not growing
3	lai	real	$m^{2/m}2$	0.0	initial leaf area index
4	bioms	real	kg/ha	0.0	initial above-ground biomass
5	phuacc	real	fraction	0.0	fraction of plant heat units accumulated at simulation start
6	pop	real	plants/m^2	0.0	plant population
7	fr_yrmat	real	fraction	1.0	fraction of current year of growth to years to maturity. Used for perennial trees
8	rsdin	real	kg/ha	10000.0	initial surface residue

Example

From `refdata/Ames_sub1/plant.ini`. First two communities, indentation trimmed for width:

```
plant.ini:
pcom_name   plt_cnt  rot_yr_ini  plt_name  lc_status  lai_init  bm_init  phu_init  plnt_pop  yrs_init  rsd
cosy_comm         2           1
                              corn        n   0.00000      0.00000   0.00000   0.00000   0.00000   10000.0
                              soyb        n   0.00000      0.00000   0.00000   0.00000   0.00000   10000.0
corn_comm         1           1
                              corn        n   0.00000      0.00000   0.00000   0.00000   0.00000   10000.0
```

soil_plant.ini

One row per initialisation record. Rows are referenced by name from `hru-data.hru` through the `soil_plant_init` field.

The number of fields read depends on `bsn_cc%nam1`. When `nam1 == 0` (the default) the reader reads 7 fields. When `nam1 /= 0` it reads an eighth field (`csc`) for the constituent-side initial-condition record. `nam1` is declared in `basin_module.f90` as "not used", so in practice the 7-field form is what is written.

Row (7 fields, or 8 with constituents)

Read into `sol_plt_ini(ii)` (`type soil_plant_initialize`).

#	Field	Type	Units	Default	Description
1	name	char(40)	none	“”	record name. Referenced from **hru-data.hru**
2	sw_frac	real	fraction	0.0	initial soil water content as a fraction of field capacity
3	nutc	char(40)	none	“”	name of an initial nutrient record (links to soil nutrient initialisation)
4	pestc	char(40)	none	“”	name of an initial pesticide record. See pest_hru.ini
5	pathc	char(40)	none	“”	name of an initial pathogen record. See path_hru.ini
6	saltc	char(40)	none	“”	name of an initial salt record. See salt_hru.ini
7	hmetc	char(40)	none	“”	name of an initial heavy-metal record. See hmet_hru.ini
8	csc	char(40)	none	“”	name of an initial constituent record. Read only when **bsn_cc%nam1 /= 0**. See cs_hru.ini

Use `null` for any solute-side column that is not active.

Example

From `refdata/Ames_sub1/soil_plant.ini`:

```
soil_plt_ini
       NAME     SW_FRAC    NUT       PEST   PATH   SALT   HMET
soilplant1         0.5    soilnut1   null   null   null   null
```

om_water.ini

One row per named record. Rows are referenced by name from the connectivity files of water bodies (aquifers in `aqu_read_init.f90`, channels, reservoirs, ponds, wetlands).

Row (19 fields)

The reader calls `read (..., om_init_name(ichi), om_init_water(ichi))`. The first field is the name. The remaining 18 fields fill all members of `type hyd_output` in declaration order.

#	Field	Type	Units	Default	Description
1	name	char(16)	none	“”	record name
2	flo	real	m^3 (or fraction for some objects)	0.0	water volume
3	sed	real	metric tons	0.0	sediment
4	orgn	real	kg N	0.0	organic N (PART_N in writer header)
5	sedp	real	kg P	0.0	organic P (PART_P)
6	no3	real	kg N	0.0	NO3-N
7	solp	real	kg P	0.0	mineral soluble P
8	chla	real	kg	0.0	chlorophyll-a
9	nh3	real	kg N	0.0	NH3
10	no2	real	kg N	0.0	NO2
11	cbod	real	kg	0.0	carbonaceous biological oxygen demand
12	dox	real	kg	0.0	dissolved oxygen
13	san	real	tons	0.0	detached sand
14	sil	real	tons	0.0	detached silt
15	cla	real	tons	0.0	detached clay
16	sag	real	tons	0.0	detached small aggregates
17	lag	real	tons	0.0	detached large aggregates
18	grv	real	tons	0.0	gravel
19	temp	real	deg C	0.0	water temperature

The `flo` field is interpreted as a fraction of principal storage for reservoirs and ponds, and as an absolute volume for channels. The interpretation is made by the consumer (`cal_allo_init.f90` and related routines), not the reader.

Example

From `SWATPlusWaterAllocation/documents/wro_test/om_water.ini`:

```
om_wat_ini Generated from ...
       NAME    VOL    SED  PART_N  PART_P    NO3   SOLP  CHL_A    NH3    NO2  CBN_BOD  DIS_OX   SAND   SIL
 aqu_default     1      0       0       0      2      0      0      0      0        0       0      0
 cha_default   0.1   1000      95       0     70      0      0      5      5        0       0      0
pond_default   0.8    100       1     0.1      1   0.03      0    0.1   0.01        0       0      0
 res_default     1    100       1     0.1      1   0.03      0    0.1   0.01        0       0      0
 wet_default   0.5      0       0       0      2      0      0      0      0        0       0      0
```

Related files

- hru-data.hru. Per-HRU pointer into `soil_plant.ini` through `soil_plant_init`.
- landuse.lum. Land-use entries point at a `plant.ini` community through `plant_cov`.
- plants.plt. Plant parameter database. Every `cpnm` in `plant.ini` must resolve here.
- aquifer.aqu, initial.aqu. Aquifers reference `om_water.ini` records for initial storage.
- *.res, *.wet, sd_channel. Water bodies reference `om_water.ini` for initial state.
- Solute-side initial conditions: pesticide module, pathogen module, metals module, salt module, cs module.

Purpose

This page documents the structural BMP parameter files referenced by HRUs and landscape units: `tiledrain.str`, `septic.str`, `filterstrip.str`, `grassedww.str`, and `bmpuser.str`. File names for the first five are set in `type input_structural` in `src/input_file_module.f90`. The page also covers two closely related files declared elsewhere: `septic.sep` (effluent quality, named in `type input_parameter_databases`), `urban.urb` (urban land-cover parameters, same type), and `satbuffer.str` (saturated buffer, file name hard-coded in the reader).

Each file is optional. If the slot in `file.cio` is `null` or the file is missing, the reader allocates a zero-length array and skips on. Every reader skips two header lines, then reads one record per parameter set.

Records in `tiledrain.str`, `filterstrip.str`, `grassedww.str`, and `bmpuser.str` are referenced by name from `hru-data.hru` (columns `tiledrain`, `vfsfield`, `grassww`, `bmpuser`). Records in `septic.str` are referenced from the management schedule when a septic operation is scheduled.

Source

- File names: src/input_file_module.f90, `type input_structural`, `type input_parameter_databases`
- Readers:
  - src/sdr_read.f90 (`tiledrain.str`)
  - src/sep_read.f90 (`septic.str`)
  - src/septic_parm_read.f90 (`septic.sep`)
  - src/scen_read_filtstrip.f90 (`filterstrip.str`)
  - src/scen_read_grwway.f90 (`grassedww.str`)
  - src/scen_read_bmpuser.f90 (`bmpuser.str`)
  - src/sat_buff_read.f90 (`satbuffer.str`)
  - src/urban_parm_read.f90 (`urban.urb`)

tiledrain.str

Subsurface drain parameter sets. Read into `sdr` (`type subsurface_drainage_parameters` in `src/hru_module.f90`). Referenced by the `tiledrain` column of `hru-data.hru`.

#	Column	Type	Units	Default	Description
1	name	char(40)	none	“null”	parameter set name
2	depth	real	mm	0.0	depth of drain tube from soil surface
3	time	real	hours	0.0	time to drain soil to field capacity
4	lag	real	hours	0.0	drain tile lag time
5	radius	real	mm	0.0	effective radius of drains
6	dist	real	mm	0.0	distance between two drain tubes or tiles
7	drain_co	real	mm/day	0.0	drainage coefficient
8	pumpcap	real	mm/hr	0.0	pump capacity
9	latksat	real	none	0.0	multiplication factor for lateral saturated hydraulic conductivity

Example from `refdata/Osu_1hru/tiledrain.str`:

```
tiledrain.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                        dp          t_fc           lag           rad          dist         drain
mw24_1000           1000.00000      24.00000      96.00000     100.00000      30.00000      10.00000
```

Editor column names (`dp`, `t_fc`, `rad`, `drain`, `pump`, `lat_ksat`) differ from source field names. The reader uses position only.

septic.str

Per-HRU septic system parameter sets (geometry, biozone properties, decay coefficients). Read into `sep` (`type septic_system` in `src/septic_data_module.f90`).

#	Column	Type	Units	Description
1	name	char(13)	none	parameter set name
2	typ	int	none	septic system type
3	yr	int	year	year the septic system became operational
4	opt	int	none	operation flag (1 = active, 2 = failing, 0 = not operated)
5	cap	real	none	number of permanent residents
6	area	real	m^2	average drainfield area per septic system
7	tfail	int	days	time until failing system gets fixed
8	z	real	mm	depth to top of biozone layer
9	thk	real	mm	thickness of biozone layer
10	strm_dist	real	km	distance to stream
11	density	real	per km^2	number of septic systems per square kilometre
12	bd	real	kg/m^3	density of biomass
13	bod_dc	real	m^3/day	BOD decay rate coefficient
14	bod_conv	real	none	mass conversion factor between bacterial growth and BOD degraded
15	fc1	real	none	linear coefficient for biozone field capacity
16	fc2	real	none	exponential coefficient for biozone field capacity
17	fecal	real	m^3/day	fecal coliform decay rate coefficient
18	plq	real	none	plaque conversion factor from TDS
19	mrt	real	none	mortality rate coefficient
20	rsp	real	none	respiration rate coefficient
21	slg1	real	none	slough-off calibration parameter
22	slg2	real	none	slough-off calibration parameter
23	nitr	real	none	nitrification rate coefficient
24	denitr	real	none	denitrification rate coefficient
25	pdistrb	real	L/kg	linear P sorption distribution coefficient
26	psorpmax	real	mg P/kg soil	maximum P sorption capacity
27	solpslp	real	none	slope of linear effluent soluble P equation
28	solpintc	real	none	intercept of linear effluent soluble P equation

Example from `refdata/Osu_1hru/septic.str`:

```
septic.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    typ         yr  operation     residents          area     t_fail       dp_bioz      tl
standard                  1          0          0       2.50000     100.00000         70     500.00000      50
failing                   1          0          0       2.50000     100.00000          1     500.00000      50
```

septic.sep

Effluent quality database. One record per effluent type. Records are referenced by index from elsewhere (no name resolution in the reader). Read into `sepdb` (`type septic_db` in `src/septic_data_module.f90`). File name is in

`in_parmdb%septic_sep`.

#	Column	Type	Units	Description
1	sepnm	char(20)	none	effluent set name
2	qs	real	m^3/day	flow rate per capita
3	bodconcs	real	mg/L	biological oxygen demand
4	tssconcs	real	mg/L	total suspended solids
5	nh4concs	real	mg/L	NH4-N concentration
6	no3concs	real	mg/L	NO3-N concentration
7	no2concs	real	mg/L	NO2-N concentration
8	orgnconcs	real	mg/L	organic N concentration
9	minps	real	mg/L	mineral P concentration
10	orgps	real	mg/L	organic P concentration
11	fcolis	real	mg/L	fecal coliform concentration

The source comment on `nh4concs` reads "concentration of total phosphorus"; the field is the NH4-N concentration based on column position and downstream use. Example from `refdata/Osu_1hru/septic.sep` (first two records shown):

```
septic.sep: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                    q_rate           bod           tss          nh4_n          no3_n          no2_n
gcon                   0.22700     170.00000      75.00000      42.40000        0.00000        0.00000
gadv                   0.22700      22.00000      14.00000      18.90000        9.60000        0.00000
```

filterstrip.str

Vegetative filter strip parameter sets. Read into `filtstrip_db` (`type filtstrip_operation` in `src/mgt_operations_module.f9`
Referenced by the `vfsfield` column of `hru-data.hru`.

#	Column	Type	Units	Default	Description
1	name	char(40)	none	“”	parameter set name
2	vfsi	int	none	0	on/off flag for vegetative filter strip
3	vfsratio	real	none	0.0	drainage-area to filter-strip-area ratio
4	vfscon	real	none	0.0	fraction of total runoff from the entire field
5	vfsch	real	none	0.0	fraction of flow entering the most concentrated 10% of the VFS that is fully channelized

Example from `refdata/Osu_1hru/filterstrip.str`:

```
filterstrip.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag       fld_vfs       con_vfs         cha_q  description
field_border             0       0.10000       0.00300       0.20000  Field_border
high_engineered          0       0.10000       0.00100       0.05000  Highly_engineered_low_channelized
```

grassedww.str

Grassed waterway parameter sets. Read into `grwaterway_db` (`type grwaterway_operation` in `src/mgt_operations_module.f9`
Referenced by the `grassww` column of `hru-data.hru`.

#	Column	Type	Units	Default	Description
1	name	char(40)	none	“”	parameter set name
2	grwat_i	int	none	0	on/off flag for waterway simulation
3	grwat_n	real	none	0.0	Manning’s n for grassed waterway
4	grwat_spcon	real	none	0.0	sediment transport coefficient
5	grwat_d	real	m	0.0	depth of grassed waterway
6	grwat_w	real	m	0.0	width of grassed waterway
7	grwat_l	real	km	0.0	length of grassed waterway
8	grwat_s	real	m/m	0.0	slope of grassed waterway

Example from `refdata/Osu_1hru/grassedww.str`:

```
grassedww.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag          mann        sed_co            dp            wd           len
grwway_high              0       0.05000       0.02000       1.00000       4.00000       0.50000       0.1
grwway_med               0       0.05000       0.02000       0.75000       3.00000       0.75000       0.0
grwway_low               0       0.05000       0.02000       0.50000       2.00000       1.00000       0.0
```

bmpuser.str

User-defined upland BMP removal-efficiency sets. Read into `bmpuser_db` (`type bmpuser_operation` in `src/mgt_operations_module.f90`). Referenced by the `bmpuser` column of `hru-data.hru`.

#	Column	Type	Units	Default	Description
1	name	char(40)	none	“”	parameter set name
2	bmp_flag	int	none	0	on/off flag
3	bmp_sed	real	percent	0.0	sediment removal
4	bmp_pp	real	percent	0.0	particulate P removal
5	bmp_sp	real	percent	0.0	soluble P removal
6	bmp_pn	real	percent	0.0	particulate N removal
7	bmp_sn	real	percent	0.0	soluble N removal
8	bmp_bac	real	percent	0.0	bacteria removal

Example from `refdata/Osu_1hru/bmpuser.str`:

```
bmpuser.str: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  flag       sed_eff      ptlp_eff      solp_eff      ptln_eff      soln_eff      bact
bmpusr1                  1       0.20000       0.20000       0.20000       0.20000       0.20000       0.2
```

Note: a second type `bmpuser_operation1` in the same module exposes a wider parameter set (surface, subsurface, tile flow and constituents). The reader populates `bmpuser_db` (`bmpuser_operation`); the wider type is not read here.

satbuffer.str

Saturated buffer source-receiver pairs. Read into `satbuff_db` (`type saturated_buffer_parameters` in `src/hru_module.f90`). The file name is hard-coded as `"satbuffer.str"` in the reader, not listed in `type input_structural`. The file is loaded only when it exists.

#	Column	Type	Description
1	name	char(40)	parameter set name
2	hru_src	int	source HRU contributing tile inflow
3	frac_src	real	fraction of source HRU contributing to tile flow
4	flocon_dtbl	char(40)	flow-control decision table name (resolved against `dtbl_flo`)
5	hru_rcv	int	receiving (buffer) HRU
6	lyr	int	soil layer for incoming tile flow (0 = surface)

After reading, the reader copies the parameter set into `hru(hru_src)%sb%sb_db` and `hru(hru_rcv)%sb%sb_db`, then resolves `flocon_dtbl` against the loaded flow-control decision tables (`dtbl_flo`) to set `hru(hru_src)%sb%dtbl`.

urban.urb

Urban land-cover parameter database (build-up, wash-off, percent impervious, urban CN). Read into `urbdb` (`type urban_db` in `src/urban_data_module.f90`). File name is in `in_parmdb%urban_urb`. Urban parameter sets are referenced from `landuse.lum` for HRUs flagged as urban.

#	Column	Type	Units	Default	Description
1	urbnm	char(16)	none	“”	urban land-cover name
2	fimp	real	fraction	0.05	fraction of HRU area that is impervious
3	fcimp	real	fraction	0.05	fraction of HRU classified as directly connected impervious
4	curbden	real	km/ha	0.0	curb length density
5	urbcoef	real	1/mm	0.0	wash-off coefficient for removal of constituents from an impervious surface
6	dirtmx	real	kg/curb-km	1000.0	maximum amount of solids allowed to build up on impervious surfaces
7	thalf	real	days	1.0	time for solids on impervious areas to build up to half the maximum
8	tnconc	real	mg N / kg sed	0.0	total nitrogen concentration in suspended-solid load from impervious areas
9	tpconc	real	mg P / kg sed	0.0	total phosphorus concentration in suspended-solid load from impervious areas
10	tno3conc	real	mg NO3-N / kg sed	0.0	NO3-N concentration in suspended-solid load from impervious areas
11	urbcn2	real	none	98.0	moisture condition II curve number for impervious areas

Example from `refdata/Osu_1hru/urban.urb` (first three records shown):

```
urban.urb: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
name                  frac_imp   frac_dc_imp      curb_den      urb_wash      dirt_max     t_halfmax     c
urhd                   0.60000       0.44000       0.24000       0.18000     225.00000       0.75000     5
urmd                   0.38000       0.30000       0.24000       0.18000     225.00000       0.75000     5
urml                   0.20000       0.17000       0.24000       0.18000     225.00000       0.75000     4
```

Related files

- file.cio. The `structural` and `hru_parm_db` lines select which of these files are read.
- hru-data.hru. References `tiledrain.str`, `filterstrip.str`, `grassedww.str`, and `bmpuser.str` by name.
- landuse.lum. References `urban.urb` entries for urban HRUs.

Purpose

The pesticide module simulates user-listed pesticides across foliage, soil, channel water, benthic sediment, and aquifers. The set of pesticides simulated is declared in constituents.cs, which populates `cs_db%num_pests`. The files below load the parameter database, the per-object initial conditions, and the parent-daughter metabolite chain.

Activation

The pesticide parameter database `pesticide.pes` is referenced from `file.cio` as `in_parmdb%pest` (default `pesticide.pes`). The initial-condition files are referenced from `file.cio` via `in_init%pest_soil` (default `pest_hru.ini`) and `in_init%pest_water` (default `pest_water.ini`).

`pest_parm_read` is called from `proc_db.f90`. `pest_metabolite_read` and `pest_hru_aqu_read` are called from `proc_read.f90`. `pest_cha_res_read` is called later from `main.f90`.

Files

pesticide.pes

Pesticide parameter database. One row per pesticide.

- Reader: src/pest_parm_read.f90
- Type: `pestdb(0:pestparm)` (`type pesticide_db`) in src/pesticide_data_module.f90. The reader also fills `pestcp(ip)%decay_f`, `decay_s`, `decay_a`, `decay_b` from the half-life columns as `exp(-0.693 / hlife)`.
- Format: title line, header line, then one line per pesticide.

#	Field	Units	Description
1	name	-	pesticide name
2	koc	mL/g	soil adsorption coefficient normalized for soil organic carbon
3	washoff	none	fraction of pesticide on foliage washed off by a rainfall event
4	foliar_hlife	days	half-life on foliage
5	soil_hlife	days	half-life in soil
6	solub	mg/L	solubility in water
7	aq_hlife	days	aquatic half-life
8	aq_volat	m/day	aquatic volatilization coefficient
9	mol_wt	g/mol	molecular weight, used for mixing velocity
10	aq_resus	m/day	aquatic resuspension velocity for sorbed pesticide
11	aq_settle	m/day	aquatic settling velocity for sorbed pesticide
12	ben_act_dep	m	depth of the active benthic layer
13	ben_bury	m/day	burial velocity in benthic sediment
14	ben_hlife	days	half-life in benthic sediment

#	Field	Units	Description
15	pl_uptake	none	fraction taken up by plant
16	descrip	-	description string

pest_hru.ini

Initial pesticide concentrations for HRU soils and plants. One block per record.

- Reader: src/pest_hru_aqu_read.f90
- Type: `pest_soil_ini` (`type cs_soil_init_concentrations`) in src/constituent_mass_module.f90. Fields: `name`, `soil(num_pests)` (ppm), `plt(num_pests)` (ppm).
- Format: one title line, then per record: a header line, a name line, then `num_pests` lines each containing `pest_name, soil_value, plt_value` in the order declared in `constituents.cs`.

The same reader populates `cs_pest_solsor(num_pests)` (allocated only).

pest_water.ini

Initial pesticide concentrations for channels and reservoirs (water column and benthic).

- Reader: src/pest_cha_res_read.f90
- Type: `pest_water_ini` (`type cs_water_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `water(num_pests)`, `benthic(num_pests)`. Record names are stored in `pest_init_name`.
- Format: title line, header line, then per record: name line, then `num_pests` pairs of lines (water value, benthic value). The values are then read again on the second pass as a single record line of `name, water(:), benthic(:)`.

pest_metabolite.pes

Parent-daughter metabolite chain. Filename is hard-coded in the reader (not driven by `file.cio`).

- Reader: src/pest_metabolite_read.f90
- Target: `pestcp(ip)%daughter(num_metab)` (`type daughter_decay_fractions`) and `pestcp(ip)%num_metab` in `src/pesticide_data_module.f90`. The daughter `%num` field is set by cross-walk against `cs_db%pests` so the metabolite must also appear in `constituents.cs`.
- Format: title line, header line, then per parent: one line `parent_name num_metab`, followed by `num_metab` lines of `daughter_name, foliar_fr, soil_fr, aq_fr, ben_fr`. The parent name must match an entry in `pesticide.pes`.

Field	Units	Description
name	-	daughter pesticide name
foliar_fr	0-1	fraction of parent on foliage that degrades to this daughter
soil_fr	0-1	fraction of parent in soil that degrades to this daughter
aq_fr	0-1	fraction of parent in aquatic phase that degrades to this daughter
ben_fr	0-1	fraction of parent in benthic phase that degrades to this daughter

Related

- constituents.cs. Declares the pesticide list and sets `cs_db%num_pests`.
- pathogen module. Same per-object initial-condition pattern.
- metals module. Same per-object initial-condition pattern.

Purpose

The pathogen module simulates user-listed pathogens across soil, foliage, channel water, benthic sediment, and aquifers. The set of pathogens simulated is declared in constituents.cs, which populates cs_db%num_paths. The files below load the parameter database and the per-object initial conditions.

Activation

The pathogen database pathogens.pth is referenced from file.cio as in_parmdb%pathcom_db (default pathogens.pth). The initial-condition files are referenced via in_init%path_soil (default path_hru.ini) and in_init%path_water (default path_water.ini).

path_parm_read is called from proc_db.f90. path_hru_aqu_read is called from proc_read.f90. path_cha_res_read is called later from main.f90.

Files

pathogens.pth

Pathogen parameter database. One row per pathogen.

- Reader: src/path_parm_read.f90
- Type: path_db(0:path) (type pathogen_db) in src/pathogen_data_module.f90.
- Format: title line, header line, then one line per pathogen.

#	Field	Units	Description
1	pathnm	-	pathogen name
2	do_soln	1/day	die-off factor in soil solution
3	gr_soln	1/day	growth factor in soil solution
4	do_sorb	1/day	die-off factor sorbed to soil particles
5	gr_sorb	1/day	growth factor sorbed to soil particles
6	kd	none	partition coefficient between solution and sorbed phase in surface runoff
7	t_adj	none	temperature adjustment factor for die-off and growth
8	washoff	none	fraction on foliage washed off by a rainfall event
9	do_plnt	1/day	die-off factor on foliage
10	gr_plnt	1/day	growth factor on foliage
11	fr_manure	none	fraction of manure with active colony forming units
12	perco	none	percolation coefficient, ratio of solution bacteria in the surface layer
13	det_thrshd	cfu/m^2	detection threshold below which soil pathogen levels are zeroed
14	do_stream	1/day	die-off factor in streams
15	gr_stream	1/day	growth factor in streams
16	do_res	1/day	die-off factor in reservoirs
17	gr_res	1/day	growth factor in reservoirs
18	conc_min	-	minimum pathogen concentration

path_hru.ini

Initial pathogen concentrations for HRU soils and plants. One block per record.

- Reader: src/path_hru_aqu_read.f90
- Type: path_soil_ini (type cs_soil_init_concentrations) in src/constituent_mass_module.f90. Fields: name, soil(num_paths), plt(num_paths) in cfu/m^2.
- Format: one title line, then per record: a header line, a name line, then one line containing name, soil(:), plt(:) for all pathogens in constituents.cs order.

The same reader allocates cs_path_solsor(num_paths).

path_water.ini

Initial pathogen concentrations for channels and reservoirs (water column and benthic).

- Reader: src/path_cha_res_read.f90
- Type: `path_water_ini` (`type cs_water_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `water(num_paths)`, `benthic(num_paths)`. Record names are stored in `path_init_name`.
- Format: title line, header line, then per record: name line, then `num_paths` pairs of lines (water value, benthic value). On the second pass the reader reads a single line `name, water(:), benthic(:)` per record.

Related

- constituents.cs. Declares the pathogen list and sets `cs_db%num_paths`.
- pesticide module. Same per-object initial-condition pattern.
- metals module. Same per-object initial-condition pattern.

Purpose

The heavy metals module simulates user-listed metals across HRU soils and plants. The set of metals simulated is declared in constituents.cs, which populates `cs_db%num_metals`. Only initial soil and plant concentrations are loaded by a dedicated reader; no separate metal parameter database is read by `hmet_hru_aqu_read`.

Activation

`in_parmdb%hmetcom_db` is defined in src/input_file_module.f90 with default `metals.mtl`. No reader in `src/` opens this filename, so the parameter database is declared in `file.cio` but not loaded.

The initial-conditions file is referenced from `file.cio` via `in_init%hmet_soil` (default `hmet_hru.ini`). `hmet_hru_aqu_read` is called from `proc_read.f90`.

Files

hmet_hru.ini

Initial metal concentrations for HRU soils and plants.

- Reader: src/hmet_hru_aqu_read.f90
- Type: `hmet_soil_ini` (`type cs_soil_init_concentrations`) in src/constituent_mass_module.f90. Fields: `name`, `soil(num_metals)` (ppm), `plt(num_metals)` (ppm).
- Format: one title line, then per record: a header line, a name line, then `num_metals` pairs of lines (`metal_name, soil_value` then `metal_name, plt_value`) in `constituents.cs` order.

The same reader allocates `cs_hmet_solsor(num_metals)`.

metals.mtl

Declared as `in_parmdb%hmetcom_db` in `input_file_module.f90` (default name `metals.mtl`). No reader in the current source opens this file. The file is reserved but inert.

Related

- constituents.cs. Declares the metal list and sets `cs_db%num_metals`. The metals block in `constit_db_read` is read but its name-database cross-walk is commented out, so listed metals only contribute the count.
- pesticide module. Same per-object initial-condition pattern.
- pathogen module. Same per-object initial-condition pattern.

Purpose

gwflow is the cell-based 2D groundwater module that replaces the lumped aquifer (*.aqu) when active. It discretises the basin into a structured or unstructured mesh and solves a horizontal groundwater balance per cell, with optional couplings to channels, HRUs/LSUs, reservoirs, wetlands, floodplains, tiles, canals, solute transport, and heat transport.

Input is split across several small *.gw files, one concern per file, written by swatplus-editor from the project database and read by the gwflow_*.f90 routines. The one exception is gwflow.con, the channel-cell connect file, which keeps the .con extension (it is a standard SWAT+ connect file listed in file.cio). The older monolithic gwflow.input master file is no longer written or required.

Activation

Set gwflow = 1 in codes.bsn. The model then expects chancell.gw to exist. basin_read_objs opens it to count channel-cell intersections, sets sp_ob%gwflow to that count, switches aqu_con to null, and points gwflow_con at gwflow.con. If chancell.gw is missing, bsn_cc%gwflow is reset to 0 and the lumped aquifer is used.

Format conventions

- **Line 1 of every file** is a description and timestamp line written by the editor; the reader skips it.
- Most tabular files have **2 preamble lines**: the description line, then a column-header row. Data starts on line 3. (gwflow.con follows the SWAT+ .con convention: a title line, then a header row.)
- Numeric tokens are whitespace separated; the reader uses list-directed input, so exact column widths do not matter. The editor right-justifies for readability.
- Optional override columns (strK, strthick, bc_type, tile_*, init_temp) carry the literal null when unset.
- Cell IDs run 1..ncell, one row per active cell, ascending; row position equals cell_id. Every file that references a cell uses that same ID space, so no cell_id may exceed ncell. For unstructured grids row/col are 0.

File overview

The writer's write() driver decides which files to emit.

File	Written when	Purpose
codes.gw	always	Feature flags and run controls (wide table)
zones.gw	always	Aquifer/streambed zone parameters
cells.gw	always	Per-cell static properties and initial state
cellcon.gw	always	Cell-to-cell connection (neighbour) lists
outputs.gw	always	Output times, observation cells, debug cell
chancell.gw	channel cells exist	Channel-to-cell exchange links (incl. depth-zone and observation flags)
gwflow.con	channel cells exist	Channel-cell spatial connections (.con style; keeps .con)
hrucell.gw	conn_type = 1 (HRU)	HRU-to-cell area mapping
lsucell.gw	conn_type = 2 (LSU)	LSU-to-cell area mapping
rescell.gw	reservoir_exchange	Reservoir-cell links
floodplain.gw	floodplain_exchange	Floodplain-cell links
wetland.gw	wetland_exchange	Wetland bed thickness
tile.gw	tile_drainage	Tile drainage parameters and per-cell flag
solute.gw	solute_transport	Solute transport configuration
cell_sol.gw	solute_transport	Per-cell initial solute concentrations
pumpex.gw	pumpex, if pumps defined	External pumping schedule
hru_pump.gw	if HRUs selected	HRUs to report pumping output for

File	Written when	Purpose

The optional flat files (`tvheads.gw`, `ponds.gw` + `pond_cell.gw` + `pond_div.gw`, `phreato.gw` + `phreato_cell.gw`, `sw_group.gw`, `chan_depth.gw`) are each written when their database table is populated. See their sections below.

The files use the `*.gw` extension (content-first SWAT+ naming, like `plants.plt`); `gwflow.con` is the only one that keeps `.con`. When a flag is off, the writer deletes any stale file.

The standalone channel-cell observation and depth-zone files from the earlier format are gone: they are now trailing columns (`obs`, `dep_zone`) of `chancell.gw`. The daily channel-depth series remains a separate file, `chan_depth.gw`.

`codes.gw`: feature flags and run controls

Wide table, same shape as `codes.bsn`: the description line, then a header row of 23 key names, then one value row, right-justified. The reader matches keys by name, so column order does not matter.

Keys in written order:

```
grid_type ncell cell_size n_rows n_cols bc_type conn_type gw_soil satx pumpex
tile res wet fp canal solute heat time_step write_day write_mon write_yr write_aa river_thresh
```

Key	Meaning
`grid_type`	`structured` or `unstructured`
`ncell`	Number of active cells
`cell_size`	Cell size, m (0 = unstructured)
`n_rows`	Grid rows (0 = unstructured)
`n_cols`	Grid cols (0 = unstructured)
`bc_type`	Default boundary condition (1 = const head, 2 = no flow)
`conn_type`	Recharge connection (1 = HRU, 2 = LSU)
`gw_soil`	Groundwater-to-soil transfer (0/1)
`satx`	Saturation-excess flow (0/1)
`pumpex`	External pumping (0/1)
`tile`	Tile drainage (0/1)
`res`	Reservoir exchange (0/1)
`wet`	Wetland exchange (0/1)
`fp`	Floodplain exchange (0/1)
`canal`	Canal seepage (0/1)
`solute`	Solute transport (0/1)
`heat`	Heat transport (0/1)
`time_step`	Solver time step (days)
`write_day`	Daily output (0/1)
`write_mon`	Monthly output (0/1)
`write_yr`	Annual output (0/1)
`write_aa`	Average-annual output (0/1)
`river_thresh`	River-bed elevation threshold (m)

Descriptions are not stored in the file; the doc-of-record for the key list is `mergeWork/inputWork/gwflow_codes_reference.md`, kept in sync by the editor. Heat is plumbed but off by default (`heat 0`).

Output frequency is hybrid: the `write_day`/`write_mon`/`write_yr`/`write_aa` flags here set the gwflow output frequency, but `print.prt` overrides them when it carries `gwflow_wb`, `gwflow_obs`, or `gwflow_pump` rows. With no such rows in `print.prt`, the `codes.gw` flags win.

`zones.gw`: aquifer and streambed zones

2 preamble lines. One row per zone, `zone_id` ascending. 6 columns:

#	Column	Meaning
1	zone_id	Zone index
2	aquifer_K	Aquifer hydraulic conductivity (m/day)
3	aquifer_Sy	Specific yield
4	streambed_K	Streambed hydraulic conductivity (m/day)
5	streambed_thick	Streambed thickness (m)
6	thermal_K	Thermal conductivity (heat batch; 0 when off)

cells.gw: cell properties and initial state

2 preamble lines. One row per active cell, id running 1..ncell. 23 columns, leading with the SWAT+ object convention id / name / gis_id:

#	Column	Meaning
1	id	Cell index, 1..ncell, equal to row position
2	name	Cell name, cellNNNN (zero-padded, e.g. cell0001)
3	gis_id	Authoritative GIS cell id from gwflowcells.shp, equal to id (1..ncell)
4	status	Cell status code
5	elev	Ground-surface elevation (m)
6	thck	Aquifer thickness (m)
7	K_zone	Aquifer K zone id
8	Sy_zone	Specific-yield zone id
9	delay	Recharge delay (days)
10	exdp	ET extinction depth (m)
11	init	Initial groundwater head (m)
12	x	Cell-centre x coordinate
13	y	Cell-centre y coordinate
14	area	Cell area (m²)
15	strK	Streambed K override (null if unset)
16	strthick	Streambed thickness override (null if unset)
17	bc_type	Per-cell boundary condition override (null if unset)
18	tile_depth	Tile depth override (null if unset)
19	tile_area	Tile area override (null if unset)
20	tile_K	Tile K override (null if unset)
21	row	Structured grid row (0 if unstructured)
22	col	Structured grid col (0 if unstructured)
23	init_temp	Initial groundwater temperature (null when heat off)

gis_id is the cell id carried straight from the QSWAT+ gwflowcells.shp layer (it equals id, since the shapefile numbers active cells 1..ncell). It is used for the gis_id output column and for cell and pump naming, replacing an earlier row/col computation. The reader consumes the columns by position, so column order is fixed.

cellcon.gw: cell-to-cell connections

2 preamble lines (description, then cell_id neighbor_tot neighbor_ids). One variable-width row per cell 1..ncell:

- cell_id
- neighbor_tot (neighbour count)
- then neighbor_tot neighbour cell_ids

This defines the solver stencil (which cells are adjacent).

outputs.gw: output control

2 preamble lines (description, then category value note). One row per item:

category	value
head_output_time	output time, YYYYDDD (one row per requested time)
observation_cell	cell_id of an observation well (one row each)
detail_debug_cell	a cell_id, or 0 for none

chancell.gw: channel-to-cell links

2 preamble lines (description, then column header). One row per channel-cell link. 7 columns:

#	Column	Meaning
1	cell_id	Cell exchanging with the channel (1..ncell)
2	elev_m	Channel-bed elevation (m)
3	channel	Channel id (matches chandeg.con)
4	riv_length_m	Channel length within the cell (m)
5	zone	Depth-zone index
6	dep_zone	Channel depth-zone index, links the cell to its series in chan_depth.gw
7	obs	1 to report gw-channel exchange output for this cell, else 0

A cell may appear on multiple rows when more than one channel crosses it. The dep_zone and obs columns fold in two files from the earlier format (the separate channel-depth-zone file and the channel-observation file).

gwflow.con: channel-cell spatial connections

SWAT+ .con-style file, keeping the .con extension. Line 1 is the title gwflow.con: channel-cell spatial connections; line 2 is the .con header:

```
NUMB NAME GISID AREA LAT LONG ELEV CELL WST CONST OVER RULE SRC_TOT OBTYPE_OUT1 OBTYPNO_OUT1 HTYPE_OUT1 FR
```

One row per channel cell. CELL is the gwflow cell_id, OBTYPNO_OUT1 is the channel connect index, with fixed sdc / tot / 1.00 in the type/total/fraction fields.

hrucell.gw: HRU-to-cell mapping

2 preamble lines (description, then column header). One row per HRU-cell overlap. 4 columns:

#	Column	Meaning
1	hru	HRU id
2	area_m2	HRU area (m^2)
3	cell_id	Cell overlapping the HRU
4	overlap_m2	Overlap area (m^2)

lsucell.gw is the LSU analogue, written when conn_type is 2.

wetland.gw: wetland bed thickness

2 preamble lines (description, then name bed_thick). One row per wetland: wetland name, bed thickness (m).

hru_pump.gw: HRU pump-observation list

Selector list of HRUs to report pumping output for. Existence-gated: the editor writes it when the source table has rows, and removes it otherwise; no codes.gw flag. Meta line, then a count, then one hru_id per line (ascending).

Other conditional files

Written only when the matching `codes.gw` flag is on. These keep a title/description line then a blank line before data (no column-header row):

- `lsucell.gw`: LSU-to-cell mapping, one row per overlap: `lsu_id  cell_id  area_m2`.
- `rescell.gw`: reservoir-cell links: bed thickness and bed K, count, then `cell_id  res_id  res_stage`. (Not present when no reservoir cells.)
- `floodplain.gw`: floodplain-cell links: count, then `cell_id  channel_id  K  area_m2`. (Not present when no floodplain cells.)
- `tile.gw`: tile drainage: a defaults row `tile_depth  tile_area  tile_k`, then a tile-group count, then a per-cell `cell_id` list.
- `solute.gw`: solute transport config: transport steps, dispersion coefficient, solute count `nsolute`, then one row per solute `name  sorption_coef  rate_const  init_conc_aquifer  init_conc_recharge`.

`cell_sol.gw`: per-cell initial solute concentrations

Written when solute transport is on. 2 preamble lines (description, then `cell_id  conc_1 … conc_nsolute`). One row per cell `1..ncell`, with one concentration column per solute in `solute.gw` order:

```
cell_sol.gw: ...
 cell_id        conc_1        conc_2   ...   conc_10
       1      12.50000       0.00000   ...   0.00000
       2       7.70000       0.00000   ...   0.00000
```

New flat optional files

These optional files were harmonized to the flat convention: a description line, a header row, then rows to end-of-file with no embedded count lines. Each is existence-gated, written from its database table when populated. The formats below match the editor's on-disk output. The deferred items `minerals.gw` (salt-mineral parameters) and `transit.gw` (transit-time tracking) are reader-supported but not exported by the editor.

`tvheads.gw`: time-varying boundary heads

One row per boundary cell: `cell_id` then one head value per simulation year (positional, `1..Nbyr`; the reader takes `Nbyr` from the run period, so the file has no count line). A `0` means no specified head that year.

```
tvheads.gw: ...
   cell_id      head_yr1      head_yr2      head_yr3      head_yr4      head_yr5      head_yr6
         1       0.00000      12.50000       0.00000       9.00000       0.00000       0.00000
         5       0.00000       0.00000       7.00000       0.00000       0.00000       4.40000
```

`ponds.gw` + `pond_cell.gw`: recharge ponds

`ponds.gw` is one row per pond: 11 fixed columns then one `unl_conc` per solute (`unl_conc_1 … unl_conc_Nsolute`, unsaturated-zone concentration of each solute):

```
ponds.gw: ...
 pond_id  area  chan  canal  unl  bed_k  wsta  evap_co  start_yr  start_mo  start_day  unl_conc_1..unl_con
       1   25000.00   5   0   1   0.000100   3   0.70000   1981   4   1    2.50  0.30  ...
```

Column	Meaning
`pond_id`	Pond index
`area`	Pond area (m²)
`chan`	Target channel id
`canal`	Target canal id
`unl`	Unsaturated-zone flag (0/1)
`bed_k`	Pond-bed hydraulic conductivity (m/day)

Column	Meaning
`wsta`	Weather-station id
`evap_co`	Evaporation coefficient
`start_yr` / `start_mo` / `start_day`	Divert start date
`unl_conc_1..N`	Unsaturated-zone concentration per solute

`pond_cell.gw` lists pond-to-cell connections, one per row: `pond_id  cell_id  conn_area` (m^2). The number of connection rows defines the per-pond cell count; there is no count line.

`pond_div.gw`: pond daily diversion series

Daily diversion volume into each pond. Dense: one row per simulation day (0-filled on days with no diversion). The columns after the date are one `div_pondN` per pond, in `ponds.gw` pond-id order:

```
pond_div.gw: ...
    year   month     day     div_pond1     div_pond2
    1980       1       1       0.00000       0.00000
    1980       1       2       0.00000       0.00000
```

- `year` / `month` / `day`: calendar date of the row. The reader keys on row order (positional), so every simulation day must be present, including leap days.
- `div_pond1 … div_pondN`: diversion volume into each pond (m^3) that day.
- Existence-gated: written when pond diversions are defined; if absent, diversion is 0.

`sw_group.gw`: channel-cell groups

Groups of channel cells for aggregated reach gw-channel exchange output. One variable-width row per group:

```
sw_group.gw: channel-cell groups
   group_id   ncell   cell_ids
   1   2   42   43
   2   2   44   45
```

- `group_id`, then `ncell` (cells in the group), then `ncell` cell ids (1..ncell space).

`phreato.gw` + `phreato_cell.gw`: phreatophyte ET

Split into a depth-ET curve and the cells it applies to.

`phreato.gw` is the depth-rate curve, one row per point:

```
phreato.gw: depth-ET curve
     depth_m     et_rate_m
   0.0   0.005
   1.0   0.003
   2.0   0.001
```

`phreato_cell.gw` lists the cells, one per row:

```
phreato_cell.gw: phreatophyte cells
   cell_id   area_m2
   22   25000000.0
   23   25000000.0
```

`chan_depth.gw`: daily channel depth per zone

Per-zone daily channel-depth series, one row per simulation day. Wide: `jday  yr` then one depth column per channel depth zone (N = number of `dep_zone` values in `chancell.gw`):

```
chan_depth.gw: daily channel depth per zone
   jday   yr   depth_z1   depth_z2
   1   1980   0.50   0.80
   2   1980   0.50   0.80
```

pumpex.gw format

External pumping schedule, enabled by `pumpex = 1` in `codes.gw`. 2 preamble lines (meta line, then a column header), then one row per pump-period. 7 columns:

```
gwflow.pumpex: external groundwater pumping
      name    cell_id        rate   yr_start   dy_start     yr_end     dy_end
  cell0700        700      150.00       1981          1       1981        180
  cell0700        700       80.00       1981        181       1981        365
  cell0912        912      250.00       1982         90       1984        270
```

#	Column	Meaning
1	`name`	Cell name (`cellNNNN`, matching `cells.gw` col 2); label only, the reader keys on `cell_id`
2	`cell_id`	Pumped cell (1..ncell)
3	`rate`	Pumping rate (m^3/day)
4	`yr_start`	Start year
5	`dy_start`	Start day-of-year (1..365)
6	`yr_end`	End year
7	`dy_end`	End day-of-year (1..365)

- One row per pump-period; rows for the same cell are consecutive, and consecutive same-cell rows form one pump.
- Dates are calendar (year + day-of-year). The reader converts each to a simulation day-count as `(year - sim_start_year) * 365 + day_of_year`, the same 365-day convention as `ponds.gw`.

The editor writes this file (`write_pumpex`) when `pumpex = 1` and pumps are defined in the project database, one row per period straight from the database. It is skipped when no pumps exist.

Example

A 2-zone structured grid (excerpts from a generated project; values are illustrative).

codes.gw (wide, abbreviated)

```
grid_type ncell cell_size n_rows n_cols ... solute heat time_step write_aa river_thresh
structured 1454   5000.00     52     53 ...      0    0      1.00        1         5.00
```

zones.gw

```
   zone_id     aquifer_K    aquifer_Sy   streambed_K streambed_thick
         1       0.50000       0.20000       0.00500         0.50000
         2       2.00000       0.20000       0.00500         0.50000
```

cells.gw (abbreviated)

```
 id  name      gis_id  status   elev   thck K_zone Sy_zone ...  row  col init_temp
  1  cell0001       1       2 395.00  34.99      1       1 ...   52    1      null
```

cellcon.gw

```
 cell_id  neighbor_tot  neighbor_ids
       1             3       2      20      21
```

chancell.gw

```
 cell_id   elev_m  channel  riv_length_m  zone  dep_zone  obs
      42   326.00       66       1853.55     1         1    1
```

Related

- codes.bsn: gwflow = 1 activates the module.
- object.cnt: sp_ob%gwflow is set from the channel-cell count and replaces sp_ob%aqu.
- aquifer.aqu, initial.aqu: not used when gwflow is active.

Purpose

The rtb-salt module simulates the eight salt ions (so4, ca, mg, na, k, cl, co3, hco3) across the soil profile, plant, channel, aquifer, urban, irrigation, fertilizer, atmospheric, and road-salt pathways. Each input file listed below feeds one piece of that state: initial concentrations, applied loads, or per-plant tolerance parameters.

Activation

Salt simulation is gated by the master constituent file constituents.cs referenced from file.cio (in_sim%cs_db, default constituents.cs). The number of salt ions in that file is held in cs_db%num_salts. If num_salts = 0, salt readers that test it (salt_roadsalt_read, salt_uptake_read, salt_urban_read) skip silently. The remaining readers test only for file existence and read whatever is present.

All readers are called from proc_read.f90 except salt_cha_read, which is called later from main.f90.

Files

salt_hru.ini

Initial salt ion concentrations and mineral fractions for HRU soils and plants.

- Reader: src/salt_hru_read.f90
- Type: salt_soil_ini (type cs_soil_init_concentrations) in src/constituent_mass_module.f90
- Format: 4 header lines, then for each entry three lines: name, then num_salts + 5 soil values, then num_salts + 5 plant values. First num_salts values are ion concentrations (ppm), next 5 are salt mineral fractions.

salt_aqu.ini

Initial salt ion concentrations and mineral fractions for aquifers.

- Reader: src/salt_aqu_read.f90
- Type: salt_aqu_ini (type salt_aqu_init_concentrations) in src/constituent_mass_module.f90. Fields: name, conc(num_salts) in g/m3, frac(5) mineral fractions.
- Format: 4 header lines, then one line per aquifer init record: name, num_salts concentrations, 5 mineral fractions.

salt_channel.ini

Initial salt ion concentrations for channels.

- Reader: src/salt_cha_read.f90
- Type: salt_cha_ini (type salt_cha_init_concentrations) in src/constituent_mass_module.f90. Fields: name, conc(num_salts) in g/m3.
- Format: 2 header lines, then one line per record: name, num_salts concentrations.

salt_irrigation

Salt ion concentrations for irrigation water sourced from outside the watershed.

- Reader: src/salt_irr_read.f90
- Type: salt_water_irr (type cs_irrigation_concentrations) in src/constituent_mass_module.f90. Fields: name, water(num_salts) in ppm.

- Format: 2 header lines, then one line per source: `name`, `num_salts` concentrations.

salt_plants

Plant salt-tolerance parameters and stress control switches.

- Reader: src/salt_plant_read.f90
- Targets in src/salt_data_module.f90:

Field	Type	Units	Description
salt_tds_ec	real	none	TDS to electrical conductivity conversion factor
salt_tol_sim	int	flag	0 = off, 1 = simulate salt effect on plant growth
salt_soil_type	int	code	1 = CaSO4 soils, 2 = NaCl soils
salt_effect	int	code	1 = applied after other stresses, 2 = combined with other stresses via min
salt_stress_a(plantparm)	real	none	a parameter in the salinity relative-yield equation
salt_stress_b(plantparm)	real	none	b parameter in the salinity relative-yield equation

Format: title line, header, `salt_tds_ec`, blank, `salt_tol_sim`, `salt_soil_type`, `salt_effect`, four skipped lines, header, then one line per plant in plant database order with `name, a, b`.

salt_road

Road salt loadings (kg/ha) per atmospheric deposition station.

- Reader: src/salt_roadsalt_read.f90
- Target: `rdapp_salt(:)%salt(:)` (`%road`, `%roadmo`, `%roadyr`, `%roadday`). Allocated for `0:atmodep_cont%num_sta` stations.
- Format: 4 commentary lines skipped, then per station and per salt ion either an annual value (`aa`), a monthly series (`mo`), or a yearly series (`yr`). For `yr`, the reader internally splits the annual mass to daily loadings on days with precipitation and sub-zero temperature.

salt_uptake

Specified daily salt uptake (kg/ha) per plant community per ion.

- Reader: src/salt_uptake_read.f90
- Targets in `src/salt_module.f90`: `salt_uptake_kg(plantparm, num_salts)`, `salt_uptake_on` (flag set to 1 when the file is present).
- Format: 3 header lines, then one line per plant: `name`, `num_salts` uptake values.

salt_urban

Salt ion concentration in suspended solids from urban impervious areas (mg salt / kg sediment).

- Reader: src/salt_urban_read.f90
- Target: `salt_urban_conc(urban, num_salts)` in `src/salt_module.f90`.
- Format: 2 header lines, then one line per urban land-use type: `urb_type`, `num_salts` concentrations. The urban name must match an entry in `urban.urb`.

salt_fertilizer.frt

Per-fertilizer salt ion loading (kg/ha).

- Reader: src/salt_fert_read.f90
- Type: `fert_salt(fertparm)` (`type fert_db_salt`) in `src/salt_module.f90`. Sets `fert_salt_flag = 1`.

#	Field	Units	Description
1	fertnm	-	fertilizer name
2	so4	kg/ha	sulfate loading
3	ca	kg/ha	calcium loading
4	mg	kg/ha	magnesium loading
5	na	kg/ha	sodium loading
6	k	kg/ha	potassium loading
7	cl	kg/ha	chloride loading
8	co3	kg/ha	carbonate loading
9	hco3	kg/ha	bicarbonate loading

Format: title, header, then one line per fertilizer in the fertilizer database order.

Related

- constituents.cs. Master constituent list. Sets `num_salts` and the ion names.
- fertilizer.frt. The base fertilizer database. `salt_fertilizer.frt` must have the same row count.
- urban.urb. Source of urban land-use names referenced by `salt_urban`.
- atmodep.cli. Defines the atmospheric deposition stations used by `salt_road`.

Purpose

The rtb-cs module simulates user-listed water-quality constituents (selenium SeO4 and SeO3, boron, and additional species defined in the constituent database). It runs in parallel with rtb-salt and shares the same master file, `constituents.cs`.

Activation

`constituents.cs` is the master file referenced from `file.cio` as `in_sim%cs_db` (default `constituents.cs`). It lists the pesticides, pathogens, metals, salt ions, and other constituents in the simulation, and populates `cs_db%num_pests`, `cs_db%num_paths`, `cs_db%num_metals`, `cs_db%num_salts`, and `cs_db%num_cs`. The cs and salt readers downstream allocate their arrays from these counts.

The readers `cs_roadsalt_read` does not exist for cs. The cs readers `cs_uptake_read`, `cs_urban_read`, and `cs_reactions_read` only proceed when `cs_db%num_cs > 0`; the remaining readers test only for file existence.

All cs readers are called from `proc_read.f90` except `cs_cha_read`, which is called later from `main.f90`.

Files

constituents.cs

Master list of constituents.

- Reader: src/constit_db_read.f90
- Type: `cs_db` (`type cs_db_type`) in src/constituent_mass_module.f90.
- Format: title, then five pairs of `count` followed by `count` names, in order: pesticides, pathogens, metals, salts, other constituents. The metals block is read but is unused downstream (the matching loop is commented out in the source). The salt names are matched against the database in salt_plants by plant-parameter index, not by name.

cs_hru.ini

Initial constituent concentrations for HRU soils and plants.

- Reader: src/cs_hru_read.f90

- Type: `cs_soil_ini` (`type cs_soil_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `soil(2*num_cs)`, `plt(2*num_cs)` in ppm.
- Format: 4 header lines, then for each entry three lines: name, soil values, plant values.

cs_aqu.ini

Initial constituent concentrations for aquifers.

- Reader: src/cs_aqu_read.f90
- Type: `cs_aqu_ini` (`type cs_aqu_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `aqu(2*num_cs)` (dissolved plus sorbed).
- Format: 3 header lines, then one line per entry: `name`, `2*num_cs` values.

cs_channel.ini

Initial constituent concentrations for channels.

- Reader: src/cs_cha_read.f90
- Type: `cs_cha_ini` (`type cs_cha_init_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `conc(num_cs)` in g/m3.
- Format: 2 header lines, then one line per record: `name`, `num_cs` concentrations.

The same reader also processes the optional `cs_streamobs` file, which lists channels for which daily concentration and load output is written to `cs_streamobs_output`.

cs_irrigation

Constituent concentrations for irrigation water sourced from outside the watershed.

- Reader: src/cs_irr_read.f90
- Type: `cs_water_irr` (`type cs_irrigation_concentrations`) in `src/constituent_mass_module.f90`. Fields: `name`, `water(num_cs)` in ppm.
- Format: 2 header lines, then one line per source: `name`, `num_cs` concentrations.

cs_plants_boron

Plant boron tolerance parameters.

- Reader: src/cs_plant_read.f90
- Targets in src/cs_data_module.f90:

Field	Type	Units	Description
bor_tol_sim	int	flag	0 = off, 1 = simulate boron effect on plant growth
bor_stress_a(plantparm)	real	none	a parameter in the boron relative-yield equation
bor_stress_b(plantparm)	real	none	b parameter in the boron relative-yield equation

Format: title, blank, `bor_tol_sim`, four header lines, then one line per plant in plant database order with `name, a, b`.

cs_uptake

Specified daily constituent uptake (kg/ha) per plant community per constituent.

- Reader: src/cs_uptake_read.f90
- Targets in src/cs_module.f90: `cs_uptake_kg(plantparm, num_cs)`, `cs_uptake_on` (flag set to 1 when the file is present).
- Format: 3 header lines, then one line per plant: `name`, `num_cs` uptake values.

cs_reactions

Selenium and boron chemical reaction parameters for soils and aquifers, organised by reaction group and shale geology.

- Reader: src/cs_reactions_read.f90
- Type: `cs_rct_soil(sp_ob%hru)` and `cs_rct_aqu(sp_ob%aqu)` (`type constituent_rct`) in `src/cs_data_module.f90`. Also fills the module-level arrays `rct(num_rct, num_groups)` and `rct_shale(num_geol_shale, 3)`.
- Format: title, header, `num_rct num_groups`, `num_rct` lines of group values, header, `num_geol_shale`, `num_geol_shale` lines of three shale parameters, header, one line per HRU (`hru_id, group, shale_fractions...`), header, one line per aquifer (`aqu_id, group, shale_fractions...`). The eight `rct` rows hold (in order): inorganic-N to Se ratio, soil oxygen, aquifer oxygen, kd_seo4, kd_seo3, kd_boron, kseo4 in soil, kseo4 in aquifer. See the source for the full assignment table.

cs_urban

Constituent concentration in suspended solids from urban impervious areas (mg cs / kg sediment).

- Reader: src/cs_urban_read.f90
- Target: `cs_urban_conc(urban, num_cs)` in `src/cs_module.f90`.
- Format: 2 header lines, then one line per urban land-use type: `urb_type`, `num_cs` concentrations. The urban name must match an entry in `urban.urb`.

fertilizer.frt_cs

Per-fertilizer constituent loading (kg/ha).

- Reader: src/cs_fert_read.f90
- Type: `fert_cs(fertparm)` (`type fert_db_cs`) in `src/cs_module.f90`. Sets `fert_cs_flag = 1`.

#	Field	Units	Description
1	fertnm	-	fertilizer name
2	seo4	kg/ha	selenate loading
3	seo3	kg/ha	selenite loading
4	boron	kg/ha	boron loading

Format: title, header, then one line per fertilizer in the fertilizer database order.

Related

- salt module. The salt readers share `constituents.cs` for ion counts and use the same per-object pattern.
- fertilizer.frt. Base fertilizer database. `fertilizer.frt_cs` must have the same row count.
- urban.urb. Source of urban land-use names for `cs_urban`.
- atmodep.cli. Atmospheric deposition stations used by the cs deposition reader `cli_read_atmodep_cs`.

Purpose

The water allocation module moves water between spatial objects (channels, reservoirs, aquifers, HRUs) and an auxiliary transport network (canals, pipes, towers, treatment plants, end-use sinks, out-of-basin sources and receivers). It is the mechanism SWAT+ uses for irrigation withdrawals, municipal and industrial supply, inter-basin transfer, and any rule-driven diversion. The driver type is `water_allocation` (`type water_allocation`, alias `wallo`) defined in `water_allocation_module.f90`. The entry-point file is `water_allocation.wro`. All other water-allocation files supply objects referenced from that file.

How it fits together

A single water allocation object (`wallo(iwro)`) owns one or more transfer objects (`wallo%trn(itrn)`). Each transfer object has:

- a transfer type (`trn_typ`): `outflo`, `ave_day`, `div_min`, `div_frac`, `dtbl_con`, `dtbl_lum`. The first five draw from the source list directly; `dtbl_lum` and `dtbl_con` defer the amount to a decision table.
- one or more source objects (`src`), each with a type from `cha`, `res`, `aqu`, `osrc`, `osrc_a`, `wtp`, `use`, `can`, plus a conveyance pointer (`pipe` or `pump`) and an optional withdrawal-limit decision table.
- exactly one receiving object (`rcv`), with a type from `hru`, `cha`, `res`, `aqu`, `wtp`, `use`, `stor` (tower), `can`, `orcv`.

The transport network is a set of independent files, each read by its own subroutine:

```
                    water_allocation.wro
                             |
            +----------------+----------------+
            |                |                |
         sources          transfer         receivers
            |                |                |
cha res aqu osrc       (rule type)      hru cha res aqu wtp
can wtp use osrc_a                       use stor can orcv
            |                                 |
            +--conveyance: pipe / pump --------+
                       |
                    canal
                (water_canal.wal)
                       |
                    pipe
                (water_pipe.wal)
                       |
                    tower
                (water_tower.wal)

wtp  : water_treat.wal     (treatment plant; lag-and-loss store)
use  : water_use.wal       (domestic/industrial/commercial sink)
osrc : out_src.wal         (out-of-basin source, recall-driven)
orcv : outside_rcv.wal     (out-of-basin receiver)
```

The water allocation routine runs per day. `wallo_control` advances the current transfer object, computes demand (`wallo_demand`), withdraws from each source in turn (`wallo_withdraw`), allows compensation between sources that share a `comp = y` flag, transports water through any pipe/pump conveyance (`wallo_transfer`), and delivers it to the receiver. Receiver-side post-processing (`wallo_treatment`, `wallo_use`, `wallo_canal`) handles lag, loss and outflow for `wtp`, `use` and `can` receivers.

Activation

The water allocation files are read at startup from `main.f90`, in this order:

```
call om_treat_read
call om_use_read
call om_osrc_read
call water_treatment_read
call water_use_read
!call water_osrc_read
call water_tower_read
call water_pipe_read
call water_canal_read
call water_allocation_read
```

The module is active whenever `water_allocation.wro` exists and is not `null`. The `wallo` array is then allocated from the count on line 2. If the file is absent the array is allocated as `wallo(0:0)` and the module is inert.

Per-step execution of `wallo_control` is triggered from `command.f90` (channel and reservoir commands) and from `sd_channel_control3.f90` (channel routing). The check `wallo(iwallo)%trn_cur <= wallo(iwallo)%trn_obs` advances the current transfer object until all are processed for the day.

Output (`water_allo_*.txt` / `.csv`) is produced when `db_mx%wallo_db > 0` and the corresponding `pco%water_allo` switch in `print.prt` is `y`.

Files

`water_allocation.wro`

Master file. Reader: src/water_allocation_read.f90. Pointed to by `in_watrts%transfer_wro` in `input_file_module.f90`.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`, the number of water allocation objects. Allocates `wallo(imax)`.
- For each allocation object:
  - Header line (skipped).
  - One record with the object header: `name`, `rule_typ`, `trn_obs`.
  - Header line (skipped).
  - `trn_obs` transfer-object records.

Allocation-object header columns (`wallo(iwro)`):

#	Field	Type	Description
1	name	char(25)	name of the water allocation object
2	rule_typ	char(25)	rule type to allocate water
3	trn_obs	int	number of transfer objects that follow

Each transfer-object record is variable-width. The reader reads it twice: first to learn `src_num`, then, after allocating `src(num_src)` and `osrc(num_src)`, to read the source and receiving fields. The full record is:

`id  trn_typ  trn_typ_name  amount  right  src_num  dtbl_src  src(1)..src(src_num)  rcv`

#	Field	Type	Description
1	id	int	transfer object index (also stored as `trn%num`)
2	trn_typ	char(10)	one of `outflo`, `ave_day`, `div_min`, `div_frac`, `dtbl_con`, `dtbl_lum`
3	trn_typ_name	char(40)	name of the dtbl row (for `dtbl_lum` / `dtbl_con`) or recall (for `outflo` with `osrc`); otherwise free text
4	amount	real	demand amount. m3/day for urban objects; mm for HRU; m3/s for `ave_day`, `div_min`, `div_frac`
5	right	char(2)	water right priority. `sr` (senior) or `jr` (junior)
6	src_num	int	number of source objects in this record
7	dtbl_src	char(25)	decision table name to allocate among sources (may be `null`)
8 to 7+8*src_num	src block	source-object tuple, repeated `src_num` times. See below.	
last three	rcv block	receiving-object tuple. See below.	

Source-object tuple (`transfer_source_objects`, 8 columns per source):

#	Field	Type	Description
1	typ	char(10)	source type: `cha`, `res`, `aqu`, `osrc`, `osrc_a`, `wtp`, `use`, `can`
2	num	int	sequential number of the source object in its own database
3	conv_typ	char(10)	conveyance type: `pipe`, `pump`, or none
4	conv_num	int	sequential number of the conveyance object
5	dtbl_lim	char(25)	decision-table name that sets the withdrawal limit (may be `null`)
6	wdraw_lim	real	actual withdrawal limit. Units depend on source type: reservoir uses principal-storage fraction; aquifer uses maximum depth (m); channel uses minimum flow (m3/s)
7	frac	real	fraction of total transfer supplied by this source
8	comp	char(1)	compensate from this source if other sources are past their threshold. `y` or `n`

Receiving-object tuple (`transfer_receiving_objects`, 3 columns):

#	Field	Type	Description
1	typ	char(10)	receiver type: `hru`, `cha`, `res`, `aqu`, `wtp`, `use`, `stor`, `can`, `orcv`
2	num	int	sequential number of the receiver in its own database
3	frac	int	for an HRU receiver, the soil layer that receives incoming tile flow. Declared `integer :: frac = 0.` in the module (initializer is a real literal).

Synthetic example (no example exists in `refdata/`):

```
water_allocation.wro: synthetic example
1
name                        rule_typ          trn_obs
muni_supply                 demand            1
id  trn_typ      trn_typ_name      amount    right  src_num  dtbl_src  [src...]
1   ave_day      none              0.10      sr     1        null      cha 5  none 0  null  0.0   1.0   n
```

`TODO: verify` exact whitespace and `null` keyword handling in `dtbl_src`. The reader uses list-directed input, so any whitespace separates fields, but the `null` token convention is inferred from other SWAT+ inputs.

`water_canal.wal`

Reader: src/water_canal_read.f90. Filename is hard-coded in the reader; not pointed to from `file.cio`.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`, the number of canal objects.
- Line 3: header (skipped).
- `imax` canal records, each read twice (back-spaced) to learn `num_aqu` before allocating `aqu_loss(num_aqu)`.

Canal record (`water_canal_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	canal name
3	w_sta	char(25)	none	nearby weather station
4	init	char(25)	none	initial concentrations record
5	dtbl	char(25)	none	decision table that determines canal outflow
6	ddown_days	real	days	days to drawdown storage to zero
7	w	real	m	top width
8	d	real	m	depth

#	Field	Type	Units	Description
9	s	real	m/m	slope
10	ss	real	m/m	side slope (trapezoidal)
11	sat_con	real	varies	parameter used to compute percolation to groundwater
12	loss_fr	real	fraction	water loss during conveyance
13	bed_thick	real	m	bed-sediment thickness for Darcy seepage in `gwflow`; 0 if not used
14	div_id	int	none	recall diversion ID (gwflow). 0 if routed by `wallo`
15	day_beg	int	Julian	canal operation start day (gwflow external)
16	day_end	int	Julian	canal operation end day (gwflow external)
17	num_aqu	int	none	number of aquifers that receive seepage loss
18 .. 17+2*num_aqu	aqu_loss block	int, real pairs	each tuple is `aqu_num` (aquifer index) and `frac` (fraction of loss to that aquifer)	

`water_pipe.wal`

Reader: src/water_pipe_read.f90. Filename hard-coded.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` pipes:
  - Header line (skipped).
  - One record, read twice (back-spaced) to learn `num_aqu`.

Pipe record (`water_transfer_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	pipe name
3	stor_mx	real	m3	maximum storage
4	ddown_days	real	days	drawdown time
5	loss_fr	real	fraction	water loss during conveyance
6	num_aqu	int	none	number of aquifers that receive loss
7 .. 6+2*num_aqu	aqu_loss block	int, real	aquifer index and loss fraction	

The `init` field exists in the type definition (`character (len=25) :: init`) but is not read by `water_pipe_read.f90`. `TODO: verify` whether this is by design or an oversight.

`water_tower.wal`

Reader: src/water_tower_read.f90. Filename hard-coded.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` towers:
  - Header line (skipped).
  - One record.

Tower record (subset of `water_transfer_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	tower name
3	stor_mx	real	m3	maximum storage
4	ddown_days	real	days	drawdown time
5	loss_fr	real	fraction	storage loss

The reader does not allocate or read `aqu_loss` for towers even though the type carries the array. The tower is treated as a passive storage node.

`water_treat.wal`

Reader: src/water_treatment_read.f90. Filename hard-coded.

Layout:

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` plants:
  - One main record.
  - If `cs_db%num_pests > 0`: a header line then one record of `num_pests` pesticide concentrations into `wtp_cs_treat(iwtp)%pest`.
  - If `cs_db%num_paths > 0`: a header line then one record of `num_paths` pathogen concentrations into `wtp_cs_treat(iwtp)%path`.

Main record (`water_treatment_use_data`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	plant name
3	stor_mx	real	m3	maximum storage
4	lag_days	real	days	treatment time (outflow lag)
5	loss_fr	real	fraction	water loss during treatment
6	org_min	char(25)	none	name of `om_treat` record for sediment, carbon, nutrients. Cross-walked to `iorg_min`
7	pests	char(25)	none	name of pesticide-set record (`ipests`)
8	paths	char(25)	none	name of pathogen-set record (`ipaths`)
9	salts	char(25)	none	name of salt-ion record (`isalts`)
10	constit	char(25)	none	name of other-constituent record (`iconstit`)
11	descrip	char(80)	none	free-text description

`init` is declared in the type but commented out in the source. The reader does not read it. The cross-walk loop sets `iorg_min` only; `ipests`, `ipaths`, `isalts`, `iconstit` are left at 0. `TODO: verify` whether the missing cross-walks are intentional.

`water_use.wal`

Reader: src/water_use_read.f90. Filename hard-coded.

Same `water_treatment_use_data` type and same layout as `water_treat.wal`, including the conditional `pest` and `path` concentration blocks. Records are stored in `wuse` instead of `wtp`. Cross-walk is against `om_use_name` rather than `om_treat_name`.

out_src.wal

Reader: src/water_osrc_read.f90. The reader checks `inquire(file='outside_src.wal')` but then opens `'out_src.wal'`. See the Important section.

Layout (as written, assuming the open succeeds):

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` sources: one main record, then conditional pest and path blocks (header + values) as in `water_treat.wal`.

Main record (`outside_basin_source`):

#	Field	Type	Units	Description
1	id	int	none	record index
2	name	char(25)	none	out-of-basin source name
3	stor_mx	real	m3	maximum storage
4	lag_days	real	days	lag
5	loss_fr	real	fraction	loss fraction

outside_rcv.wal

Reader: src/water_orcv_read.f90. Filename hard-coded. The reader is not called from `main.f90`. See the Important section.

Layout (as written):

- Line 1: title (skipped).
- Line 2: `imax`.
- Line 3: header (skipped).
- For each of `imax` receivers: one record.

Main record (`outside_basin_receive`):

#	Field	Type	Description
1	id	int	record index
2	name	char(25)	out-of-basin receiver name
3	filename	char(25)	name of the output file written for this receiver

element.wro and water_rights.wro

Declared in `input_file_module.f90` (`in_watrts%element`, `in_watrts%water_rights`). No reader in `water_*_read.f90` or anywhere else under `src/` references either filename. `TODO: verify` whether these are legacy placeholders or are read by code outside the water-allocation subroutines.

Related

- `*.dtl` decision tables drive transfers via the `water_rights` action. `dtbl_lum` rows are looked up by `trn_typ_name` from `dtbl_lum` and supply irrigation amount; `dtbl_con` rows are looked up from `dtbl_flo` and supply flow-control amount, source availability and source/receiver allocation. `dtbl_src` and `dtbl_lim` are matched to decision-table names at run time.
- `object.cnt` has a `wro` column (position 21). It is currently noted in the source as "not used"; the actual count of water allocation objects is taken from line 2 of `water_allocation.wro`.
- `recall_db` is cross-walked for `osrc` sources (daily/monthly/yearly recall) and `exco_db` for `osrc_a` (annual constant from an `exco` record).

- [om_treat.om, om_use.om, om_osrc.om] supply the organic and mineral concentration sets named in org_min fields of water_treat.wal, water_use.wal and out_src.wal.

SWAT+ writes one text file per (scope, group, interval) combination. Every output is toggled in print.prt. This section documents the file groups, their columns, and where they are written in the source.

Naming convention

All time-step output files follow the pattern:

<scope>_<group>_<interval>.txt

When csvout = y in print.prt, a matching .csv variant is also written.

Scopes (the spatial aggregation level):

Scope	Meaning
hru_	one row per Hydrologic Response Unit
hru-lte_	one row per low-resolution HRU (lite model)
lsunit_	one row per landscape unit (ls_unit.def)
ru_	one row per routing unit
region_	one row per region defined for regional output
basin_	one row, the basin-wide sum or average
channel_	one row per channel object (legacy cha)
channel_sd / sd_chan	one row per channel object (sd_channel routing)
aquifer_	one row per aquifer object
reservoir_	one row per reservoir object
wetland_	one row per wetland object
recall_	one row per recall object (point source or inlet)
crop_yld_	one row per HRU per harvested plant

Groups (the variable family):

Group	Meaning
_wb	water balance (precipitation, ET, runoff, percolation, storage)
_nb	nutrient balance (mineral and organic N and P pools)
_ls	losses from the landscape (sediment, nutrient yields)
_pw	plant and weather (LAI, biomass, yield, temperature, stress)
_cha	channel routing (legacy: flow, sediment, nutrients)
_sd_cha	sd_channel routing (water body storage, in, out)
_aqu	aquifer storage and fluxes
_res	reservoir storage and fluxes
_pest	pesticide
_salt	salt ions
_cs	constituents
_carbon	soil carbon pools
_chanbud	channel sediment budget
_chamorph	channel morphology

Intervals (the time aggregation):

Suffix	Meaning
_day	one row per simulated day
_mon	one row per month
_yr	one row per year
_aa	one row, average annual over the print window

Each interval is enabled independently in `print.prt`. A row in `print.prt` named `hru_wb` with `y` in the `avann` column produces `hru_wb_aa.txt`; with `y` in the `daily` column it also produces `hru_wb_day.txt`. The file is opened in `output_landscape_init.f90` only if the matching switch is `y`.

Per-group pages

- Naming convention. The full grammar and the `print.prt` mapping.
- Water balance. The `*_wb_*.txt` columns: precipitation, ET, runoff, percolation, soil and snow storage.
- Channel and sediment. The legacy `channel_*.txt` and the sd_channel `channel_sd_*.txt` columns, including sediment and nutrient routing.
- Aquifer, reservoir, plant. The `aquifer_*.txt`, `reservoir_*.txt`, `crop_yld_*.txt`, and plant/water-stress (`*_pw_*.txt`) columns.
- Checker and diagnostics. The always-on diagnostic files: `checker.out`, `diagnostics.out`, `simulation.out`, `files_out.out`.
- gwflow. 2D groundwater outputs: basin and per-cell water balance, heat, six solute species, observation wells, and pumping.
- Post-processing helpers. The two Python scripts shipped in `test/` for regression checks and CSV conversion.

Output index file

When SWAT+ runs, it writes a master index `files_out.out` listing every output file actually opened during that run. The first column is a short tag (`HRU`, `BASIN`, `CHK`, `CHN`, `AQU`, `RES`, …) and the second column is the filename. Open `files_out.out` first to see exactly which outputs your `print.prt` produced.

Filename grammar

`<scope>_<group>_<interval>.<ext>`

- `scope` identifies the spatial unit.
- `group` identifies the variable family.
- `interval` identifies the time aggregation.
- `ext` is `txt` (always) and `csv` (when `csvout = y` in `print.prt`).

Examples that the source actually opens:

```
hru_wb_day.txt
hru_wb_mon.txt
hru_wb_yr.txt
hru_wb_aa.txt
basin_wb_aa.txt
lsunit_wb_aa.txt
hru-lte_wb_aa.txt
hru_nb_aa.txt
hru_ls_aa.txt
hru_pw_aa.txt
basin_aqu_aa.txt
basin_res_aa.txt
basin_cha_aa.txt
basin_sd_cha_aa.txt
channel_sd_day.txt
channel_day.txt
aquifer_day.txt
reservoir_day.txt
crop_yld_aa.txt
crop_yld_yr.txt
```

Scopes

The scope prefix maps to a writer subroutine:

Scope	Writer	Loop body
`hru_`	`hru_output.f90`	one record per HRU
`hru-lte_`	`hru_lte_output.f90`	one record per HRU-LTE
`lsunit_`	`lsu_output.f90`	one record per landscape unit
`ru_`	`ru_output.f90`	one record per routing unit
`region_`	`lsreg_output.f90`	one record per region per landuse
`basin_`	`basin_output.f90`, `basin_aquifer_output.f90`, `basin_channel_output.f90`, `basin_sdchannel_output.f90`, `basin_reservoir_output.f90`	one record (basin sum or average)
`channel_`	`channel_output.f90`	one record per legacy channel object
`channel_sd` (file group `sd_cha`)	`sd_channel_output.f90`	one record per sd_channel object
`aquifer_`	`aquifer_output.f90`	one record per aquifer
`reservoir_`	`reservoir_output.f90`	one record per reservoir
`wetland_`	`wetland_output.f90`	one record per wetland
`recall_`	`recall_output.f90`	one record per recall object
`crop_yld_`	`hru_output.f90` (last block)	one record per HRU per plant

Groups

The group token identifies the column family. Each writer holds its own Fortran derived type and the columns are the fields of that type in declaration order.

Group	Type	Module
`_wb`	`output_waterbal`	`src/output_landscape_module.f90`
`_nb`	`output_nutbal`	`src/output_landscape_module.f90`
`_ls`	`output_losses`	`src/output_landscape_module.f90`
`_pw`	`output_plantweather`	`src/output_landscape_module.f90`
`_cha`	`ch_output`	`src/channel_module.f90`
`_sd_cha`	`water_body` plus two `hyd_output` records (in, out)	`src/water_body_module.f90`, `src/hydrograph_module.f90`
`_aqu`	`aquifer_dynamic`	`src/aquifer_module.f90`
`_res`	`water_body` plus two `hyd_output` records (in, out)	`src/water_body_module.f90`, `src/hydrograph_module.f90`
`_pest`	pesticide processes types	`src/output_ls_pesticide_module.f90`, others
`_salt`	salt ion types	`src/salt_module.f90`
`_cs`	constituents types	`src/cs_module.f90`
`_carbon`, `_soilcarb`, `_rescarb`, `_plcarb`	carbon pool types	`src/output_landscape_module.f90`, `src/carbon_module.f90`

Intervals

Suffix	Meaning	`print.prt` column
`_day`	one row per day	`daily`
`_mon`	one row per month	`monthly`
`_yr`	one row per year	`yearly`
`_aa`	one row, average annual over the print window	`avann`

The print window is the simulation range minus `nyskip` warm-up years (set in the first table of `print.prt`).

print.prt toggle

`print.prt` has a table of object groups, one row per group, with four columns (`daily`, `monthly`, `yearly`, `avann`). Each cell is `y` or `n`. The file is opened only if the cell is `y`. The toggles set `pco%<group>%<d|m|y|a>` (read in `print_read.f90`) and the file is opened in `output_landscape_init.f90` or one of the `header_*.f90` files.

Example: in `refdata/Ames_sub1/print.prt`, the row

```
hru_wb                          n             n             n             y
```

opens `hru_wb_aa.txt` only. The other three (`day`, `mon`, `yr`) are not written.

A separate row, `csvout`, switches on the matching `.csv` variant for every enabled file.

`crop_yld` in `print.prt` accepts `n`, `a` (annual only), `y` (yearly only), or `b` (both). Other groups use the four-column form.

Subdaily output

When `time%step > 1` (subdaily simulation), additional `*_subday.txt` files are written. The current subdaily channel writer is in `sd_channel_output.f90` (`channel_sd_subday.txt`, unit 2508).

Always-on outputs

A few files are not controlled by `print.prt` and are written every run:

- `simulation.out`. Daily progress log written from `main.f90` and `time_control.f90`.
- `files_out.out`. Index of every output file opened, written from `proc_bsn.f90`.
- `diagnostics.out`. Warnings about missing input references, opened in `proc_bsn.f90`.
- `checker.out`. HRU-level parameter sanity dump, written in `proc_hru.f90`.
- `erosion.out`. USLE inputs and outputs per HRU.
- `success.fin`. Empty marker created on successful completion.

See Checker and diagnostics for the column-level breakdown.

Purpose

Water balance files report the major flows and storages of the HRU-to-channel hydrology: precipitation, snow, ET, surface runoff, lateral and tile flow, percolation, soil-water storage, and snow pack. One file is written per scope (`hru`, `hru-lte`, `lsunit`, `ru`, `region`, `basin`) per interval (`day`, `mon`, `yr`, `aa`), toggled in `print.prt`.

Source

- HRU writer: `src/hru_output.f90`, units 2000 (day), 2001 (mon), 2002 (yr), 2003 (aa).
- Landscape unit writer: `src/lsu_output.f90`.
- Routing unit writer: `src/ru_output.f90`.
- Basin writer: `src/basin_output.f90`.
- Region writer: `src/lsreg_output.f90`.
- File opening and header writes: `src/output_landscape_init.f90`.
- Record type: `src/output_landscape_module.f90`, `type output_waterbal`.

Format

- Line 1: title (project name, version banner).
- Line 2: column header (`wb_hdr`).
- Line 3: units (`wb_hdr_units`).
- Lines 4+: data records. One record per object (HRU, lsunit, …) per timestep.

The leading seven columns are the same for every landscape output:

#	Column	Type	Description
1	jday	int	Julian day of year
2	mon	int	calendar month
3	day	int	day of month
4	yr	int	calendar year
5	unit	int	spatial id (HRU number, lsunit number, …)
6	gis_id	int	GIS id from the object table
7	name	char	object name (`ob(iob)%name`)

The remaining columns come from `output_waterbal`, in declaration order. All flows are mm of water depth over the unit area; CN is dimensionless.

#	Column	Units	Description
8	precip	mm	precipitation (rain + snow)
9	snofall	mm	precipitation falling as snow, sleet, or freezing rain
10	snomlt	mm	snow or melting ice
11	surq_gen	mm	surface runoff generated on the landscape
12	latq	mm	lateral soil flow
13	wateryld	mm	water yield (surq + latq + tile)
14	perc	mm	percolation out of soil profile into vadose zone
15	et	mm	actual evapotranspiration
16	ecanopy	mm	canopy evaporation (not reported in this column in current source)
17	eplant	mm	plant transpiration
18	esoil	mm	soil evaporation
19	surq_cont	mm	surface runoff leaving the landscape
20	cn	none	average curve number for the timestep
21	sw_init	mm	soil water content at start of timestep
22	sw_final	mm	soil water content at end of timestep
23	sw_ave	mm	average soil water content over the timestep
24	sw_300	mm	final soil water content in upper 300 mm
25	sno_init	mm	snow water equivalent at start of timestep
26	sno_final	mm	snow water equivalent at end of timestep
27	snopack	mm	average snow water equivalent over the timestep
28	pet	mm	potential evapotranspiration
29	qtile	mm	subsurface tile flow leaving the landscape
30	irr	mm	irrigation water applied
31	surq_runon	mm	surface runoff received from upland landscape
32	latq_runon	mm	lateral flow received from upland landscape
33	overbank	mm	overbank flooding from channels
34	surq_cha	mm	surface runoff flowing into channels
35	surq_res	mm	surface runoff flowing into reservoirs
36	surq_ls	mm	surface runoff flowing onto a landscape element
37	latq_cha	mm	lateral flow into channels
38	latq_res	mm	lateral flow into reservoirs
39	latq_ls	mm	lateral flow into a landscape element
40	gwsoilq	mm	groundwater transferred to soil profile (gwflow)
41	satex	mm	saturation excess flow developed from high water table (gwflow)
42	satex_chan	mm	saturation excess flow reaching the main channel (gwflow)
43	sw_change (delsw)	mm	change in soil water volume
44	lagsurf	mm	surface runoff in transit to channel
45	laglatq	mm	lateral flow in transit to channel
46	lagsatex	mm	saturation excess flow in transit to channel
47	wet_evap	mm	evaporation from wetland surface
48	wet_out (wet_oflo)	mm	outflow (spill) from wetland

#	Column	Units	Description
49	wet_stor	mm	wetland storage at end of timestep

The HRU writer also appends two trailing string fields after the numeric columns, written by `hru_output.f90`:

Trailing	Description
plant_cov	name of the plant community from `lum.dtl`
mgt_ops	name of the management schedule from `lum.dtl`

Aggregation

- Daily (`_day`): values for the day. Files use unit 2000 (HRU), 2050 (basin), 2140 (lsunit).
- Monthly (`_mon`): daily values summed; `sw`, `sno`, and `cn` reset their initial/final markers at month boundaries. Unit 2001 (HRU), 2051 (basin), 2141 (lsunit).
- Yearly (`_yr`): monthly values summed; storage variables divided by 12 where appropriate. Unit 2002 (HRU), 2052 (basin), 2142 (lsunit).
- Annual average (`_aa`): yearly values divided by `time%yrs_prt`. Unit 2003 (HRU), 2053 (basin), 2143 (lsunit).

Example

`refdata/Ames_sub1/basin_wb_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
  jday    mon    day     yr    unit  gis_id  name                  precip     snofall      snomlt    surq_gen
                                                                     mm          mm          mm          mm
   366     12     31   2020          1       1  demo                1905.214    220.690     219.997      59.5
```

The 42 numeric columns after `name` map one-to-one onto fields 8 to 49 above. The trailing `Original Simulation` token is the scenario name (basin scope substitutes it for the HRU `plant_cov`/`mgt_ops` pair).

Related files

- Naming convention.
- Channel and sediment outputs.
- `print.prt`. Toggles each file group on or off.

Purpose

SWAT+ supports two channel-routing implementations, each with its own output set.

- **Legacy channel** (`channel_*.txt`, `basin_cha_*.txt`). One row per channel object, holding inflow / outflow of water, sediment, sediment by particle size, nutrients, pesticides, bacteria, and conservative metals.
- **sd_channel** (`channel_sd_*.txt`, `basin_sd_cha_*.txt`). Storage-and-discharge routing. Three blocks of columns: a `water_body` block (area, precip, evap, seep, storage), followed by an inflow `hyd_output` record and an outflow `hyd_output` record.

The channel sediment budget and morphology have their own writers (`sd_chanbud_output.f90`, `sd_chanmorph_output.f90`) and produce `*_chanbud_*.txt` and `*_chamorph_*.txt`.

Legacy channel files

Source

- Writer: `src/channel_output.f90`, units 2480 (day), 2481 (mon), 2482 (yr), 2483 (aa).

- Basin sum: src/basin_channel_output.f90.
- File opening: src/header_channel.f90.
- Record type: src/channel_module.f90, `type ch_output`.

Format

Each line is the seven leading columns (jday, mon, day, yr, unit, gis_id, name), then the 60 fields of `ch_output` in declaration order.

#	Column	Units	Description
1	flo_in	ha-m	streamflow into reach
2	flo_out	ha-m	streamflow out of reach
3	evap	m^3/s	rate of water loss by evaporation
4	tloss	m^3/s	rate of transmission loss through streambed
5	sed_in	tons	sediment transported into reach
6	sed_out	tons	sediment transported out of reach
7	sed_conc	mg/L	sediment concentration in reach
8	orgn_in	kg N	organic nitrogen in
9	orgn_out	kg N	organic nitrogen out
10	orgp_in	kg P	organic phosphorus in
11	orgp_out	kg P	organic phosphorus out
12	no3_in	kg N	nitrate in
13	no3_out	kg N	nitrate out
14	nh4_in	kg	ammonium in
15	nh4_out	kg	ammonium out
16	no2_in	kg	nitrite in
17	no2_out	kg	nitrite out
18	solp_in	kg P	soluble phosphorus in
19	solp_out	kg P	soluble phosphorus out
20	chla_in	kg	chlorophyll-a in
21	chla_out	kg	chlorophyll-a out
22	cbod_in	kg	carbonaceous BOD in
23	cbod_out	kg	carbonaceous BOD out
24	dis_in	kg	dissolved oxygen in
25	dis_out	kg	dissolved oxygen out
26	solpst_in	mg pst	soluble pesticide in
27	solpst_out	mg pst	soluble pesticide out
28	sorbpst_in	mg pst	sorbed pesticide in
29	sorbpst_out	mg pst	sorbed pesticide out
30	react	mg pst	pesticide loss by reaction in water
31	volat	mg	pesticide loss by volatilization
32	setlpst	mg pst	pesticide settling from water to bed
33	resuspst	mg	pesticide resuspended from bed
34	difus	mg	pesticide diffused from water to bed
35	reactb	mg	pesticide reaction loss in bed
36	bury	mg	pesticide buried
37	sedpest	mg	pesticide in bed sediment
38	bacp	# cfu/100mL	persistent bacteria out
39	baclp	# cfu/100mL	less persistent bacteria out
40	met1	kg	conservative metal #1 out
41	met2	kg	conservative metal #2 out
42	met3	kg	conservative metal #3 out
43	sand_in	tons	sand in
44	sand_out	tons	sand out
45	silt_in	tons	silt in
46	silt_out	tons	silt out
47	clay_in	tons	clay in
48	clay_out	tons	clay out
49	smag_in	tons	small aggregate in
50	smag_out	tons	small aggregate out
51	lag_in	tons	large aggregate in

#	Column	Units	Description
52	lag_out	tons	large aggregate out
53	grvl_in	tons	gravel in
54	grvl_out	tons	gravel out
55	bnk_ero	tons	bank erosion
56	ch_deg	tons	channel degradation
57	ch_dep	tons	channel deposition
58	fp_dep	tons	floodplain deposition
59	tot_ssed	mg/L	total suspended sediments

Example header

`refdata/Ames_sub1/basin_cha_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
  jday     mon    day    yr     unit  gis_id  name                 flo_in        flo_out             evap
                                                                      ha-m           ha-m             ha-m
```

The header reports `evap` and `tloss` units as `ha-m`, but `ch_output` carries them as `m^3/s` (rate). The unit row in the file does not match the source comment.

sd_channel files

Source

- Writer: src/sd_channel_output.f90, units 2500 (day), 2501 (mon), 2502 (yr), 2503 (aa); subdaily on unit 2508.
- Basin sum: src/basin_sdchannel_output.f90.
- File opening: src/header_sd_channel.f90.
- Record types: src/water_body_module.f90, `type water_body`, and src/hydrograph_module.f90, `type hyd_output`.

Format

Each record is the seven leading columns, then four blocks of fields written in this order by `sd_channel_output.f90`:

```
write (2500,100) time%day, time%mo, time%day_mo, time%yrc, ichan, ob(iob)%gis_id, ob(iob)%name,
 ↪  &
  ch_wat_d(ichan)%area_ha, ch_wat_d(ichan)%precip, ch_wat_d(ichan)%evap, ch_wat_d(ichan)%seep,
 ↪  &
  ch_stor(ichan), ch_in_d(ichan), ch_out_d(ichan), wtemp
```

Block 1: water body status (4 fields).

#	Column	Units	Description
1	area	ha	channel water surface area
2	precip	m^3	precipitation on water surface
3	evap	m^3	evaporation from water surface
4	seep	m^3	seepage from streambed

Block 2: channel storage `ch_stor` (one `hyd_output` record, 18 fields). These are the `_stor` columns in the file header.

Block 3: inflow `ch_in_d` (one `hyd_output` record, 18 fields). These are the `_in` columns.

Block 4: outflow `ch_out_d` (one `hyd_output` record, 18 fields). These are the `_out` columns.

Block 5: water temperature `wtemp` (deg C), one trailing scalar.

The 18 fields of `hyd_output`, in declaration order:

Field	Units	Description
flo	m^3 (storage), m^3/s (in / out flow rate)	volume or flow rate
sed	metric tons	sediment
orgn	kg N	organic N
sedp	kg P	organic P
no3	kg N	NO3-N
solp	kg P	soluble P
chla	kg	chlorophyll-a
nh3	kg N	NH3
no2	kg N	NO2
cbod	kg	carbonaceous BOD
dox	kg	dissolved oxygen
san	tons	detached sand
sil	tons	detached silt
cla	tons	detached clay
sag	tons	detached small aggregate
lag	tons	detached large aggregate
grv	tons	gravel
temp	deg C	temperature

The line ends with `wtemp` after the `out` block.

Example header

`refdata/Ames_sub1/basin_sd_cha_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
   jday    mon    day     yr     unit   gis_id   name                    area         precip           evap
                                                                          ha            m^3            m^3
```

Each block ends with a `null`-labelled column. That column is the `temp` field of `hyd_output`. The writer passes `ch_in_d(ichan)`, `ch_out_d(ichan)`, and `ch_stor(ichan)` as whole records, so the 18th field of each block is `temp`. The basin-wide water temperature is the trailing `wtemp` scalar written after the outflow block.

Channel sediment budget and morphology

- `sd_chanbud_*.txt`. Sediment budget per channel: bank erosion, bed erosion, deposition. Writer: `src/sd_chanbud_output.f90`. Basin sum in `src/basin_chanbud_output.f90`.
- `sd_chamorph_*.txt`. Channel morphology: width, depth, slope. Writer: `src/sd_chanmorph_output.f90`. Basin sum in `src/basin_chanmorph_output.f90`.

Related files

- Aquifer, reservoir, plant outputs.
- Water balance outputs.

This page covers four output families:

- Aquifer storage and fluxes (`aquifer_*.txt`, `basin_aqu_*.txt`).
- Reservoir storage and fluxes (`reservoir_*.txt`, `basin_res_*.txt`).
- Crop yields (`crop_yld_aa.txt`, `crop_yld_yr.txt`, `basin_crop_yld_*.txt`).
- Plant and weather (`hru_pw_*.txt`, `lsunit_pw_*.txt`, `basin_pw_*.txt`).

Aquifer

Source

- Writer: src/aquifer_output.f90, units 2520 (day), 2521 (mon), 2522 (yr), 2523 (aa).
- Basin sum: src/basin_aquifer_output.f90, units 2090-2093.
- File opening: src/header_aquifer.f90, src/header_write.f90.
- Record type: src/aquifer_module.f90, `type aquifer_dynamic`.

Format

Seven leading columns (jday, mon, day, yr, unit, gis_id, name), then the 17 fields of `aquifer_dynamic` in declaration order.

#	Column	Units	Description
1	flo	mm	lateral flow from aquifer
2	dep_wt	m	average depth from surface elevation to water table
3	stor	mm	average water storage in aquifer in timestep
4	rchrg	mm	recharge entering aquifer from other objects
5	seep	mm	seepage from bottom of aquifer
6	revap	mm	plant water uptake plus evaporation from aquifer
7	no3_st	kg/ha N	current total NO3-N mass in aquifer
8	minp	kg/ha P	mineral P transported in return (lateral) flow
9	cbn (header label: `orgn`)	percent	organic carbon in aquifer (static). Header mislabels this column.
10	orgn (header label: `orgp`)	kg/ha N	organic nitrogen in aquifer (static). Header mislabels this column.
11	no3_rchg	kg/ha N	NO3-N flowing into aquifer from another object
12	no3_loss	kg/ha	NO3-N loss
13	no3_lat	kg/ha N	nitrate load to reach in groundwater
14	no3_seep	kg/ha N	nitrate seepage to next object
15	flo_cha	mm	aquifer return flow into channels
16	flo_res	mm	aquifer return flow into reservoirs
17	flo_ls	mm	aquifer return flow into a landscape element

Example header

`refdata/Ames_sub1/basin_aqu_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
   jday    mon    day     yr    unit  gis_id  name                          flo          dep_wt            stor
                                                                             mm               m              mm
```

The `aqu_header` strings in `aquifer_module.f90` declare columns `orgn` (position 9) and `orgp` (position 10), but the `aquifer_dynamic` type fields in those positions are `cbn` (organic carbon percent) and `orgn` (organic nitrogen). The writer pushes whole records in declaration order, so the column under the header `orgn` is carbon and the column under `orgp` is organic nitrogen.

Aggregation

- Daily, monthly, yearly, annual average. Aggregation rules: monthly `stor`, `dep_wt`, `no3_st` are divided by the number of days in the month so they remain averages; yearly divides those three by 12; annual average divides everything by `time%yrs_prt`.

Reservoir

Source

- Writer: src/reservoir_output.f90, units 2540 (day), 2541 (mon), 2542 (yr), 2543 (aa).

- Basin sum: src/basin_reservoir_output.f90.
- File opening: src/header_reservoir.f90.
- Record types: `type water_body` in src/water_body_module.f90, `type hyd_output` in src/hydrograph_module.f90.

Format

The writer call is:

```
write (2540,100) time%day, time%mo, time%day_mo, time%yrc, j, ob(iob)%gis_id, ob(iob)%name, &
  res_wat_d(j), res(j), res_in_d(j), res_out_d(j)
```

Seven leading columns, then four blocks. Each `hyd_output` block contributes its 18 fields. `res(j)` is the full `reservoir` derived type and contributes the storage fields written into the `_stor` columns.

Block	Source	Columns
1	`res_wat_d(j)` (`water_body`)	area (ha), precip (m^3), evap (m^3), seep (m^3)
2	`res(j)` storage as `hyd_output` (`_stor` columns)	18 storage fields: flo, sed, orgn, sedp, no3, solp, chla, nh3, no2, cbod, dox, san, sil, cla, sag, lag, grv, temp
3	`res_in_d(j)` (`hyd_output`, `_in` columns)	18 inflow fields
4	`res_out_d(j)` (`hyd_output`, `_out` columns)	18 outflow fields

See Channel and sediment outputs for the 18 fields of `hyd_output` and their units; the layout is identical to the sd_channel writer (inflow is `m^3/s`, outflow is `m^3/s`, storage is `m^3`).

Example header

`refdata/Ames_sub1/basin_res_aa.txt`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
   jday    mon    day     yr      unit    gis_id   name                     area          precip            evap
```

The basin-level reservoir output uses `m^3` for the inflow / outflow columns instead of the `m^3/s` used by `basin_sd_cha_aa.txt`, because the reservoir code does not divide by seconds. The header reports `m^3` accordingly.

Crop yield

Source

- Writer: bottom block of src/hru_output.f90, units 4008 (aa) and 4010 (yr). Triggered only at `time%end_sim == 1` (for `_aa`) or `time%end_yr == 1` (for `_yr`).
- Basin variant: `basin_crop_yld_*.txt`, written in the basin output path.
- File opening: src/output_landscape_init.f90.

Format

Different shape from the other output files: each line is one HRU per plant, not one HRU per timestep.

#	Column	Units	Description
1	jday	int	last simulated day
2	mon	int	last simulated month
3	day	int	day of month
4	yr	int	year
5	unit	int	HRU number
6	PLANTNM	char	plant name (`pldb(idp)%plantnm`)
7	MASS	kg/ha	total yield mass (dry weight)
8	C	kg/ha	carbon in yield
9	N	kg/ha	nitrogen in yield

#	Column	Units	Description
10	P	kg/ha	phosphorus in yield

`crop_yld_aa.txt` reports the per-harvest average (yield divided by number of harvests). `crop_yld_yr.txt` reports the yield for that year.

Example

`refdata/Ames_sub1/crop_yld_aa.txt`:

```
 demo
  SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328

                                                                    --YIELD (kg/ha)--
  jday    mon    day     yr     unit PLANTNM                  MASS          C           N           P
   366     12     31   2020        1     corn                                3358.149    1511.167
   366     12     31   2020        1     soyb                                   0.000       0.000
```

Switched on in `print.prt` by the `crop_yld` row (`a` for annual average only, `y` for yearly only, `b` for both).

Plant and weather

Source

- HRU writer: src/hru_output.f90, units 2040 (day), 2041 (mon), 2042 (yr), 2043 (aa).
- File opening: src/output_landscape_init.f90.
- Record type: src/output_landscape_module.f90, `type output_plantweather`.

Format

Seven leading columns, then the 25 fields of `output_plantweather` in declaration order.

#	Column	Units	Description
1	lai	$m^{2/m}2$	leaf area index
2	bioms	kg/ha	total plant biomass
3	yield	kg/ha	harvested biomass yield (dry weight)
4	residue	kg/ha	surface residue cover
5	sol_tmp	deg C	temperature of soil layer 2
6	strsw	days	water stress (drought)
7	strsa	days	excess water stress (aeration)
8	strstmp	days	temperature stress
9	strsn	days	nitrogen stress
10	strsp	days	phosphorus stress
11	strss	days	salinity stress
12	nplnt	kg N/ha	plant N uptake
13	percn	kg N/ha	NO3-N leached past soil profile
14	pplnt	kg P/ha	plant P uptake
15	tmx	deg C	maximum air temperature
16	tmn	deg C	minimum air temperature
17	tmpav	deg C	average air temperature
18	solrad	MJ/m^2	solar radiation
19	wndspd	m/s	windspeed
20	rhum	none	relative humidity
21	phubase0	deg C/deg C	base-zero potential heat units
22	lai_max	$m^{2/m}2$	maximum LAI during timestep
23	bm_max	kg/ha	maximum total biomass during timestep
24	bm_grow	kg/ha	total biomass growth during timestep
25	c_gro	kg/ha	total plant carbon growth during timestep

#	Column	Units	Description

The HRU writer appends `plant_cov` and `mgt_ops` as trailing string fields, just like the water-balance writer.

Aggregation

- Daily values come straight from `hpw_d`.
- Monthly: daily values summed, then divided by days-in-month for the columns that represent averages (`lai`, `tmpav`, etc.), preserving running max for `bm_max` and `lai_max`.
- Yearly: same pattern at year boundary.
- Annual average: divided by `time%yrs_prt` at end of simulation.

Related files

- Water balance outputs.
- Channel and sediment outputs.

Purpose

The carbon output family reports the state and fluxes of the SWAT+ Century-style carbon module: per-HRU and per-LSU gain and loss totals, soil C transformations, per-layer pool composition, plant carbon state, and process drivers. The family supersedes the old sprawl of 47 files in 26 base names. Every file is opt-in through `print.prt`: each carbon family has its own row with the standard four-slot `d / m / y / a` layout.

The outputs are produced only when `bsn_cc%cswat == 2` (the dynamic carbon model; the activation value moved from 1 to 2). See `codes.bsn` and the carbon model theory.

Source

- HRU writer: `src/hru_carbon_output.f90` (gain/loss and transformations) and `src/soil_nutcarb_write.f90` (per-layer pools, fluxes, drivers, dynamics, soil snapshot).
- LSU writer: `src/lsu_carbon_output.f90`.
- File opening and headers: `src/output_landscape_init.f90`.
- Header types and variable-name lists: `src/output_landscape_module.f90` (`output_carb_gl_header`, `output_hscf_header`, …); `src/carbon_module.f90` (`cpool_vars`, `n_p_pool_vars`, `cflux_vars`, `carb_drv_vars`, `carb_dyn_vars`, `soil_snap_vars`).
- Per-family `print.prt` parsing: `src/basin_print_codes_read.f90`, `hru_cb_*` and `lsu_cb_*` case branches.

File naming

Every file follows the canonical convention `<family>_{day,mon,yr,aa}.{txt,csv}`. The `_day` variant is daily, `_mon` is end-of-month, `_yr` is end-of-year, `_aa` is the average-annual record written once at end-of-simulation. The `hru_soil_snap` family uses `_tot` instead of `_aa` because an annual average of a snapshot pool is degenerate; the `_tot` file carries one begsim row and one endsim row, so `endsim_c - begsim_c` is the simulated SOC change per layer (see the carbon outputs reference).

CSV variants are emitted in parallel when `csvout = y` in `print.prt`. Both variants share the same column header and units row.

Per-family print.prt gating

Each carbon family has its own row in the per-object output block of `print.prt`. The `basin_print_codes_read.f90` `case (...)` table accepts these names; each row is one object label followed by four flags (`d m y a`):

print.prt row	Files gated	Output scope
hru_cb_gl	hru_carb_gl_*.{txt,csv}	HRU
hru_cb_trf	hru_scf_*.{txt,csv}	HRU
hru_cb_lyr	hru_cbn_lyr_*.{txt,csv}	HRU
hru_cb_cpool	hru_cpool_stat_*.{txt,csv}	HRU
hru_cb_npool	hru_n_p_pool_stat_*.{txt,csv}	HRU
hru_cb_plt	hru_plc_stat_*.{txt,csv}	HRU
hru_cb_flux	hru_cflux_stat_*.{txt,csv}	HRU
hru_cb_drv	hru_carb_drv_*.{txt,csv}	HRU
hru_cb_dyn	hru_carb_dyn_*.{txt,csv}	HRU
hru_cb_snap	hru_soil_snap_*.{txt,csv} (uses _tot instead of _aa)	HRU
lsu_cb_gl	lsu_carb_gl_*.{txt,csv}	LSU
lsu_cb_trf	lsu_scf_*.{txt,csv}	LSU
lsu_cb_plt	lsu_plc_stat_*.{txt,csv}	LSU

The output restructure split the previous coarse hru_cb flag into the ten per-family flags above. The old hru_cb and hru_cb_vars rows now drive a parallel set of legacy CSU output files (see below), reinstated so the CSU team's existing workflows keep working alongside the standardised per-family files.

The LSU files are emitted only when db_mx%lsu_out > 0 (the dataset contains routing units), regardless of the lsu_cb_* flag. Datasets without rout_unit.con populated do not create empty LSU carbon files.

Legacy CSU carbon outputs

A second, older set of carbon output files runs alongside the standardised per-family files above. These are not exposed in SWAT+ Editor, so regular users do not see them; the CSU team adds the driving rows to print.prt by hand. This legacy path is temporary and is marked for removal in revision 63.

Two print.prt rows drive it, both read by basin_print_codes_read.f90 and name-driven (a print.prt without them produces none of these files):

print.prt row	When	Files
hru_cb	always when present	hru_cbn_lyr, hru_seq_lyr, hru_n_p_pool_stat
hru_cb = l (layer detail)	flag letter l turns diagnostics on	adds hru_cpool_stat, hru_cflux_stat, hru_plc_stat, hru_begsim_soil_prop, hru_endsim_soil_prop, basin_carbon_all
hru_cb = y (profile)	flag letter y	light set only (diagnostics off)
hru_cb_vars	requires carbon on (cswat == 2)	hru_carbvars, hru_org_allo_vars, hru_org_ratio_vars, hru_org_trans_vars

Each file is written as .txt, plus .csv when csvout = y in print.prt.

The legacy and standardised files hold the same values in a different layout. For example, legacy hru_cbn_lyr (totals) plus hru_seq_lyr (sequestered) together equal the merged new hru_cbn_lyr_yr, which carries totals, sequestered, and depths in one wide table.

The internal cbn_diagnostics switch (which gates the heavy diagnostic files) is no longer read from an input file; it is set at startup from the hru_cb flag letter (l = on, y = off). Legacy output uses Fortran file units 8348-8387.

Note

The cbn_diagnostics flag in the old carb_coefs.cbn is retired. Output gating now lives entirely in print.prt.

HRU files

hru_carb_gl_* (family hru_cb_gl)

Merged HRU carbon gain and loss totals. 21 value columns covering soil C balance, residue C balance, and plant C balance. Replaces the former `hru_soilcarb_*`, `hru_rescarb_*`, and `hru_plcarb_*` triplet. Units are `kg C/ha` for every value column.

Columns (in declaration order in `output_carb_gl_header`): `jday`, `mon`, `day`, `yr`, `unit`, `gis_id`, `name`, then `sed_c`, `surq_c`, `latq_c`, `perc_c`, `res_decay`, `man_app_c`, `man_graze_c`, `rsp_c`, `soil_emit_c`, `plant_surf_c`, `plant_root_c`, `rsd_surfdecay_c`, `rsd_rootdecay_c`, `harv_stov_c`, `rsd_emit_c`, `npp_c`, `harv_abgr_c`, `harv_root_c`, `drop_c`, `grazeat_c`, `plant_emit_c`.

The output restructure broke out four sub-fields that the previous output silently summed under a single label: `plant_surf_c`, `plant_root_c`, `harv_abgr_c`, `harv_root_c`.

hru_scf_* (family hru_cb_trf)

HRU-level soil carbon transformations (the 13 inter-pool fluxes). One row per HRU per timestep.

Columns: `jday`, `mon`, `day`, `yr`, `unit`, `gis_id`, `name`, then `meta_micr`, `str_micr`, `str_hs`, `co2_meta`, `co2_str`, `micr_hs`, `micr_hp`, `hs_micr`, `hs_hp`, `hp_micr`, `co2_micr`, `co2_hs`, `co2_hp`.

hru_cbn_lyr_* (family hru_cb_lyr)

Per-layer total soil carbon, written wide (one row per HRU per timestep with per-layer columns). The number of layers in the output is `cb_n_layers`. When the optional `carbon_layers.prt` is absent this defaults to the largest soil layer count across all HRUs, so the deepest profile is written in full; when the file is present it fixes the count to the value given. Each row carries that HRU's own layer depths, and layers a soil does not have are filled with the sentinel `-99`.

Header is built at first data write and carries the structure `layer depth (mm) | total_c_lyr1 | total_c_lyr2 | ... | total_c_lyrN`. Units: `Mg C/ha` per layer column.

hru_cpool_stat_* (family hru_cb_cpool)

Per-layer C pools. Wide layout, with the columns in `cpool_vars` (`src/carbon_module.f90:20`) expanded to `<var>_lyr1 ... <var>_lyrN`:

```
residue_c, structural_c, metabolic_c, hs_c, hp_c, microbial_c,
lignin_c, nonlignin_c, root_mass, soil_water
```

10 variables x `cb_n_layers` columns. Units are kg C/ha for the C pools, kg/ha for `root_mass`, mm for `soil_water`.

hru_n_p_pool_stat_* (family hru_cb_npool)

Per-layer N and P content of the carbon pools. Variables in `n_p_pool_vars`:

```
tot_pool_n, residue_n, structural_n, metabolic_n, hs_n, hp_n,
microbial_n, lignin_n, nonlignin_n,
tot_pool_p, residue_p, structural_p, metabolic_p, hs_p, hp_p,
microbial_p, lignin_p, nonlignin_p
```

18 variables x `cb_n_layers` columns. Units kg N/ha or kg P/ha as the column name implies.

hru_plc_stat_* (family hru_cb_plt)

HRU-level plant carbon state. No per-layer expansion. Seven flat columns:

```
total_c, ab_gr_c, leaf_c, stem_c, seed_c, root_c, surf_rsd_c
```

Units kg C/ha.

hru_cflux_stat_* (family hru_cb_flux)

Per-layer C and N fluxes from the Century pool transformations. Variables in `cflux_vars` (37 entries):

```
cfmets1, cfstrs1, cfstrs2,                          (C flows from litter)
efmets1, efstrs1, efstrs2,                          (E = effective C)
immmets1, immstrs1, immstrs2,                       (immobilisation)
mnrmets1, mnrstrs1, mnrstrs2,                       (mineralisation)
co2fmet, co2fstr,                                   (CO2 from litter)
cfs1s2, cfs1s3, cfs2s1, cfs2s3, cfs3s1,             (inter-SOC pool C flows)
efs1s2, efs1s3, efs2s1, efs2s3, efs3s1,
imms1s2, imms1s3, imms2s1, imms2s3, imms3s1,
mnrs1s2, mnrs1s3, mnrs2s1, mnrs2s3, mnrs3s1,
co2fs1, co2fs2, co2fs3                              (CO2 from SOC pools)
```

37 variables x `cb_n_layers` columns. Pool indices: `s1` = microbial, `s2` = slow humus, `s3` = passive humus, `met` = metabolic litter, `str` = structural litter.

hru_carb_drv_* (family hru_cb_drv)

Per-layer environmental drivers and intermediate factors for the Century transformations. Variables in `carb_drv_vars` (14 entries):

```
sut, tillagef, cons_bmix, tillagef_biomix, tillagef_tillmix,
till_eff, cdg, ox, cs, no3, nh4, co2_resp, soil_temp, emix
```

14 variables x `cb_n_layers` columns. `sut`, `cdg`, `ox`, `cs` are dimensionless factors; `no3`, `nh4` are kg/ha; `co2_resp` is kg C/ha; `soil_temp` is deg C; `tillagef*`, `till_eff`, `cons_bmix`, `emix` are dimensionless.

hru_carb_dyn_* (family hru_cb_dyn)

Per-layer carbon pool dynamics: allocation fractions and the C-to-N ratios of each pool transformation. Variables in `carb_dyn_vars` (21 entries):

```
asp, abp, abco2, a1co2, asco2, apco2,               (CO2 / passive allocations)
ncbm, nchp, nchs,                                   (N:C of microbial, passive, slow)
bmctp, bmntp, hsctp, hsntp, hpctp, hpntp,           (pool C and N totals)
lmctp, lmntp, lsctp, lslctp, lslnctp, lsntp         (litter pool totals)
```

21 variables x `cb_n_layers` columns. Allocation fractions are dimensionless; pool totals are kg C/ha or kg N/ha.

hru_soil_snap_* (family hru_cb_snap)

Per-layer soil snapshot. Uses `_tot` in place of `_aa` because an annual average of a snapshot is degenerate.

Variables in `soil_snap_vars` (13 entries):

```
bd, awc, soil_k, tot_c, clay, silt, sand, rock,
alb, usle_k, ec, caco3, ph
```

13 variables x `cb_n_layers` columns.

The `_day`, `_mon`, `_yr` files emit end-of-period rows on the usual cadence. The `_tot` file is special: it emits one row at simulation begin (`begsim`) and one at simulation end (`endsim`). Both rows carry pool values in the same unit (`kg C/ha` for `tot_c`), so `endsim_tot_c_lyrK - begsim_tot_c_lyrK` is the simulated SOC change for layer K. The previous design wrote a percentage on the begsim row and a mass on the endsim row, which made the delta meaningless.

LSU files

The LSU files are area-weighted HRU aggregates produced by `lsu_carbon_output.f90`. Per-layer families are **not** aggregated to LSU because different HRUs within an LSU can have different soil profiles.

`lsu_carb_gl_*` (family `lsu_cb_gl`)

LSU-level carbon gain and loss totals, area-weighted from the HRU `hru_carb_gl_*` values. Reuses the `output_carb_gl_header` columns (same 21 value columns as the HRU file).

`lsu_scf_*` (family `lsu_cb_trf`)

LSU-level soil C transformations, area-weighted from the HRU values. Reuses the `output_hscf_header` columns.

`lsu_plc_stat_*` (family `lsu_cb_plt`)

LSU-level plant carbon, summed across HRUs with the `ru_frac` weight. Single value column `plant_c` (kg C/ha), printed once per LSU per timestep.

Optional supporting file

`carbon_layers.prt`

Optional file that fixes `cb_n_layers`, the number of soil layers expanded in every wide per-layer carbon file. It is only needed when you want a specific column count; if it is absent the model uses the largest soil layer count across all HRUs (see below). Format:

- Line 1: title (free text, skipped).
- Line 2: header (free text, skipped).
- Line 3: single integer giving the layer count.

Only line 3 carries data. The reader skips lines 1 and 2 entirely, so their text is for the reader's own benefit and may say anything. An integer below 1 is rejected and the automatic default is used instead.

Example

```
carbon_layers.prt: layers to expand in per-layer carbon output
n_layers
10
```

Here line 1 is a free-text title, line 2 is a free-text header, and line 3 is the only value the reader uses. With `10`, every per-layer carbon file carries ten layer columns regardless of the soils present, and any unused slots are filled by `-99`. Use this to force a fixed column count, for example to truncate output to the top layers or to keep a constant width across runs.

If the file is absent, `cb_n_layers` defaults to the largest soil layer count across all HRUs. This means the deepest profile is written in full and no real layer is truncated; HRUs with fewer layers than that maximum get the sentinel `-99` in their extra slots. (Earlier versions defaulted to a fixed 7, which silently truncated deeper soils.) Reader: src/carbon_layers_read.f90. Not listed in `file.cio`.

Related pages

- Carbon (model theory). Pool definitions and Century-style transformation equations.
- `carbon.bsn`. Basin-wide scalar inputs.
- `carbon_lyr.bsn`. Per-layer inputs.
- `print.prt`. Output gating.
- Naming convention. Common naming rules across output files.

Purpose

The gwflow module writes its own set of output files when `bsn_cc%gwflow = 1` in `codes.bsn`. Outputs are toggled in `print.prt` via six rows: `gwflow_wb`, `gwflow_flux`, `gwflow_heat`, `gwflow_solute`, `gwflow_obs`, `gwflow_pump`. Each row has the standard daily/monthly/yearly/average-annual flags.

Source

- Writer: src/gwflow_output.f90
- Types: src/gwflow_module.f90, `type groundwater_ss` for fluxes
- Simulation-time writes (cell-level): `src/gwflow_simulate.f90`

Output files

Water balance

File	Scope	Aggregation
`gwflow_basin_wb_day.txt`	basin total	daily
`gwflow_basin_wb_mon.txt`	basin total	monthly
`gwflow_basin_wb_yr.txt`	basin total	yearly
`gwflow_basin_wb_aa.txt`	basin total	annual average
`gwflow_cell_wb_day.txt`	per cell	daily
`gwflow_cell_wb_mon.txt`	per cell	monthly
`gwflow_cell_wb_yr.txt`	per cell	yearly
`gwflow_cell_wb_aa.txt`	per cell	annual average

Columns (in declaration order from `type groundwater_ss`, units `m^3`):

#	Name	Description
1	chng	change in storage (grid summaries only)
2	rech	recharge
3	gwet	groundwater ET
4	gwsw	groundwater discharge to channels
5	swgw	channel seepage to groundwater
6	satx	saturation excess flow
7	soil	groundwater added to the soil profile
8	latl	lateral flow between cells
9	disp	dispersion (heat and solute transport)
10	bndr	boundary exchange
11	ppag	allocation-driven pumping (irrigation)
12	ppdf	pumping deficit (unmet demand)
13	ppex	external pumping
14	tile	tile drainage outflow
15	resv	reservoir exchange
16	wetl	wetland exchange
17	fpln	floodplain exchange
18	canl	canal exchange
19	pond	recharge pond seepage
20	phyt	phreatophyte transpiration
21	totl	sum of inputs and outputs

Per-cell files prepend a cell id and timestamp; basin files aggregate across all cells. The full per-cell water-balance header (`gwflow_cell_wb_*`), verified against `gwflow_output.f90`, is:

```
jday mon day yr unit gis_id name head wt_depth recharge gw_et gw_sw sw_gw sat_excess soil lateral pump_all
```

- `gis_id` is the authoritative cell id carried from the QSWAT+ shapefile; `name` is `cellNNNN` (a 12-character field).
- `head` and `wt_depth` are in metres; the flux columns (`recharge` through `phytorem`) are in m^3/day for daily files (monthly files carry the monthly-average daily rate).
- `wt_depth` = cell land-surface elevation − head (depth of the water table below the cell's representative ground surface). It can look deeper at channel/river cells on coarse grids: the cell surface represents the valley, not the streambed, so the depth is measured from the valley rim while the head sits near the channel. This is a grid-resolution reporting effect, not a model error; the head field itself is correct.

Observation wells

`gwflow_obs_{day,mon,yr,aa}.txt` report state at each observation cell (set via the observation list) for every timestep. Columns:

Column	Units	Meaning
`head`	m	Groundwater head
`wt_depth`	m	Water-table depth below ground (land-surface elevation − head)
`temp`	degC	Groundwater temperature, `-99` when heat transport is off
`no3`	mg/L	Nitrate concentration, `-99` when solute transport is off
`p`	mg/L	Phosphorus concentration, `-99` when solute transport is off

Heat

When the heat transport sub-module is active (`gwflow.heat` is present), the following are written:

File	Aggregation
`gwflow_basin_heat_day.txt`	daily
`gwflow_basin_heat_yr.txt`	yearly
`gwflow_basin_heat_aa.txt`	annual average

Columns are the same fields as water balance, reinterpreted as heat fluxes (the writer reuses `type groundwater_ss`). Refer to the model theory page on Aquifers for the heat transport equations.

Solutes

When solute transport is active (`gwflow.solutes` listing constituents), per-species water-balance-style files are written. Six species are emitted directly by the writer:

Species	Files
Calcium (`ca`)	`gwflow_basin_sol_ca_{day,mon,yr,aa}.txt`
Chloride (`cl`)	`gwflow_basin_sol_cl_{day,mon,yr,aa}.txt`
Potassium (`k`)	`gwflow_basin_sol_k_{day,mon,yr,aa}.txt`
Magnesium (`mg`)	`gwflow_basin_sol_mg_{day,mon,yr,aa}.txt`
Sodium (`na`)	`gwflow_basin_sol_na_{day,mon,yr,aa}.txt`
Phosphorus (`p`)	`gwflow_basin_sol_p_{day,mon,yr,aa}.txt`

Columns follow the same `groundwater_ss` field set, interpreted as solute mass fluxes (kg).

Cell definition

`gwflow_cell_definition.txt` is written once at the start of the run. It maps the grid coordinates to cell IDs and is the key for interpreting any per-cell output.

Always-on log

`gwflow_record` is a plain-text log opened when `bsn_cc%gwflow = 1`. The simulation banner and per-feature notes are written there. Useful for confirming the gwflow module activated and which sub-features (pumps, ponds, tiles, etc.) loaded.

print.prt toggles

Toggle	Files produced
`gwflow_wb`	basin and cell water balance
`gwflow_flux`	gwflow canal, pond, tile, gwsw, channel observation diagnostics
`gwflow_heat`	basin heat balance
`gwflow_solute`	basin solute balance for each species
`gwflow_obs`	observation well output (when `gwflow.hru_pump_observe` or observation file is set)
`gwflow_pump`	per-HRU pumping output

The aggregation flags on each row (`d`, `m`, `y`, `a`) control which suffix variants are written.

Related pages

- `gwflow` input reference
- `codes.bsn` (`gwflow` switch)
- `print.prt` (output toggles)
- Model theory: Aquifers

These files are written every run, regardless of `print.prt`. Check them first when validating a simulation.

simulation.out

Source

Opened in `src/main.f90` on unit 9003 (first 30 lines). Daily progress lines written from `src/time_control.f90`. Climate-file open events written from `src/proc_date_time.f90`.

Content

- Banner: SWAT+ version, revision, compiler, OS, build date.
- One line per climate file as it is opened.
- One line per simulated day:

  `<scenario name>  <mon>  <day>  <year>  Yr  <yr>  of  <yr_total>  Time  hh:mm:ss`
- On successful completion, a line `Execution successfully completed` and a `Date of Sim` stamp.

Example

```
                  SWAT+
            Revision 61.0.1-33-g68b1328
     Soil & Water Assessment Tool
IntelLLVM (2024.0.0), 2024-08-27 08:49:23, Linux
    Program reading . . . executing


  Date of Sim   8/27/2024 Time   8:49:51
 reading from pet file                  Time   8:49:51
 reading from precipitation file        Time   8:49:51
 ...
  Original Simulation        1   1  1975 Yr    1 of   46 Time   8:49:51
  ...
  Original Simulation       12  31  2020 Yr   46 of   46 Time   8:49:53
```

```
 Execution successfully completed

  Date of Sim    8/27/2024 Time    8:49:53
```

If `Execution successfully completed` is absent, the run aborted. Search upward in the file for the last day printed: the failure is on the next simulated day.

diagnostics.out

Source

Opened in src/proc_bsn.f90 on unit 9001. Lines are appended by every `*_read.f90` that detects a missing input reference (`grep -rn "write *(9001"` lists about 60 call sites in the source).

Content

One warning per line. Each warning is the name of a missing reference and the input file that pointed to it. Example:

```
DIAGNOSTICS.OUT FILE
corn_lum                                CORN_rot                                not found in management.
agrr_lum                                AGRR_rot                                not found in management.
soyb_lum                                SOYB_rot                                not found in management.
ames.tem                                          file not found (tgage)
file not found (basins_carbon.tes)
```

Read this file end-to-end after every run. Lines here mean SWAT+ silently substituted a default or skipped a setting because the referenced object was not in the database it was supposed to be in. Common categories:

- `<name> not found in management.sch`. A land use management points at a schedule that does not exist.
- `<name> not found (initial.cha)`, `(hydrology.cha)`, `(sediment.cha)`, `(nutrients.cha)`. A channel object points to a property set that does not exist.
- `file not found (<gage>)`. A weather station refers to a gage file that is missing.
- `file not found (<file>)`. A control input (carbon coefficients, snow database) is missing.

An empty `diagnostics.out` (only the header) means no missing references were detected.

checker.out

Source

Opened in src/proc_hru.f90 on unit 4000. Writes one row per HRU after model setup. Header types are `output_checker_header` and `output_checker_unit` in src/hydrograph_module.f90.

Format

- Line 1: scenario name and SWAT+ version.
- Line 2: column header (`chk_hdr`).
- Line 3: unit row (`chk_unit`). The last cell carries the legend `0=notile;1=tile;`.
- Lines 4+: one row per HRU.

#	Column	Units	Description
1	sname	name	soil name
2	hydgrp	A/B/C/D	hydrologic soil group
3	zmx	mm	maximum rooting depth (`soil%zmx`)
4	usle_k	none	USLE soil erodibility factor
5	sumfc	mm	sum of field capacity over profile
6	sumul	mm	sum of saturation (upper limit) over profile
7	usle_p	none	USLE practice factor

#	Column	Units	Description
8	usle_ls	none	USLE slope-length-steepness factor
9	esco	none	soil evaporation compensation factor
10	epco	none	plant uptake compensation factor
11	cn3_swf	none	curve-number soil-water adjustment
12	perco	none	percolation coefficient
13	latq_co	none	lateral flow coefficient
14	tiledrain	0 or 1	tile drainage flag

Example

`refdata/Ames_sub1/checker.out`:

```
 demo                        SWAT+ 2024-08-27        MODULAR Rev 2024.61.0.1-33-g68b1328
                                            zmx       usle_k       sumfc       sumul      usle_p     usle_ls
 sname            hydgrp                    (mm)                    (mm)        (mm)
soil_01           B                     2000.0000       0.0000    361.5000    616.1781      1.0000      0.3638
soil_02           B                     2000.0000       0.0000    361.5000    445.6733      1.0000      0.3638
```

Check `usle_k`, `zmx`, `sumfc`, and `sumul` for unrealistic values (zero, negative, or extreme). A `zmx` of 2000 mm on every HRU usually means the soil definitions were not parsed and the model fell back to a default.

files_out.out

Source

Opened in src/proc_bsn.f90 on unit 9000. Every `open_output_file` call that opens an output writes one line here, naming the file and a short tag.

Format

- Line 1: header `files_out.out - OUTPUT FILES WRITTEN`.
- Lines 2+: two columns, tab/space separated.

Column	Description
tag	`CHK`, `HRU`, `BASIN`, `CHN`, `AQU`, `RES`, `MGT`, …
filename	the file SWAT+ opened

This is the master inventory: anything actually produced by this run is listed here. If you expect a file and it is missing from `files_out.out`, the corresponding switch in `print.prt` was `n`.

erosion.out

Source

Opened in src/proc_hru.f90 on unit 4001. Writes one row per HRU of the USLE inputs and computed factors. Always on; not switchable in `print.prt`.

success.fin

Created in src/main.f90 on unit 107. An empty file written only at the end of a successful run. External tools (calibration drivers, test harnesses) test for its existence to confirm the run completed.

erosion.txt

Opened in `main.f90` on unit 888. Internal debug log of erosion calculations. Not part of the documented output set.

Recommended checks after every run

1. Open `simulation.out` and confirm the last line is `Execution successfully completed`. If not, look at the last printed day.
2. Open `diagnostics.out`. Every line is a problem. Resolve each one before trusting any output.
3. Open `checker.out`. Confirm `zmx`, `sumfc`, `sumul` are sane per HRU. Confirm `tiledrain` flag matches your design.
4. Confirm `success.fin` exists.
5. Open `files_out.out`. Confirm every output file your `print.prt` requested is listed.

Related files

- Naming convention.
- Post-processing helpers. Includes a parser for `checker.out`.

Two Python scripts are bundled in `test/` of the SWAT+ source repository. Each is a small standalone tool. They are not installed by the build; copy them into your working tree or invoke them directly.

test/spcheck.py

A regression-test driver. Runs a SWAT+ executable against a scenario directory, then performs a field-by-field numerical comparison against a golden output.

Subcommands

`spcheck.py` has three subcommands.

run

```
spcheck.py run <executable> <scenario>
```

Workflow:

1. Looks up `data/<scenario>`.
2. Creates `runs/<scenario>/<NN>#<timestamp>#<executable>/` and copies the scenario into it.
3. Runs `<executable>` (from `build/<executable>`) with that directory as cwd.
4. If the executable returns non-zero, renames the run directory to end in `-FAILED`.

`NN` is the next two-digit serial number (`01`, `02`, …).

compare

```
spcheck.py compare <scenario> <a> <b> [--abserr 1e-8] [--relerr 0.05] [--nlines N] [--nerrorlines N]
```

Compares the outputs of two prior runs of the same scenario, identified by their serial numbers. Reads the list of output files from `<scenario>/.testfiles.txt` (one filename per line, `#` starts a comment).

For each pair of files, line-by-line, field-by-field:

- Splits the line into tokens by whitespace.
- Skips non-numeric tokens. Skips lines containing `MODULAR` (the version banner).
- Compares each float field: a difference is flagged when

  ```
  abs(f1 - f2) >= aerr + rerr * abs(f2)
  ```

- Defaults: `aerr = 1e-8`, `rerr = 0.05` (5 percent).
- For each diff, prints the line numbers, the field positions, the two values, and the absolute and relative error.

ctest

`spcheck.py ctest <model_path> <data_dir> <tmp_dir> [--abserr ...] [--relerr ...] [--nlines N] [--nerrorlin`

For automated test runs. Copies `<data_dir>` into `<tmp_dir>/<scenario_name>`, runs `<model_path>` in that tmp directory, then compares the newly produced outputs against the originals in `<data_dir>`. Exits non-zero if any comparison errors are above the thresholds.

What it is good for

- Detecting numerical regressions when changing source code.
- Comparing two executables (different compilers, different revisions) against the same scenario.
- Running CI tests with a small abserr/rerr tolerance to flag any unexpected drift.

What it does not do

- Does not understand the column structure of SWAT+ outputs. Tokens are compared by position only.
- Skips integer fields (calls `int()` first; floats only).
- Trusts that both files have the same line count and the same number of float tokens per line. Differences in structure produce a `danger` return code (-1) for the line and the comparison stops on that line.

test/swat_output_to_csv_files.py

A pandas-based parser for SWAT+ text outputs. Maps every known output filename to a read specification (rows to skip, header row, column names, separator), then loads it with `pandas.read_csv`.

How it is structured

A single module-level dictionary `spec_dict` keys every supported filename to a dict of `pandas.read_csv` arguments.

Two default specifications are reused for most files:

```
default_spec = {
    "skiprows": [0, 2],
    "header":   [0],
    "column_names": None,
    "delim_whitespace": True
}

default_spec2 = {
    "skiprows": [0],
    "header":   [0],
    "column_names": None,
    "delim_whitespace": True
}
```

`default_spec` skips line 0 (the SWAT+ banner) and line 2 (the units row), so line 1 becomes the column header. This matches the standard SWAT+ output layout: title, header, units, data.

`default_spec2` keeps the units row in the data. Used for files that do not have a units row (`lu_change_out.txt`, `basin_crop_yld_*.txt`).

`crop_yld_aa.txt` and `crop_yld_yr.txt` get `skiprows=2` (no list) because their banner is two lines long.

`checker.out` and `files_out.out` get explicit `column_names` lists because the file headers do not parse cleanly as a single row of column names.

Files covered

The script supports these outputs out of the box:

```
files_out.out, checker.out
hru_orgc.txt, hru_wb_aa.txt, hru_ncycle_aa.txt, hru_nb_aa.txt,
hru_carbon_aa.txt, hru_nut_carb_gl_aa.txt, hru_ls_aa.txt, hru_pw_aa.txt
basin_wb_aa.txt, basin_nb_aa.txt, basin_carbon_aa.txt,
basin_ls_aa.txt, basin_pw_aa.txt
crop_yld_yr.txt, crop_yld_aa.txt
lu_change_out.txt, basin_crop_yld_yr.txt, basin_crop_yld_aa.txt
hydout_aa.txt, hydin_aa.txt
deposition_aa.txt, wetland_aa.txt
basin_aqu_aa.txt, basin_res_aa.txt
recall_aa.txt
basin_cha_aa.txt, basin_sd_cha_aa.txt, basin_sd_chamorph_aa.txt
basin_psc_aa.txt
ru_aa.txt
```

Entry point

```python
def run():
    fname = "files_out.out"
    df = read_output(fname, spec_dict, True)
    for index, row in df.iterrows():
        fname = row["filename"]
        if fname == "mgt_out.txt" or fname == "yield.out":
            continue
        df = read_output(fname, spec_dict, True)
```

When run as `python swat_output_to_csv_files.py`, the script:

1. Reads `files_out.out` from the current directory.
2. For every output filename listed there (other than `mgt_out.txt` and `yield.out`), loads the file with the matching spec and writes a tab-separated `.csv` alongside the original.

`read_output(fname, spec_dict, write_csv=False)` is the unit entry point. Pass `write_csv=True` to write `<fname>.csv`; the function always returns the loaded DataFrame.

Limitations

- The script must be run in the directory containing the SWAT+ outputs.
- If a filename is not in `spec_dict`, the script prints `Swat+ output filename not found in read specification dictionary.` and calls `exit(1)`.
- `pandas.read_csv(..., delim_whitespace=True)` is deprecated in newer pandas versions in favour of `sep=r"\s+"`. Expect a `FutureWarning`.
- The output `.csv` files are tab-separated despite the `.csv` extension (`output_seperator = "\t"`).

Typical usage

```
cd <scenario directory>
python /path/to/swat_output_to_csv_files.py
# produces hru_wb_aa.txt.csv, basin_wb_aa.txt.csv, ... in the same dir
```

Or, in a notebook, import the module and call `read_output` directly on the file of interest to get a DataFrame in memory:

```python
import swat_output_to_csv_files as s
df = s.read_output("basin_wb_aa.txt", s.spec_dict, write_csv=False)
```

Related files

- Checker and diagnostics. `files_out.out` is the entry point for the CSV script.
- Naming convention. Explains which file is which.

This section is for people working on the SWAT+ Fortran source: building from a clone, running the test suite, and submitting changes back to `swat-model/swatplus`.

If you only need to run a prebuilt SWAT+ executable, see Install instead.

Pages

- Build with CMake. The base out-of-source build flow that every platform uses. Covers configuration, compiler selection (gfortran, ifort, ifx), debug vs release, and installation.
- Visual Studio on Windows. End-to-end workflow for Visual Studio 2022 with the Intel oneAPI Fortran compiler. Cloning, configuring, debugging against an input dataset.
- VS Code Codespaces. Running a SWAT+ build and debug session inside a GitHub Codespaces virtual instance. Useful when you do not want to install a Fortran toolchain locally.
- Testing. The scenario regression test suite that compares model output against golden datasets in `data/`. Driven by `ctest`.
- Tagging and releases. How the build system pulls the version from `git describe` and projects it into `main.f90` and the executable name. Covers the release tag flow.
- Coding conventions. Fortran 90/95 style rules for SWAT+ source contributions: portability, readability, robustness, arrays, I/O, and obsolescent features to avoid.
- Contributing. Fork, branch, pull request, and the CI checks that run on every PR.
- Code of conduct. The Contributor Covenant, applied to the SWAT+ project community.

SWAT+ is built out-of-source. All intermediate and final files go into a separate build directory so the source tree stays clean. The same source tree can host multiple build flavors (different compilers, debug vs release) at the same time.

The build is driven by `CMakeLists.txt` at the repository root. It globs every `.f90` under `src/` and links a single executable.

```
file(GLOB sources src/*.f90)
add_executable(${SWATPLUS_EXE} ${sources})
```

Generate the build files

Make `swatplus` your current working directory.

```
cd <dir>/swatplus
```

Configure the build. This processes `CMakeLists.txt` and writes a build system into `build/`. It does not compile yet.

```
cmake -S . -B build
```

The short form is equivalent.

```
cmake -B build
```

Pick a compiler

Default is `gfortran`. To force it explicitly:

```
cmake -B build -D CMAKE_Fortran_COMPILER=gfortran
```

For Intel Fortran, `ifort` was deprecated in 2024 and removed from the 2025 Intel oneAPI release. Use `ifx` instead.

```
cmake -B build -D CMAKE_Fortran_COMPILER=ifx
```

On Windows, the Intel compilers need a generator toolset:

```
cmake -B build -D CMAKE_GENERATOR_TOOLSET=fortran=ifx
cmake -B build -G "Ninja" -D CMAKE_Fortran_COMPILER=gfortran
```

Note: on Windows, use the default `cmd` shell, not PowerShell, for the command-line build.

Debug vs release

For a debug build with symbols:

```
cmake -B build -D CMAKE_BUILD_TYPE=Debug
```

For a release build:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
```

Compile and link

From the `swatplus` directory:

```
cmake --build build
```

Parallelize across CPU cores with `-j`. Eight is a reasonable default on most laptops.

```
cmake --build build -j 8
```

The executable lands in `build/` as `swatplus-<version>-<arch>.exe` on Windows and without an extension on Linux and macOS. The version part comes from `git describe --tags`. See Tagging and releases.

To remove all build artifacts and start over:

```
rm -rf build
```

On Windows `cmd`:

```
rd /s /q build
```

To clean just the object files and module files while keeping the build configuration:

```
cmake --build build --target clean
```

Install

Copy the built executable to a directory of your choice with `cmake --install`. The `--prefix` flag sets the base directory.

```
cmake --install build --prefix ~/bin
```

If you omit `--prefix`, CMake installs to the system default binary location. You can also skip this step and copy the executable yourself.

Linux note on libgfortran

If your Linux distribution does not ship the static `libgfortran.a`, remove the `link_libraries("static")` line from `CMakeLists.txt` and build against the dynamic `libgfortran.so`. The resulting binary then needs the dynamic libraries present at runtime. Use `ldd <binary>` to list them.

If you hit a platform-specific compile failure, open an issue on GitHub with your OS, the error output, and any fix you found.

This page covers the full workflow for working on SWAT+ in Visual Studio 2022 on Windows: installing the required tools, cloning the repository, configuring the CMake build, and running a debug session against an input dataset.

Screenshots referenced below live in the SWAT+ source repo's `doc/` folder.

Required software

Install in this order. The Intel compiler requires Visual Studio to be present first.

1. Visual Studio Community 2022. During install, in the **Workloads** panel, check **Desktop development with C++**. That workload pulls in the CMake integration that SWAT+ needs.
2. Intel Fortran Compiler for Windows. Use the standalone installer. The current Intel oneAPI release ships `ifx`. The older `ifort` was deprecated in 2024 and is gone from the 2025 release. SWAT+ produces the same results with either Intel compiler.
3. Git for Windows. Visual Studio bundles its own Git, but SWAT+ uses the standalone `git` during the CMake build to read the current tag for the version string.

Reboot after the three installs finish.

Optional: install Python (from the Microsoft Store) and the Python workload in Visual Studio if you want to run the integrated test suite from inside the IDE.

Starting Visual Studio

The Intel compiler only works with the Visual Studio CMake integration if Visual Studio is launched from the Intel oneAPI command prompt. Follow these steps every time.

1. Open the Windows Start Menu, type `intel`, and select **Intel oneAPI command prompt for Intel 64 for Visual Studio 2022**. Pin it if you use it often.
2. The terminal opens and prints the oneAPI initialization output:

   ```
   :: initializing oneAPI environment...
      Initializing Visual Studio command-line environment...
      Visual Studio version 17.13.0 environment configured.
      "C:\Program Files\Microsoft Visual Studio\2022\Community\"
      Visual Studio command-line environment initialized for: 'x64'
   :  compiler -- latest
   :  debugger -- latest
   :: oneAPI environment initialized ::
   ```

3. At the prompt, run:

   ```
   devenv
   ```

   This launches Visual Studio with the Intel environment variables set.

If you start Visual Studio by clicking the desktop icon directly, the Intel compiler will not be found and the build will fail.

Cloning the repository

Visual Studio can clone the SWAT+ repository and open it as a CMake project in one step.

1. On the Visual Studio start screen, click **Clone a Repository**.
2. Click the GitHub icon (lower left) to browse your repositories. Sign in if prompted. Select your fork of `swatplus` and click **Open**. Adjust the local path if needed.
3. Visual Studio clones the repository and loads it into Solution Explorer as a CMake project. It runs the CMake configure step against whichever Fortran compiler it finds (`ifx`, `ifort`, or `gfortran`). The 2025 Intel compiler only ships `ifx`, so any `ifort` configurations will fail on that version.

Configuring, compiling, running

Configure

SWAT+ is a CMake project. The Visual Studio toolbar exposes the available compiler and build-type combinations through a drop-down. Switching the selection triggers a reconfigure. The Output window prints the CMake settings used, which is the place to verify which compiler picked up.

To force a fresh configure, right-click `CMakeLists.txt` in Solution Explorer and select **Delete Cache and Reconfigure**.

Build

Use **Build > Build All** or **Build > Rebuild All**. The executable name encodes the compiler, architecture, and version. The version is read from the latest Git tag automatically. See Tagging and releases.

The Intel-built Windows executable is dynamically linked. To run it on a machine without Visual Studio, ship the Intel redistributable libraries alongside it.

Run and debug

You need to tell the project which folder contains the input dataset to run against.

1. Right-click `CMakeLists.txt` in Solution Explorer and select **Add Debug Configuration**. If Co-Pilot is hiding Solution Explorer, close that pane first.
2. In the dialog, pick **Default** and click **Select**. Visual Studio creates `launch.vs.json` if it does not exist and opens it for editing.

The default file looks like this:

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "CMakeLists.txt"
    }
  ]
}
```

Add a `currentDir` entry pointing at your input dataset. The path can be absolute or relative to the `swatplus` project root. The `name` field is what shows up in the toolbar.

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "Ames_testing",
      "currentDir": "C:/work/sp/Ames_testing"
    }
  ]
}
```

Do not change `project` or `type`. You can add more configurations by repeating **Add Debug Configuration** and editing the new block.

Pick your configuration from the toolbar drop-down and run **Debug > Start Debugging** or **Debug > Start Without Debugging**. Breakpoints, conditional watches, and step-through all work.

Resources

- Intel C++ and Fortran Compilers Redistributable Libraries by Version
- Intel Fortran Compiler
- Intel Fortran Compiler for oneAPI release notes
- Visual Studio documentation
- CMake

The SWAT+ repository ships a `.devcontainer/` folder that lets GitHub Codespaces spin up a ready-to-use Visual Studio Code instance in the browser. No local Fortran toolchain needed.

When a Codespaces instance starts, it:

- loads your fork into a remote working folder
- installs the Intel `ifx` and `gfortran` compilers
- configures VS Code to use the CMake build system
- installs the VS Code extensions needed to debug Fortran
- copies the `refdata/Ames_sub1` dataset into `/workdata/Ames_sub1`
- creates an empty `/workdata/my_data` folder for your own inputs

Screenshots referenced below live in the SWAT+ source repo's `doc/` folder.

Cost

GitHub gives every account 120 free CPU hours per month across all Codespaces instances. A 2-CPU instance running for 60 hours uses 120 hours of quota. Beyond that, you pay per active hour. A small per-hour storage fee applies whether the instance is running or stopped.

Instance states:

- **Active.** The instance is in use (interactive or compiling). After 30 minutes of inactivity it goes to **Inactive**. The timeout is configurable.
- **Inactive.** Not running. Storage fee only.
- **Shutdown.** You stopped it manually. Same as Inactive billing-wise.
- **Deleted.** No further charges. A new instance has to be created from scratch to come back.

Starting an instance

1. Sign in to GitHub and go to your `swatplus` fork.
2. Open the hamburger menu (top left) and click **Codespaces**.
3. Click **New codespace** (top right).
4. Pick the repository, the branch, and the CPU count. Two CPUs is enough for SWAT+. Click **Create codespace**.

The instance takes a couple of minutes to come up. You should see an install progress bar in the lower right. When it disappears, a drop-down at the top of the editor asks which Fortran compiler to use. Pick one.

Building

Click the CMake icon in the left sidebar (the triangle-with-wrench) to open the CMake pane.

1. Under **Project Status > Configure**, check the Fortran compiler shown. To change it, hover and click the pencil icon, then pick another.

2. Right-click `CMakeLists.txt` in the CMake pane and select **Clean and Reconfigure**. Do this every time you switch compilers, and any time you add or remove a Fortran source file.
3. Right-click `CMakeLists.txt` again and select **Build All Projects**. The Output panel at the bottom should end with:

   ```
   [build] Build finished with exit code 0
   ```

Running and debugging

Click the debug icon in the left sidebar (the triangle with a bug). Pick `Ames_sub1` from the drop-down and click the green triangle.

After a fresh build, VS Code may ask which executable to run. Any of the debug-build options works. A release build is not debuggable.

Watch model output in the **Terminal** tab at the bottom. Set breakpoints, conditional breakpoints, or just let it run. On a runtime exception, the debugger stops at the failure point and you can inspect variable values.

Using your own dataset

The `/workdata/my_data` folder starts empty. To populate it:

1. In the VS Code Explorer, right-click `/workdata/my_data` and select **Upload**.
2. Allow the cookie prompt if your browser shows one (GitHub needs it to open your local file picker).
3. Select all the files in your input dataset and upload.
4. Switch the debug drop-down to `my_data` and click the green triangle.

To add a third dataset, create a new folder under `/workdata/`, then edit `.vscode/launch.json`. The default file already has blocks for `Ames_sub1` and `my_data`. Copy one block, change `name` and `cwd`, and add a comma between blocks.

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Ames_sub1",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/Ames_sub1",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        },
```

```
        {
            "name": "walnut_creek",
            "type": "cppdbg",
            "request": "launch",
            "program": "${command:cmake.launchTargetPath}",
            "args": [],
            "stopAtEntry": false,
            "cwd": "${workspaceFolder}/workdata/walnut_creek",
            "environment": [
                {
                    "name": "PATH",
                    "value": "${env:PATH}:${command:cmake.getLaunchTargetDirectory}"
                }
            ],
            "externalConsole": false,
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

Stopping the instance

To stop billing for active usage, click **Codespaces** in the bottom-left status bar and select **Stop Current Codespace**. It takes a minute or two.

Deleting the instance

After stopping, go back to the browser tab where you created the instance (or open the Codespaces panel from your GitHub repository). Find the instance at the bottom, click the ellipsis on its row, and select **Delete**. Storage charges stop after deletion. A new instance has to be created from scratch the next time you want to work.

SWAT+ ships a scenario test suite that runs the model against reference input datasets and compares the output files to known-good outputs. The tests catch regressions when source changes shift numeric results outside an accepted tolerance.

Tests are wired into the CMake build and driven by `ctest`.

How it works

The test data lives in `data/`. Each subfolder is a scenario containing input files and the golden output files. The reference output files are committed copies of model output that was reviewed and accepted as correct.

The driver script `test/check.py` does the actual comparison. The wiring sits in `CMakeLists.txt`:

```
find_package(Python REQUIRED)

set(check_py "${PROJECT_SOURCE_DIR}/test/check.py")
set(exe_path "${PROJECT_BINARY_DIR}/${SWATPLUS_EXE}")
set(test_dir "${PROJECT_BINARY_DIR}/data")
set(ref_dir  "${PROJECT_SOURCE_DIR}/data")
```

```
# error tolerances
set(rel_err "0.01")
set(abs_err "1e-8")

add_test(Ithaca_sub6 python3 ${check_py} ${exe_path} ${ref_dir}/Ithaca_sub6 ${test_dir}
 ↪  ${abs_err} ${rel_err})
add_test(Ames_sub1   python3 ${check_py} ${exe_path} ${ref_dir}/Ames_sub1   ${test_dir}
 ↪  ${abs_err} ${rel_err})
```

Two scenarios are registered by default: `Ames_sub1` and `Ithaca_sub6`. Default tolerances are 1% relative error and `1e-8` absolute error. Adding another scenario means adding another `add_test` line that points at a new subfolder in `data/`.

For each scenario, `check.py`:

1. Copies the scenario folder from `data/` into the build directory.
2. Runs the SWAT+ executable inside that copy.
3. Reads `.testfiles.txt` in the scenario, which lists the output file names to check (for example, `wb.txt` and `soc.txt`).
4. For each listed file, compares the new output to the golden copy line by line and column by column. Only floating-point values on matching lines and columns are compared.
5. Two values are considered equal if:

   ```
   abs(v1 - v2) <= abs_err + rel_err * abs(v2)
   ```

6. Prints the count of differing values and the maximum relative and absolute error. The scenario fails if any pair of values does not satisfy the tolerance.

Running the tests

Python 3 has to be on `PATH`. From the `swatplus` directory:

```
cmake -B build
cmake --build build
cd build
ctest
```

`ctest` prints a summary at the end. Detailed per-test logs land in `build/Testing/Temporary/`. The most recent run is captured in `LastTest.log`.

To run a subset:

```
ctest -R Ames_sub1
```

To see test output as it runs:

```
ctest --output-on-failure
```

`make test` from inside `build/` works as an alias.

Adding a new scenario

1. Create a new subfolder under `data/`, for example `data/MyBasin`. Put a full SWAT+ input set in it.
2. Run the model in that folder and review the outputs you want to lock down.
3. Commit those output files alongside the inputs.
4. Add a `.testfiles.txt` listing the output filenames (one per line) that `check.py` should compare.
5. Add an `add_test` line to `CMakeLists.txt`.
6. Reconfigure (`cmake -B build`) and run `ctest`.

CI

The same tests run in GitHub Actions on every push to `main`. See `.github/workflows/test.yml` in the source repository.

A SWAT+ version number is a Git tag on the `main` branch. No source file edits are needed to cut a release. The build system reads the tag and projects it into `src/main.f90` and the executable name.

How the version is read

`CMakeLists.txt` runs `git describe --tags` at configure time. If the current commit is a tagged one, that tag is the version. If not, the output is the most recent reachable tag plus the number of commits since and the abbreviated object name (for example, `61.1-1-g92b4413`).

```
execute_process(
  COMMAND git describe --tags
  WORKING_DIRECTORY ${CMAKE_SOURCE_DIR}
  OUTPUT_VARIABLE TAG
  OUTPUT_STRIP_TRAILING_WHITESPACE
)

set(SWAT_VERSION ${TAG})
set(SWATPLUS_EXE "swatplus-${SWAT_VERSION}.exe")
```

The version becomes part of the executable file name. Building at tag `61.0` produces `swatplus-61.0.exe`. Building one commit past tag `61.1` produces `swatplus-61.1-1-g92b4413.exe`.

Listing available tags

```
git tag
git log
```

Example `git log` output with two tags reachable from `HEAD`:

```
commit 92b44138a3fdb0ada27ed86e2a3a378188f70aef (HEAD -> main, origin/main)
Author: od <odavid@colostate.edu>
Date:   Tue Mar 5 21:43:50 2024 -0700

    fixed 'typo'

commit c5ff4b1083c5b320c72d02ec8b728811bed4441a (tag: 61.1)
...

commit df07e3f572adc21834f05c6711df3566e0231c2f (tag: 61.0)
...
```

Building at a specific tag

Check out the tag, then run the normal CMake build.

```
git checkout 61.0
cmake -B build
cmake --build build
```

The executable will be `swatplus-61.0.exe` (or no extension on Linux and macOS). Switch back to the branch head with `git switch main` and rebuild to get the post-tag version string.

How the version reaches the source

The Fortran source uses a template, `src/main.f90.in`. CMake processes it during configure and writes the final `src/main.f90` with substituted values:

```
prog = " SWAT+ @TODAY@        MODULAR Rev @YEAR@.@SWAT_VERSION@"
    write (*,1000)
    open (9003,file='simulation.out')
    write (9003,1000)
1000 format(1x,"                  SWAT+                ",/,             &
   &           "             Revision @SWAT_VERSION@   ",/,             &
   &           "      Soil & Water Assessment Tool     ",/,             &
   &           "@CMAKE_Fortran_COMPILER_ID@ (@CMAKE_Fortran_COMPILER_VERSION@), @ISO@,
    ↪  @CMAKE_HOST_SYSTEM_NAME@",/,             &
   &           "    Program reading . . . executing",/)
```

The substitution block in `CMakeLists.txt`:

```
if (EXISTS "${PROJECT_SOURCE_DIR}/src/main.f90.in")
    string (TIMESTAMP ISO   "%Y-%m-%d %H:%M:%S")
    string (TIMESTAMP TODAY "%b %d %Y")
    string (TIMESTAMP YEAR  "%Y")

    configure_file(
        "${PROJECT_SOURCE_DIR}/src/main.f90.in"
        "${PROJECT_SOURCE_DIR}/src/main.f90"
    )
endif()
```

After `cmake -B build`, `src/main.f90` contains the resolved values:

```
prog = " SWAT+ Mar 04 2024        MODULAR Rev 2024.61.0.0"

    write (*,1000)
    open (9003,file='simulation.out')
    write (9003,1000)
1000 format(1x,"                  SWAT+                ",/,             &
   &           "             Revision 61.0.0   ",/,             &
   &           "      Soil & Water Assessment Tool     ",/,             &
   &           "GNU (11.4.0), 2024-03-04 14:31:18, Linux",/,             &
   &           "    Program reading . . . executing",/)
```

Do not edit `src/main.f90` directly. It is overwritten every build. Edit `src/main.f90.in` instead.

Why tag-based versioning

- The executable's version always matches the tagged source commit it was built from.
- Version propagation into source and into the binary file name happens automatically.
- GitHub builds source archives and release binaries for each tag through the release workflow in `.github/workflows/build.yml`.

Conventions for writing SWAT+ Fortran source. The goal is portable, readable, and robust code that compiles cleanly on every supported compiler and operating system.

Each item is tagged with one of three levels:

- **Required.** Mandatory. Pull requests that violate these will not be merged.
- **Recommended.** Strongly preferred. Deviations need justification.
- **Encouraged.** Optional. Apply for consistency where reasonable.

The list below is ordered by decreasing impact on code quality. Most subjective style items (indent width, capitalization) sit in the lower tiers.

Interoperability and portability

Required

- Source must conform to the ISO Fortran 95 standard.
- No compiler-specific or platform-specific extensions.
- No use of compiler-dependent error specifier values (`IOSTAT`, `STAT`).
- The source must compile with `gfortran` from GCC, even if the production build uses the Intel compiler.
- `EXIT(N)` is the F90/95 standard exit. Prefer it to `STOP`. `STOP` does not always return an error code. If you need an error code passed back to a script, use `EXIT`, and keep its call sites in one central place.

Recommended

- For floating-point precision and integer size, use the F90/95 `KIND` feature. Do not depend on a vendor-supplied default precision flag.
- Do not use tab characters. Tabs are not part of the Fortran character set and break portability.

Encouraged

- For code that interacts with external frameworks, declare every variable with a `KIND` type to make integration easier.

Readability

Required

- Free-form source.
- Consistent indentation, at least two spaces per level.
- Organize source into modules.
- Do not use Fortran keywords (`DATA`, etc.) as variable names.
- Use meaningful names for variables and parameters. Recognized abbreviations are fine when full names would be unwieldy.
- Every externally-called function or subroutine carries a header describing what it does, its arguments, and the author. Small internal routines can use a few descriptive comments instead.
- No magic numbers. Physical constants (pi, gas constants) go in `PARAMETER` statements, not hard-coded into expressions.
- Do not hard-code numeric literals in argument lists. The compiler's default precision for literals is not guaranteed.
- No `GOTO`. If you must use one, document why.

Recommended

- Name your loops and other constructs (subroutines, functions, modules, interfaces) where the name aids readability, especially in nested loops.
- Comment input, output, and local variables. Grouping comments across similar variables is fine when their names speak for themselves.
- Comment to mark major functional sections.
- Do not use Fortran statement names or intrinsic function names as symbols.
- Prefer named parameters over numeric literals: `REAL, PARAMETER :: PI = 3.14159, ONE = 1.0`.

Encouraged

- In new code, follow your own consistent style. When editing existing code, match the style around you.
- Indent comments the same way as code.

- Group related procedures and data into modules.
- Keep lines to 132 characters or fewer.
- Use `<, >, <=, >=, ==, /=` instead of `.lt., .gt., .le., .ge., .eq., .ne.`.
- Name modules the same as the file that contains them. Avoid putting multiple modules in one file.
- Use blanks around operators, around `=`, and in declarations to separate syntax elements.
- Always use the `::` notation in declarations, even when there are no attributes.
- Line up attributes, variable names, and trailing comments vertically in declarations.
- Remove unused variables.
- Remove debug code once the bug is fixed.

Robustness

Required

- `IMPLICIT NONE` in every program, module, and procedure.
- Use `PRIVATE` in modules by default, then explicitly mark what should be `PUBLIC`. Exception: modules that exist only to expose public data (constants, for example) can stay fully public.
- Initialize every variable. Do not rely on the machine's default.
- Do not initialize a variable of one type with a value of another type.

Recommended

- Do not use `==` or `/=` on floating-point values. Compare against an explicit tolerance instead (an epsilon check).
- In mixed-type expressions and assignments, write the type conversion explicitly. Do not let the compiler do it implicitly.
- No `INCLUDE` files. Use `USE` with modules.
- Derived types belong in their own module, together with the procedures that operate on them.
- Each procedure should do one thing.
- Module-level (global) public variables should hold only static or rarely-changing data.

Encouraged

- Always use parentheses to make expression evaluation order explicit.
- Prefer derived types (`%` field access) over loose collections of arrays.

Arrays

Required

- Subscript expressions must be integer.
- Do not assume any particular array passing mechanism when passing arrays as arguments.

Recommended

- Use array operations and intrinsics where they apply.
- Pass arrays as assumed shape.

Encouraged

- Declare `DIMENSION` on every non-scalar.

Dynamic memory and pointers

Required

- Prefer `ALLOCATABLE` arrays over pointers where possible. Reduces leak and fragmentation risk.

- Pointers are allowed when a subroutine needs to return a declared array to its caller.
- Initialize every pointer to `NULL()` in its declaration: `INTEGER, POINTER :: x => NULL()`.
- Prefer automatic arrays for transient allocations. `ALLOCATABLE` and `POINTER` arrays need explicit `DEALLOCATE`.

Recommended

- Always deallocate. Especially inside subroutines and loops.
- Check the `STAT` argument on `ALLOCATE` and `DEALLOCATE`.
- Do not repeatedly `ALLOCATE`, `DEALLOCATE`, then `ALLOCATE` a larger block. Heap fragmentation follows.

Encouraged

- Dynamic allocation is preferred over fixed-size declarations with worst-case dimensions.
- Use automatic arrays in subroutines for simplicity.

Looping

Required

- No `GOTO` for loop exit or continuation. Use `EXIT` or `CYCLE`.

Recommended

- No numbered `DO` loops (`DO 10 ... 10 CONTINUE`).

Functions and procedures

Required

- Avoid `SAVE`. Hold state in module variables instead.
- No `ENTRY` statements in functions.
- Functions must not return pointers.
- Do not name a user function the same as an intrinsic (`SUM`, etc.).
- A procedure that returns a single value should be a function. The value can be a derived type.
- Communicate through arguments or through the procedure's own module variables.

Recommended

- Every dummy argument that is not a pointer carries an `INTENT`.
- Avoid type-specific intrinsics (`AMAX`, `DMAX`). Use the generic (`MAX`).
- Do not declare static-dimensioned array arguments.
- Validate argument values where it matters.

Encouraged

- On an error, print a message that includes the routine name. You may terminate inside the routine, or pass an error flag back through the argument list.
- Use overloading when the same operation works on multiple types.
- Use modules or `CONTAINS` for explicit interfaces.
- Avoid external routines. They need separate interface blocks that drift out of sync.

I/O

Required

- External-file I/O statements carry `err=`, `end=`, and `iostat=` as appropriate.
- All global variables, if any, are initialized at startup.

Recommended

- Avoid `NAMELIST` I/O.
- Use `WRITE` rather than `PRINT` for non-terminal I/O.
- Use character parameters or in-line format specifiers in `READ` and `WRITE`. Do not use labeled `FORMAT` statements.

Obsolescent features to avoid

Required

- No common blocks. Use modules.
- No assigned or computed `GO TO`. Use `CASE`.
- No arithmetic `IF`. Use a block `IF`.
- Use `REAL`, not `DOUBLE PRECISION`.
- Avoid `DATA`, `ASSIGN`, labeled `DO`, `BACKSPACE`, blank `COMMON`, `BLOCK DATA`.
- Do not branch to an `END IF` from outside its `IF` block.
- No non-integer `DO` loop counters.
- No Hollerith constants.
- No `PAUSE`.
- No multiple `RETURN`, no alternate `RETURN`.

Recommended

- No `EQUIVALENCE`. Use pointers or derived types.

Encouraged

- Do not implicitly reshape arrays when passing them. This was common in F77 and is forbidden by F90 but still possible through external routines without interface blocks.

Source files

Required

- Document every function interface: argument name, type, units, description, constraints, defaults.
- No `INCLUDE`. Use `USE`.
- Keep lines (including comments) under 80 columns where the language allows it.

Recommended

- Keep individual procedures under 300-500 effective lines.
- Separate statement blocks with blank lines or a marker character in column 1.
- Indent consistently.
- Module and subprogram names are lower case. The file is named after the contained module or subprogram, with `.f90`.

Encouraged

- Separate argument declarations from local declarations visually.
- Use descriptive unique names. Keep them under 12-15 characters.
- Indent continuation lines so a multi-line expression lines up readably.
- Comments start with a marker character (`!`). A standalone comment line starts the marker in column 1.

References

- SAS Coding Conventions

- WRF Coding Conventions
- USGS Coding Standards

Code Of Conduct

The SWAT+ project follows the Contributor Covenant, version 2.1.

The full text lives in the source repository as `CODE_OF_CONDUCT.md`. To report a violation, contact the maintainers listed there.

Anyone contributing to SWAT+ (issues, pull requests, code review, mailing-list discussions) is expected to follow it.

Where work happens

- **Code**: https://github.com/swat-model/swatplus
- **Issues**: https://github.com/swat-model/swatplus/issues
- **Pull requests**: https://github.com/swat-model/swatplus/pulls
- **Release notes**: https://github.com/swat-model/swatplus/releases

Reporting a bug

Open an issue with:

- the SWAT+ version (`swatplus -v` or the executable filename)
- the operating system and compiler
- a minimal project that reproduces the problem (or, if possible, a pointer to one of the reference datasets in `refdata/` plus the changes that trigger the bug)
- the contents of `simulation.out`, `diagnostics.out`, and `checker.out` from the failed run.

Bugs that affect water or mass balance, output formats, or input parsing are highest priority.

Submitting a change

1. **Fork** `swat-model/swatplus` on GitHub.
2. **Branch** from `main`. Use a descriptive name (`fix-aquifer-init`, `add-soil-temp-output`).
3. **Build and test locally** before pushing. See Build from source (CMake) and Testing.
4. **Open a pull request** against `swat-model/swatplus:main`. The PR description should state:
   - what the change does
   - why it is needed (link an issue if there is one)
   - which reference datasets you ran and what changed in their outputs
   - any new input or output fields and how existing input files should be updated
5. **Address review comments**. Force-push is acceptable on your branch during review.

What reviewers look for

- Code compiles with both gfortran and ifx (the CI matrix).
- Reference-dataset outputs are unchanged for unrelated runs (no regression).
- New parameters have defaults that preserve existing behaviour when not set.

- Input-file changes are accompanied by SWAT+ Editor changes or a migration note.
- Style follows Coding conventions.

CI

The repository runs GitHub Actions on every pull request. The matrix builds with multiple compilers and runs the scenario tests in `test/`. See `.github/workflows/` for the exact configuration. A red check blocks merging.

Documentation changes

This documentation site lives at https://github.com/swat-model/swatplus-documentation. Documentation pull requests follow the same workflow as code. Every input-reference and output-reference page is verified against the SWAT+ Fortran source; see `SOURCE-OF-TRUTH.md` for the procedure if you are touching one of those pages.

SWAT+ supports two calibration modes that are configured and run from inside the model.

- **Hard calibration** fits named model parameters to observed time series (streamflow, sediment, nutrient loads). Parameter values are perturbed within absolute bounds, the model is re-run, and the simulation is compared with measured data. Implemented in `calhard_*.f90`. Driven by `cal_parms.cal` (parameter definitions and bounds) and `calibration.cal` (the change list applied each run).
- **Soft calibration** adjusts process outputs toward regional water-balance and yield targets without exposing the underlying parameters as a sample space. Targets are expressed as fractions of precipitation for the water balance, channel sediment budgets, and plant growth. Implemented in `calsoft_*.f90`. Driven by `codes.sft`, `wb_parms.sft`, `water_balance.sft`, `ch_sed_budget.sft`, `ch_sed_parms.sft`, `plant_parms.sft`, and `plant_gro.sft`.

The two modes share the same internal parameter selector (`cal_parm_select.f90`) and the same conditional change machinery (`cal_conditions.f90`).

Pages

- Hard calibration. Parameter perturbation, sources, the role of `calibration.cal`.
- Soft calibration. Regional water balance and yield targets, sources, the role of `codes.sft`.
- Parameter change files. Format of `cal_parms.cal` and `calibration.cal`, verified against the readers.
- Conditions and regionalization. Applying a change only to HRUs that match a condition (HSG, texture, plant, land use, calibration group).
- Sensitivity workflow. Driving SWAT+ from an external tool to compute sensitivity indices, using the same `cal_parms.cal` to define the sample space.

Soft calibration nudges model outputs toward regional targets expressed as fractions of precipitation (water balance), per-stream-order channel sediment budgets, and per-region plant yields. The targets are user-supplied and the model adjusts a small fixed set of parameters internally until each component falls inside its target window. Soft calibration is run separately from a normal simulation; it loops over the relevant land-use regions and process components, updates parameters, and re-runs the affected balance routine.

Components

The soft codes file (`codes.sft`) is read by `calsoft_read_codes.f90` into `cal_codes` (`type soft_calibration_codes` in `calibration_data_module.f90`). Each switch enables one component of the soft calibration:

Code	Switch	Component	Driver routine
hyd_hru	n, a, b, y	HRU water balance by land use in each region. a calibrates all components (ET, surface runoff, lateral flow, percolation). b calibrates baseflow and total runoff only.	calsoft_hyd.f90, calsoft_hyd_bfr.f90
hyd_hrul	y/n	hru_lte water balance by land use in each region	caltsoft_hyd.f90
plt	y/n	Plant growth (LAI, harvest index) by region	calsoft_plant.f90
sed	y/n	Upland sediment yield by land use in each region	calsoft_sed.f90
nut	y/n	HRU nutrient balance by land use in each region	(gated in calsoft_control.f90)
chsed	y/n	Channel widening and bank accretion by stream order	(gated in calsoft_control.f90)
chnut	y/n	Channel nutrient balance by stream order	(gated in calsoft_control.f90)
res	y/n	Reservoir budgets by reservoir	(gated in calsoft_control.f90)

If any switch is non-default the master flag cal_soft is set to "y" and calsoft_control is called instead of (or alongside) the normal simulation.

HRU water balance

calsoft_hyd.f90 is the full water-balance loop. It iterates over land-use regions, reads the regional water-balance targets (water_balance.sft) and the per-parameter perturbation envelope (wb_parms.sft), and adjusts the active parameter set until each component is within tolerance. The components and their helper routines are:

- PET, calsoft_hyd_bfr_pet.f90
- ET, calsoft_hyd_bfr_et.f90
- Surface runoff, calsoft_hyd_bfr_surq.f90
- Lateral flow, calsoft_hyd_bfr_latq.f90
- Percolation, calsoft_hyd_bfr_perc.f90

The shared baseflow ratio loop is calsoft_hyd_bfr.f90.

Adjustable parameters for the upland water balance are read into ls_prms (type soft_calib_parms). Each entry carries a parameter name, the change type (absval, abschg, or pctchg), and neg/pos perturbation limits with absolute floor/ceiling values.

Channel sediment

ch_sed_budget.sft carries the per-stream-order sediment budget targets. ch_sed_parms.sft carries the perturbation envelope for channel sediment parameters, read into ch_prms. The channel sediment block is gated in calsoft_control.f90 but the call to a calsoft_chsed driver is commented out in the current source; only the per-order accounting loop runs.

Plant growth

plant_parms.sft defines the plant parameter envelope (LAI potential, harvest index) and plant_gro.sft carries the per-region plant target table. The driver is calsoft_plant.f90; calsoft_plant_zero.f90 zeros the running statistics between iterations. After soft calibration writes the new plant parameter values, pl_write_parms_cal dumps a fresh plants.plt and a hydrology-cal.hyd snapshot.

Outputs

`calsoft_control.f90` writes:

- `hydrology-cal.hyd` (unit 5001) with per-HRU hydrology parameters after the run.
- A channel sediment summary (unit 5000) when `chsed = y`.

These outputs are intended to be copied back into the model inputs to lock in the soft-calibrated values.

Input files

File	Read by	Purpose
`codes.sft`	`calsoft_read_codes.f90`	Component switches.
`wb_parms.sft`	(read in `calsoft_init`)	Upland water-balance parameter envelope.
`water_balance.sft`	(read in `calsoft_init`)	Regional water-balance targets (SR, LF, PC, ET, TF as fractions of precipitation).
`ch_sed_budget.sft`	(read in `calsoft_init`)	Channel sediment budget targets by stream order.
`ch_sed_parms.sft`	(read in `calsoft_init`)	Channel sediment parameter envelope.
`plant_parms.sft`	(read in `calsoft_init`)	Plant parameter envelope.
`plant_gro.sft`	(read in `calsoft_init`)	Per-region plant yield targets.

All `*.sft` slots are declared in `input_file_module.f90`, `type input_chg`. Set the slot to `null` (or remove the file) to skip a component.

Related pages

- Hard calibration. Use after soft calibration to fit remaining parameters to observed time series.
- Parameter change files. `cal_parms.cal` defines the parameter universe and bounds shared with hard calibration.

Hard calibration applies a list of named parameter changes from `calibration.cal`, re-initializes all model objects, then re-runs the time loop. Each change names a parameter, a change type (`absval`, `abschg`, `pctchg`), a value, a list of objects (HRUs, channels, aquifers, plants, …) to apply the change to, and optional conditions. The parameter universe and the absolute min/max for each parameter are defined in `cal_parms.cal`.

Source

- Driver: `src/calhard_control.f90`
- Parameter selector: `src/cal_parm_select.f90`
- Change application with conditions: `src/cal_conditions.f90`
- Parameter database reader: `src/cal_parm_read.f90`
- Change list reader: `src/cal_parmchg_read.f90`

Workflow

1. The model is run once normally. Output baseline is captured.
2. The user edits `calibration.cal` with the parameter changes to test (or an external driver writes it).
3. SWAT+ is re-launched. During startup, `cal_parm_read` loads `cal_parms.cal` into the `cal_parms` array and `cal_parmchg_read` loads `calibration.cal` into the `cal_upd` array. Each `cal_upd` entry is crosswalked to its `cal_parms` entry by name to pick up the absolute bounds and object type.
4. Inside the simulation, `cal_conditions` is called once. For every entry in `cal_upd`, it evaluates the conditions, then walks the element list and calls `cal_parm_select` for each affected object. `cal_parm_select` does the actual write into the live model state (HRU, channel, aquifer, plant, soil layer, climate series).

5. The master switch `cal_hard = "y"` triggers a second pass: `calhard_control` calls `re_initialize` then `time_control`, re-running the full simulation with the modified parameters.

The change type drives how the value is applied to the existing parameter:

chg_typ	Effect
absval	The new value replaces the existing one.
abschg	The value is added to the existing one (signed).
pctchg	The value is interpreted as a percent change.

After every change the result is clamped to the `[absmin, absmax]` range from `cal_parms.cal`.

Object types

`cal_parms.cal` tags each parameter with an object type (`ob_typ`). The type determines which collection in the model state is iterated when the change is applied:

Code	Collection	Examples
hru	HRUs	cn2, ovn, lat_ttime, canmx
hlt	hru_lte	parameters of the simplified HRU
sol	Soil layers within an HRU (lyr1/lyr2 select the range)	awc, bd, k
lyr	Soil layers	layer-targeted variants
plt	Plants	lai_pot, harv_idx, epco, phu_mat
aqu	Aquifers	alpha, revap_co, flo_min
cha, sdc, rte	Channels (legacy, degraded, routed)	channel routing and sediment parameters
swq	Channel water quality	nutrient parameters
res	Reservoirs	res_pvol, surface area parameters
rdt	Reservoir decision tables	drawdown_days, withdraw_rate
ru	Routing units	RU-level parameters
bsn	Basin	basin-wide scalars in parameters.bsn
pcp, tmp, cli	Climate gauges and series	precip, temp adjustments over date ranges
gwf	gwflow cells	gwflow parameters

The list above is taken from the `select case (cal_parms(ipar)%ob_typ)` block in `cal_parmchg_read.f90` and the dispatch in `cal_conditions.f90`.

Re-initialization

`calhard_control.f90` calls `re_initialize` before re-running. This resets simulation state (water content, plant state, accumulators) so the second run is comparable to the baseline rather than continuing from baseline end-state.

Related pages

- Parameter change files. Column-level format of `cal_parms.cal` and `calibration.cal`.
- Conditions and regionalization. Limiting a change to HRUs that match a condition.
- Sensitivity workflow. Driving hard calibration from an external sampler.
- Soft calibration. Adjust process outputs toward regional targets before fitting to time series.

Purpose

`cal_parms.cal` is the dictionary of every parameter that calibration can touch: its name, the object type it lives on, and the absolute range it may be moved through. `calibration.cal` is the change list applied during a hard-calibration run: each row picks a parameter from the dictionary, sets a change type and magnitude, optionally restricts the change to a list of objects, and optionally attaches conditions. Both files feed the same parameter selector (`cal_parm_select`) used by hard calibration and by parameter changes inside soft calibration.

Source

- `cal_parms.cal` reader: src/cal_parm_read.f90
- `calibration.cal` reader: src/cal_parmchg_read.f90
- Type definitions: src/calibration_data_module.f90, `type calibration_parameters` (rows of `cal_parms.cal`) and `type update_parameters` (rows of `calibration.cal`).
- File slot names: src/input_file_module.f90, `type input_chg`. Defaults: `cal_parms = "cal_parms.cal"`, `cal_upd = "calibration.cal"`.

cal_parms.cal

`cal_parm_read.f90` reads:

- Line 1: title (skipped).
- Line 2: integer `mchg_par`, the number of parameter records that follow.
- Line 3: header (skipped).
- Lines 4 .. 3 + `mchg_par`: one record per parameter, read list-directed into `cal_parms(i)`.

Record layout (`type calibration_parameters`):

#	Field	Type	Description
1	`name`	char(25)	Parameter name. Must match a case in `cal_parm_select.f90`. Examples: `cn2`, `awc`, `alpha`, `epco`, `lat_ttime`.
2	`ob_typ`	char(25)	Object type the parameter is attached to: `hru`, `hlt`, `sol`, `lyr`, `plt`, `aqu`, `cha`, `sdc`, `rte`, `swq`, `res`, `rdt`, `ru`, `bsn`, `pcp`, `tmp`, `gwf`. Determines which collection is iterated.
3	`absmin`	real	Lower bound. Any change is clamped to this value.
4	`absmax`	real	Upper bound. Any change is clamped to this value.
5	`units`	char(25)	Units string for the parameter (`mm`, `m`, `days`, `null`, ...). Documentation only; the reader does not interpret it.

Example

`refdata/Osu_1hru/cal_parms.cal`:

```
cal_parms.cal: written by SWAT+ editor v2.2.0 on 2023-03-22 04:25 for SWAT+ rev.60.5.4
186
name                        obj_typ       abs_min       abs_max             units
cn2                             hru      35.00000      95.00000              null
cn3_swf                         hru       0.00000       1.00000              null
usle_p                          hru       0.00000       1.00000              null
ovn                             hru       0.01000      30.00000              null
elev                            hru       0.00000    5000.00000                 m
slope                           hru       0.00010       0.90000               m/m
slope_len                       hru      10.00000     150.00000                 m
lat_ttime                       hru       0.50000     180.00000              days
```

The count line on line 2 (`186`) drives the allocation of `cal_parms(0:mchg_par)`. The header on line 3 (`name`, `obj_typ`, `abs_min`, `abs_max`, `units`) is skipped; values are read by position.

calibration.cal

`cal_parmchg_read.f90` reads:

- Line 1: title (skipped).
- Line 2: integer `mcal`, the number of change records.
- Line 3: header (skipped).
- Then `mcal` change records, each followed by `conds` condition lines (often zero).

Change record

Each change record is one logical line, read list-directed in two passes. The first pass reads the fixed prefix; if the prefix indicates an object list is present, the line is backspaced and re-read to pick up the integers.

Fixed prefix:

#	Field	Type	Description
1	`name`	char(25)	Parameter name. Must match a `name` in `cal_parms.cal`. The reader crosswalks this to `num_db`.
2	`chg_typ`	char(16)	Change operator. One of `absval` (replace), `abschg` (add), `pctchg` (percent change).
3	`val`	real	Magnitude of the change, interpreted per `chg_typ`.
4	`conds`	int	Number of condition lines that follow this record. May be zero.
5	`lyr1`	int	First soil layer in range (0 means all layers). For reservoir decision tables, reused as a storage-zone flag.
6	`lyr2`	int	Last soil layer in range (0 means through last layer).
7	`year1`	int	First year (for climate parameter changes).
8	`year2`	int	Last year (for climate parameter changes).
9	`day1`	int	First day of year (for climate parameter changes).
10	`day2`	int	Last day of year (for climate parameter changes).
11	`nspu`	int	Number of object indices listed after this prefix. `0` means apply to every object of this parameter's `ob_typ`.

If `nspu > 0`, the same line continues with `nspu` integers giving the object indices to update. When `nspu == 0` the reader allocates a `num` array that lists every object of the matching type (every HRU, every channel, every plant, and so on) so the change applies globally.

Condition lines

A condition line takes one of two forms, distinguished by the literal token in column 1:

- `range <var> <val1> <val2>`. Restricts the change to objects whose `<var>` falls between `val1` and `val2`. Used in `cal_conditions.f90` for `res_pvol`.
- `<var> <alt> <targ> <targc>`. Read as a `calibration_conditions` record. Used for categorical filters: `hsg`, `texture`, `plant`, `pl_class`, `landuse`, `cal_group`, `res_typ`.

See Conditions and regionalization for the supported variable list.

Example

`refdata/Osu_1hru/calibration.cal`:

```
  hru-new.cal developed from soft data calibration
    13
 NAME           CHG_TYP                  VAL    CONDS  LYR1  LYR2   YEAR1  YEAR2   DAY1   DAY2  OBJ_TOT
cn2         pctchg                 7.714000E+00  0          0          0          0          0
petco       pctchg                -5.273950E+01  0          0          0          0          0
latq_co     pctchg                -3.732030E+01  0          0          0          0          0
alpha       pctchg                 6.859000E+00  0          0          0          0          0
epco        pctchg                 6.534100E+01  0          0          0          0          0
awc         pctchg                -6.256550E+01  0          0          0          0          0
esco        pctchg                 3.305500E+01  0          0          0          0          0
perco       pctchg                 1.434000E+00  0          0          0          0          0
cn3_swf     pctchg                -7.710000E-02  0          0          0          0          0
canmx       pctchg                -6.680230E+01  0          0          0          0          0
lat_len     pctchg                -6.241200E+00  0          0          0          0          0
revap_co    pctchg                -5.854530E+01  0          0          0          0          0
surlag      pctchg                -2.125720E+01  0          0          0          0          0
```

In this file every record has `conds = 0` and `nspu = 0`, so each change applies to every HRU. The trailing integer count is `11` per row: `conds`, `lyr1`, `lyr2`, `year1`, `year2`, `day1`, `day2`, and `nspu` (eight integers), padded by the editor with extra zeros that the list-directed reader ignores.

Carbon parameters

SWAT+ wires 40 carbon-related parameters in `cal_parm_select.f90`. Each entry below maps a calibration name to its target field, suggests a working min/max range, and notes the object type (`bsn`, `lyr`, `plt`, `hru`) that must appear in the `ob_typ` column of `cal_parms.cal`. Bounds come from the sptAPI bundled `cal_parms.cal` (261 entries), with the temperature triple tightened to physical bounds to physically realistic ranges.

Basin scalars from `carbon.bsn` (18 entries)

name	ob_typ	units	abs_min	abs_max	description
init_seq	bsn	fraction	0.5	1.0	initial sequestration fraction (org_frac%frac_seq)
init_microb	bsn	fraction	0.0	0.1	initial microbial-humus fraction (org_frac%frac_hum_microb)
init_slow	bsn	fraction	0.2	0.6	initial slow-humus fraction (org_frac%frac_hum_slow)
init_passive	bsn	fraction	0.2	0.6	initial passive-humus fraction (org_frac%frac_hum_passive)
koc_c	bsn	none	500	1500	KOC for C transport (cb_wtr_coef%prmt_21)
solc_ratio	bsn	0..1	0.1	1.0	runoff:percolate soluble C ratio (cb_wtr_coef%prmt_44)
manure_c_frac	bsn	none	0.0	1.0	manure C partition between fresh and stable pools (man_coef%rtof)
bio_consol	bsn	none	0.05	0.5	biological consolidation factor (bio_consf)
till_consol	bsn	none	0.05	0.5	tillage consolidation factor (till_consf)
sfc_rsd_photodeg	bsn	1/day	0.0	0.05	surface residue photodegradation rate (photo_degrade_factor)
n_act_frac	bsn	fraction	0.005	0.04	fraction of organic N in active humus pool (n_act_frac)
cnr_cap	bsn	none	100	1000	upper cap on residue C:N ratio
cnr_ref	bsn	none	10	50	reference C:N ratio where decomp factor = 1
cpr_cap	bsn	none	1000	10000	upper cap on residue C:P ratio
cpr_ref	bsn	none	100	400	reference C:P ratio where decomp factor = 1
t_cbn_min	bsn	deg C	-5	5	minimum temperature bound for tmpf2 (org_con%tn); tightened to physical bounds
t_cbn_opt	bsn	deg C	25	40	optimum temperature for tmpf2 (org_con%top); tightened to physical bounds
t_cbn_max	bsn	deg C	45	55	maximum temperature bound for tmpf2 (org_con%tx); tightened to physical bounds

Note

The `min < opt < max` ordering invariant is not enforced by `cal_parms.cal` itself; calibration tools may still produce invalid combinations. The `fcgd` clamp ensures invalid orderings degrade gracefully (return 0) rather than crashing.

Per-layer fields from `carbon_lyr.bsn` (11 entries)

Each entry addresses `carbdb(ly)` or `org_allo(ly)` using the `lyr1`/`lyr2` slot of the `calibration.cal` record. Bounds are guarded to `[1, size(carbdb)]` inside the `case` branch, so out-of-range layer indices are silently no-op.

name	ob_typ	units	abs_min	abs_max	description
hp_rate	lyr	1/day	1.0e-06	1.0e-04	passive humus turnover rate (carbdb(ly)%hp_rate)
hs_rate	lyr	1/day	1.0e-05	1.0e-03	slow humus turnover rate (carbdb(ly)%hs_rate)
microb_rate	lyr	1/day	0.005	0.05	microbial biomass turnover rate (carbdb(ly)%microb_rate)
meta_rate	lyr	1/day	0.01	0.1	metabolic litter turnover rate (carbdb(ly)%meta_rate)
str_rate	lyr	1/day	0.005	0.05	structural litter turnover rate (carbdb(ly)%str_rate)
microb_top_rate	lyr	1/day	0.005	0.05	top-layer microbial activity adjustment (carbdb(ly)%microb_top_rate)
hs_hp	lyr	none	0.01	0.2	slow-to-passive humus allocation (carbdb(ly)%hs_hp)
a1co2	lyr	fraction	0.3	0.8	metabolic litter to CO2 (org_allo(ly)%a1co2)
asco2	lyr	fraction	0.3	0.8	slow humus to CO2 (org_allo(ly)%asco2)
apco2	lyr	fraction	0.3	0.8	passive humus to CO2 (org_allo(ly)%apco2)
abco2	lyr	fraction	0.0	0.8	microbial biomass to CO2 (org_allo(ly)%abco2)

Biomixing and tillage curve coefficients (6 entries)

Curve fit coefficients used by `mgt_tillfactor.f90` tillf option 4. The `zz_` prefix that the older code carried was dropped to match the column names in `carbon.bsn`.

name	ob_typ	units	abs_min	abs_max	description
bmix_a	bsn	none	1.0	5.0	biological mixing coefficient A
bmix_b	bsn	none	1.0	10.0	biological mixing coefficient B
bmix_c	bsn	none	-10.0	-1.0	biological mixing coefficient C
tillmix_a	bsn	none	1.0	5.0	tillage mixing coefficient A
tillmix_b	bsn	none	5.0	25.0	tillage mixing coefficient B
tillmix_c	bsn	none	-10.0	-1.0	tillage mixing coefficient C

Plant carbon parameters (5 entries)

Carbon-relevant parameters on the plant database (`pldb(ielem)`). These were not previously exposed even though plant biomass drives the whole NPP and residue chain.

name	ob_typ	units	abs_min	abs_max	description
bio_e	plt	(kg/ha)/(MJ/m^2)	10.0	60.0	biomass-energy ratio (drives NPP)
hvsti	plt	fraction	0.1	0.8	harvest index (controls residue vs grain C split)
rdmx	plt	m	0.5	3.0	maximum root depth
t_opt	plt	deg C	15	35	optimum plant growth temperature
t_base	plt	deg C	0	15	base plant growth temperature

The existing `usle_c` plant parameter is retained from earlier releases.

> **Note**
>
> For the `init_*` parameters to take effect, `proc_cal` must run **before** `soil_nutcarb_init`. The current `main.f90` already orders these correctly; do not reorder. Previously the order was reversed and `init_*` calibration was a silent no-op because the pools were already sized using pre-cal values.

Related files

- `carbon.bsn`. Basin-wide carbon scalar inputs whose fields these parameters target.
- `carbon_lyr.bsn`. Per-layer carbon inputs.
- Hard calibration. How these files are consumed during a calibration run.
- Conditions and regionalization. Condition-line vocabulary.

A change in `calibration.cal` can carry condition lines that restrict where the change is applied. The conditions are tested object by object in `cal_conditions.f90`. If every condition evaluates true for a given object, the change is passed to `cal_parm_select` for that object; otherwise the object is skipped.

Source

- Condition evaluation: `src/cal_conditions.f90`
- Condition reader: `src/cal_cond_read.f90`
- Parameter dispatch: `src/cal_parm_select.f90`
- Condition record type: `src/calibration_data_module.f90`, `type calibration_conditions` and `type update_conditional`.

Two condition syntaxes

`cal_parmchg_read.f90` peeks at the first token of each condition line:

- If the token is the literal word `range`, the line is read as `range <var> <val1> <val2>`. The condition holds when the value of `<var>` for the current object falls inside `[val1, val2]`. `val1` and `val2` are stored on the parent change record (shared across that record's range conditions).
- Otherwise the line is read as a `calibration_conditions` record: `<var> <alt> <targ> <targc>`. `targ` is a real, `targc` is a string. The reader stores all four fields but `cal_conditions.f90` only uses `var`, `targc`, and `targ` depending on the variable.

Supported variables

The `select case (cal_upd(ichg_par)%cond(ic)%var)` block in `cal_conditions.f90` accepts:

Variable	Object type	Form	Semantics
`res_pvol`	reservoirs	range	Reservoir principal storage volume (compared in units of 10000).
`hsg`	HRU soils	categorical (`targc`)	Hydrologic soil group must equal `targc` (`A`, `B`, `C`, `D`).
`res_typ`	reservoirs	categorical (`targc`)	Reservoir decision-table name must equal `targc`.
`texture`	HRU soils	categorical (`targc`)	Soil texture string must equal `targc`.
`plant`	HRU plant community	categorical (`targc`)	The HRU's plant community must contain a plant named `targc`.
`pl_class`	HRU plants	categorical (`targc`)	Plant class for the HRU must equal `targc`.
`landuse`	HRU land use management	categorical (`targc`)	`hru%land_use_mgt_c` must equal `targc`.
`cal_group`	HRU calibration tag	categorical (`targc`)	`hru%cal_group` must equal `targc`.

A change with multiple conditions implements an AND: all must hold for the object to be modified.

Regionalization

Two patterns produce a regional effect:

- **Implicit through `cal_group`.** Tag every HRU with a `cal_group` name (set on the HRU input). Then a change with a `cal_group <name>` condition runs only on that group. This is how named region-based hard calibration is typically done.
- **Explicit object list.** Put the HRU (or channel, or aquifer) indices directly on the change record (`nspu > 0`). No conditions needed. This is what soft calibration writes when it commits its perturbations.

The two can combine: an explicit object list with one or more conditions further filters the list.

Decision-table conditional updates

`cal_cond_read.f90` reads a separate file, `scen_dtl.upd`, that schedules updates by decision table. Each row gives a maximum number of times the table may fire, a table type, and the name of a `dtbl` entry in `conditional.ctl`. The reader crosswalks the table name into `dtbl_scen` and stores the result in `upd_cond` (`type update_conditional`). The same decision-table machinery used elsewhere in SWAT+ (land-use change scenarios, management triggers) is reused here.

Parameter dispatch

Once conditions pass, the change is sent to `cal_parm_select.f90` with the parameter name, change type, magnitude, absolute bounds, the target element, and the soil layer (for layer-targeted soil parameters). `cal_parm_select` is a long `select case (chg_parm)` that writes the change into the live model state via `chg_par` (the change operator) and calls the relevant re-initializer (`curno` for `cn2`, `soil_awc_init` for `awc`, and so on).

Related pages

- Parameter change files. The columns that hold these conditions on disk.
- Hard calibration. When conditions are evaluated in the calibration run.

SWAT+ has no built-in sensitivity analysis. Sensitivity is run by an external tool that writes `calibration.cal`, launches the SWAT+ executable, reads an objective value from the simulation output, and repeats. The parameter list and bounds are taken from the same `cal_parms.cal` used by hard calibration.

Pattern

1. Pick the parameters to test. Pull their absolute ranges from `cal_parms.cal` or override them with a tighter calibration range. Each parameter is one dimension of the sample space.
2. Generate a sample matrix using a sensitivity method (Sobol, FAST, Morris, LH-OAT, …). Each row of the matrix is one parameter vector.
3. For each row: write `calibration.cal` with one `pctchg` (or `absval`, `abschg`) record per parameter, run SWAT+, read the chosen objective (NSE, KGE, percent bias, total flow, …) from the output files, and record it.
4. Feed the sample matrix and the model performance vector into the sensitivity method to compute indices: first-order, total-order, second-order interaction, or elementary effects depending on the method.

The expensive step is the simulation loop. A sample size in the hundreds to low thousands is common.

sensitivityAPI

`sensitivityAPI` is a command-line tool that implements steps 2 and 4 of the pattern. It operates on a SWAT+

TxtInOut directory: it reads a parameter CSV (`par_data.stb`), writes a sample matrix (`par_sample.stb`), and after the user has run the simulations and collected performance values into `perf_report.stb`, computes sensitivity indices.

Supported methods:

Method	Sampling	Indices
`FAST`	frequency-based	S1, ST
`sobol`	Saltelli quasi-random	S1, ST, S2
`RBD_FAST`	Latin Hypercube	S1
`DMIM`	Latin Hypercube	S1 (delta)
`lhoat`	LH-OAT hybrid	S1 (elementary effects)

Usage:

```
sensitivityAPI generate_sample sobol /path/to/txtinout 1024
# run the simulations here, fill perf_report.stb
sensitivityAPI analyse_sensitivity sobol /path/to/txtinout
```

The orchestration (writing `calibration.cal` per row, launching SWAT+, parsing the output) is done by a driver such as sptAPI or spToolboX. `sensitivityAPI` is only the sampler and the post-processor.

Roll your own

Any language with a numerical library can do the same. The minimum loop:

1. Parse `cal_parms.cal` to get bounds for the parameters you care about.
2. Generate `N` parameter vectors with a Python (`SALib`), R (`sensitivity`), or in-house sampler.
3. For each vector, overwrite `calibration.cal` with the corresponding `pctchg` records and call the SWAT+ binary with the TxtInOut directory as the working directory.
4. Read the metric from `channel_sd_day.txt` (or whichever output your objective uses), record it.
5. Hand the matrix and the vector to your sensitivity routine.

Related pages

- Parameter change files. Format of `cal_parms.cal` and `calibration.cal`.
- Hard calibration. The mechanism each sensitivity run drives, one parameter vector at a time.

This section takes the small datasets that ship in `swatplus/refdata/` and walks them apart piece by piece. Each tutorial assumes you have installed SWAT+ and read the Getting Started pages.

Pages

- Ames_sub1 walkthrough. A one-subbasin, twelve-HRU watershed near Ames, Iowa, simulated 1975 to 2020. Open every input group, run, and read the outputs.
- Osu_1hru walkthrough. A single-HRU paddy near Osu, Korea, simulated 2010 to 2023. Same structure as Ames at the smallest possible scale.
- Carbon notebook. The Jupyter notebook in `swatplus/jupyter_notebooks/carbon.ipynb` that plots the soil-carbon outputs.
- Cookbook recipes. Short, copy-and-paste edits: add an HRU, switch the PET method, switch to Green-Ampt, change the simulation period, turn on the carbon model.

`Ames_sub1` is the smallest realistic SWAT+ project in the source tree. One subbasin near Ames, Iowa. Twelve HRUs. Forty-six years of daily climate (1975 to 2020). The folder is at `swatplus/refdata/Ames_sub1`.

This page opens every file in the project, in the order SWAT+ reads them, explains what is in it, and then runs the model and reads the outputs. Use it as the second pass over a SWAT+ project after the Quick start.

Prerequisites

- A working SWAT+ executable. See Install.
- A local copy of `refdata/Ames_sub1`.

file.cio: the index of every other input

`file.cio` tells SWAT+ which other files exist. It is the first file the model reads. The literal token `null` means "this slot is not used in this project".

Open `Ames_sub1/file.cio`. The relevant rows for this project are:

```
simulation        time.sim          print.prt         null              object.cnt        null
basin             codes.bsn         parameters.bsn
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli
 ↪  tmp.cli
connect           hru.con
hru               hru-data.hru
hydrology         hydrology.hyd     topography.hyd
hru_parm_db       plants.plt        fertilizer.frt    tillage.til       pesticide.pes     ...
 ↪  snow.sno
ops               harv.ops          graze.ops         irr.ops           chem_app.ops
 ↪  fire.ops   sweep.ops
lum               landuse.lum       management.sch    cntable.lum       cons_practice.lum
 ↪  ovn_table.lum
init              plant.ini         soil_plant.ini
soils             soils.sol         nutrients.sol
decision_table    lum.dtl
regions           ls_unit.ele       ...  aqu_catunit.ele
```

Most slots are `null`. Ames_sub1 has no channels, no reservoirs, no routing units, no aquifer file. It is a pure HRU water-balance run. The `connect` row points to `hru.con` only.

Reference: file.cio.

simulation group

time.sim

```
day_start  yrc_start   day_end   yrc_end      step
       0       1975         0      2020         0
```

`step = 0` means a daily time step. `day_start = 0` and `day_end = 0` mean run from January 1 of `yrc_start` to December 31 of `yrc_end`.

Reference: time.sim.

print.prt

The print configuration. The first block sets the warm-up and date window:

```
nyskip      day_start  yrc_start  day_end   yrc_end   interval
0           0               1975         0     2020          1
```

`nyskip = 0` means no warm-up years are discarded from output. The objects table that follows toggles each output file. In Ames_sub1, every row is set to `aa` only:

```
objects                   daily       monthly        yearly         avann
basin_wb                      n             n             n             y
hru_wb                        n             n             n             y
...
```

That is why the run produces `*_aa.txt` files and no daily or monthly outputs.

Reference: print.prt.

object.cnt

Counts of objects to allocate. SWAT+ reads this once and sizes its arrays accordingly.

Reference: object.cnt.

basin group

codes.bsn

Method switches that apply to the whole basin. Open `codes.bsn` and read row 3:

```
pet  event  crack  swift_out  sed_det  rte_cha  deg_cha  wq_cha  nostress  cn  c_fact  carbon
 ↪  lapse  uhyd  sed_cha  tiledrain  wtable  soil_p  gampt  atmo_dep  stor_max  i_fpwet
 ↪  gw_flow  idc_till
  2      0      0          0        0        0        0       0         0   2       0       1
 ↪  0     1        0          0       0       0      0         0         0        0        0
 ↪  3
```

Notable settings for this project:

- `pet = 2`: Hargreaves PET.
- `carbon = 1`: reserved C-FARM slot, which runs no carbon model. The dynamic carbon model now activates on `carbon = 2`; this dataset must be changed to `carbon = 2` to run it.
- `uhyd = 1`: gamma-function unit hydrograph.
- `gampt = 0`: curve-number infiltration (the default).

Reference: codes.bsn.

parameters.bsn

Numeric constants used basin-wide (sediment routing coefficients, plant-water-stress thresholds, surface-runoff lag, and so on). The defaults in this file are the same defaults a fresh QSWAT+ project produces.

Reference: parameters.bsn.

climate group

The Ames station has only two climate files: `ames.pcp` (precipitation) and `ames.tem` (temperature, legacy `.tem` extension). The `weather-sta.cli` file lists one station, `weather-wgn.cli` holds the monthly statistical generator for it, and `pcp.cli` and `tmp.cli` are index files that point at `ames.pcp` and `ames.tem` respectively.

PET is computed by Hargreaves (`pet = 2` above), so there is no PET file. Solar radiation, humidity, and wind are generated from the weather generator.

References: *.cli, *.pcp, *.tmp, *.tem.

connect, hru, hydrology groups

hru.con

The connectivity file for HRU objects. One row per HRU with its area, coordinates, elevation, and the object the HRU drains to. In Ames_sub1 every HRU drains to itself (`hyd_typ = tot` to a downstream object of `null`), so the basin output is the sum of HRU contributions with no routing.

Reference: *.con.

hru-data.hru

The pointer table. Twelve rows, one per HRU. Each row names the topography record, hydrology record, soil, land-use management, soil/plant initialization, surface storage, snow, and field for that HRU:

```
id  name      topo          hydro     soil      lu_mgt           soil_plant_init  surf_stor
 ↪  snow     field
 1  hru0001   topohru0001   hyd0001   soil_01-h1   cosy_lum      soilplant1       null
 ↪  snow001  null
 2  hru0002   topohru0002   hyd0002   soil_02      mntill_corn_lum soilplant1     null
 ↪  snow001  null
...
12  hru0012   topohru0012   hyd0012   soil_12      cosy_lum      soilplant1       null
 ↪  snow001  null
```

Eleven of the twelve HRUs are `cosy_lum` (corn-soy rotation). HRU 2 is `mntill_corn_lum` (minimum-till continuous corn). That single contrast is the point of the dataset: the other HRUs differ only in soil.

Reference: hru-data.hru.

hydrology.hyd, topography.hyd

Per-HRU hydrology and topography parameters. `hydrology.hyd` carries the curve number adjustment, lateral and tile flow coefficients, perco coefficient, ESCO, EPCO, and so on. `topography.hyd` carries slope, slope length, and field length.

hru_parm_db, ops, lum groups

These are the parameter and operation databases the HRUs and management schedules look things up in.

- `plants.plt`: plant-growth parameters per land cover code.
- `fertilizer.frt`, `tillage.til`, `pesticide.pes`, `urban.urb`, `septic.sep`, `snow.sno`: input databases.
- `harv.ops`, `graze.ops`, `irr.ops`, `chem_app.ops`, `fire.ops`, `sweep.ops`: management operation databases.
- `landuse.lum`: each row binds one `lu_mgt` name (used in `hru-data.hru`) to a CN, a management schedule, a conservation practice, and an n-table.
- `management.sch`: ordered list of operations per management name. The Ames dataset has two: `cosy_lum` (corn-soy rotation) and one continuous-corn variant.
- `cntable.lum`, `cons_practice.lum`, `ovn_table.lum`: lookups referenced by `landuse.lum`.

References: plants.plt, fertilizer.frt, tillage.til, snow.sno, *.ops, landuse.lum.

init, soils, decision_table groups

- `plant.ini` and `soil_plant.ini`: starting conditions for plant biomass, residue, soil water content, soil nutrient pools.
- `soils.sol` and `nutrients.sol`: layered soil profiles and per-layer nutrient pools for each named soil.
- `lum.dtl`: decision tables that govern automatic operations (planting, harvesting) inside the management schedule.

References: soils.sol, *.dtl.

carbon database

The dataset ships `carbon = 1` in `codes.bsn`, which is now the reserved C-FARM slot and runs no carbon model. Set `carbon = 2` to activate the dynamic CENTURY/SWAT-C model; coefficients then come from `carbon.bsn` and `carbon_lyr.bsn`, and initial carbon pools from `soil_plant.ini`.

Reference: carb_coefs.cbn.

Run the model

From inside the `Ames_sub1` folder:

```
./swatplus-<version>-linux-x86_64
```

The console will fill with `Original Simulation  mm  dd  yyyy Yr N of 46`. The full run takes a few seconds on a recent laptop.

Read the outputs

When the run finishes, the folder contains a new set of files. Start with:

simulation.out

The last lines should be:

```
Original Simulation       12  31  2020 Yr   46 of   46 Time   8:49:53

Execution successfully completed
```

If you do not see `Execution successfully completed`, the run aborted. Look at the last simulated day printed and check `diagnostics.out`.

diagnostics.out

Every line here is a missing reference. The shipped run prints:

```
DIAGNOSTICS.OUT FILE
corn_lum                CORN_rot     not found in management.sch
agrr_lum                AGRR_rot     not found in management.sch
soyb_lum                SOYB_rot     not found in management.sch
oats_lum                OATS_rot     not found in management.sch
wwht_lum                WWHT_rot     not found in management.sch
ames.tem                             file not found (tgage)
                                     file not found (basins_carbon.tes)
```

The first five lines are land-use entries in `landuse.lum` that reference schedules not present in `management.sch`. They are unused in the run (none of the HRUs use those `lu_mgt` names), so the outputs are still correct. The `ames.tem` line refers to a station gage slot in `weather-sta.cli` that was set; the file exists, but the reader did not match it to the `tgage` slot. `basins_carbon.tes` is an optional carbon test file.

Reference: Checker and diagnostics.

checker.out

One row per HRU of the resolved soil parameters:

```
sname     hydgrp  zmx(mm)  usle_k  sumfc(mm)  sumul(mm)  usle_p  usle_ls  esco   epco   cn3_swf
 ↪  perco  latq_co  tiledrain
soil_01   B       2000.0   0.0     361.5      616.18     1.0     0.3638   0.95   1.0    0.0
 ↪  0.5    0.01     0
soil_02   B       2000.0   0.0     361.5      445.67     1.0     0.3638   0.95   1.0    0.0
 ↪  0.5    0.01     0
...
```

In this dataset `usle_k = 0.0` on every HRU. That is a known artifact of the Ames soil definitions; sediment yield will be zero. The `zmx` is 2000 mm on every HRU; the soil rooting depth is taken from `soils.sol`. The `tiledrain` flag is `0` on every HRU.

basin_wb_aa.txt

The basin-average annual water balance. Columns are documented in Water balance outputs. A typical line for Ames_sub1:

```
precip   snofall  snomlt   surq_gen  latq   wateryld  perc   et       pet
1905.2   220.7    219.997  59.5      0.06   59.6      0.003  1846.4   3464.5
```

Precipitation is 1905 mm, ET is 1846 mm, percolation is essentially zero, and surface runoff is 60 mm. The result is dominated by ET because the basin is small and well-drained and the simulation is at HRU scale only.

Other outputs

- `hru_wb_aa.txt`: same fields as `basin_wb_aa.txt`, one row per HRU.
- `hru_nb_aa.txt`, `hru_ls_aa.txt`, `hru_pw_aa.txt`: nutrient balance, landscape balance, plant and water stress, per HRU.
- `crop_yld_aa.txt`: average annual yield by HRU and crop.
- `basin_carbon_aa.txt`, `hru_carbon_aa.txt`: carbon outputs (only when `carbon = 2` activates the carbon model).

What to change next

- Switch the PET method by editing `codes.bsn` (`pet = 0` for Priestley-Taylor, `pet = 1` for Penman-Monteith). See the cookbook.
- Turn on daily output by setting `daily` to `y` in `print.prt` for the rows you care about.
- Change `nyskip` in `print.prt` to discard the first few years as warm-up.
- Read the Carbon notebook for plotting the carbon outputs.

`Osu_1hru` is the smallest SWAT+ project that exercises every input group. One HRU, ten hectares, near Osu, Korea. The land use is `rice140_lum` (a 140-day rice rotation). The folder is at `swatplus/refdata/Osu_1hru`.

Use this dataset when you want to see every input file in `file.cio` populated, but only one HRU to track.

Prerequisites

- A working SWAT+ executable. See Install.
- A local copy of `refdata/Osu_1hru`.

file.cio: every group used

Unlike Ames_sub1, this project populates almost every slot. Open `Osu_1hru/file.cio`:

```
simulation        time.sim          print.prt         null              object.cnt
basin             codes.bsn         parameters.bsn
climate           weather-sta.cli   weather-wgn.cli   null              pcp.cli
 ↪  tmp.cli           slr.cli           null              wnd.cli
connect           hru.con           null              rout_unit.con     null
 ↪  aquifer.con       null              null              null      recall.con
 ↪  null              null              null              chandeg.con
channel           initial.cha       null              null              null
 ↪  nutrients.cha     channel-lte.cha   hyd-sed-lte.cha
reservoir         initial.res       null              null              sediment.res
 ↪  nutrients.res     weir.res          wetland.wet       hydrology.wet
routing_unit      rout_unit.def     rout_unit.ele     rout_unit.rtu
hru               hru-data.hru
aquifer           initial.aqu       aquifer.aqu
hydrology         hydrology.hyd     topography.hyd    field.fld
```

The project has channels, aquifers, routing units, reservoirs, recall, and a wetland. Most of those tables are populated with template entries from the SWAT+ Editor; the single HRU in `hru.con` is what actually runs.

Reference: file.cio.

time.sim and print.prt

`time.sim`:

```
day_start  yrc_start   day_end   yrc_end      step
        1       2010       365      2023         0
```

January 1 2010 through December 31 2023, daily.

`print.prt` has `nyskip = 3`, so the first three years are discarded. The objects table sets `basin_wb` to `yearly` and `avann`. Every other object is `avann` only.

References: time.sim, print.prt.

codes.bsn

```
pet  event  crack  rtu_wq  sed_det  rte_cha  deg_cha  wq_cha  nostress  cn  c_fact  carbon
 ↪  lapse  uhyd  sed_cha  tiledrain  wtable  soil_p  gampt  atmo_dep  stor_max  i_fpwet
  1      0      0       1         0        0       0       1         0   0       0       0
 ↪  0     1        0          0       0       0      0          a        0        0
```

Settings worth noting:

- `pet = 1`: Penman-Monteith PET. Needed because the climate has solar radiation, humidity, and wind.
- `rtu_wq = 1`: routing-unit water quality on.
- `wq_cha = 1`: in-stream water quality on.
- `carbon = 0`: legacy static mineralization (no C-FARM, no Century).
- `gampt = 0`: curve-number infiltration.

Reference: codes.bsn.

climate

Five climate files for the Imsil station:

- `Imsilpcp.pcp`: daily precipitation.
- `Imsiltmp.tmp`: daily max and min temperature.
- `Imsilsol.slr`: daily solar radiation.
- `Imsilwind.wnd`: daily wind speed.

(Relative humidity is generated from the weather generator.) The index files `pcp.cli`, `tmp.cli`, `slr.cli`, `wnd.cli` each list one filename, and `weather-sta.cli` binds those four index files to the station `s35610n127290e`.

References: *.cli, *.pcp, *.tmp.

hru.con: a single HRU

```
id  name     gis_id  area  lat       lon         elev    hru  wst              cst  ovfl  rule
 ↪  out_tot
 1  hru0001       1    10  35.65311  127.36763   350.97   1   s35610n127290e   0    0     0
 ↪  0
```

Area is 10 ha. `hru = 1` and `wst = "s35610n127290e"` bind this row to HRU 1 and to the weather station. `out_tot = 0` means no downstream routing object: the run produces HRU-scale outputs only.

Reference: *.con.

hru-data.hru

The single HRU points at:

```
id  name      topo          hydro      soil        lu_mgt        soil_plant_init  surf_stor
 ↪  snow     field
 1  hru0001   topohru0001   hyd0001    PadHOEGOG   rice140_lum   soilplant1       paddy0001
 ↪  snow001  null
```

- `soil = PadHOEGOG`: a paddy soil. See `soils.sol`.
- `lu_mgt = rice140_lum`: maps in `landuse.lum` to plant community `rice140_comm` and management schedule `rice140_rot`.
- `surf_stor = paddy0001`: surface-storage record for ponded water. This is what makes it a paddy.

Reference: hru-data.hru.

landuse.lum and management.sch

`landuse.lum` row for `rice140_lum` binds the HRU to:

- `plnt_com = rice140_comm` (plant community in `plant.ini`).
- `mgt = rice140_rot` (management schedule in `management.sch`).
- `cn2 = rc_strow_g` (curve-number table entry in `cntable.lum`).
- `cons_prac = cross_slope`.
- `ov_mann = convtill_res`.

`management.sch` then describes the 140-day rice rotation: paddy-fill, transplant, fertilize, harvest, drain.

Reference: landuse.lum.

aquifer.aqu and aquifer.con

`aquifer.aqu` defines twenty shallow aquifer records (`aqu011` through `aqu102`) plus one deep aquifer (`aqu_deep010`). They share identical parameters. The single HRU links to one of them through `aquifer.con` so the percolation has somewhere to go.

Reference: *.aqu.

channel-lte.cha and hyd-sed-lte.cha

A template channel table is present so the file structure validates, but with `out_tot = 0` on the HRU, no flow reaches a channel and the channel outputs will be zero.

Reference: *.cha.

Run the model

```
cd path/to/Osu_1hru
./swatplus-<version>-linux-x86_64
```

The run takes a few seconds because there is only one HRU. Watch the console for `Original Simulation` lines.

Read the outputs

simulation.out

The last line should be `Execution successfully completed`.

diagnostics.out

Every line is a missing reference. Expect a long list because the Editor wrote many template channel and aquifer rows whose companion property records do not all exist. The HRU run itself is unaffected.

Reference: Checker and diagnostics.

checker.out

A single row for `PadHOEGOG`. Confirm `zmx`, `sumfc`, `sumul`, `hydgrp` are reasonable for a paddy.

basin_wb_aa.txt and basin_wb_yr.txt

Annual and average-annual water balance for the basin. Because there is one HRU, basin equals HRU. Compare `basin_wb_yr.txt` (per-year values) to `basin_wb_aa.txt` (the average over years 4 through 14, since `nyskip = 3`).

hru_wb_aa.txt

The water-balance row for HRU 1. Look at `irr` (irrigation depth, large for a paddy), `surq_gen` (surface runoff), `et`, `perc`.

Plant outputs

`crop_yld_aa.txt` reports rice yields by HRU and crop. `hru_pw_aa.txt` carries plant biomass and water-stress days.

What to change next

- Switch to Hargreaves PET (`pet = 2` in `codes.bsn`). The model will stop reading the solar, humidity, and wind files.
- Turn on daily output for `basin_wb` to inspect the paddy fill and drain pattern through the season.
- Change the simulation period in `time.sim` to a single year for a fast iteration loop.

See the cookbook for these edits.

The SWAT+ source tree ships one analysis notebook: `jupyter_notebooks/carbon.ipynb`. It reads the carbon-output files for a single HRU and draws time-series plots of every carbon pool the C-FARM / Century model tracks. Use it after running a project with the carbon model enabled.

What it reads

The notebook reads the following files from a SWAT+ output folder:

- `hru-data.hru` and `landuse.lum`. Used to look up the soil name, the `lu_mgt` name, and the management identifier for a chosen HRU.
- `mgt_out.txt`. Operation log: planting, harvest, tillage, fertilization. The notebook overlays these as event markers on every plot.
- `hru_cbn_lyr.txt`. Total soil carbon per soil layer.
- `hru_seq_lyr.txt`. Sequestered carbon per soil layer.
- `hru_rsdc_stat.txt`. Surface residue and soil carbon pools.
- `hru_plc_stat.txt`. Plant carbon pools.
- `hru_cpool_stat.txt`. Residue and soil carbon pools by layer (metabolic, structural, lignin, microbial, slow humus, passive humus).
- `hru_org_trans_vars.txt`. Organic-matter transformation variables.
- `hru_cflux_stat.txt`. Carbon flux statistics including CO2.

The notebook expects these to be produced by a run where `print.prt` lists each of those output rows with at least one frequency turned on. Daily output is what the notebook is designed for.

What it draws

The notebook is organised by output file. Each section opens with an "Initialize" cell that loads the file into a DataFrame, then one or more graph cells produce a Plotly figure. Sections, in order:

1. Total carbon by layer depth (`hru_cbn_lyr`).
2. Total sequestered carbon and 300 mm sequestered (`hru_seq_lyr`).
3. Surface residue and soil carbon pools at 10 mm depth (`hru_rsdc_stat`).
4. Soil carbon pools at a user-specified depth.
5. Live root mass by soil layer.
6. Plant stats from `hru_plc_stat`.
7. Residue and soil carbon pools (`hru_cpool_stat`) at 10 mm.
8. Same pools at a user-specified depth.
9. Soil residue, metabolic, structural, lignin, microbial, slow humus, and passive humus pools by soil layer.
10. Soil water content by soil layer.
11. Organic-matter transformation variables (`hru_org_trans_vars`).
12. Carbon fluxes (`hru_cflux_stat`), non-CO2 and CO2.

Every plot shows the chosen variable as a time series. PLANT, HARVEST, and TILLAGE events from `mgt_out.txt` are overlaid as vertical markers so you can read the management context off the figure.

How to run it

1. Run SWAT+ on a project with the carbon model on. In `codes.bsn`, set `carbon = 2` to activate the dynamic CENTURY/SWAT-C model (`carbon = 1` is the reserved C-FARM slot and runs no carbon model). The `Ames_sub1` reference dataset ships `carbon = 1`, so change it to `carbon = 2`.
2. Turn on the relevant outputs in `print.prt`. At minimum, `hru_cb` and the carbon-pool outputs need `daily` set to `y`.
3. Open `carbon.ipynb` in JupyterLab or VS Code.
4. Edit the first code cell. Set:
   - `hru_id` to the HRU you want to plot.
   - `wdir` and `sdir` so that `f"{wdir}/{sdir}"` is the folder that holds the SWAT+ outputs.
5. Run all cells (Run All).

The notebook uses `pandas`, `plotly`, and the standard library. Install with:

```
pip install pandas plotly
```

What you should see

Each graph cell renders a Plotly figure inline. Event markers from `mgt_out.txt` line up the plots with the management schedule. The "Initialize" cells print "File not found" and stop rendering that section if the corresponding output file is missing; this is expected when an output is not enabled in `print.prt`.

Related

- Theory: Carbon covers what the pools and fluxes represent.
- print.prt for the output switches.
- Cookbook: enabling the carbon model for the input edits.

Five common edits to a working SWAT+ project. Each recipe lists the file to open, the line to change, and what to check after running. Every recipe assumes you start from a project that already runs successfully.

Add a new HRU

You need a new entry in `hru.con`, `hru-data.hru`, and a row in any companion table the new HRU points to that does not yet contain the records (`topography.hyd`, `hydrology.hyd`, `soils.sol`, …).

1. Open `hru.con`. Append a new row, copying an existing row and bumping the `id`. Edit `area`, `lat`, `lon`, `elev`, and `hru` (the HRU index). Keep `wst` pointed at an existing weather station.
2. Open `hru-data.hru`. Append a row with the same `id` and a new `name`. Point its `topo`, `hydro`, `soil`, `lu_mgt`, `soil_plant_init`, `surf_stor`, `snow`, `field` columns at names that exist in the corresponding tables.
3. Open `object.cnt` and update the `hru` count and `obj` count to reflect the new total.
4. Run. After the run, check `checker.out` for a new row matching the soil name. If `usle_k`, `zmx`, `sumfc` look wrong, your `soils.sol` row is incomplete.

References: hru-data.hru, *.con, object.cnt.

Swap the PET method

`codes.bsn`, field 3 (`pet`):

- `0`: Priestley-Taylor.
- `1`: Penman-Monteith.
- `2`: Hargreaves.
- `3`: read from `pet.cli`.

Edit the value, save, run.

Penman-Monteith needs solar radiation, relative humidity, and wind. If those climate files are absent, the weather generator will fill them in, and the result will not match a station-driven Penman-Monteith run.

Hargreaves needs only minimum and maximum temperature. It is the right choice for sites with minimal climate data.

If you set `pet = 3`, also point `pet_file` (field 1 of `codes.bsn`) at a valid `pet.cli` and provide the daily-PET file it references.

Reference: codes.bsn.

Switch to Green-Ampt infiltration

`codes.bsn`, field 21 (`gampt`):

- `0`: curve-number infiltration (default).
- `1`: Green-Ampt with Mein-Larson.

Green-Ampt runs on a sub-daily time step. Set `step` in `time.sim` to a non-zero value (the source supports `step = 24` for hourly). Your precipitation file must contain sub-daily values; a daily `.pcp` will not be disaggregated for you.

After the run, look at `basin_wb_aa.txt`. The split between `surq_gen` and `perc` shifts when the method changes; if it does not, the run is still using curve number.

References: codes.bsn, time.sim.

Change the simulation period

`time.sim`:

```
day_start  yrc_start  day_end  yrc_end  step
        0       1975        0     2020     0
```

- `day_start = 0`: start January 1 of `yrc_start`.
- `day_start = N > 0`: start on Julian day N of `yrc_start`.
- `day_end = 0`: end December 31 of `yrc_end`.
- `day_end = N > 0`: end on Julian day N of `yrc_end`.

To shorten a run, change `yrc_start` and `yrc_end`. To set a warm-up that is excluded from the average-annual outputs, leave `time.sim` alone and set `nyskip` in `print.prt`.

Your climate files must contain data for every day of the simulation period. If they do not, SWAT+ writes a "file not found" or runs off the end of the file.

References: time.sim, print.prt.

Enable the carbon model

`codes.bsn`, field 14 (`carbon`):

- `0`: static (legacy mineralization).
- `1`: C-FARM.
- `2`: Century.

For `1` or `2`, also:

1. Provide a `carb_coefs.cbn` file in the project folder. The Ames_sub1 dataset ships one you can copy. If `cswat /= 0` and no `carb_coefs.cbn` is present, SWAT+ uses internal defaults and writes a notice to `diagnostics.out`.
2. Confirm `soil_plant.ini` (or `soils.sol`, depending on tool version) carries initial organic carbon by layer. Without realistic initial pools, the first decade of output is transient as pools spin up.
3. Turn on carbon outputs in `print.prt`. Set `hru_cb` and the per-pool outputs (`hru_cbn_lyr`, `hru_cpool_stat`, `hru_cflux_stat`, …) to `daily` if you want to use the carbon notebook.

After the run, `basin_carbon_aa.txt`, `hru_carbon_aa.txt`, and the per-pool files appear in the output folder. If they are missing, the print switches were not on.

References: codes.bsn, carb_coefs.cbn, print.prt.

The authoritative API reference for the SWAT+ Fortran source is generated by FORD and published at:

https://swat-model.github.io/swatplus

That site contains:

- Every module, derived type, subroutine, and function declared in `swatplus/src/`.
- Call and caller graphs.
- Source-listing pages for each `.f90` file.
- The leading comment block of every routine, indexed and searchable.

FORD reads the Fortran source directly. Whatever is in the source is reflected on the site; the docs do not duplicate this information.

How it is built

The FORD configuration lives in `ford.md` in the root of the SWAT+ source repository. Excerpt:

```
project: SWAT+
output_dir: /drive/c/repositories/swatplus/build/doc
src_dir: src
version: <git describe>
graph: true
coloured_edges: true
```

`ford.md.in` is the template; CMake substitutes the version from `git describe` and writes the resolved `ford.md`. The build command is:

```
ford ford.md
```

The resulting HTML tree is published from the `gh-pages` branch by a GitHub Action.

When to use FORD vs this site

- Use **this site** for the input file formats, output file formats, theory, calibration workflow, and how to run the model.
- Use **the FORD site** when you need to know what a specific subroutine does, what types a routine reads and writes, or where in the call tree a given variable is touched. It is the right place to start before modifying SWAT+ source.

Related

- Developer guide: how to clone, build, test, and contribute.
- Coding conventions: the style FORD picks up from comments.

When a SWAT+ run does not produce what you expect, the first three files to read are `simulation.out`, `diagnostics.out`, and `checker.out`. Their format is described in Checker and diagnostics.

Pages

- Common runtime errors. The error strings the model writes to its diagnostic files, what they mean, and what to change.
- Water balance and mass balance issues. When `basin_wb_aa.txt` looks wrong: climate mapping, curve numbers, missing management, bad soils.
- Performance tips. Build flags, daily versus sub-daily routing, the cost of enabling many outputs.

When in doubt

1. Confirm the executable matches the input format. Use the SWAT+ revision printed on the first line of `simulation.out`.
2. Read `diagnostics.out` end to end. Every line is a missing reference that SWAT+ replaced with a default or skipped.
3. Read `checker.out`. The per-HRU `zmx`, `sumfc`, `sumul`, `tiledrain` columns are the fastest way to spot a soil or option that did not load.
4. Confirm `success.fin` exists. If it does not, the run aborted.

SWAT+ writes runtime messages to three files. The location of the message tells you the type of problem:

- `simulation.out`: console-style status. Successful runs end with `Execution successfully completed`. Failed runs stop mid-stream.
- `diagnostics.out`: missing references and ignored options. Each line is a name that one input file pointed at but that does not exist in the file it was supposed to be in. The run continues, but with a default substituted.
- `checker.out`: resolved per-HRU parameters. Bad values here often mean a soil or option silently failed to load.
- console output to stdout: a small number of `Error:` lines written directly by some readers. These often precede an abort.

Source: error and warning text strings traced to the SWAT+ source at `/drive/c/repositories/swatplus/src/`. Search the source with `grep -rn "write (9001" src/` to see every diagnostics line, and `grep -rn '"Error' src/` to see the stderr lines.

Missing input references (diagnostics.out)

These all share the form `<name> not found in <file>` or `<name> not found (<file>)`. The reader detected that an input pointed at a name that the corresponding database does not contain.

`<name> not found in management.sch`

Source: `landuse_read.f90` line 147.

A row in `landuse.lum` named a management schedule that does not exist in `management.sch`. Fix: open `landuse.lum`, find the row whose `mgt` column holds `<name>`, and either add a matching schedule to `management.sch` or point the row at one that exists.

This is a warning, not a stop. The land use will run without scheduled operations, which means no plant, no fertilize, no harvest. The HRU will not behave as intended.

<name> not found in plant.ini

Source: `landuse_read.f90` line 144.

Same pattern, different column. Fix the `plnt_com` column of the offending `landuse.lum` row, or add a row to `plant.ini`.

<name> not found (landuse.lum)

Source: `hru_read.f90` line 77.

An HRU in `hru-data.hru` named a `lu_mgt` that does not exist in `landuse.lum`. The HRU runs with no land use, which is rarely what you want.

<name> not found (soils.sol)

Source: `hru_read.f90` line 157.

An HRU named a soil that is not in `soils.sol`. The HRU will fall back to a default soil. `checker.out` is the fastest place to confirm: look for `zmx = 2000`, `usle_k = 0`, or any of your soils missing entirely.

<name> not found (topography.hyd), (hydrograph.hyd), (snow.sno)

Same family. The pointer in `hru-data.hru` does not match any record in the named file. Verify the name in both files matches exactly. Names are case-sensitive.

<name> not found (initial.cha), (hydrology.cha), (sediment.cha), (nutrients.cha)

Source: `ch_read.f90` lines 97 to 100.

A channel definition references a property record that is missing. The channel will use defaults. If your project has no channels, ignore.

<name> not found (res-init), (res-hyd), (res-release), (res-sed), (res-nut)

Source: `res_read.f90` and `wet_initial.f90`. Same pattern for reservoirs and wetlands. Fix the corresponding pointer.

file not found (wgn), (pgage), (tgage), (sgage), (hgage), (wgage), (petgage), (atmodep)

Source: `cli_staread.f90` lines 72 to 95.

A weather station in `weather-sta.cli` named a gage that does not match any row in the corresponding `.cli` index file. Verify the index file lists the station file under the same name. The Ames_sub1 shipped dataset prints `ames.tem  file not found (tgage)` because the editor wrote the station with a `.tem` extension that the `.tmp` index does not contain.

file not found (basins_carbon.tes)

Source: `carbon_read.f90` line 19.

Optional carbon-test file. Safe to ignore unless you are running the carbon test harness.

WARNING: The variable <name> in the input file carb_coefs.cbn is not a recognized variable.

Source: carbon_coef_read.f90 line 141.

The reader did not recognize a variable name in carb_coefs.cbn. Check spelling against the names in carbon_coef_read.f90. The variable is ignored.

Warning: Input file named <name> is missing or null.

Source: utils.f90 line 69. Emitted by the generic CSV/table reader when a referenced file is null or missing. Usually benign because most slots in file.cio accept null.

Warning: unknown column header named <name>

Source: utils.f90 line 874. The reader's column-name table did not include the header it found. The column is skipped. Often caused by a file written by a newer or older editor than the executable expects.

Errors written to stdout

Error: The output object <name> in the input file print.prt is not a valid object.

Source: basin_print_codes_read.f90 line 665.

A row in print.prt names an object that is not in the recognized list. Fix the row name. The list of valid objects is in basin_module.f90.

Error: <name> print object is duplicated in the input file print.prt. Aborting

Source: basin_module.f90 line 429.

A print.prt row appears twice. Delete the duplicate.

Error: The number of nmbr_soil_test_layers has not been specified

Source: carbon_coef_read.f90 line 120. The carb_coefs.cbn file is missing the nmbr_soil_test_layers declaration. Add it before the soil-test block.

Error: The number input soil test layers exceeds nmbr_soil_test_layers of <n>

Source: carbon_coef_read.f90 line 127. You declared nmbr_soil_test_layers = n and then provided more than n soil-test rows. Match them.

Failure modes that are not text errors

Run stops mid-day with no Execution successfully completed

Look at the last Original Simulation mm dd yyyy line in simulation.out. The crash is on or near that date. Common causes:

- Climate file is shorter than the simulation period.
- A divide-by-zero in a hydrology routine (most often caused by zero soil-layer depths or zero plant biomass).
- The compiler-generated STOP from a numerical routine (look for an integer code in console output).

success.fin is missing

The run did not complete. The model writes success.fin only on a clean exit.

Output files are not written

The corresponding row in `print.prt` is set to `n` for that interval. Cross-check against `files_out.out`, which lists every output file the run actually opened.

`Permission denied` (Linux, macOS)

The executable file is not marked executable. Run `chmod +x swatplus-<version>-linux-x86_64`.

`Error opening file ...`

The model is not running from inside the project folder. `cd` into the folder, or pass a full path to a launcher that does so.

Related

- Checker and diagnostics for the file structures.
- Water balance issues for output sanity checks.

When `basin_wb_aa.txt` does not match expectation, the cause is almost always in one of four places: climate input mapping, curve numbers, the management schedule, or soil parameters. Inspect them in that order. Compare the offending values in `checker.out` and `diagnostics.out` against the source of truth in your input files.

What to compare

The basin-average annual water balance from `basin_wb_aa.txt` should approximately satisfy:

`precip = et + surq_gen + latq + perc + snopack_storage + soil_water_change`

Where `wateryld = surq_gen + latq + qtile`. Big discrepancies in any one term point at a specific input group.

Symptom: precip is wrong

The reported `precip` does not match the rainfall in the source `.pcp` file.

- Open `weather-sta.cli`. Confirm the station name and that `pgage` points at the right `.cli` index.
- Open the `.cli` index. Confirm it lists the `.pcp` filename that contains the right data.
- Open the `.pcp` file and check the first record. The header carries the `nbyr`, `tstep`, `lat`, `lon`, and `elev` for the station. Lat and lon mismatched with `hru.con` can mean the wrong station was assigned to the HRU.

If `diagnostics.out` contains `<name> file not found (pgage)`, the climate mapping failed entirely and SWAT+ generated rainfall from the weather generator. The reported `precip` will not match any observed dataset.

References: *.cli, *.pcp.

Symptom: surq_gen is too high or too low

Surface runoff is controlled by:

1. The curve number for the (soil hydrologic group, land cover) pair, taken from `cntable.lum` and modified by `cn3_swf` in `hydrology.hyd`.
2. The soil-water content, computed from `soils.sol` field capacity and saturation.
3. The infiltration method: `gampt = 0` (curve number) or `gampt = 1` (Green-Ampt) in `codes.bsn`.

Checks:

- Open `checker.out`. Confirm `hydgrp` (A, B, C, D) per soil matches what your `soils.sol` declares. A misclassification flips runoff by tens of mm.
- Open `cntable.lum`. The CN values are looked up by the `cn2` column of the offending `landuse.lum` row. Cross-check the value against a CN table for that land cover / hydrologic group pair.

- Check `cn3_swf` in `hydrology.hyd`. Defaults to zero; a value near 1 will damp runoff response to wet conditions.
- Confirm `gampt` in `codes.bsn` is what you intended.

References: Checker and diagnostics, codes.bsn, landuse.lum.

Symptom: ET is too high or too low

ET depends on the PET method and on plant cover.

- `codes.bsn` field 3 (`pet`): 0 Priestley-Taylor, 1 Penman-Monteith, 2 Hargreaves, 3 file. Penman-Monteith needs solar radiation, humidity, and wind; if those are missing, the model uses generated values.
- `esco` and `epco` in `hydrology.hyd` (per HRU) control soil evaporation and plant uptake compensation. Defaults in `checker.out` are 0.95 and 1.0; deviations from those change the ET partition.
- Plant cover: if the management schedule is wrong, plants never grow, `eplant` stays low, and the run produces more bare-soil evaporation and runoff.

Open `checker.out` and confirm `esco` and `epco` match what you expect. Open `hru_pw_aa.txt` and inspect biomass and LAI; near-zero biomass on a cropped HRU means management did not fire.

References: codes.bsn, hru.md.

Symptom: management never fires

When `landuse.lum` points at a schedule that does not exist, `diagnostics.out` writes:

```
<lum_name>  <mgt_name>  not found in management.sch
```

The HRU runs with no operations: no plant, no fertilize, no harvest. `hru_pw_aa.txt` will show zero biomass and zero yield. `crop_yld_aa.txt` will be empty or report a residual.

Fix by either matching `landuse.lum%mgt` to a schedule name in `management.sch`, or adding the schedule.

Reference: *.dtl covers decision-table-driven operations.

Symptom: percolation is zero or saturated

Percolation depends on the soil profile in `soils.sol`:

- `sumfc` (column in `checker.out`): the total field capacity over the profile. A near-zero value means the soil-layer field capacities did not load.
- `sumul`: the saturation upper limit. Should be larger than `sumfc`.
- The `perco` coefficient in `hydrology.hyd` scales the percolation rate. Default 0.5; setting to 0 stops percolation.

A `zmx = 2000` row in `checker.out` for every HRU usually means the per-soil `soils.sol` block was not parsed. Look for warnings in `diagnostics.out` referencing the soil name.

References: soils.sol, checker.out.

Symptom: bad latq, qtile, or surq_runon

These depend on:

- `latq_co` in `hydrology.hyd`. The lateral-flow coefficient. Zero turns lateral flow off. Default in the shipped Ames data is 0.01.
- `tiledrain` flag in `hydrology.hyd`. Confirm `checker.out` shows `0` (no tile) or `1` (tile) as intended.
- Tile drainage parameters: see the `tile` block in `hydrology.hyd` and the `tiledrain.str` structural file.

Symptom: ru_aa.txt and basin_wb_aa.txt disagree

The routing-unit output sums HRU contributions through routing. The basin output sums them directly. A large disagreement means routing is doing something unexpected. Check:

- `rout_unit.def`, `rout_unit.ele`, `rout_unit.rtu`. The routing definition and area weights.
- `chandeg.con` and the channel-routing inputs. A misconfigured channel that has no length or no slope produces bizarre routed values.

Workflow

1. Run the model. Confirm `success.fin` exists.
2. Read `diagnostics.out` end to end. Resolve every line.
3. Open `checker.out`. Per HRU, sanity-check `hydgrp`, `zmx`, `sumfc`, `sumul`, `usle_k`, `esco`, `epco`, `perco`, `latq_co`, `tiledrain`.
4. Open `basin_wb_aa.txt`. Compare each term against your expectation. If precip is wrong, fix climate mapping. If ET is wrong, check PET and esco/epco. If surface runoff is wrong, check CN. If percolation is wrong, check soils.

Related

- Checker and diagnostics for the structure of the diagnostic files.
- Common runtime errors for the exact warning strings.
- Water balance outputs for the column definitions.

SWAT+ is a serial Fortran program. There is no MPI, no OpenMP. The run time is set by the simulation length, the number of objects (HRUs, channels, aquifers), the time step, and how much output is written. The build is parallel; the model is not.

Build with `-j`

CMake passes the parallel-jobs flag to the underlying build tool. Use it on the build step:

```
cmake -B build
cmake --build build -j 8
```

`-j 8` runs eight compile jobs at once. Use a value close to your physical core count. The build is CPU-bound; oversubscribing the I/O does not help.

A clean build of SWAT+ on a recent laptop takes a couple of minutes with `-j 8` and around ten minutes with `-j 1`.

Choose an optimized build type

For production runs, build in Release mode (debug symbols and runtime checks off):

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j 8
```

This is the default in the SWAT+ `CMakeLists.txt`. If you have configured a Debug build, expect runs to be five to ten times slower.

For development, use Debug or RelWithDebInfo so floating-point checks are enabled.

Daily versus sub-daily routing

`time.sim` field `step`:

- `step = 0`: daily. Fastest.
- `step > 0`: sub-daily. Required for Green-Ampt infiltration (`gampt = 1` in `codes.bsn`) and for sub-daily channel routing.

Sub-daily routing multiplies the inner-loop work by the number of sub-daily steps per day. A 46-year daily run that finishes in ten seconds will take several minutes at hourly. Use sub-daily only when the science needs it. Green-Ampt and hourly stage data are the most common reasons.

References: time.sim, codes.bsn.

The cost of enabling many outputs

Every row in `print.prt` that is set to `daily` writes one record per object per day. The cost grows with the simulation length, the number of objects, and the number of rows.

Concretely:

- 46 years of daily output for 12 HRUs and 10 output rows is 12 * 46 * 365 * 10 = around 2 million records. The model produces these in seconds, but parsing them downstream can take longer than the simulation itself.
- The `aa` (average-annual) interval is essentially free. One record per object at end of run.
- `mon` (monthly) is 12 per year. `yr` (yearly) is 1 per year.

If you only need average-annual results, leave everything at `aa` (as the Ames_sub1 default does). Turn `daily` on only for the variables you will plot.

The CSV-output flag (`csvout = y` in `print.prt`) also adds cost: every output is written twice (once as fixed-width, once as CSV). Disable when not needed.

Reference: print.prt.

Object count

Run time scales roughly linearly with the number of HRUs, channels, aquifers, and routing units in the project. A 12-HRU project (Ames_sub1) finishes in a few seconds. A 5000-HRU regional project takes minutes per simulated decade.

If you are iterating on inputs, prototype on a small subset and scale up only when the inputs are stable.

I/O

SWAT+ reads inputs once at the start and writes outputs progressively during the run. Putting the project folder on a local SSD instead of a network share matters more than CPU when many outputs are enabled.

Profiling

Compile with `-pg` (gfortran) or the Intel compiler's profile flags, run a representative scenario, and inspect with `gprof` or the Intel tools. Most time in a typical project goes to `pl_grow` (plant growth), the soil-water routines, and the channel routing. Adding more daily output rows shifts the balance toward the I/O routines.

Summary

Lever	Effect
`cmake --build build -j N`	Build time, no run-time effect.
`CMAKE_BUILD_TYPE=Release`	Five to ten times faster runs.
`step = 0` in `time.sim`	Daily, fastest.
`step > 0` in `time.sim`	Sub-daily, multiplies inner-loop cost.
Many `daily` rows in `print.prt`	More disk and downstream parsing cost.

Lever	Effect
Use `aa` only	Cheapest output mode.
Local SSD vs network share	Real difference when many outputs are on.